FPT UNIVERSITY

Capstone Project Report

Report 4 – Software Design Document

– Can Tho, October 2026 –

Table of Contents

Definition and Acronyms **10**

I. Record of Changes **12**

II. Software Design Document **13**

1. System Design **13**

1.1 System Architecture 13

1.2 Package Diagram 16

2. Database Design **21**

2.1 Table Relationships 21

2.2 Module Entity–Relationship Diagrams 30

3. Detailed Design **43**

3.1 Verify JWT 43

3.2 Authentication 45

3.3 Role Management 52

3.4 Job Browsing 59

3.5 Job Management 61

3.6 Bookmarked Jobs Management 68

3.7 Forum Management 72

3.8 Manage Conversation 91

3.9 Saved Search Management 99

3.10 Job Alert Management 105

3.11 Permission Management 111

3.12 Session Management 113

3.13 Forum Category Management 116

3.14 Manage Moderation 121

3.15 Manage ID Verification 125

3.16 Manage Forum Trash 129

3.17 Proposal Management 133

3.18 User Management 143

3.19 Banner Management 152

3.20 Skill Management 158

3.21 Expert Profile Management 165
3.22 Job Category Management 167
4. Reference 174

Table of Figures

Figure 1 . System Architecture Diagram 13
Figure 2 . Package diagram 17
Figure 3 . Entity–relationship diagram of the Frevia database 21
Figure 4 . ERD1 — Authentication & Sessions 31
Figure 5 . ERD2 — Role-Based Access Control 32
Figure 6 . ERD3 — Profile Core 33
Figure 7 . ERD4 — Profile History & Social Links 33
Figure 8 . ERD5 — Identity Verification & Profile Visits 34
Figure 9 . ERD6 — Portfolio & Skills 35
Figure 10 . ERD7 — Favorites & Follows 35
Figure 11 . ERD8 — Job Posting 36
Figure 12 . ERD9 — Job Discovery & Alerts 37
Figure 13 . ERD10 — Proposal 37
Figure 14 . ERD11 — Contract 38
Figure 15 . ERD12 — Milestone & Project Progress 38
Figure 16 . ERD13 — Milestone Submission 39
Figure 17 . ERD14 — Escrow, Payments & Certificates 39
Figure 18 . ERD15 — Dispute Core 40
Figure 19 . ERD16 — Dispute Resolution 40
Figure 20 . ERD17 — Reviews & Reputation 41
Figure 21 . ERD18 — Forum Posts & Comments 41
Figure 22 . ERD19 — Forum Likes 41
Figure 23 . ERD20 — Forum Reports & Moderation 42
Figure 24 . ERD21 — Messaging & Notifications 42
Figure 25 . ERD22 — Administration & System 43
Figure 26 . Class diagram of Verify JWT 44
Figure 27 . Sequence diagram of UC-0 Verify JWT 45
Figure 28 . Class diagram of Authentication 46
Figure 29 . Sequence diagram of UC-01 Login with Google 47
Figure 30 . Sequence diagram of UC-02 Login 48
Figure 31 . Sequence diagram of UC-03 Logout 49
Figure 32 . Sequence diagram of UC-04 Register 50

Figure 33 . Sequence diagram of UC-05 Forgot Password 51
Figure 34 . Class diagram of Role Management 52
Figure 35 . Sequence diagram of UC-06.01 View Role List 53
Figure 36 . Sequence diagram of UC-06.02 View Role Detail 54
Figure 37 . Sequence diagram of UC-06.03 Create Role 55
Figure 38 . Sequence diagram of UC-06.04 Update Role 56
Figure 39 . Sequence diagram of UC-06.05 Delete Role 57
Figure 40 . Sequence diagram of UC-06.06 Update Role Permissions 58
Figure 41 . Class diagram of Job Browsing 59
Figure 42 . Sequence diagram of UC-10.01 View Job List 60
Figure 43 . Sequence diagram of UC-10.02 View Job Detail 61
Figure 44 . Class diagram of Job Management 62
Figure 45 . Sequence diagram of UC-11.01 View My Jobs 63
Figure 46 . Sequence diagram of UC-11.02 Create Job 64
Figure 47 . Sequence diagram of UC-11.03 Publish Job 65
Figure 48 . Sequence diagram of UC-11.04 Update Job 66
Figure 49 . Sequence diagram of UC-11.05 Delete Job 67
Figure 50 . Sequence diagram of UC-11.06 Change Job Status 68
Figure 51 . Class diagram of Bookmarked Jobs Management 69
Figure 52 . Sequence diagram of UC-12.01 View Bookmarked Job List 70
Figure 53 . Sequence diagram of UC-12.02 Bookmark Job 71
Figure 54 . Sequence diagram of UC-12.03 Remove Bookmark 72
Figure 55 . Class diagram of Forum Management 73
Figure 56 . Sequence diagram of UC-14.01 View Forum Category List 74
Figure 57 . Sequence diagram of UC-14.02 View Forum Category Detail 75
Figure 58 . Sequence diagram of UC-14.03 View Post List 76
Figure 59 . Sequence diagram of UC-14.04 View Post Detail 77
Figure 60 . Sequence diagram of UC-14.05 Create Post 78
Figure 61 . Sequence diagram of UC-14.06 Update Post 79
Figure 62 . Sequence diagram of UC-14.07 Delete Post 80
Figure 63 . Sequence diagram of UC-14.08 Like Post 81
Figure 64 . Sequence diagram of UC-14.09 Unlike Post 82
Figure 65 . Sequence diagram of UC-14.10 View Post Likes 83

Figure 66 . Sequence diagram of UC-14.11 View Post Comments 84
Figure 67 . Sequence diagram of UC-14.12 Create Comment 85
Figure 68 . Sequence diagram of UC-14.13 Update Comment 86
Figure 69 . Sequence diagram of UC-14.14 Delete Comment 87
Figure 70 . Sequence diagram of UC-14.15 Like Comment 88
Figure 71 . Sequence diagram of UC-14.16 Unlike Comment 89
Figure 72 . Sequence diagram of UC-14.17 Report Content 90
Figure 73 . Class diagram of Manage Conversation 91
Figure 74 . Sequence diagram of UC-27.01 View Conversation List 92
Figure 75 . Sequence diagram of UC-27.02 Start Conversation 93
Figure 76 . Sequence diagram of UC-27.03 View Conversation Detail 94
Figure 77 . Sequence diagram of UC-27.04 Send Message 95
Figure 78 . Sequence diagram of UC-27.05 Mark Message as Read 96
Figure 79 . Sequence diagram of UC-27.06 Delete/Recall Message 97
Figure 80 . Sequence diagram of UC-27.07 Pin/Unpin Conversation 98
Figure 81 . Sequence diagram of UC-27.08 Send File 99
Figure 82 . Class diagram of Saved Search Management 100
Figure 83 . Sequence diagram of UC-22.01 View Saved Search List 101
Figure 84 . Sequence diagram of UC-22.02 View Saved Search Detail 102
Figure 85 . Sequence diagram of UC-22.03 Create Saved Search 103
Figure 86 . Sequence diagram of UC-22.04 Update Saved Search 104
Figure 87 . Sequence diagram of UC-22.05 Delete Saved Search 105
Figure 88 . Class diagram of Job Alert Management 106
Figure 89 . Sequence diagram of UC-23.01 View Job Alert List 107
Figure 90 . Sequence diagram of UC-23.02 View Job Alert Detail 108
Figure 91 . Sequence diagram of UC-23.03 Create Job Alert 109
Figure 92 . Sequence diagram of UC-23.04 Update Job Alert 110
Figure 93 . Sequence diagram of UC-23.05 Delete Job Alert 111
Figure 94 . Class diagram of Permission Management 112
Figure 95 . Sequence diagram of UC-15.01 View Permission List 112
Figure 96 . Sequence diagram of UC-15.02 View Permission Detail 113
Figure 97 . Class diagram of Session Management 114
Figure 98 . Sequence diagram of UC-16.01 View Session List 114

Figure 99 . Sequence diagram of UC-16.02 View Session Detail 115
Figure 100 . Sequence diagram of UC-16.03 Revoke Session 116
Figure 101 . Class diagram of Forum Category Management 117
Figure 102 . Sequence diagram of UC-17.01 Create Forum Category 118
Figure 103 . Sequence diagram of UC-17.02 Update Forum Category 119
Figure 104 . Sequence diagram of UC-17.03 Delete Forum Category 120
Figure 105 . Sequence diagram of UC-17.06 Restore Forum Category 121
Figure 106 . Class diagram of Manage Moderation 122
Figure 107 . Sequence diagram of UC-34.01 View Moderation Queue 122
Figure 108 . Sequence diagram of UC-34.02 View Moderation Item Detail 123
Figure 109 . Sequence diagram of UC-34.03 Resolve Moderation Item 124
Figure 110 . Class diagram of Manage ID Verification 125
Figure 111 . Sequence diagram of UC-35.01 View ID Verification Request List 125
Figure 112 . Sequence diagram of UC-35.02 View ID Verification Request Detail 126
Figure 113 . Sequence diagram of UC-35.03 Approve ID Verification 127
Figure 114 . Sequence diagram of UC-35.04 Reject ID Verification 128
Figure 115 . Class diagram of Manage Forum Trash 129
Figure 116 . Sequence diagram of UC-36.01 View Deleted Posts/Comments List 130
Figure 117 . Sequence diagram of UC-36.02 View Deleted Posts/Comments Detail . . . 131
Figure 118 . Sequence diagram of UC-36.03 Restore Deleted Posts/Comments 132
Figure 119 . Class diagram of Freelancer Proposal Management 133
Figure 120 . Class diagram of Client Proposal Management 134
Figure 121 . Sequence diagram of UC-32.01 View My Proposal List 135
Figure 122 . Sequence diagram of UC-32.02 View My Proposal Detail 136
Figure 123 . Sequence diagram of UC-32.03 Submit Proposal 137
Figure 124 . Sequence diagram of UC-32.04 Update Proposal 138
Figure 125 . Sequence diagram of UC-32.05 Withdraw Proposal 139
Figure 126 . Sequence diagram of UC-32.06 Accept Proposal 140
Figure 127 . Sequence diagram of UC-32.07 Reject Proposal 141
Figure 128 . Sequence diagram of UC-32.08 View Job Proposal List 142
Figure 129 . Sequence diagram of UC-32.09 View Proposal Detail 143
Figure 130 . Class diagram of User Management 144
Figure 131 . Sequence diagram of UC-28.01 View User List 145

Figure 132 . Sequence diagram of UC-28.02 View User Detail 146
Figure 133 . Sequence diagram of UC-28.03 Create User 147
Figure 134 . Sequence diagram of UC-28.04 Update User 148
Figure 135 . Sequence diagram of UC-28.05 Update Client Profile 149
Figure 136 . Sequence diagram of UC-28.06 Update Freelancer Basic Info 150
Figure 137 . Sequence diagram of UC-28.07 Update Freelancer Skills 151
Figure 138 . Sequence diagram of UC-28.08 Update Freelancer Portfolio 152
Figure 139 . Class diagram of Banner Management 153
Figure 140 . Sequence diagram of UC-37.01 View Banner List 154
Figure 141 . Sequence diagram of UC-37.02 View Banner Detail 155
Figure 142 . Sequence diagram of UC-37.03 Create Banner 156
Figure 143 . Sequence diagram of UC-37.04 Update Banner 157
Figure 144 . Sequence diagram of UC-37.05 Delete Banner 158
Figure 145 . Class diagram of Skill Management 159
Figure 146 . Sequence diagram of UC-38.01 View Skill List 160
Figure 147 . Sequence diagram of UC-38.02 View Skill Detail 161
Figure 148 . Sequence diagram of UC-38.03 Create Skill 162
Figure 149 . Sequence diagram of UC-38.04 Update Skill 163
Figure 150 . Sequence diagram of UC-38.05 Delete Skill 164
Figure 151 . Sequence diagram of UC-38.06 Restore Skill 165
Figure 152 . Class diagram of Expert Profile Management 166
Figure 153 . Sequence diagram of UC-28.09 Update Expert Profile 167
Figure 154 . Class diagram of Job Category Management 168
Figure 155 . Sequence diagram of UC-46.01 View Job Category List 168
Figure 156 . Sequence diagram of UC-46.02 View Job Category Detail 169
Figure 157 . Sequence diagram of UC-46.03 Create Job Category 170
Figure 158 . Sequence diagram of UC-46.04 Update Job Category 171
Figure 159 . Sequence diagram of UC-46.05 Delete Job Category 172
Figure 160 . Sequence diagram of UC-46.06 Restore Job Category 173

Table of Tables

Table 1 . Frontend package description (`apps/web`) 18
Table 2 . Backend package description (`apps/api`) 18
Table 3 . Shared and tooling packages description (`packages/`) 20
Table 4 . AI microservices package description (`ai-services/`) 20
Table 5 . Table descriptions of the Frevia database 23

Definition and Acronyms

Acronym	Definition
AI	Artificial Intelligence
API	Application Programming Interface
AWS	Amazon Web Services
CDN	Content Delivery Network
CI/CD	Continuous Integration / Continuous Deployment
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
CV	Curriculum Vitae
DTO	Data Transfer Object
eKYC	Electronic Know Your Customer
ERD	Entity–Relationship Diagram
HTML	HyperText Markup Language
HTTP / HTTPS	HyperText Transfer Protocol / HTTP Secure
JWT	JSON Web Token
ORM	Object-Relational Mapping
OTP	One-Time Password
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDK	Software Development Kit
SCM	Source Code Management
SQL	Structured Query Language
SRS	Software Requirements Specification
SSL / TLS	Secure Sockets Layer / Transport Layer Security
UI	User Interface
UML	Unified Modeling Language

Acronym	Definition
URI / URL	Uniform Resource Identifier / Uniform Resource Locator
UX	User Experience
WBS	Work Breakdown Structure
WebSocket	Full-Duplex Bidirectional Communication Protocol

I. Record of Changes

Date	A* M, D	In charge	Change Description

**A - Added M - Modified D - Deleted*

II. Software Design Document

1. System Design

1.1 System Architecture

The Frevia system follows an N-layer architecture, separating the client applications, presentation layer, business logic layer, data access layer, shared library layer, and external integration systems (Figure 1). The system also natively integrates with PostgreSQL, Cloudinary, Stripe, Google OAuth 2.0, Nodemailer, Infisical, and local Python FastAPI AI microservices backed by Ollama LLM.

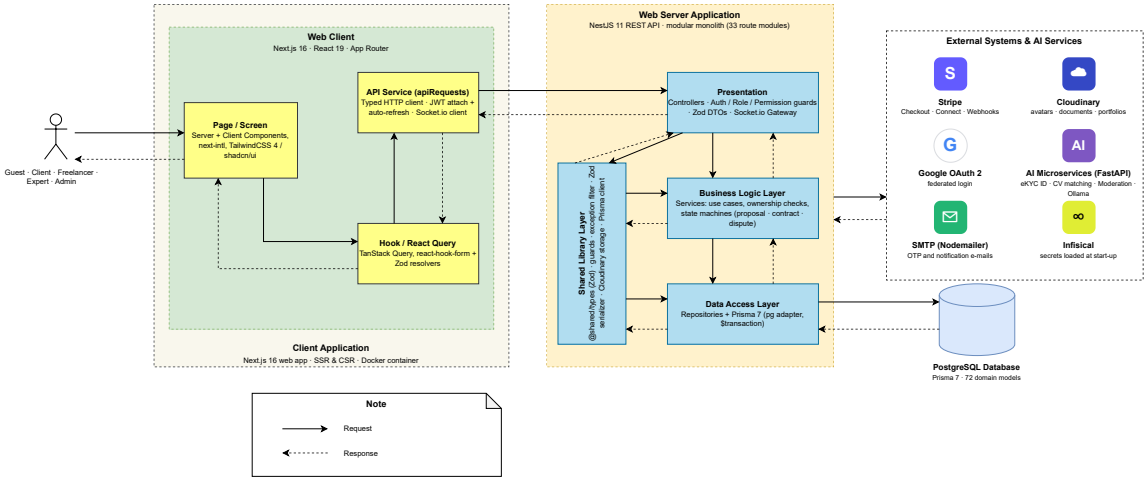

Figure 1: System Architecture Diagram

Client Application

Frevia delivers a unified, responsive web application catering to five actor roles: Guest, Client, Freelancer, Expert, and Administrator.

- **Web Client:** Developed with Next.js 16 (React 19) and TypeScript utilizing the App Router architecture. It consists of:
  - *Pages and Screens:* Built with React Server Components (RSC) for optimal search engine indexing and rapid initial page loads, complemented by Client Components for interactive user interfaces. Styled using TailwindCSS 4 and the `@repo/ui` component library (based on shadcn/ui), and localized into English and Vietnamese via `next-intl`.
  - *Hooks and State Management:* Driven by TanStack Query (React Query) for server state caching, background revalidation, and optimistic mutations. Client-side form validation is managed via `react-hook-form` paired with Zod schemas.
  - *API Services (`apiRequests/`):* A typed HTTP client layer encapsulating `lib/http.ts`. It automatically binds JWT access tokens to request headers, transparently handles token

refreshes upon expiration, and interfaces with the Socket.io client for real-time messaging and notifications.

The client application transmits requests exclusively over HTTPS REST APIs and displays returned responses. It contains no direct database access logic and enforces no authoritative business rules.

Presentation Layer

The Presentation Layer is implemented within the NestJS 11 Web Server Application (`apps/api`). It serves as the entry boundary for incoming client requests and real-time events:

- **Controllers:** Expose structured RESTful API endpoints (versioned under `/api/v1`), receive client requests, bind validated request payloads, and delegate processing to domain services.
- **Authentication and Authorization Guards:** Intercept requests to verify JWT tokens and enforce role-based access control (`RoleGuard`) and fine-grained permission checks (`PermissionGuard`).
- **Validation and Serialization:** Validates incoming request payloads using shared Zod DTOs and sanitizes outgoing responses using the global `ZodSerializerInterceptor`.
- **Socket.io Gateway:** Manages real-time bidirectional WebSocket connections for peer-to-peer conversations and immediate notification pushes.

Business Logic Layer

The Business Logic Layer represents the core of the Frevia platform, encapsulating business rules, ownership verifications, and deterministic domain state machines across 33 route modules:

- **Authentication and Access Control Module:** Manages user registration, OTP verification, credential login, Google OAuth federated sign-in, token refresh, device sessions, roles, and granular permissions.
- **User and Profile Management Module:** Handles administrator user accounts, account profiles, freelancer professional portfolios, CV uploads, and expert consultation profiles.
- **Job and Proposal Management Module:** Governs client job postings, faceted job browsing, bookmarking, freelancer proposal submissions, interviews, and proposal acceptance/rejection workflows.
- **Contract and Milestone Module:** Enforces contract lifecycle transitions (draft, agreement, digital signature, completion, cancellation) and milestone delivery reviews.
- **Payment and Escrow Module:** Integrates Stripe Checkout for milestone escrow funding, platform fee collections, dispute fee handling, and automated Stripe Connect payouts to freelancer bank accounts.

- **Communication and Community Module:** Facilitates real-time direct messaging, multi-category discussion forums, post interactions, and automated user notification feeds.
- **Dispute Arbitration Module:** Provides formal dispute filing, evidence attachment, and administrator arbitration workflows.

Data Access Layer

The Data Access Layer encapsulates all persistence logic using the Repository Pattern combined with Prisma 7 ORM and the native PostgreSQL driver adapter (`@prisma/adapter-pg`):

- **Repositories:** Implement dedicated query methods, data filtering, pagination helpers, and soft-delete semantics (`isDeleted`). Controllers and services are strictly decoupled from raw database queries.
- **Transaction Management:** Wraps complex multi-table workflows (such as escrow fund transfers, contract creation, and user onboarding cascades) inside atomic `$transaction` blocks to maintain absolute relational integrity.

Shared Library Layer

The Shared Library Layer provides shared definitions and cross-cutting utilities utilized across the application tiers:

- **Contract Single Source of Truth (`@shared/types`):** Monorepo package defining shared Zod validation schemas, TypeScript interfaces, system constants, database enumerations, and standardized error codes.
- **Backend Cross-Cutting Infrastructure (`src/shared`):** Contains global exception filters, authentication decorators, Cloudinary storage wrappers, Nodemailer integration, and the shared Prisma client service.

Data Storage

- **PostgreSQL Database:** Stores all persistent platform data across 72 relational domain models and 43 enumerations, maintaining relational integrity with foreign keys, compound indexes, and audit timestamps.
- **Cloudinary Object Storage:** Dedicated cloud repository storing binary multimedia files, including user profile avatars, citizen identity document photographs (CCCD), candidate resumes, and contract deliverable archives.

External Systems and Internal AI Services

- **Google OAuth 2.0:** Facilitates federated single sign-on. The frontend retrieves a Google authorization credential, which the backend verifies to establish an authenticated Frevia session.

- **Stripe:** Processes online financial transactions, managing escrow funding via Stripe Checkout, platform fee collection, dispute deposits, and automated freelancer payouts via Stripe Connect. Status updates are synchronized securely via cryptographic webhooks.
- **Internal AI Microservices (FastAPI & Python):**
  - *eKYC ID Verification (`id-verification`, port 8001):* Employs RapidOCR for card text extraction, InsightFace for biometric facial embeddings, and MediaPipe for liveness detection to detect fraudulent identity documents.
  - *Section-Aware CV and Job Matching (`cv-matching`, port 8000):* BGE-M3 embeddings, ChromaDB vector indexing, and local Ollama LLM inference to parse candidate CVs and compute semantic candidate-to-job matching scores.
  - *AI Content Moderation (`ai-moderation`, port 8001):* Automatically scores community forum posts and comments, quarantining toxic or abusive content into the administrative review queue.
- **SMTP Mail Service (Nodemailer):** Delivers transactional notifications, one-time passwords (OTP), and contract status alerts.
- **Infisical:** Secure secrets manager that injects environment variables and credentials into application containers at startup.

Request-Response Data Flow

A typical user request originates on a Next.js Page or Screen, triggers a TanStack Query hook, and invokes a typed API method in `apiRequests/`. The service binds the JWT access token and issues an HTTPS REST call to the NestJS backend. The request passes through the Authentication Guard, Role/Permission Guards, and Zod DTO validation before reaching the appropriate Controller. The Controller delegates execution to the Service Layer, which validates business rules and state machines, then commands the Repository to execute queries through Prisma 7 to PostgreSQL. The resulting entity follows the reverse path, is sanitized by the Zod serializer interceptor, and is returned as a uniform JSON response to update the client-side cache and re-render the user interface. Real-time events follow an asynchronous parallel path via the Socket.io Gateway.

1.2 Package Diagram

The Frevia repository is organized as a high-cohesion monorepo managed with **pnpm workspaces** and **Turborepo** (Figure 2). This structure cleanly separates the presentation tier (`apps/web`), the application tier (`apps/api`), the shared workspace packages (`packages/*`), and the independent AI microservices (`ai-services/`).

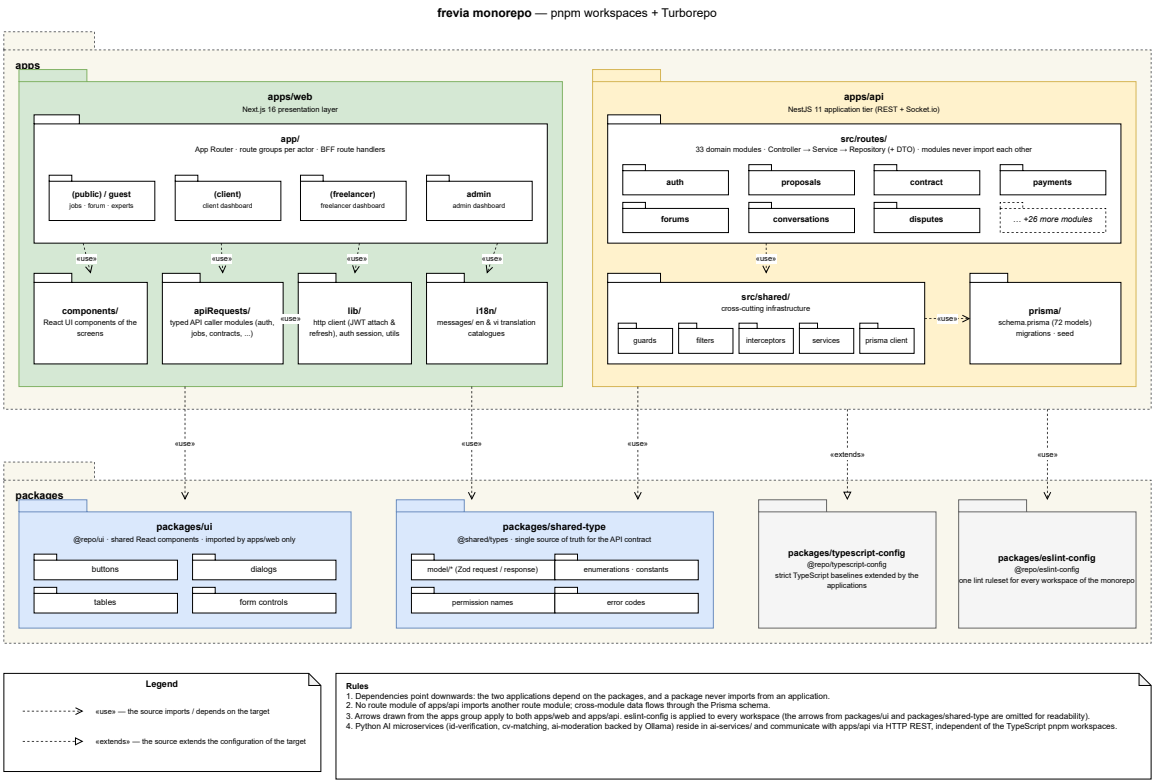

Figure 2: Package diagram

Monorepo Architecture Rules

The monorepo enforces four key architectural rules illustrated in Figure 2:

1. **Strict Downward Dependencies:** Applications (`apps/web` and `apps/api`) depend on shared packages (`packages/*`); a shared package never imports code or types from an application.

2. **Module Isolation:** Domain route modules inside `apps/api/src/routes/` are strictly decoupled and never import controllers, services, or repositories from sibling modules. Cross-module data flows exclusively through database relationships at the Prisma ORM schema level.

3. **Universal Configuration:** Common configurations for linting (`@repo/eslint-config`) and TypeScript compilation (`@repo/typescript-config`) are inherited by all workspaces to guarantee uniform code quality.

4. **Independent AI Service Boundaries:** Python-based AI microservices (`id-verification`, `cv-matching`, `ai-moderation` backed by Ollama) reside in `ai-services/` and communicate with the backend via standard HTTP REST endpoints, maintaining complete independence from the TypeScript pnpm workspaces.

Package Descriptions

Tables 1, 2, 3, and 4 detail the responsibilities of every package in the Frevia project.

Table 1: Frontend package description (`apps/web`)

No	Package	Description
01	`app`	Next.js App Router entry points, layout definitions, error boundaries, and route groups (`(auth)`, `(client)`, `(freelancer)`, `admin/`, and public pages).
02	`components`	Reusable UI components, page section blocks, modals, and screen layouts specific to the web application.
03	`apiRequests`	Typed HTTP client callers for all domain modules (auth, jobs, proposals, contracts, payments, etc.) that attach the JWT token and coordinate API communication.
04	`lib`	Core HTTP fetch wrapper (`http.ts`), authentication session token management, form validation resolvers, string utilities, and formatters.
05	`i18n`	Internationalization configuration and localization message catalogues (`messages/en.json`, `messages/vi.json`) powered by `next-intl`.
06	`hooks`	Custom React hooks for server state caching (TanStack Query), form management, debouncing, and authentication monitoring.
07	`configs`	Client application environment configuration variables, API endpoints, and system runtime constants.
08	`providers`	Application-wide React context providers, including QueryClientProvider, NextIntlClientProvider, and theme state providers.

Table 2: Backend package description (`apps/api`)

No	Package	Description
01	`src/routes`	33 domain modules structured into Controller, Service, Repository, and DTO layers.
02	`src/routes/auth`	User registration, OTP delivery, login, Google OAuth, token refresh, and role switching.
03	`src/routes/sessions`	Device session telemetry tracking, session listing, and remote revocation.
04	`src/routes/account-profile`, `identity-verifications`	Account profile management, avatar uploads, citizen identity document submission (CCCD), and administrator verification reviews.
05	`src/routes/users`	Administrator user management, account status control, and role assignments.
06	`src/routes/roles`, `permissions`	Role-based access control (RBAC) administration and granular permission mapping.

No	Package	Description
07	src/routes/profiles, portfolios, profile-revisions	Freelancer professional resumes, CV analysis integration, portfolio showcases, skill tags, and profile revision audit history.
08	src/routes/expert-profile	Expert directory listings, consultation profiles, and domain certifications.
09	src/routes/browse-job	Public job search, keyword filtering, category navigation, and slug-based job details.
10	src/routes/manage-job	Client job posting, draft editing, applicant review, and bookmark management.
11	src/routes/job-categories, job-categories-admin, skills-admin	Job category taxonomy, category hierarchy management, and administrative skill tags.
12	src/routes/proposals	Freelancer bid submissions, cover letters, proposed terms, and proposal lifecycle management.
13	src/routes/contract, milestone	Contract generation, digital signing, milestone deliverable submissions, and progress reviews.
14	src/routes/milestone-file, milestone-submission, shared-file	Milestone file attachments, deliverable review submissions, and shared project documents.
15	src/routes/payments	Stripe Connect onboarding, milestone escrow deposits, fund releases, platform fee deductions, and webhook handlers.
16	src/routes/disputes	Contract dispute arbitration, claim filing, evidence submission, and administrator resolution.
17	src/routes/reviews	Bilateral post-contract rating reviews, score aggregations, and public responses.
18	src/routes/conversations	Direct real-time messaging, conversation threads, and message history persistence.
19	src/routes/notifications	User notification feed generation, unread status tracking, and push alerts.
20	src/routes/forums, forums-admin	Public discussion forums, posts, comments, likes, reports, AI moderation queues, and soft-delete trash recovery.
21	src/routes/saved-search, job-alert	Saved job filter criteria and automated alert notifications for matching postings.
22	src/routes/recommendations	AI-assisted candidate and job recommendations based on semantic CV matching.
23	src/routes/banners, banners-admin	Promotional banner delivery, placement targeting, and administrative banner management.
24	src/shared/guards	JWT authentication guards, role-based authorization guards, and permission guards.
25	src/shared/filters	Global exception filter converting internal exceptions into standard JSON error bodies.

No	Package	Description
26	src/shared/interceptors	Zod response serialization and transformation interceptor.
27	src/shared/services	Cross-cutting infrastructure services including Cloudinary media storage, Nodemailer email dispatch, and AI moderation client.
28	src/shared/repositories	Base repository definitions and database query execution helpers.
29	prisma	Prisma 7 ORM persistence configuration, schema.prisma (72 models, 43 enums), database migrations, and seed scripts.

Table 3: Shared and tooling packages description (packages/)

No	Package	Description
01	packages/ui	Shared React component library (@repo/ui) styled with TailwindCSS 4 and shadcn/ui (buttons, dialogs, form controls, tables) imported exclusively by the web application.
02	packages/shared-type	API contract single source of truth (@shared/types) providing Zod validation models, TypeScript interfaces, database enumerations, permission names, and error codes to both applications.
03	packages/typescript-config	Shared TypeScript configuration baselines (@repo/typescript-config) that the applications extend for strict type checking.
04	packages/eslint-config	Centralized ESLint configuration ruleset (@repo/eslint-config) enforced across every monorepo package.

Table 4: AI microservices package description (ai-services/)

No	Package	Description
01	id-verification	FastAPI microservice (port 8001) performing Vietnamese citizen identity card (CCCD) OCR via RapidOCR, facial biometric embedding via InsightFace, and anti-spoofing liveness verification via MediaPipe.
02	cv-matching	FastAPI microservice (port 8000) performing section-aware candidate CV parsing, dense semantic vector embedding with BGE-M3, ChromaDB vector indexing, and candidate-job matching supported by Ollama LLM.
03	ollama	Containerized local Large Language Model runtime (port 11434) running instruction-tuned models (e.g., Qwen) for structured CV extraction and semantic matching.
04	ai-moderation	FastAPI microservice (port 8001) providing automated toxic content classification, sensitive keyword detection, and moderation scoring for forum posts and user submissions.

2. Database Design

Frevia persists every domain object in a single PostgreSQL database whose schema is declared with Prisma 7 in `apps/api/prisma/schema.prisma`. The schema contains 72 models together with 43 enumerations, every table has a surrogate integer primary key generated by the database (nine membership tables use a composite key instead), and deletion is either restricted by a foreign-key rule (identity and money-related rows are never removed) or implemented as a soft delete through a `deletedAt` column (content that an administrator must be able to restore: jobs, forum categories and posts, skills, banners, contract term templates). Binary files are not stored in the database: only their Cloudinary URL is kept in the owning row.

Figure 3 shows the entity–relationship diagram of the whole schema. Every table is drawn as a frameless box: the header carries the table name, each of the following rows is one column, and the key columns are flagged with `PK` (primary key) or `FK` (foreign key). A relationship is a line that leaves the many-side table with a crow's foot and ends on the one-side table with a single bar, so it reads from the row that holds the foreign key to the row it references.

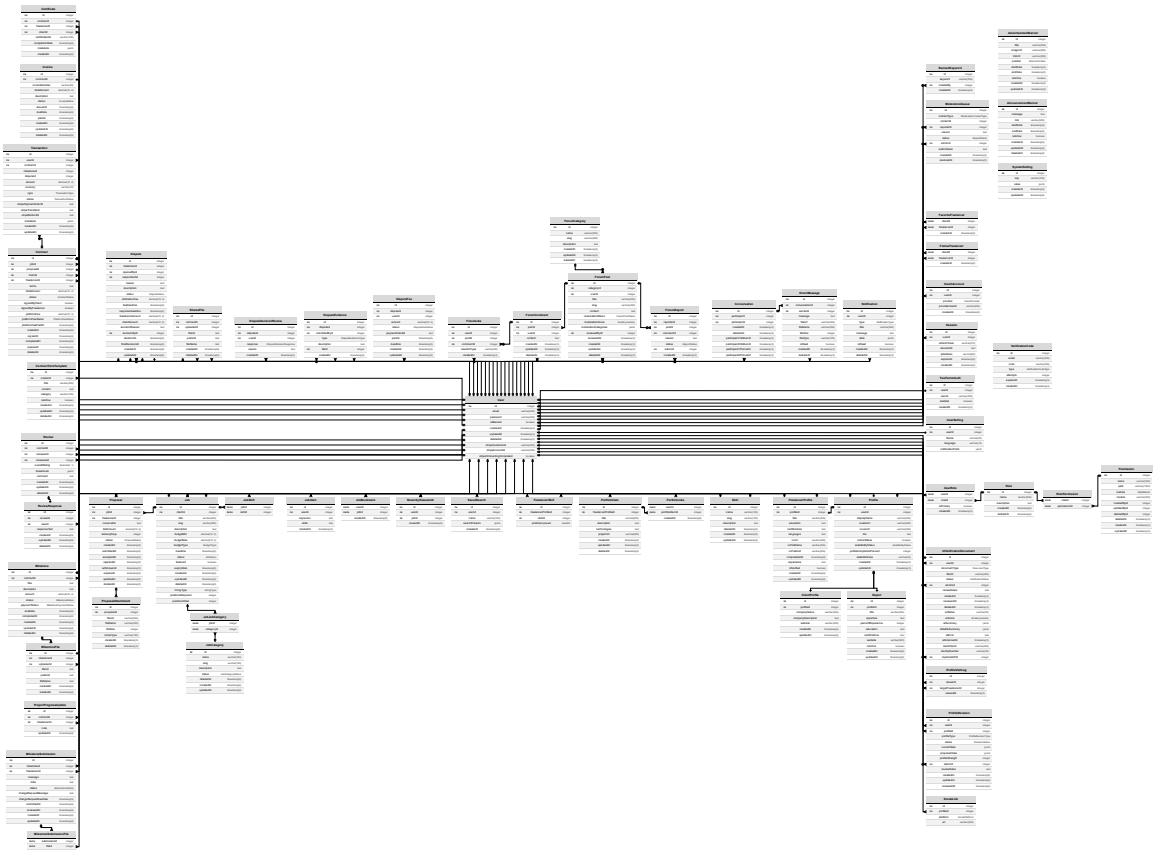

Figure 3: Entity–relationship diagram of the Frevia database

2.1 Table Relationships

The 64 tables are grouped into nine families; inside a family the relationships are one-to-many, and membership tables carry the many-to-many links:

- **Identity and access** — `User` is the root of the schema. `UserRole` links an account to a `Role`, and `RolePermission` links a role to a `Permission`, which yields the RBAC model enforced by the guards. Authentication adds `Session` (one row per device), `VerificationCode` (OTP for registration and password reset), `OauthAccount` (Google login), `TwoFactorAuth` and `UserSetting`.

- **Profiles** — `Profile` is shared by every account; `FreelancerProfile` and `ClientProfile` extend it with role-specific columns, `Expert` publishes an expert entry in the public directory, and `FreelancerSkill` joins a freelancer to the `Skill` catalogue. `PortfolioItem`, `SocialLink`, `Certificate`, `IdVerificationDocument` and `ProfileRevision` hang from the profile.

- **Jobs** — `Job` belongs to a client and is classified through `JobJobCategory` (`JobCategory`) and `JobSkill` (`Skill`). `JobBookmark`, `RecentlyViewedJob`, `SavedSearch` and `JobAlert` record the browsing behaviour of a freelancer.

- **Proposals and contracts** — `Proposal` references one `Job` and one freelancer; accepting it creates exactly one `Contract`, which carries `ContractTermTemplate` snapshots and a list of `Milestone` rows. `ProjectProgressUpdate`, `SharedFile` and `MilestoneFile` document the work in progress, while `MilestoneSubmission` (with `MilestoneSubmissionFile`) records the deliverable and its review decision.

- **Payments** — `Transaction` stores every Stripe movement (funding, release, refund, platform fee, dispute fee) with its payment-intent identifier, and `Invoice` holds the invoice issued per contract. Money is never derived: the escrow balance of a milestone is the sum of its transactions.

- **Disputes** — `Dispute` is opened on a contract or a milestone, `DisputeEvidence` collects the attachments of both parties, `DisputeFee` records the fee charged to open it, and `DisputeDecisionReview` stores the request of a party that asks for the decision to be reconsidered.

- **Reviews** — one `Review` per completed contract, answered by at most one `ReviewResponse`.

- **Messaging and notifications** — `Conversation` groups the `DirectMessage` rows exchanged by two accounts (pin and read state are kept on the message and on the conversation), and `Notification` feeds the in-app bell.

- **Community forum and configuration** — `ForumCategory` contains `ForumPost`, a post contains `ForumComment`, and `ForumLike` / `ForumReport` attach reactions and reports to posts or comments. Administrators work through `ModerationQueue` and `BannedKeyword`; platform-wide configuration lives in `SystemSetting`, `AnnouncementBanner` and `AdvertisementBanner`.

Table Descriptions

Table 5 describes the purpose of every table of the schema, in the order in which the models are declared in `schema.prisma`, together with its primary key and its foreign keys.

Table 5: Table descriptions of the Frevia database

No	Table	Description
01	`Role`	Role catalogue of the RBAC model (admin, client, freelancer, expert); carries the role code, name and description. • **Primary keys:** `id` • **Foreign keys:** —
02	`UserRole`	Assignment of a role to an account, so one account can hold several roles and switch between them. • **Primary keys:** `userId`, `roleId` • **Foreign keys:** `userId` → `User`, `roleId` → `Role`
03	`Permission`	Catalogue of route-level permissions (`module.action` names) used by the permission guard. • **Primary keys:** `id` • **Foreign keys:** —
04	`RolePermission`	Permissions granted to a role; drives the effective permission list of every signed-in account. • **Primary keys:** `roleId`, `permissionId` • **Foreign keys:** `roleId` → `Role`, `permissionId` → `Permission`
05	`User`	Account identity: e-mail, username, bcrypt password hash, status, verification and ban flags, preferred language and timestamps. • **Primary keys:** `id` • **Foreign keys:** —
06	`VerificationCode`	One-time code (OTP) issued for e-mail verification or password reset, with its purpose, expiry and consumed flag. • **Primary keys:** `id` • **Foreign keys:** —
07	`Session`	Server-side session per device: refresh-token hash, device/user-agent information, last activity and revocation time. • **Primary keys:** `id` • **Foreign keys:** `userId` → `User`

No	Table	Description
08	OauthAccount	Link between an account and an external identity provider (Google) for federated login. • **Primary keys:** id • **Foreign keys:** userId → User
09	TwoFactorAuth	Two-factor authentication secret and enabled flag of an account (one row per account). • **Primary keys:** id • **Foreign keys:** userId → User
10	UserSetting	Per-account preferences: interface theme, language and the JSON notification preferences. • **Primary keys:** id • **Foreign keys:** userId → User
11	Profile	Base profile shared by all accounts: display name, avatar, cover, headline and biography. • **Primary keys:** id • **Foreign keys:** userId → User
12	FreelancerProfile	Professional data of a freelancer: professional title, overview, hourly rate, availability and total earnings. • **Primary keys:** id • **Foreign keys:** profileId → Profile
13	FreelancerSkill	Many-to-many link between a freelancer profile and a skill, with the self-declared proficiency level. • **Primary keys:** id • **Foreign keys:** freelancerProfileId → FreelancerProfile, skillId → Skill
14	ClientProfile	Client-side data: company name, company size, industry and billing information. • **Primary keys:** id • **Foreign keys:** profileId → Profile
15	Expert	Expert entry published in the public expert directory, with specialisation and verification state. • **Primary keys:** id • **Foreign keys:** profileId → Profile
16	ProfileRevision	Pending change request on a profile, reviewed and approved or rejected by an administrator. • **Primary keys:** id • **Foreign keys:** userId → User, adminId → User, profileId → Profile

No	Table	Description
17	PortfolioItem	Portfolio entry of a profile: title, description, media URL, project link and ordering position. • **Primary keys:** id • **Foreign keys:** freelancerProfileId → FreelancerProfile
18	SocialLink	External link of a profile (website, LinkedIn, GitHub, Behance) shown on the public profile. • **Primary keys:** id • **Foreign keys:** profileId → Profile
19	IdVerificationDocument	Identity document uploaded for eKYC, with the AI check result and the administrator decision. • **Primary keys:** id • **Foreign keys:** userId → User, adminId → User, duplicateOfId → IdVerificationDocument
20	ProfileVisitLog	View log of a freelancer profile (viewerId, targetFreelancerId, viewedAt); source of the profile statistics. • **Primary keys:** id • **Foreign keys:** viewerId → User, targetFreelancerId → User
21	JobCategory	Hierarchical job category (parent reference, slug, name, icon) with soft-delete support. • **Primary keys:** id • **Foreign keys:** —
22	Job	Job posting: slug, title, description, budget range, duration, deadline, visibility, status and the owning client. • **Primary keys:** id • **Foreign keys:** clientId → User
23	JobJobCategory	Many-to-many link between a job and the job categories it is published in. • **Primary keys:** jobId, categoryId • **Foreign keys:** jobId → Job, categoryId → JobCategory
24	Skill	Skill catalogue shared by freelancer profiles and jobs, with soft-delete support. • **Primary keys:** id • **Foreign keys:** —
25	JobSkill	Skills required by a job, with the required proficiency level. • **Primary keys:** jobId, skillId • **Foreign keys:** jobId → Job, skillId → Skill

No	Table	Description
26	JobBookmark	Jobs saved by a freelancer. • **Primary keys:** userId, jobId • **Foreign keys:** userId → User, jobId → Job
27	RecentlyViewedJob	Recently opened job postings of an account, with the viewing timestamp. • **Primary keys:** id • **Foreign keys:** userId → User, jobId → Job
28	SavedSearch	Stored job filter criteria (keyword, categories, budget, skills) that the freelancer can reapply. • **Primary keys:** id • **Foreign keys:** userId → User
29	JobAlert	Alert configuration attached to a saved search (frequency, channel, active flag). • **Primary keys:** id • **Foreign keys:** userId → User
30	Proposal	Proposal submitted by a freelancer for a job: cover letter, bid amount, estimated duration and lifecycle status. • **Primary keys:** id • **Foreign keys:** jobId → Job, freelancerId → User
31	ProposalAttachment	File attached to a proposal (portfolio sample, reference document) stored in Cloudinary. • **Primary keys:** id • **Foreign keys:** proposalId → Proposal
32	Contract	Contract created when a proposal is accepted: client, freelancer, total amount, terms, signature state, start/end dates and status. • **Primary keys:** id • **Foreign keys:** jobId → Job, proposalId → Proposal, clientId → User, freelancerId → User
33	ContractTermTemplate	Reusable contract term template owned by a user, with category, content and soft-delete support. • **Primary keys:** id • **Foreign keys:** creatorId → User
34	Milestone	Milestone inside a contract: title, description, amount, due date, escrow status, progress percentage and approval state. • **Primary keys:** id • **Foreign keys:** contractId → Contract

No	Table	Description
35	MilestoneFile	Reference file attached to a milestone and shared with both parties. • **Primary keys:** id • **Foreign keys:** milestoneId → Milestone, uploaderId → User
36	MilestoneSubmission	Work submitted for a milestone, with the submitted note, timestamp and the client review decision. • **Primary keys:** id • **Foreign keys:** milestoneId → Milestone, freelancerId → User
37	MilestoneSubmissionFile	Join table linking a submission to the milestone files it delivers. • **Primary keys:** submissionId, fileId • **Foreign keys:** submissionId → MilestoneSubmission, fileId → MilestoneFile
38	Invoice	Invoice of a contract (one per contract): invoice number, total amount, status, issued, due and paid dates. • **Primary keys:** id • **Foreign keys:** contractId → Contract
39	Certificate	Completion certificate generated when a contract is finished: unique certificate UID, completion date and JSON metadata. • **Primary keys:** id • **Foreign keys:** contractId → Contract, freelancerId → User, clientId → User
40	Dispute	Dispute opened on a contract or a milestone: reason, description, opened by, status, resolution and decision timestamps. • **Primary keys:** id • **Foreign keys:** milestoneId → Milestone, openedById → User, respondentId → User, decisionById → User
41	DisputeFee	Fee charged to open a dispute, with its amount, payment state and the transaction that settled it. • **Primary keys:** id • **Foreign keys:** disputeId → Dispute, userId → User
42	DisputeEvidence	Evidence attached by either party to a dispute (file, description, uploader). • **Primary keys:** id • **Foreign keys:** disputeId → Dispute, submittedById → User, fileId → SharedFile

No	Table	Description
43	DisputeDecisionReview	Request of a party to reconsider a dispute decision, with the response, the reason and the timestamp. • **Primary keys:** id • **Foreign keys:** disputeId → Dispute, userId → User
44	Transaction	Payment movement reconciled with Stripe (funding, release, refund, platform fee, dispute fee): type, amount, status, payment-intent id and timestamps. • **Primary keys:** id • **Foreign keys:** userId → User, contractId → Contract
45	Review	Review written after a contract is completed: rating, comment, author, recipient and the reviewed contract. • **Primary keys:** id • **Foreign keys:** contractId → Contract, reviewerId → User, revieweeId → User
46	ReviewResponse	Reply of the reviewed party to a review (at most one per review). • **Primary keys:** id • **Foreign keys:** reviewId → Review, userId → User
47	ForumCategory	Forum category: slug, name, description, display order, post counter and soft-delete support. • **Primary keys:** id • **Foreign keys:** —
48	ForumPost	Forum post: title, content, author, category, moderation status, AI score, reviewer and soft-delete support. • **Primary keys:** id • **Foreign keys:** categoryId → ForumCategory, userId → User, reviewedById → User
49	ForumComment	Comment on a forum post; comments are one level deep and can be soft-deleted. • **Primary keys:** id • **Foreign keys:** postId → ForumPost, userId → User
50	ForumLike	Reaction of a member on a post or a comment; a member can like a target only once. • **Primary keys:** id • **Foreign keys:** userId → User, postId → ForumPost, commentId → ForumComment

No	Table	Description
51	ForumReport	Content report filed by a member: reason, description, status and the administrator who handled it. • **Primary keys:** id • **Foreign keys:** reporterId → User, postId → ForumPost, commentId → ForumComment, adminId → User
52	Notification	In-app notification of an account: type, title, body, target link and read state. • **Primary keys:** id • **Foreign keys:** userId → User
53	AnnouncementBanner	Platform announcement shown in the application: message, link, display window, active flag and soft delete. • **Primary keys:** id • **Foreign keys:** —
54	ModerationQueue	Queue of AI-flagged or user-reported content waiting for an administrator decision, with the item type, the score and the assignee. • **Primary keys:** id • **Foreign keys:** reporterId → User, adminId → User
55	SystemSetting	Platform-wide configuration values stored as key/value rows. • **Primary keys:** id • **Foreign keys:** —
56	BannedKeyword	Keyword list used by the moderation filter before the AI service is called. • **Primary keys:** id • **Foreign keys:** createdBy → User
57	AdvertisementBanner	Advertising banner with image, target URL, position, schedule and click counter. • **Primary keys:** id • **Foreign keys:** —
58	FavoriteFreelancer	Freelancers marked as favourite by a client account. • **Primary keys:** clientId, freelancerId • **Foreign keys:** clientId → User, freelancerId → User
59	FollowFreelancer	Follow relationship between two accounts, used by the activity feed. • **Primary keys:** clientId, freelancerId • **Foreign keys:** clientId → User, freelancerId → User

No	Table	Description
60	PortfolioLike	Like given on a portfolio item. • **Primary keys:** userId, portfolioItemId • **Foreign keys:** userId → User, portfolioItemId → PortfolioItem
61	ProjectProgressUpdate	Progress note posted by the freelancer on a contract, with the update timestamp. • **Primary keys:** id • **Foreign keys:** contractId → Contract, freelancerId → User
62	SharedFile	File shared in the workspace of a contract (both parties can upload and read). • **Primary keys:** id • **Foreign keys:** contractId → Contract, uploaderId → User
63	Conversation	Direct conversation between two accounts, with last-message cache and the pin state of each participant. • **Primary keys:** id • **Foreign keys:** participant1 → User, participant2 → User
64	DirectMessage	Message of a conversation: sender, content, attachments, read state, recall (soft delete) flag and timestamp. • **Primary keys:** id • **Foreign keys:** conversationId → Conversation, senderId → User

2.2 Module Entity–Relationship Diagrams

The whole schema is easier to navigate when it is split along the functional modules used in Report 3. Each figure below shows one module together with the tables that the module references, so every foreign key of the module is visible in the same picture. The notation is the one of Figure 3: the crow's foot marks the many side of a relationship and the single bar marks the one side.

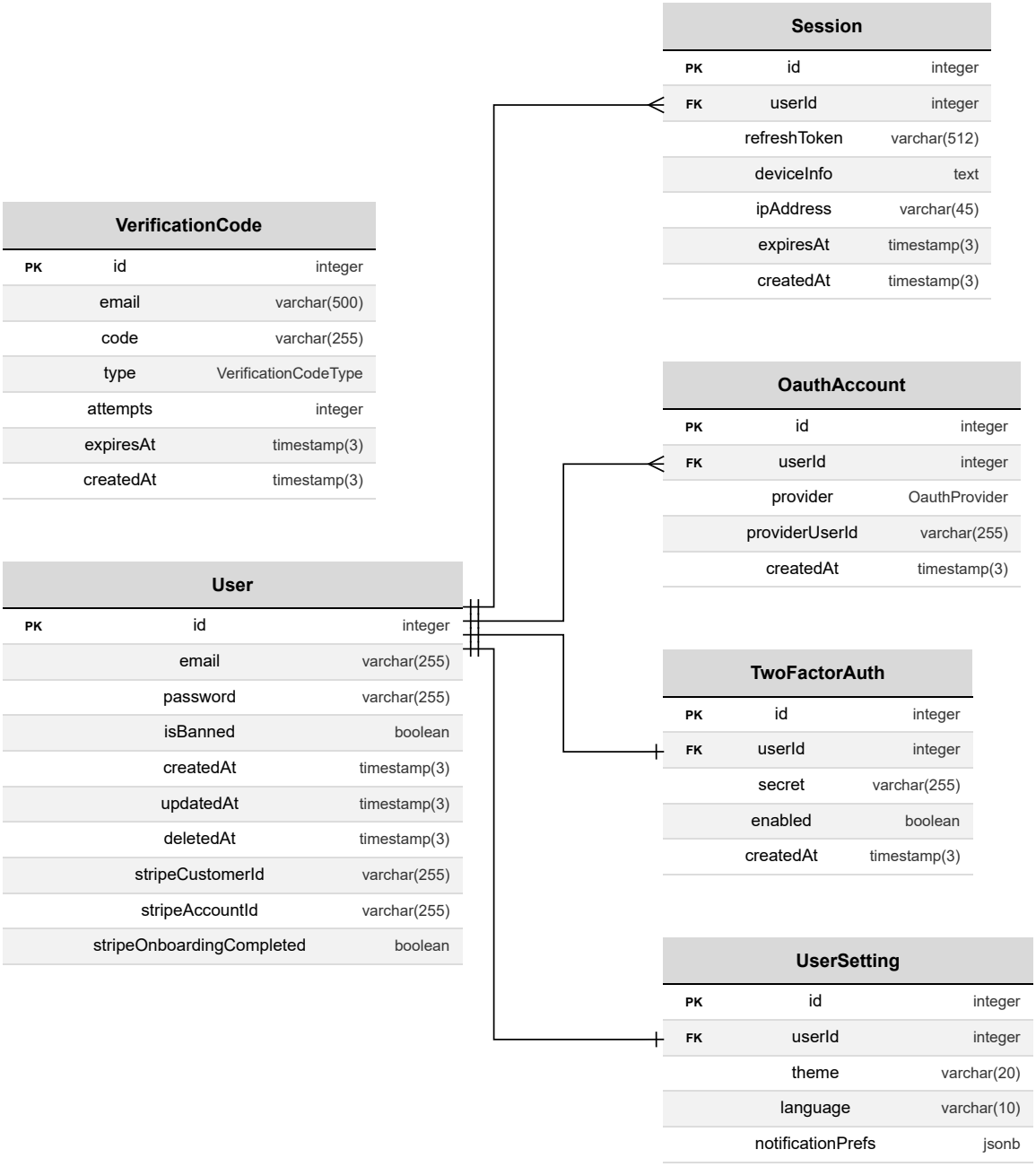

Figure 4: ERD1 — Authentication & Sessions

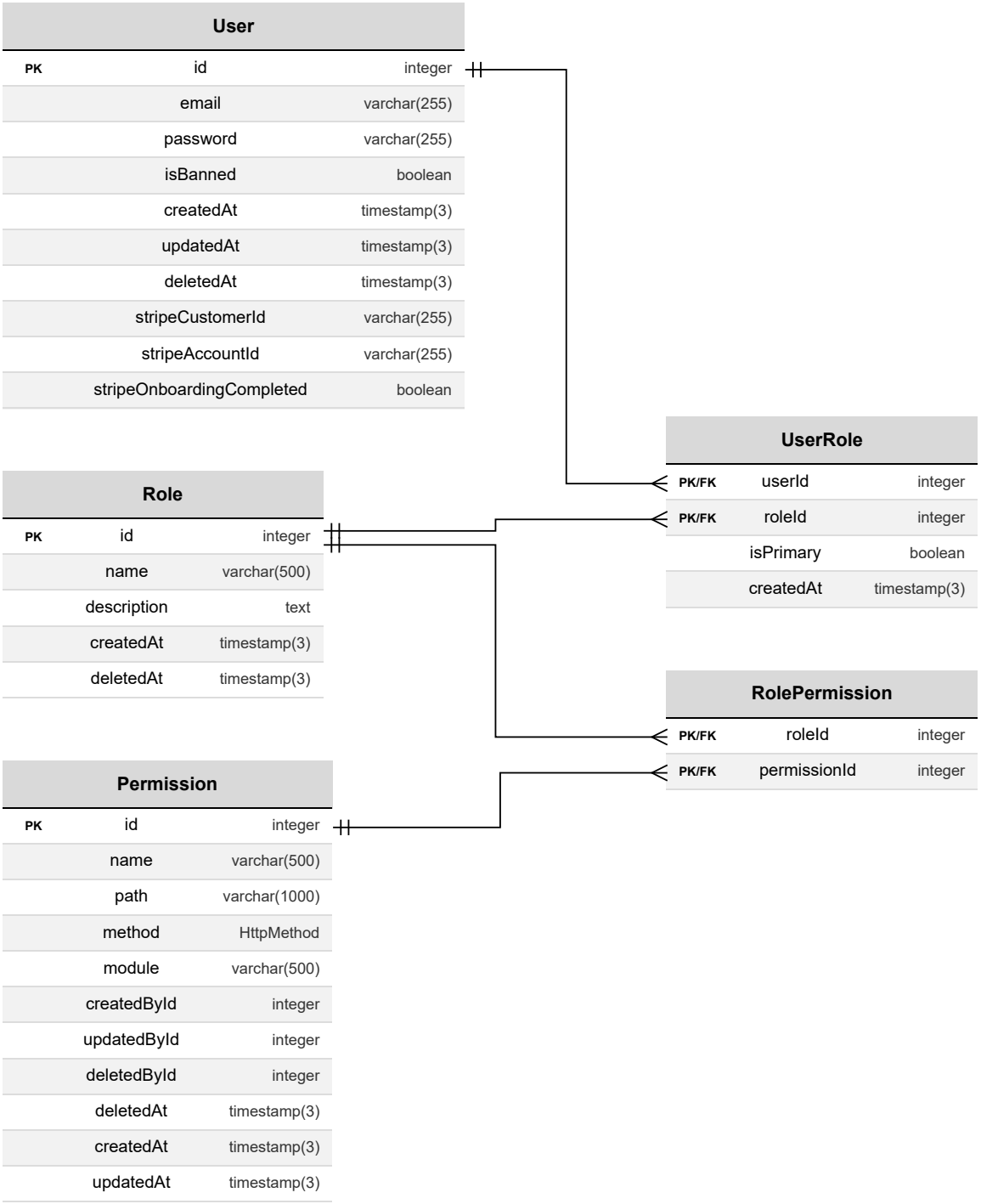

Figure 5: ERD2 — Role-Based Access Control

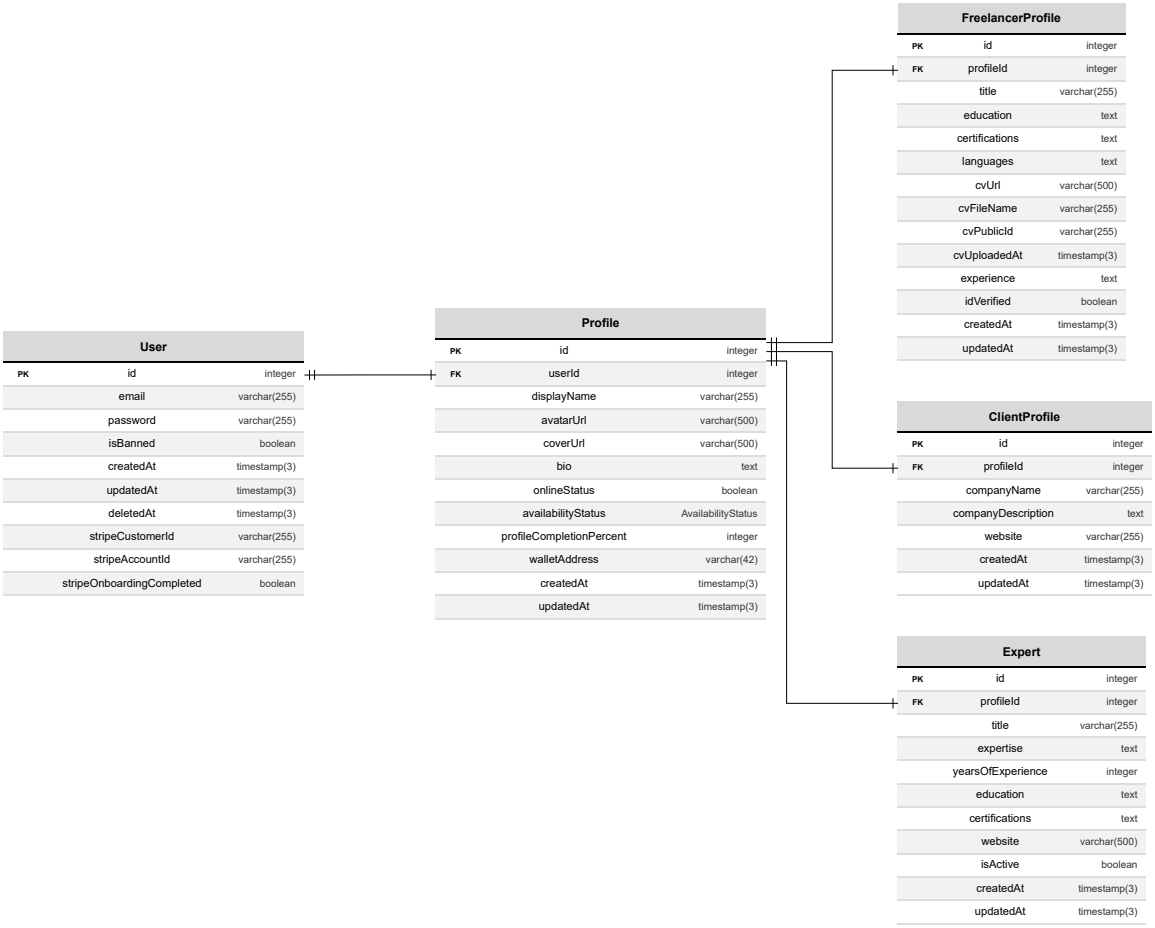

Figure 6: ERD3 — Profile Core

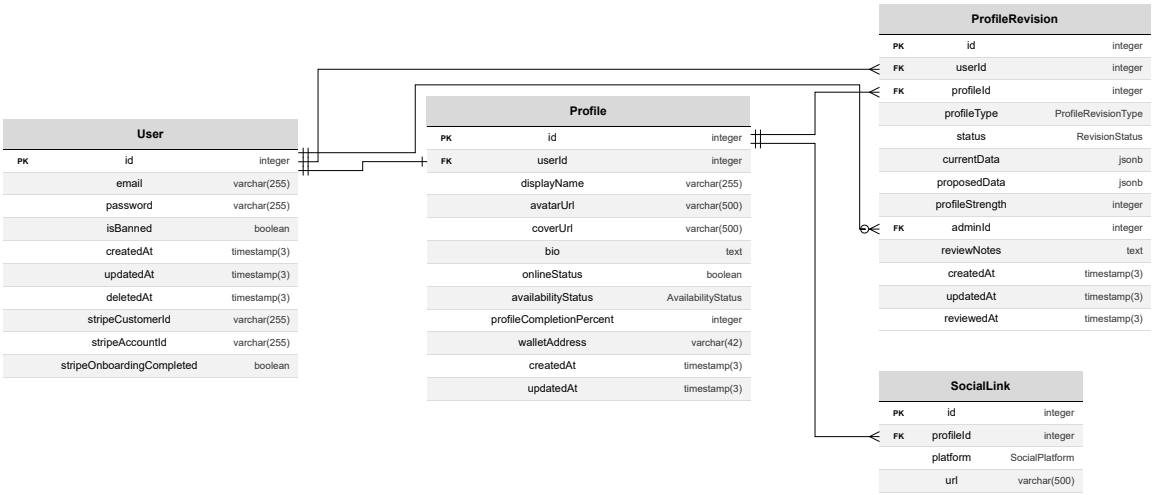

Figure 7: ERD4 — Profile History & Social Links

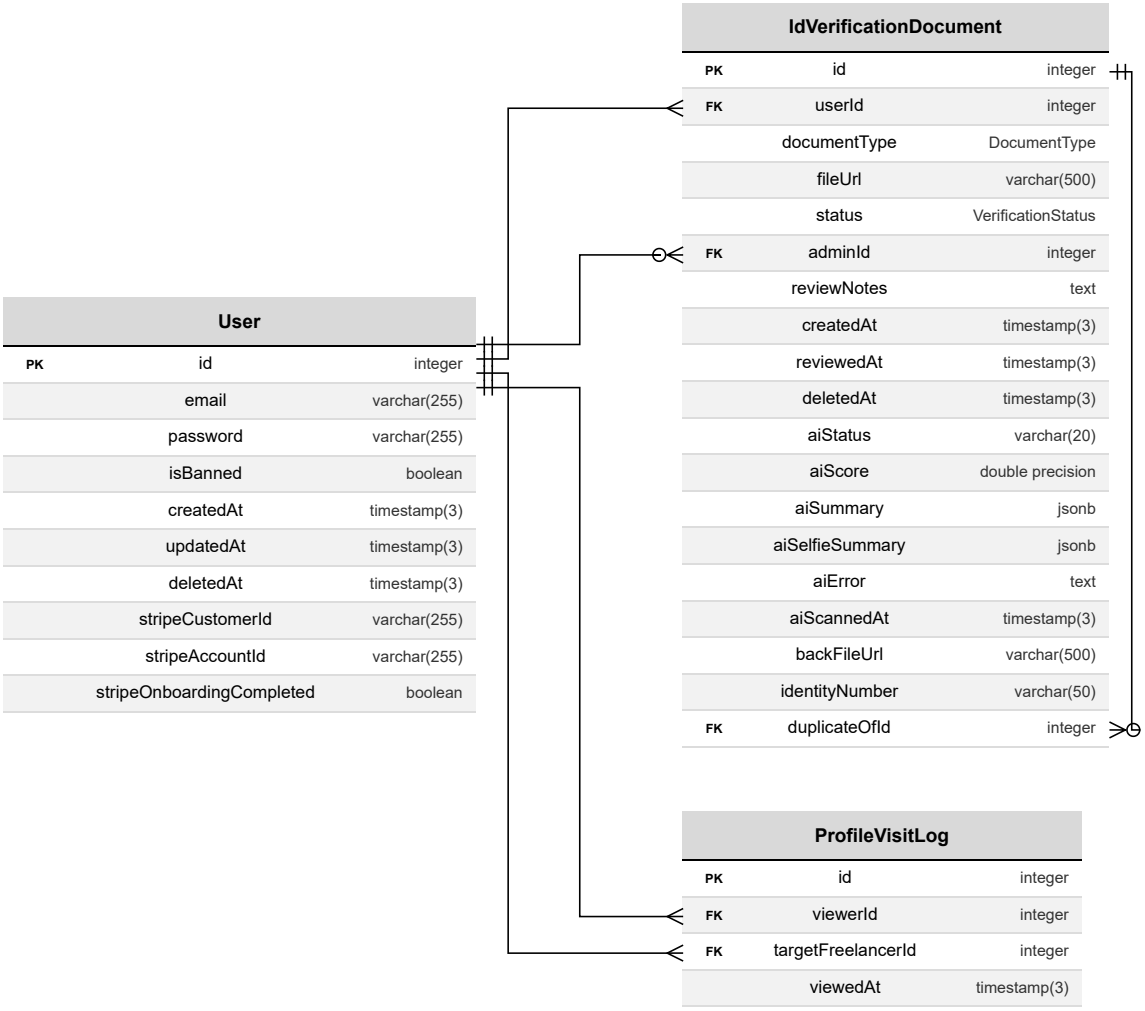

Figure 8: ERD5 — Identity Verification & Profile Visits

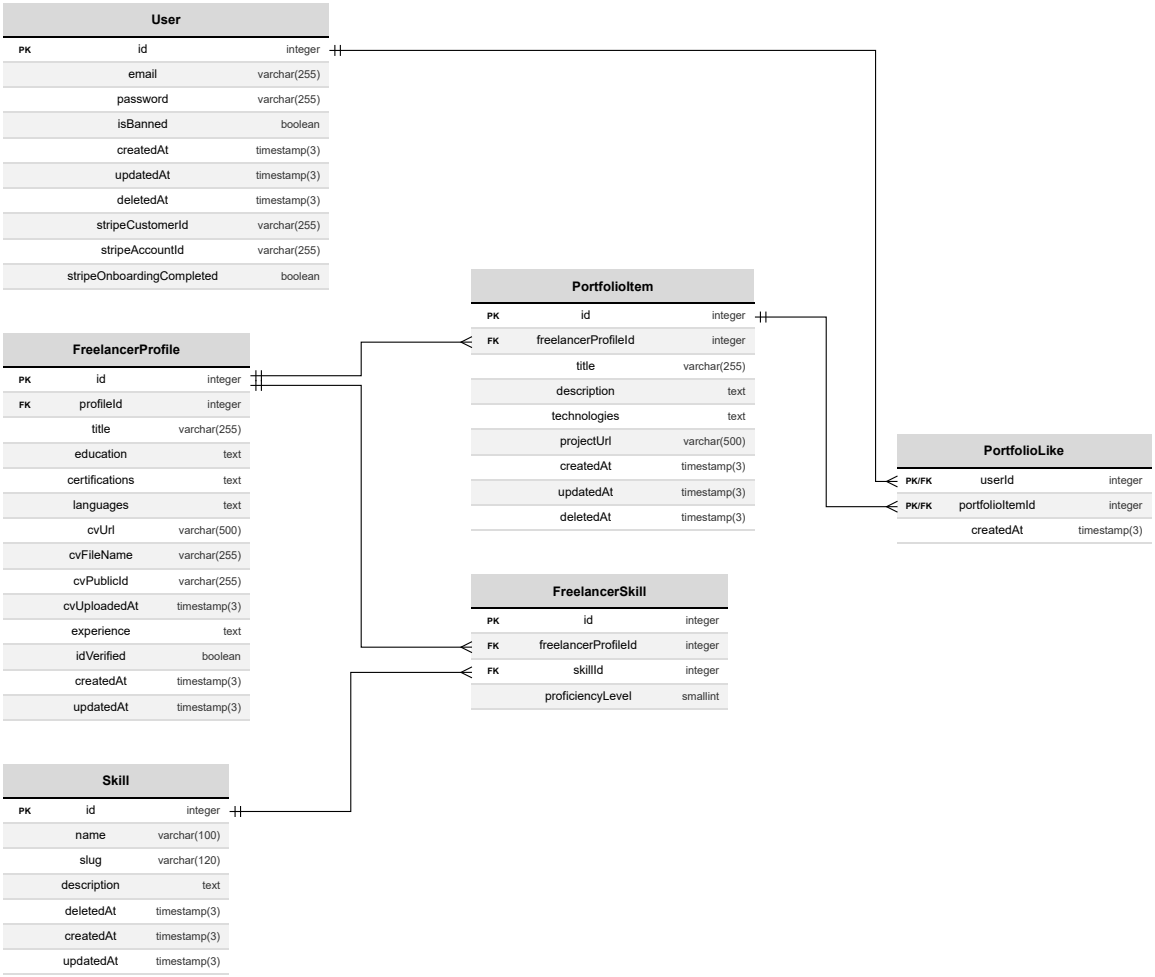

Figure 9: ERD6 — Portfolio & Skills

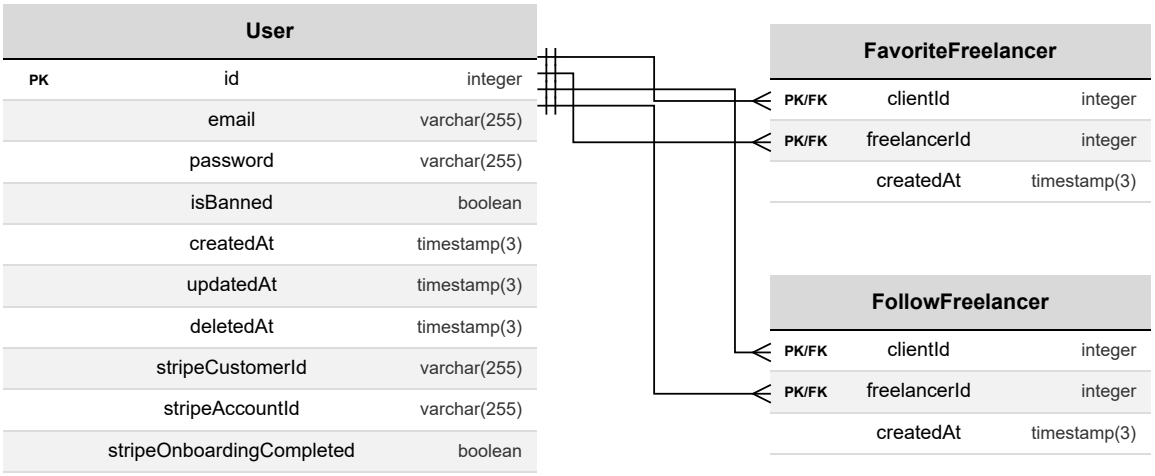

Figure 10: ERD7 — Favorites & Follows

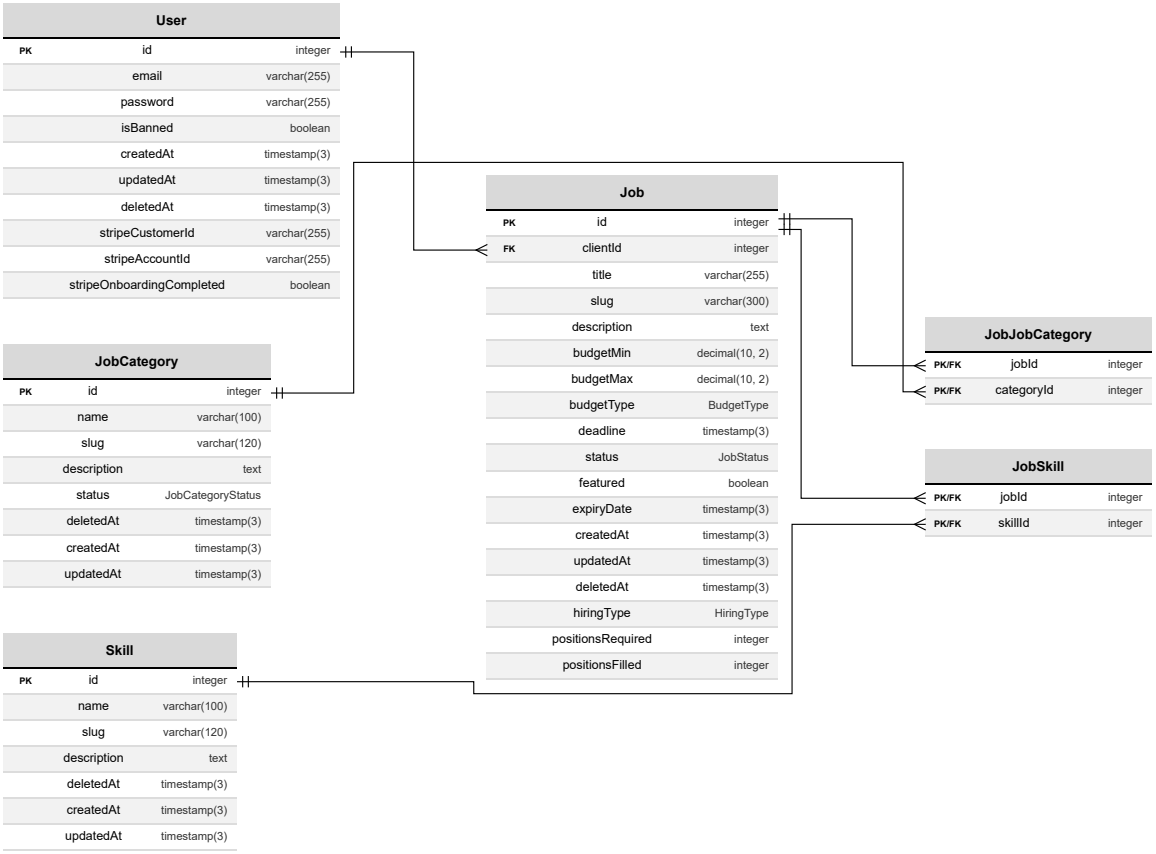

Figure 11: ERD8 — Job Posting

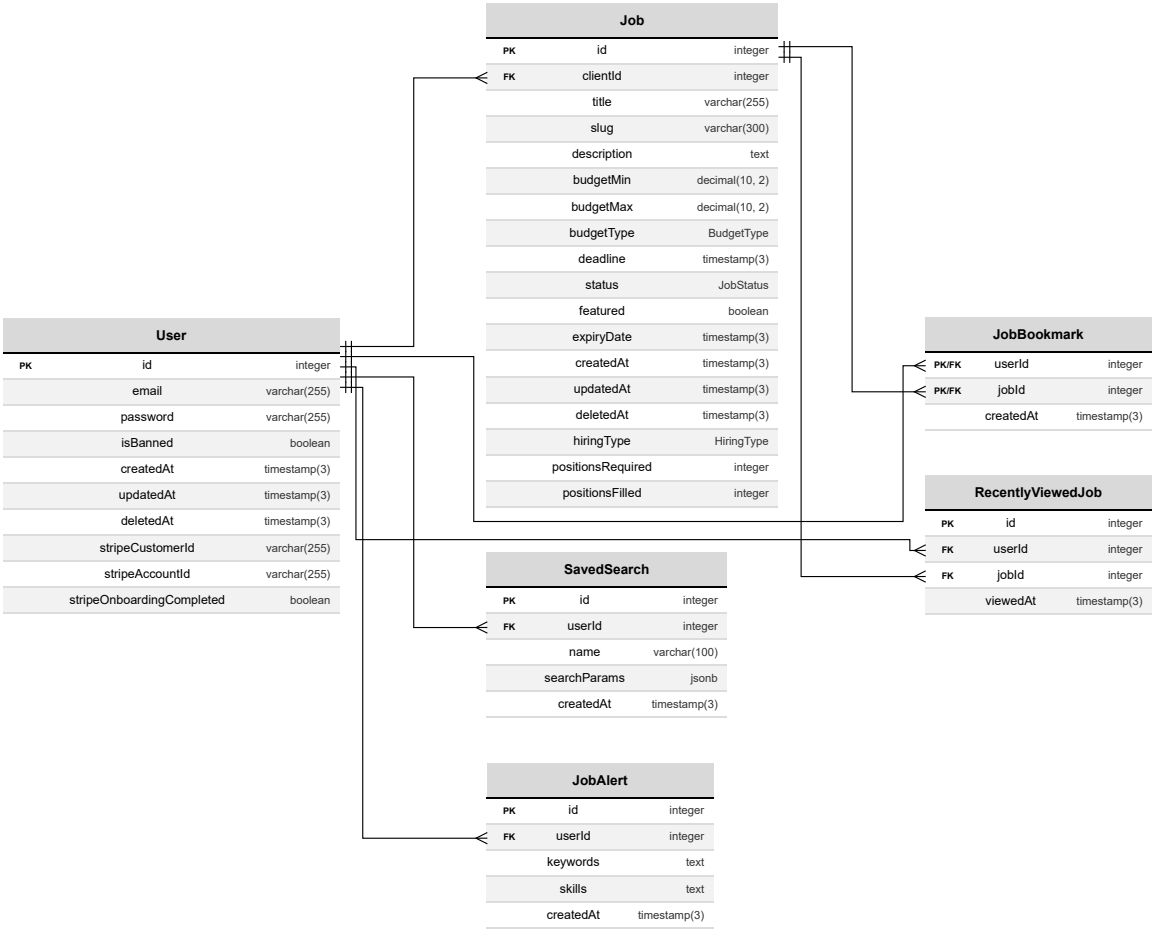

Figure 12: ERD9 — Job Discovery & Alerts

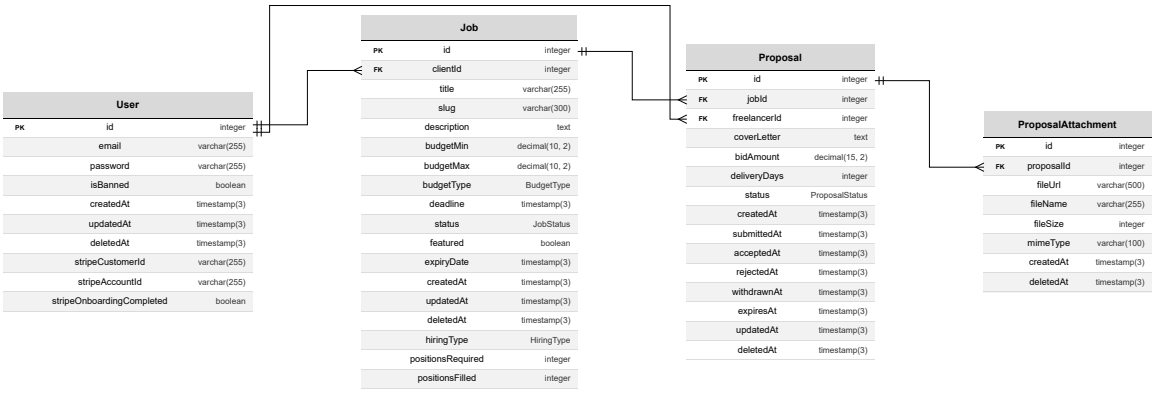

Figure 13: ERD10 — Proposal

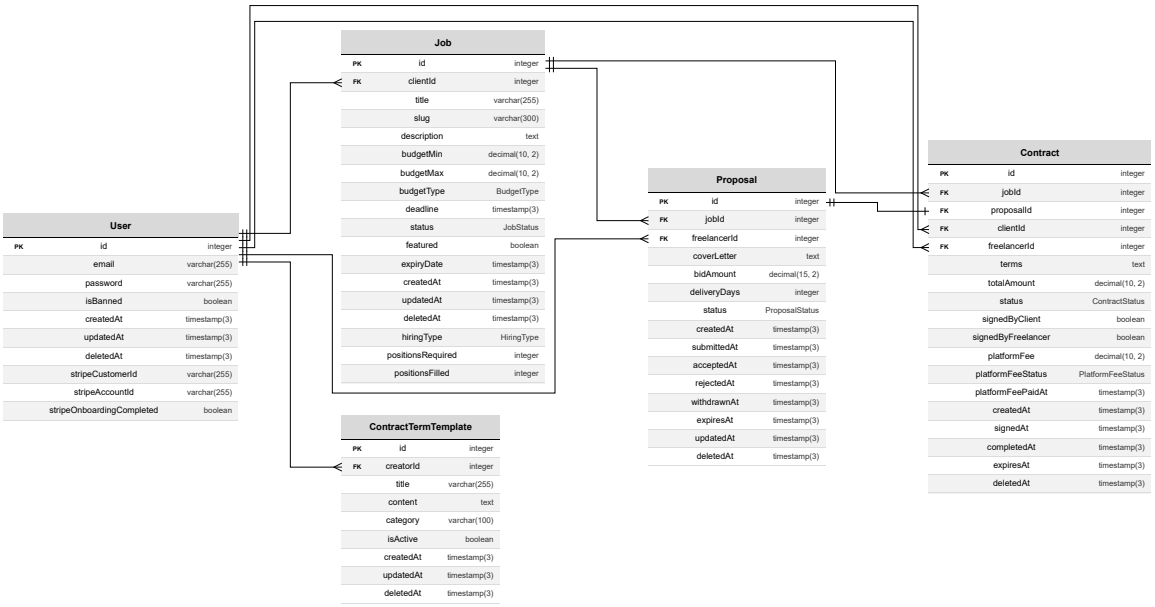

Figure 14: ERD11 — Contract

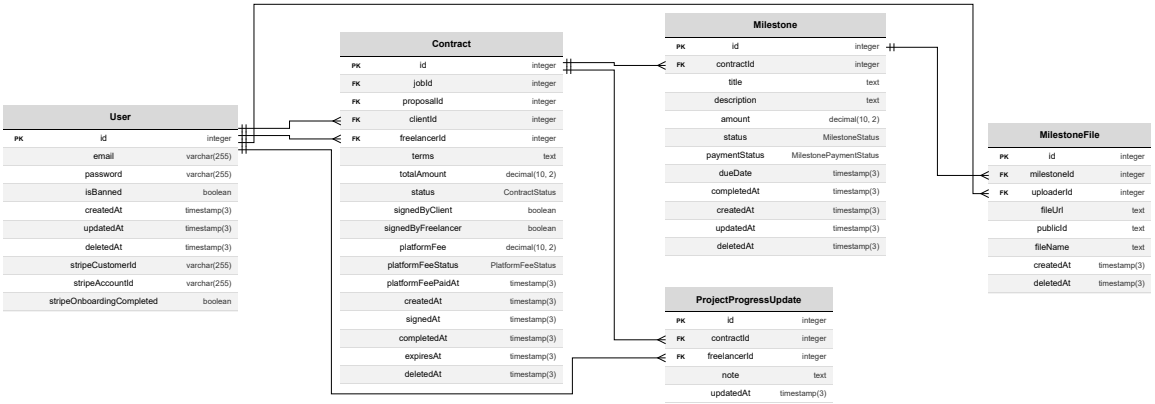

Figure 15: ERD12 — Milestone & Project Progress

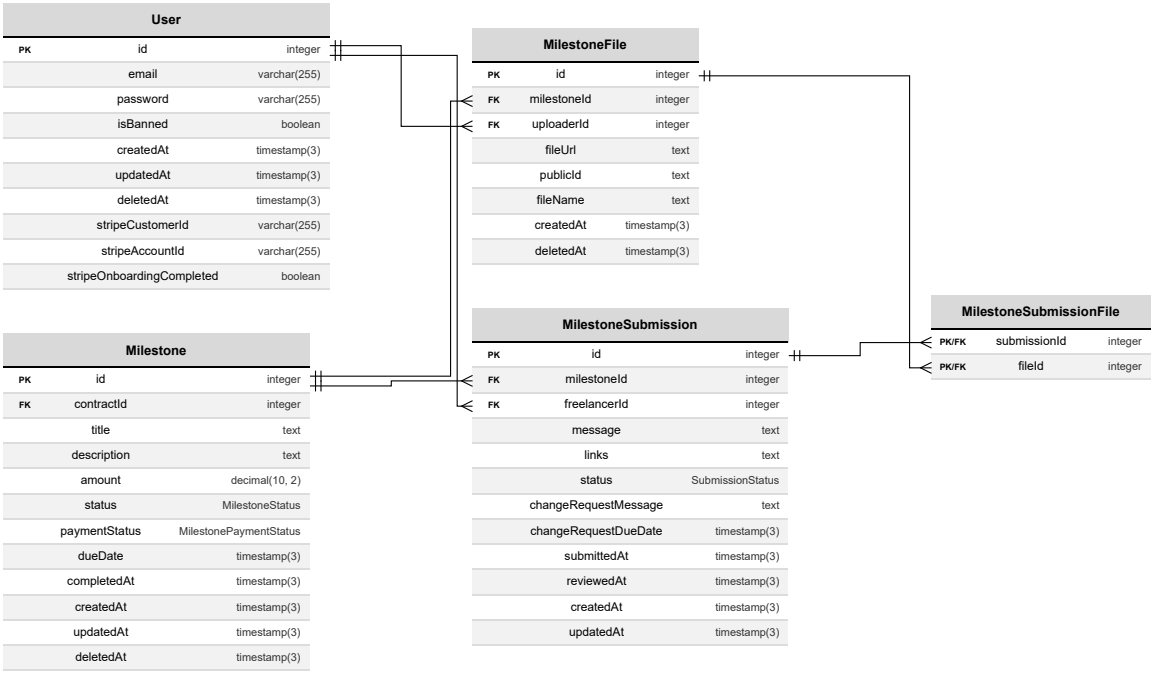

Figure 16: ERD13 — Milestone Submission

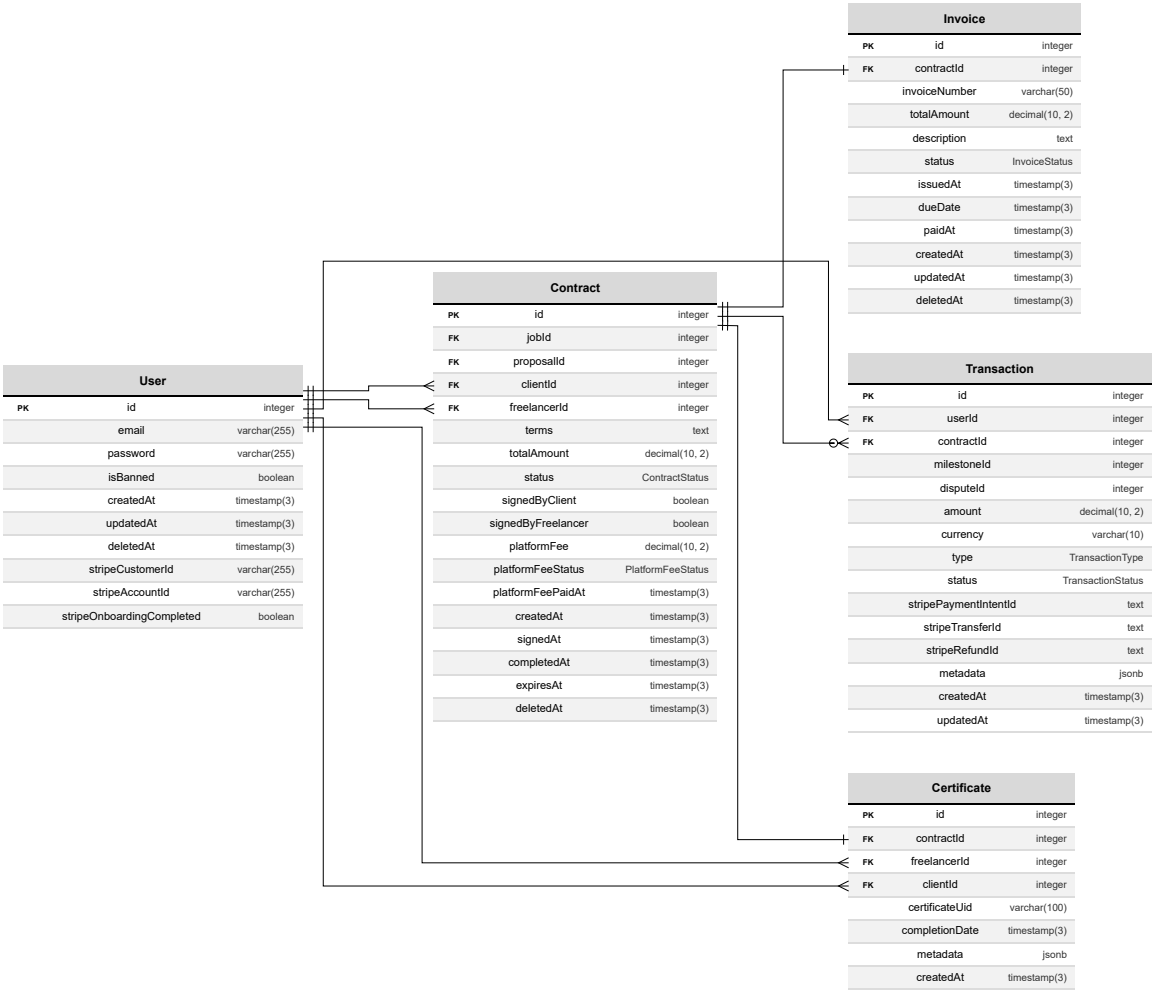

Figure 17: ERD14 — Escrow, Payments & Certificates

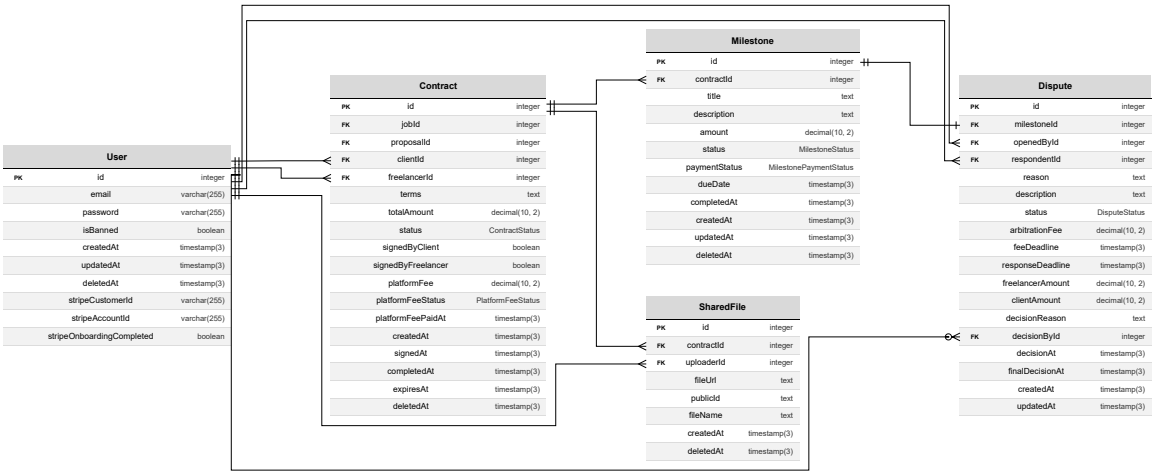

Figure 18: ERD15 — Dispute Core

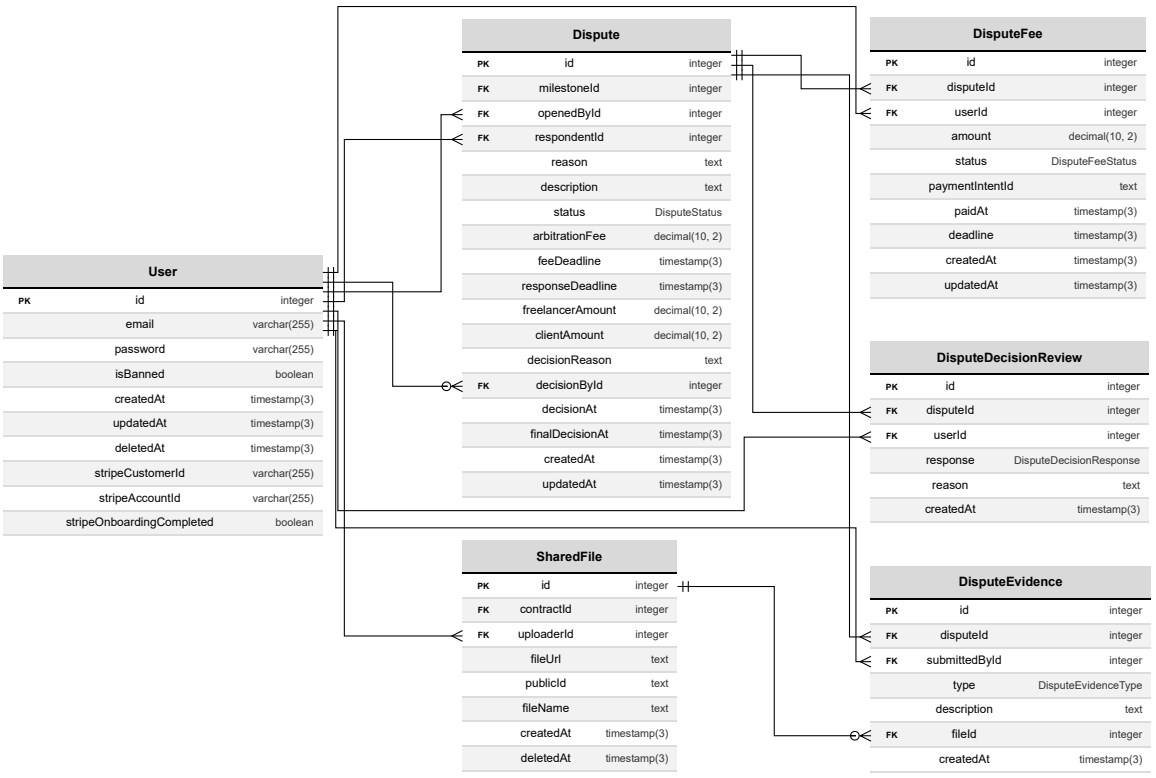

Figure 19: ERD16 — Dispute Resolution

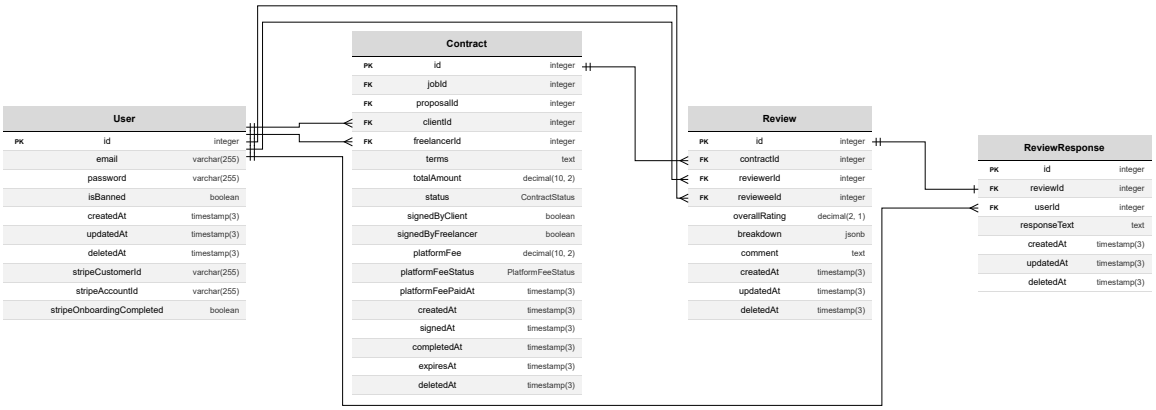

Figure 20: ERD17 — Reviews & Reputation

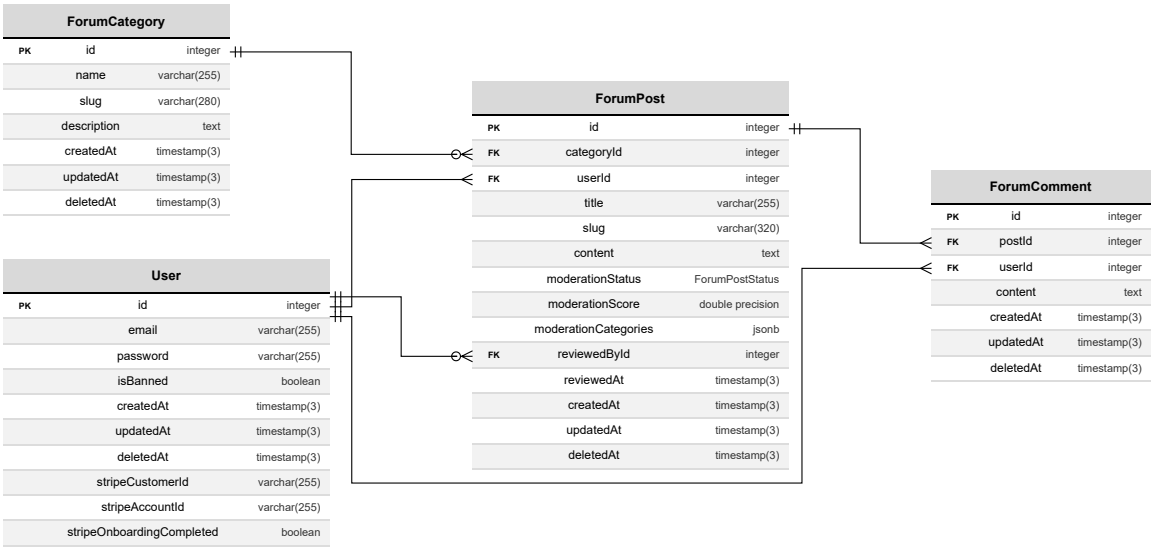

Figure 21: ERD18 — Forum Posts & Comments

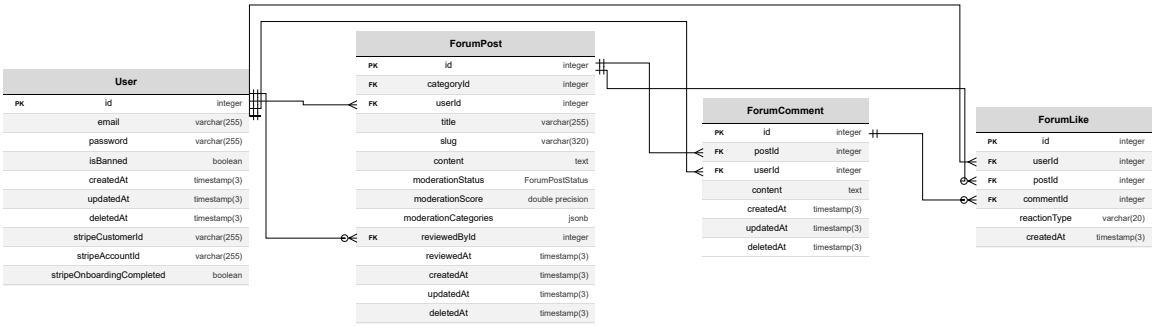

Figure 22: ERD19 — Forum Likes

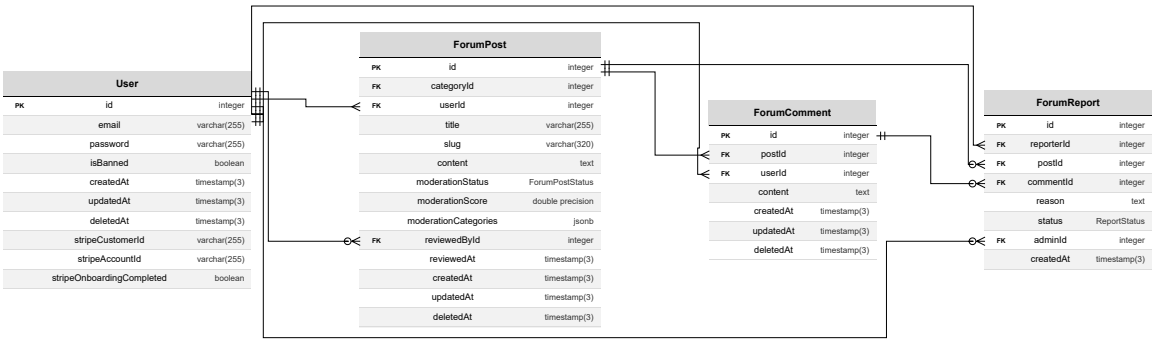

Figure 23: ERD20 — Forum Reports & Moderation

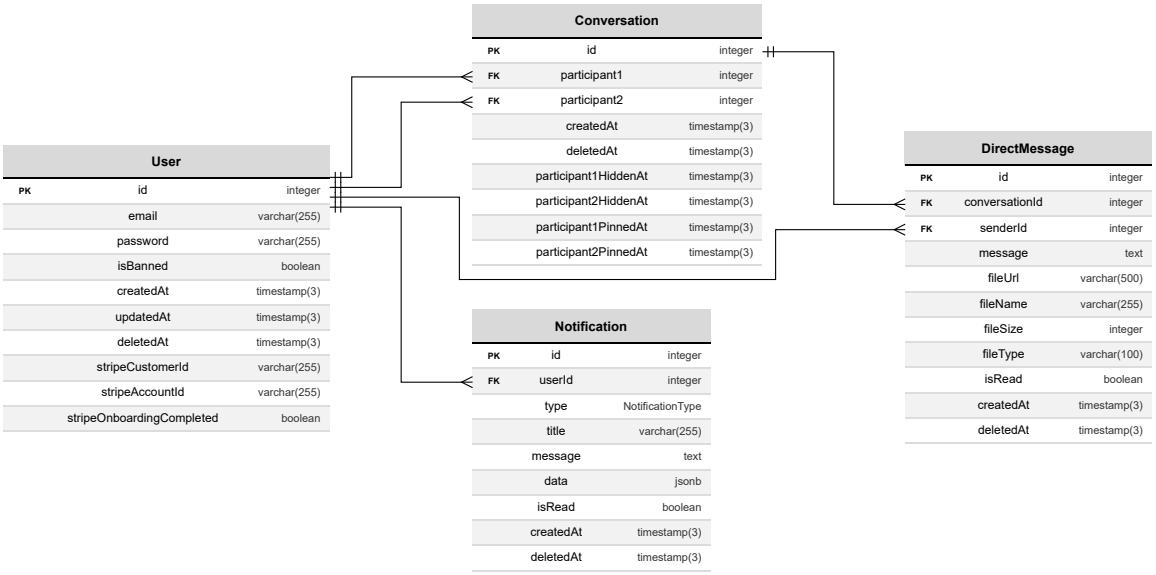

Figure 24: ERD21 — Messaging & Notifications

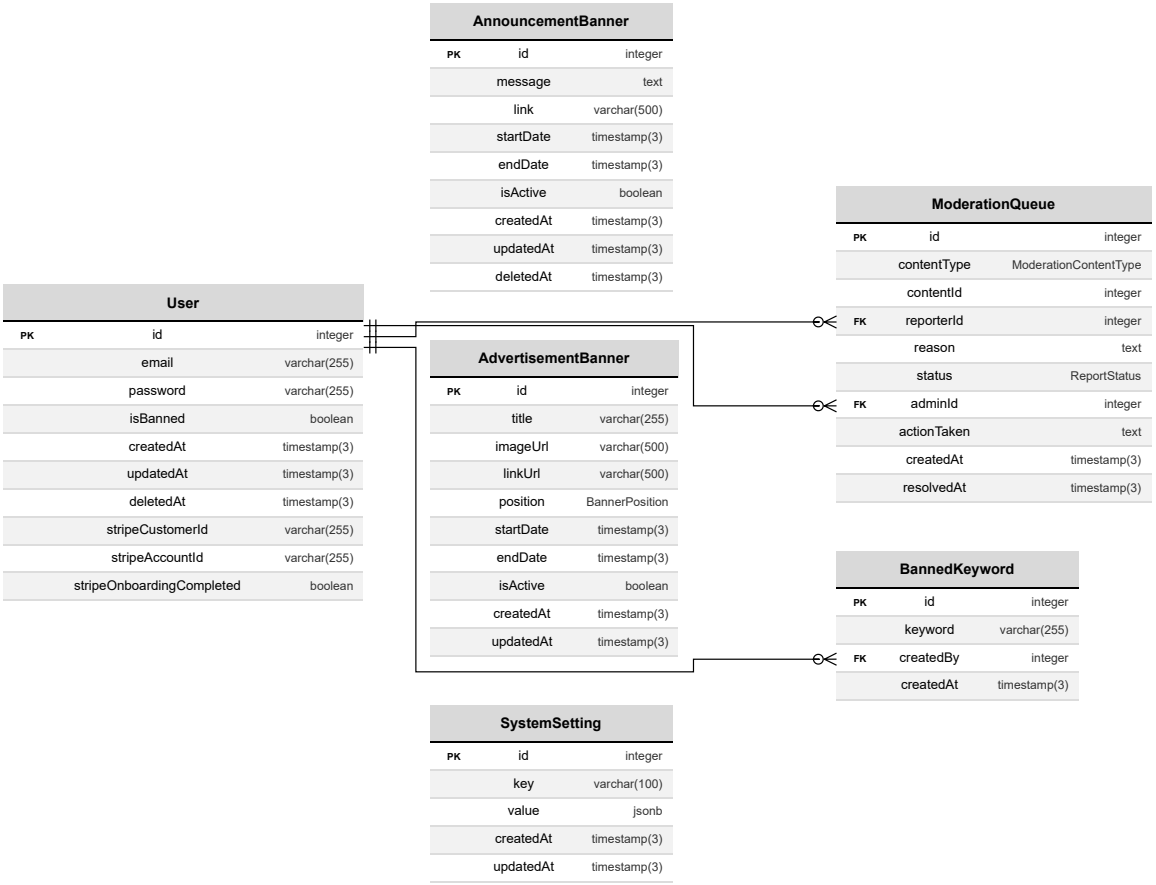

Figure 25: ERD22 — Administration & System

3. Detailed Design

This chapter presents the detailed design of every feature of the system. Each section (3.x) covers one feature and contains the class diagram of the feature (3.x.1) followed by its sequence diagrams (3.x.2, 3.x.3, ...), one for each use case of the feature. Where several functions are built from the same classes and follow the same interaction pattern, the diagrams are presented once for the representative function and the other functions refer to that section.

3.1 Verify JWT

Verify JWT is the shared security guard protecting API endpoints across all modules. Every incoming HTTP request with a Bearer token is intercepted, validated for cryptographic signature and expiration time, and verified against the database to guarantee the associated session remains active and has not been revoked.

3.1.0.1 Class Diagram

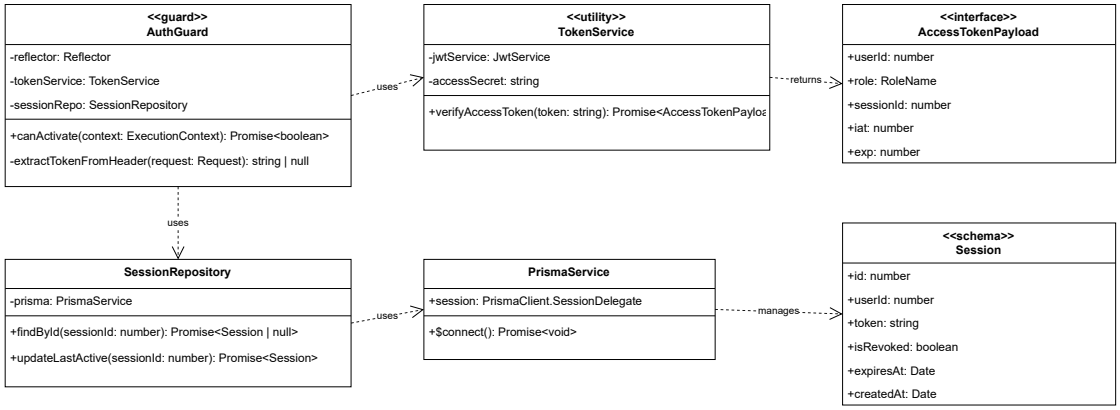

Figure 26: Class diagram of Verify JWT

3.1.0.2 UC-0 – Verify JWT

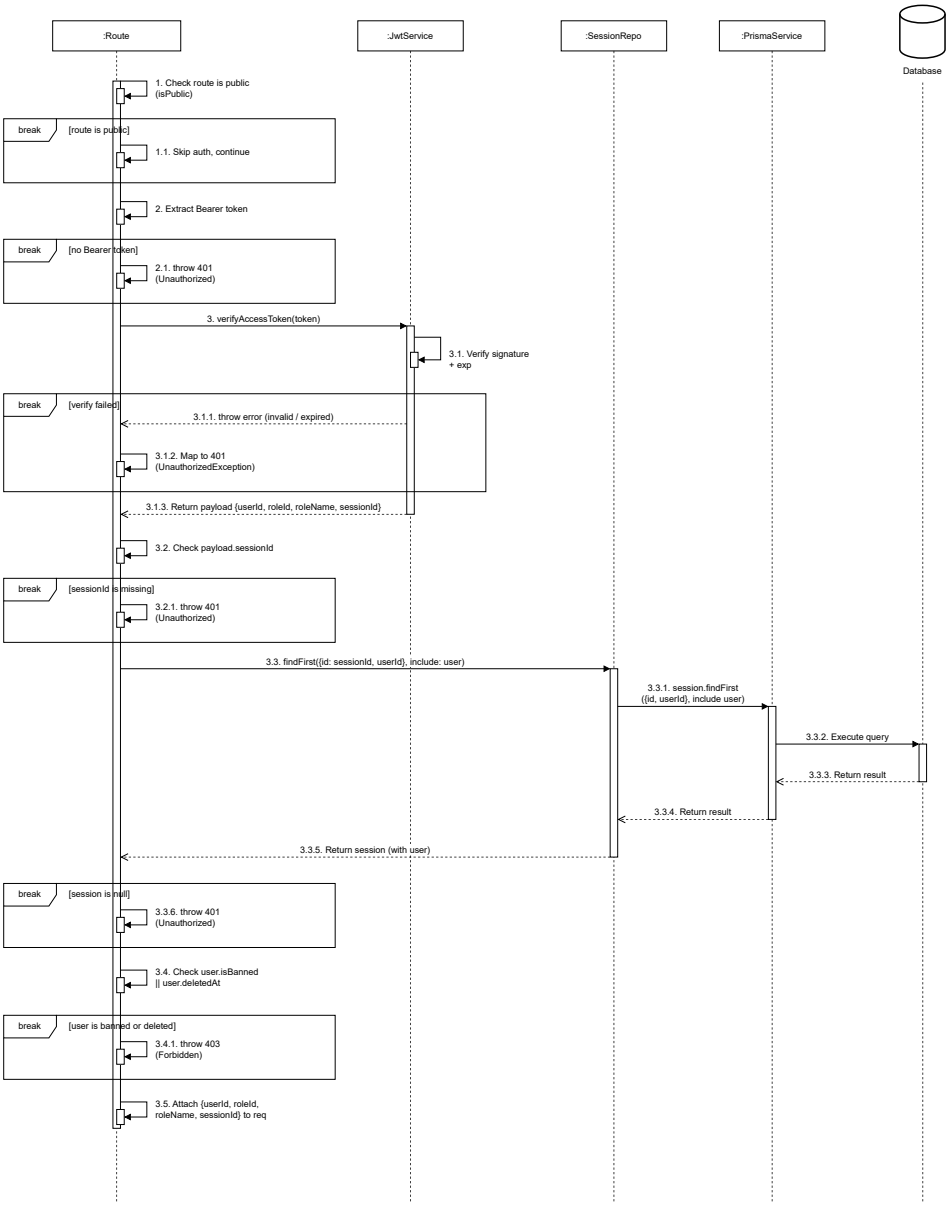

Figure 27: Sequence diagram of UC-0 Verify JWT

3.2 Authentication

The authentication module covers account registration, email/password and Google OAuth login, logout, password recovery, and the JWT/session flow shared by all protected endpoints.

3.2.0.1 Class Diagram

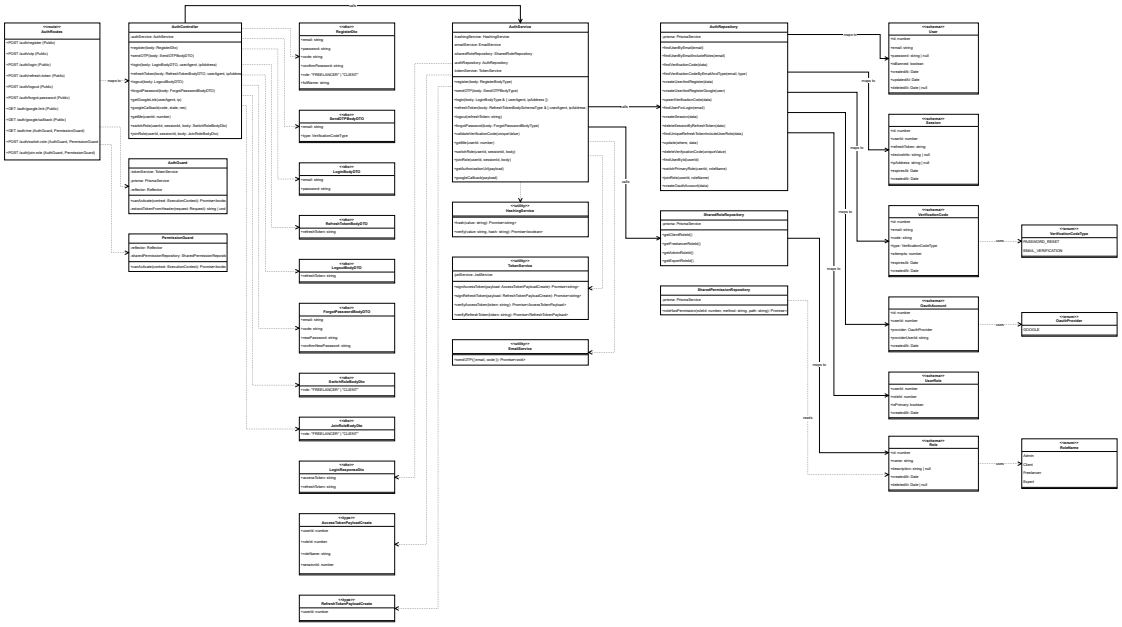

Figure 28: Class diagram of Authentication

3.2.0.2 UC-01 – Login with Google

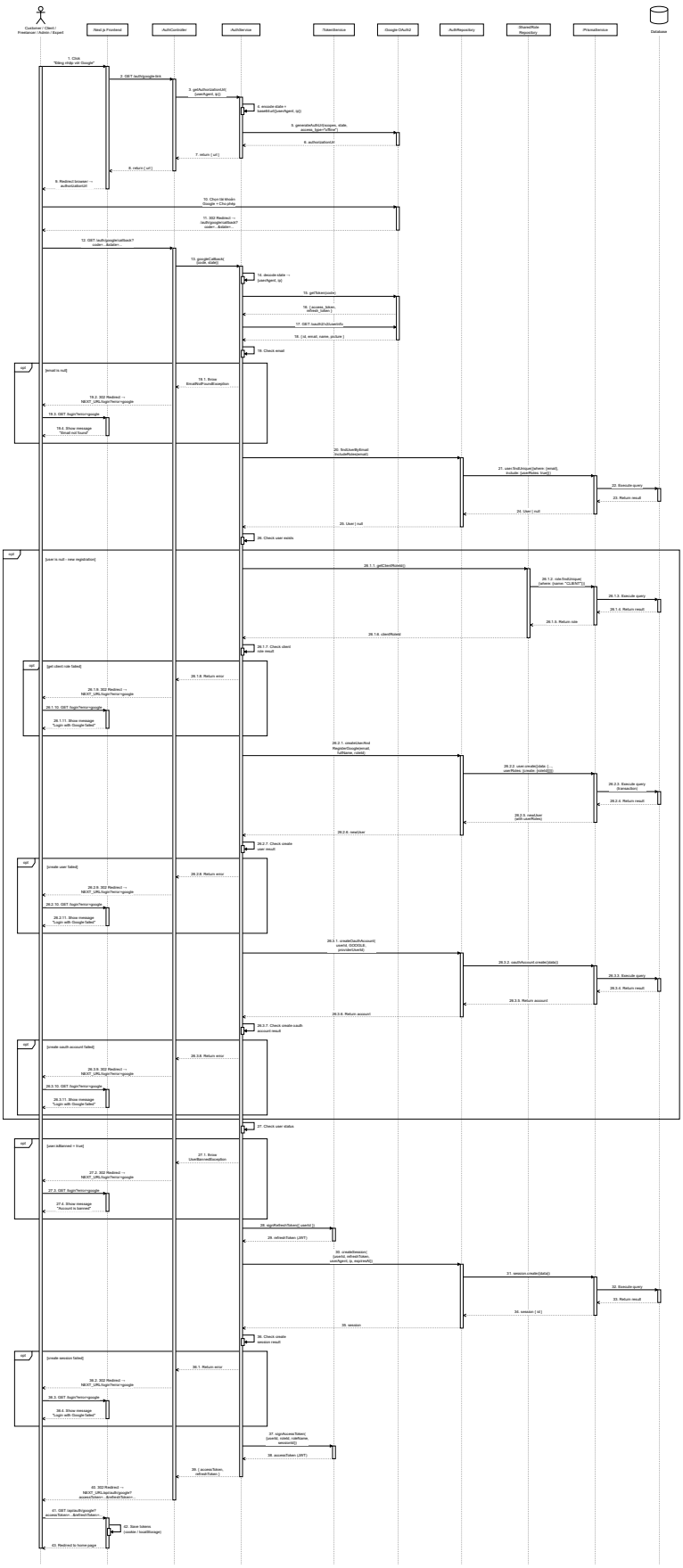

Figure 29: Sequence diagram of UC-01 Login with Google

3.2.0.3 UC-02 – Login

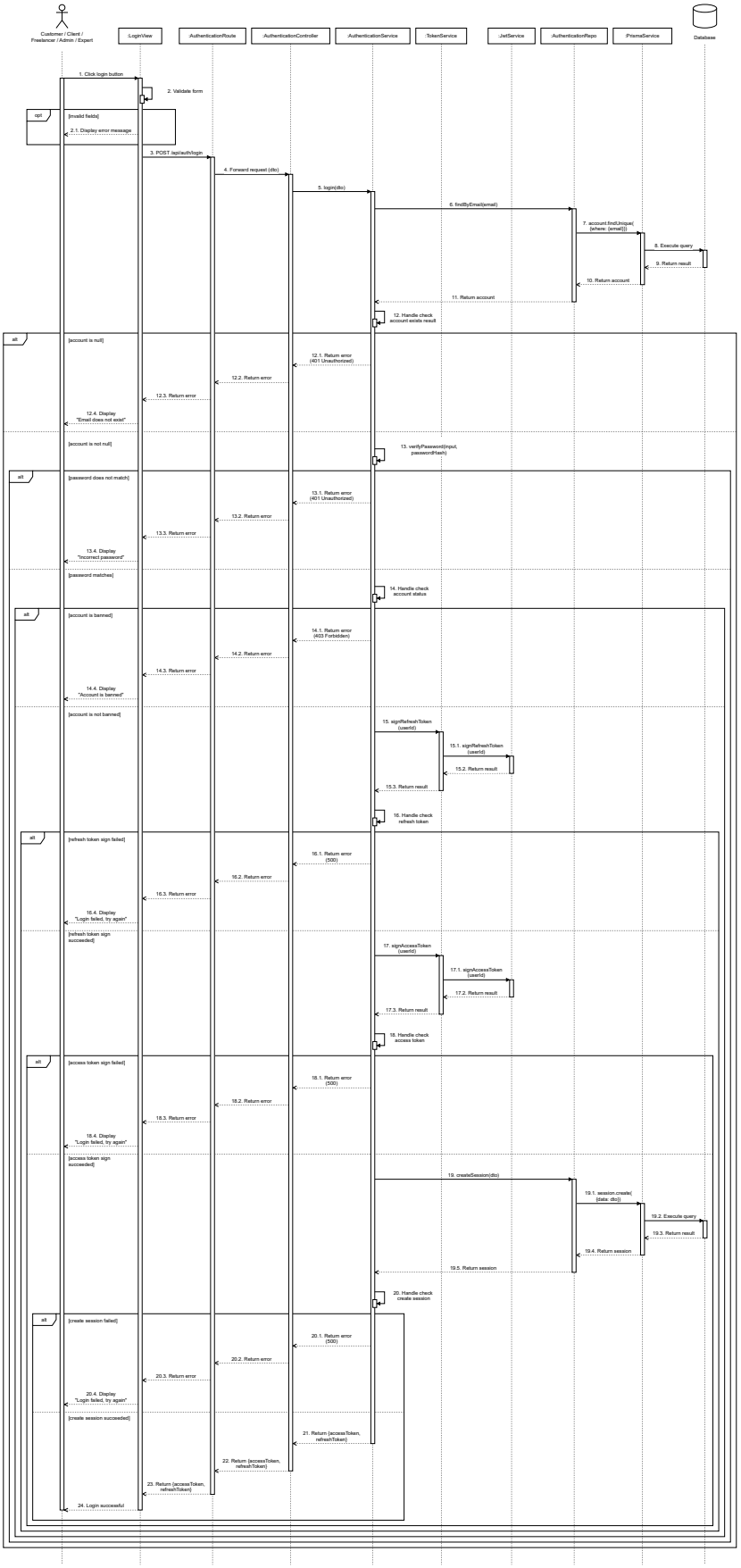

Figure 30: Sequence diagram of UC-02 Login

3.2.0.4 UC-03 – Logout

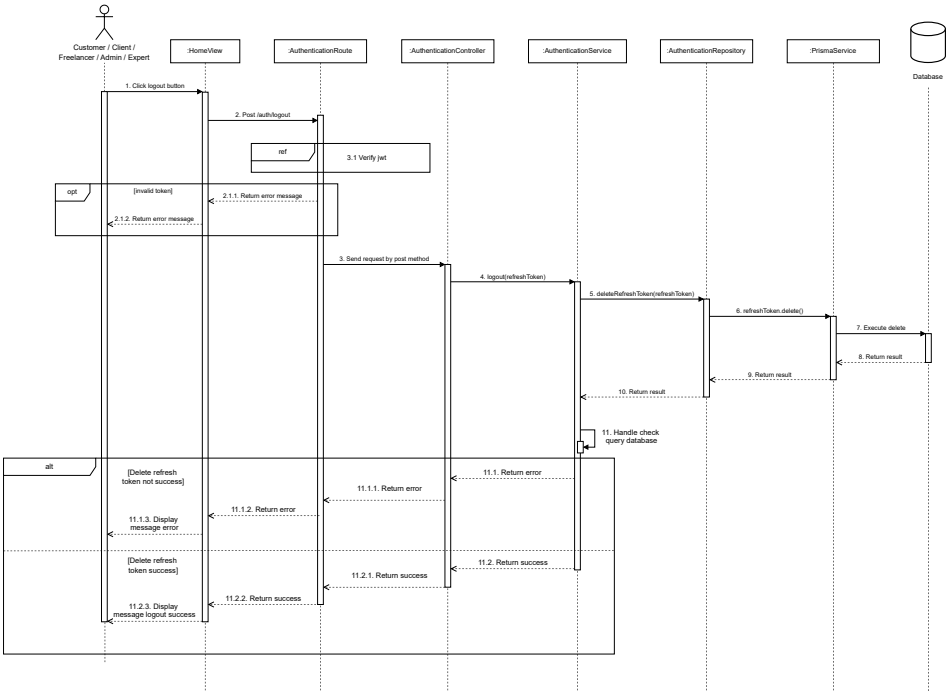

Figure 31: Sequence diagram of UC-03 Logout

3.2.0.5 UC-04 – Register

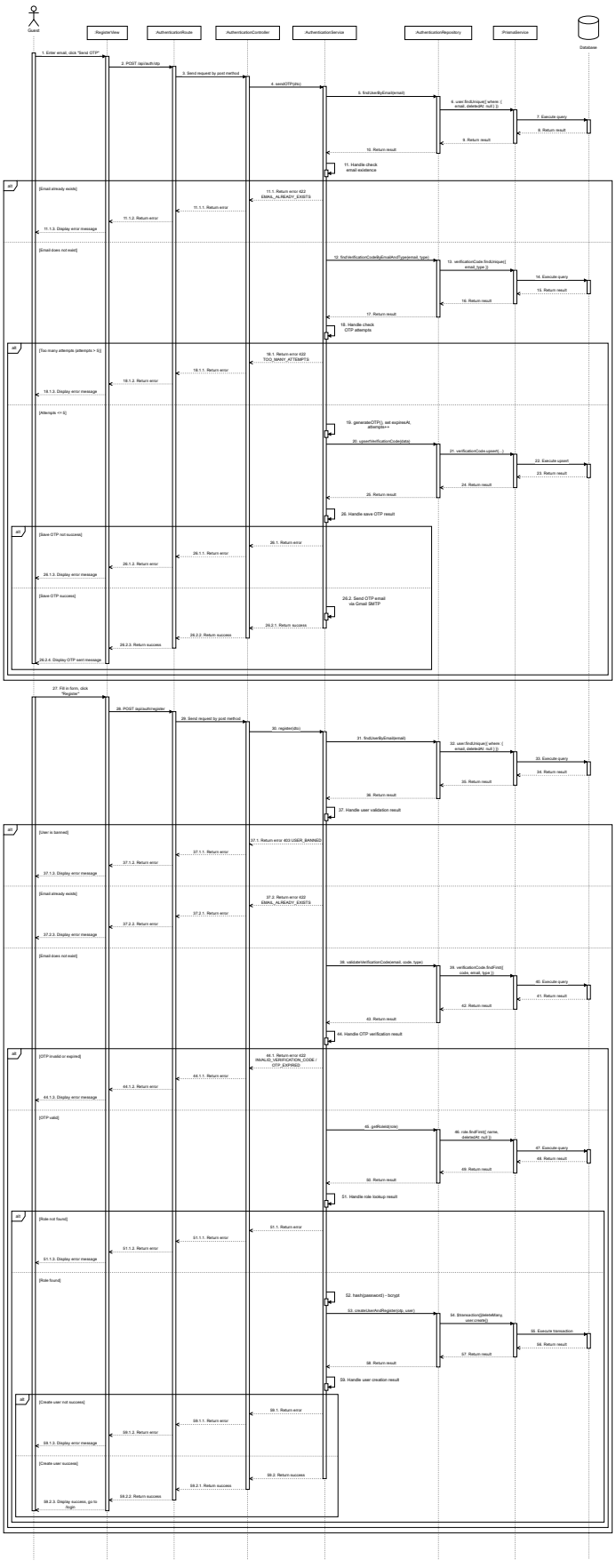

Figure 32: Sequence diagram of UC-04 Register

3.2.0.6 UC-05 – Forgot Password

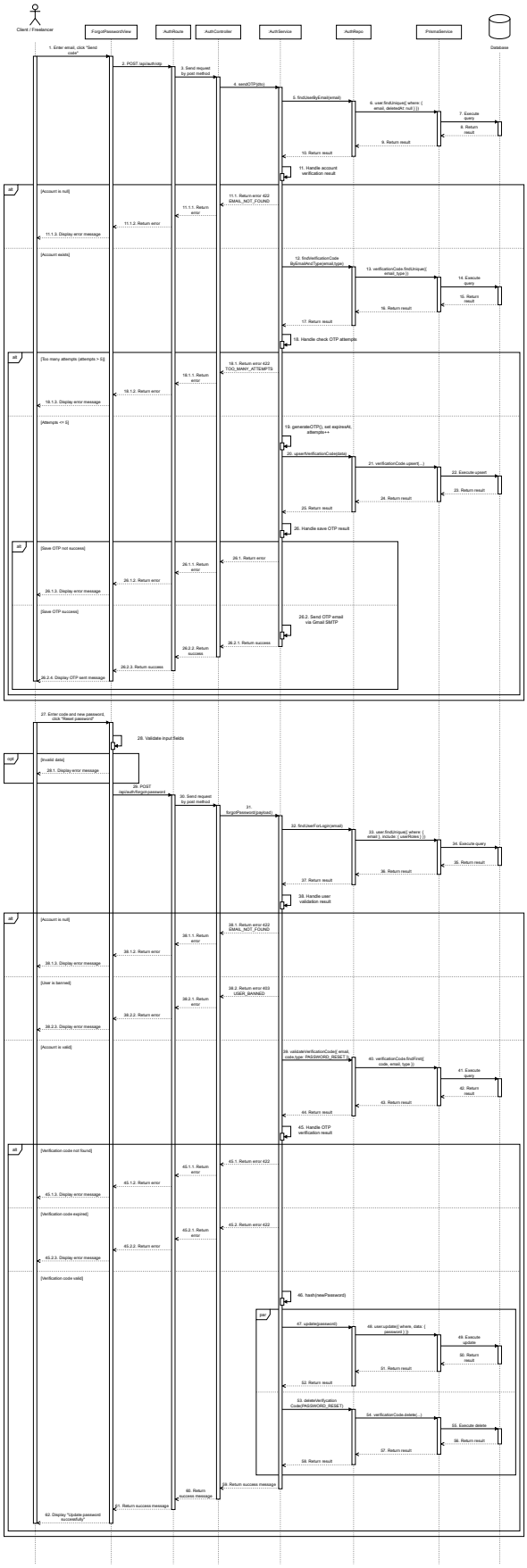

Figure 33: Sequence diagram of UC-05 Forgot Password

3.3 Role Management

Role Management lets an administrator inspect the role catalogue, maintain role metadata, and replace the permission set assigned to a role.

3.3.0.1 Class Diagram

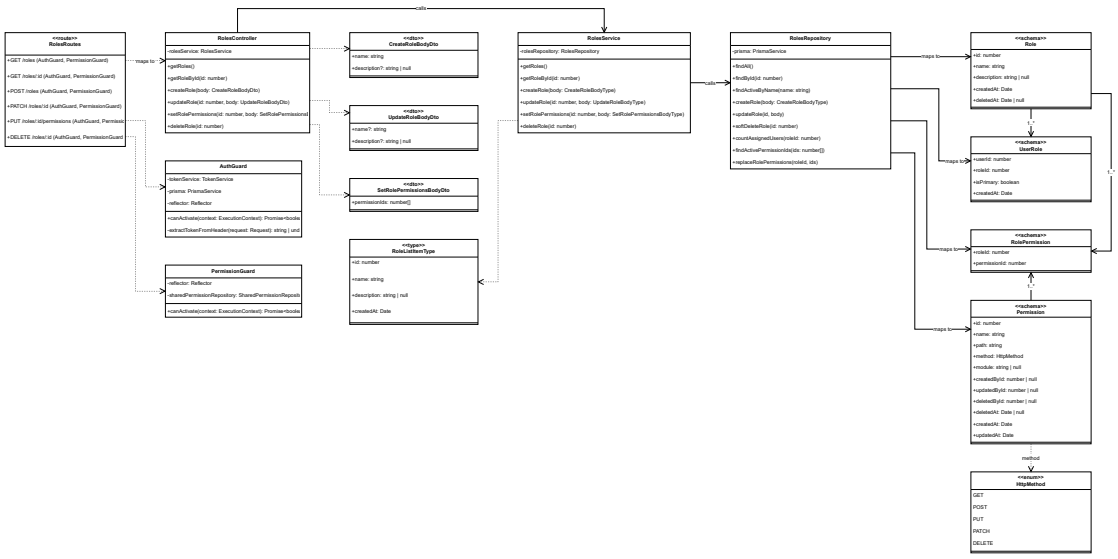

Figure 34: Class diagram of Role Management

3.3.0.2 UC-06.01 – View Role List

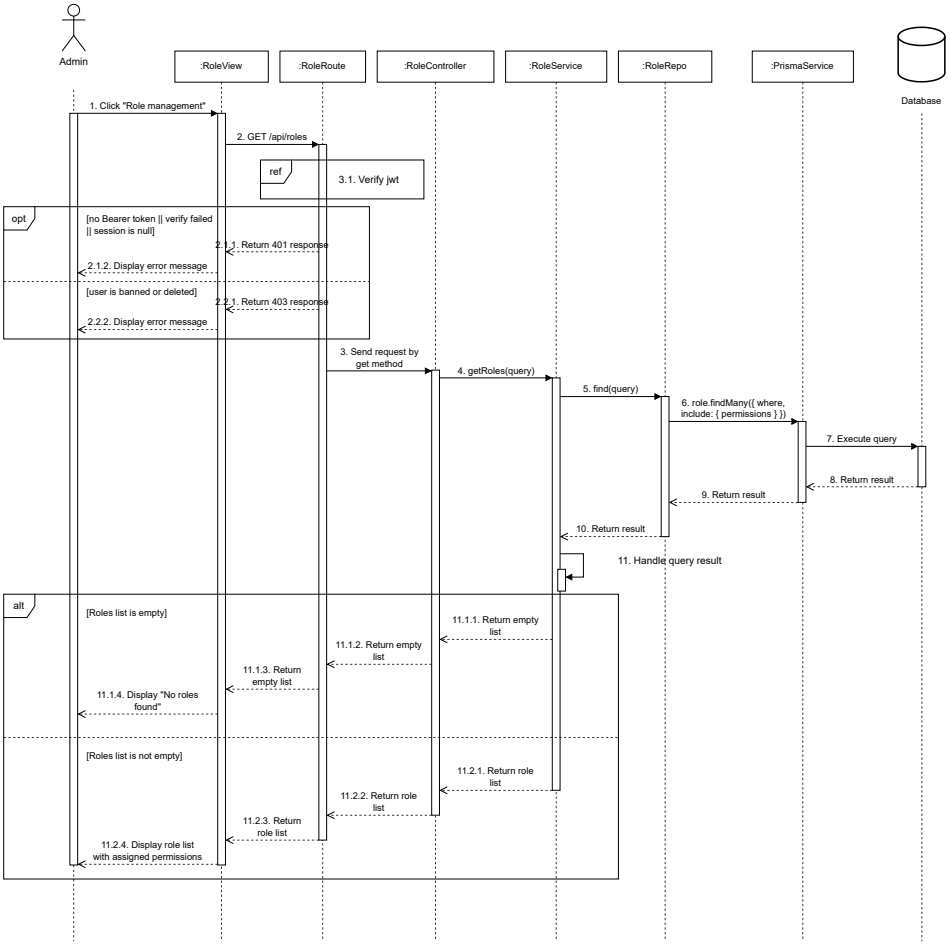

Figure 35: Sequence diagram of UC-06.01 View Role List

3.3.0.3 UC-06.02 – View Role Detail

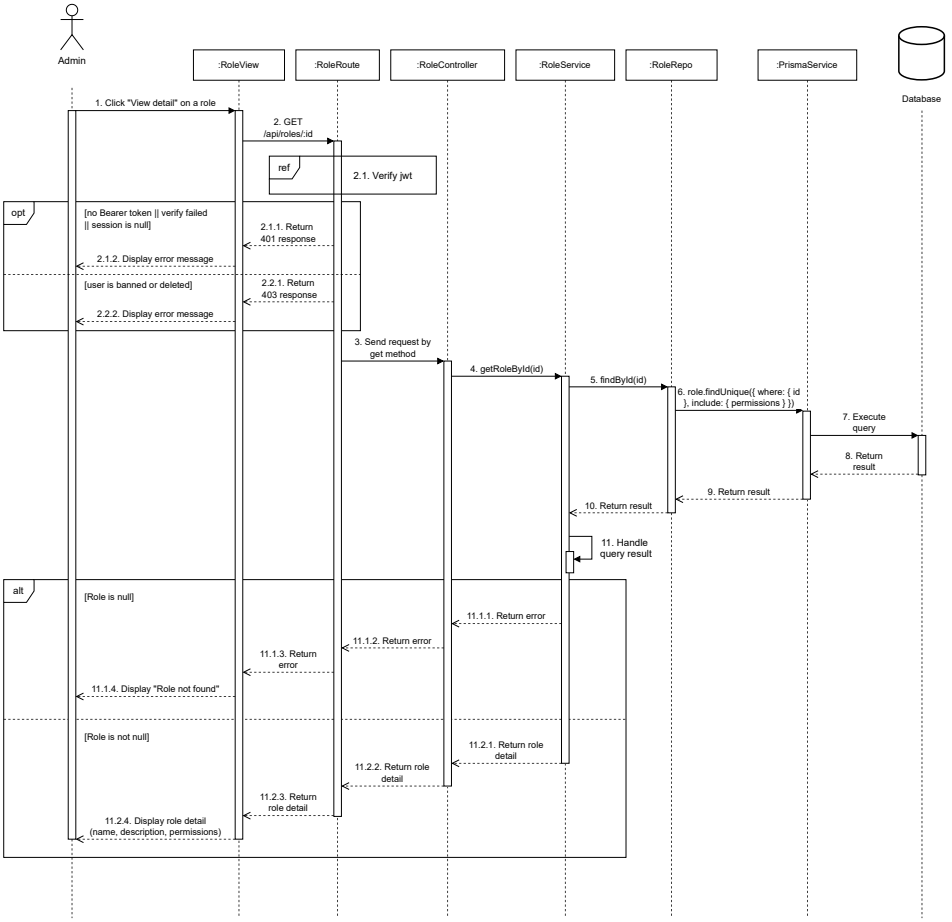

Figure 36: Sequence diagram of UC-06.02 View Role Detail

3.3.0.4 UC-06.03 – Create Role

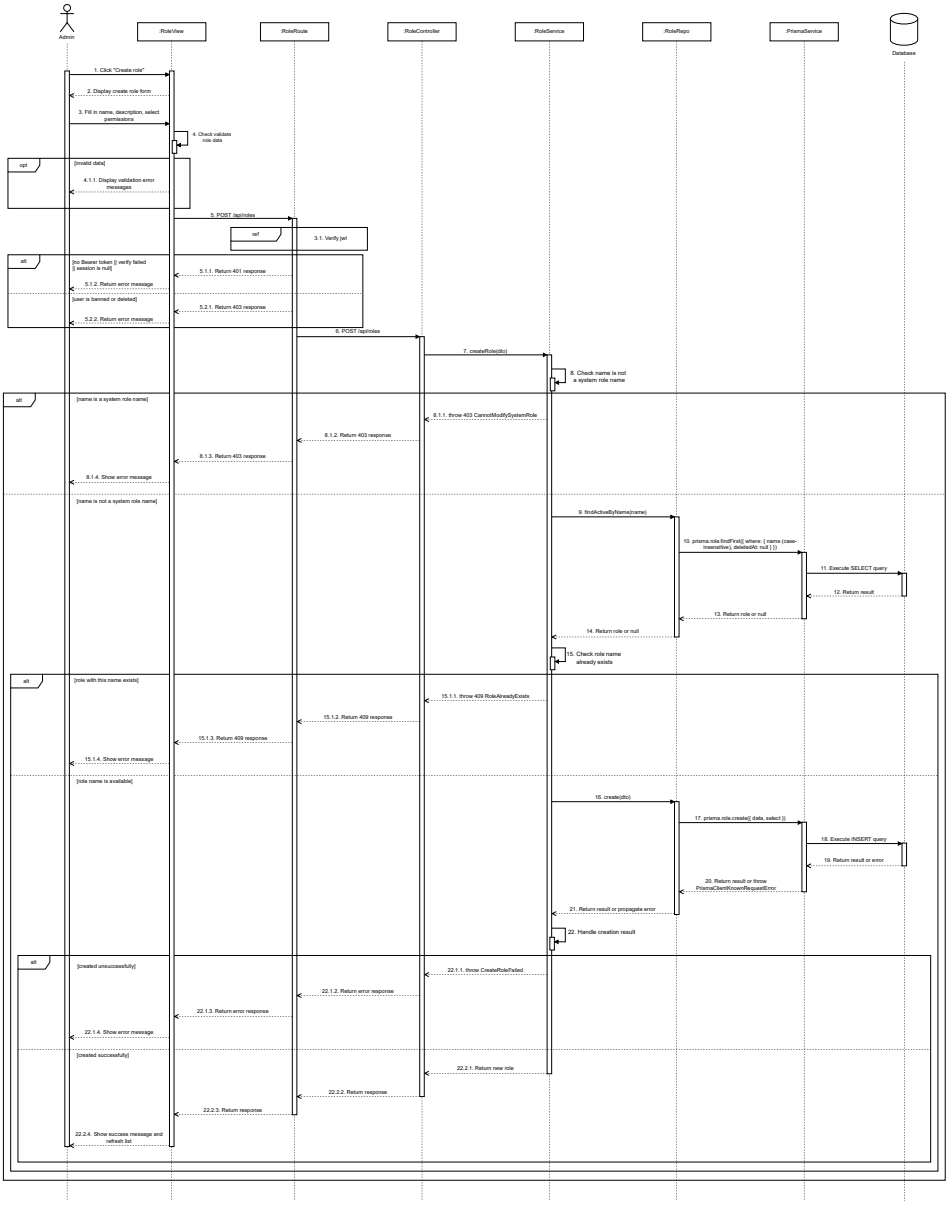

Figure 37: Sequence diagram of UC-06.03 Create Role

3.3.0.5 UC-06.04 – Update Role

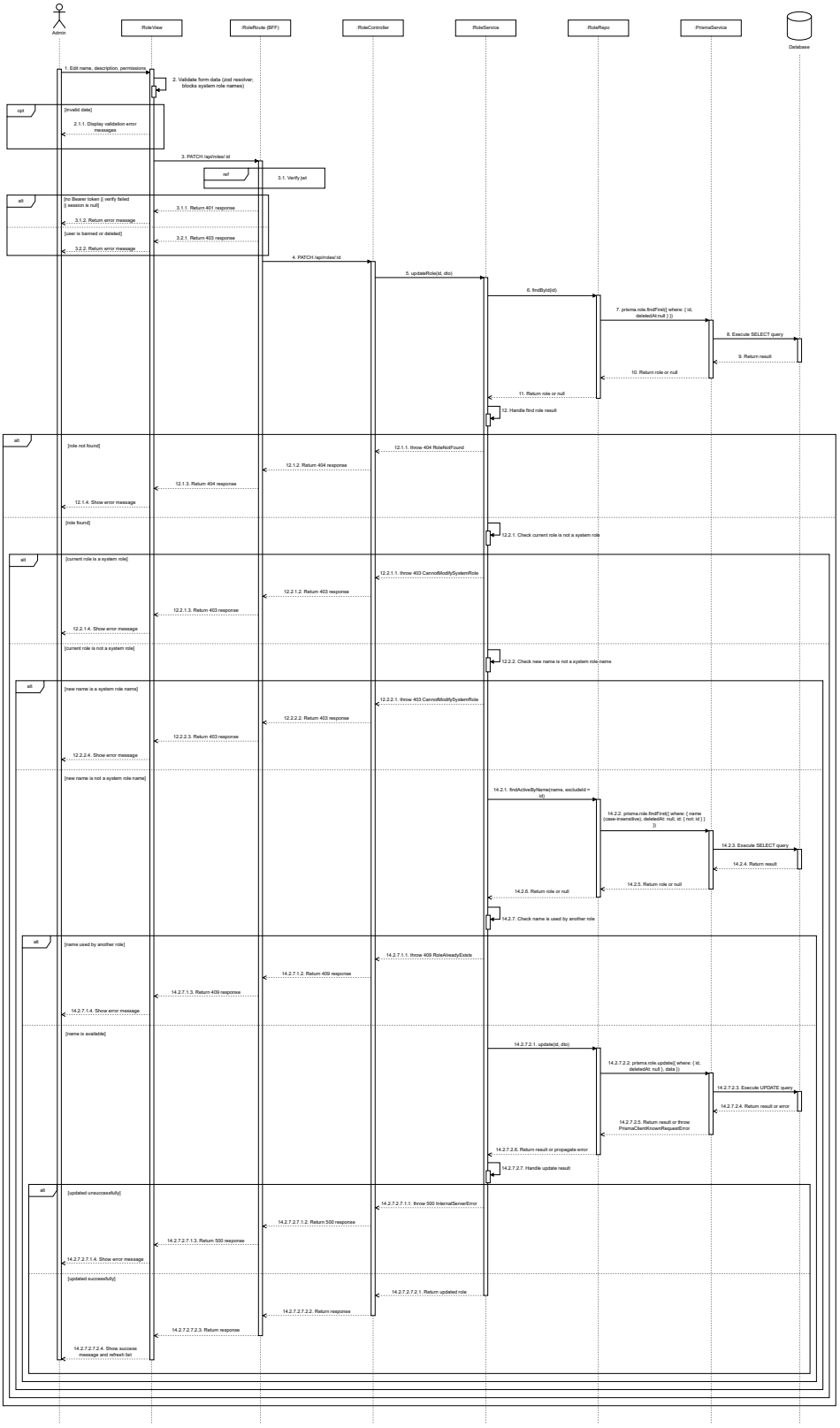

Figure 38: Sequence diagram of UC-06.04 Update Role

3.3.0.6 UC-06.05 – Delete Role

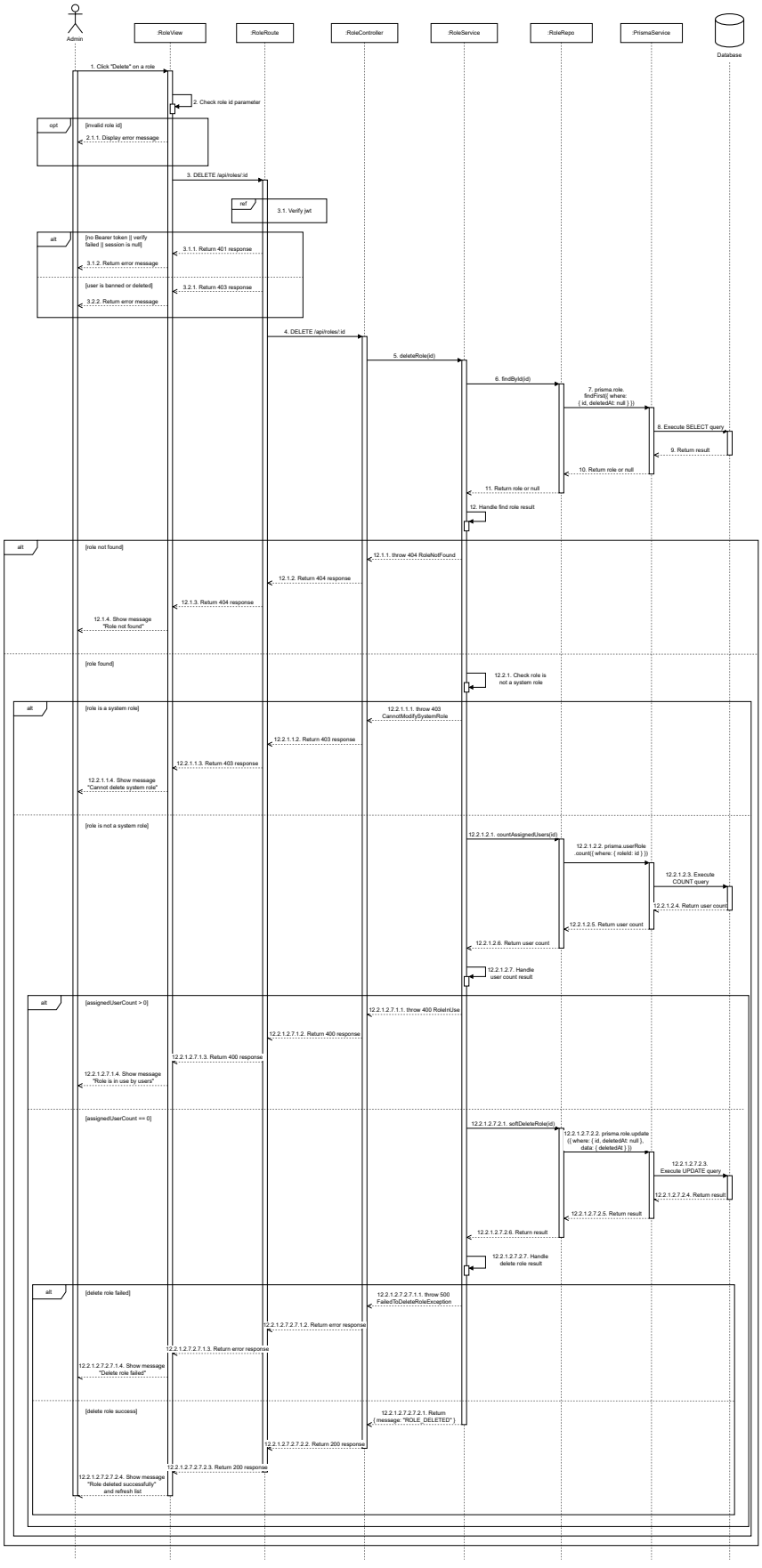

Figure 39: Sequence diagram of UC-06.05 Delete Role

3.3.0.7 UC-06.06 – Update Role Permissions

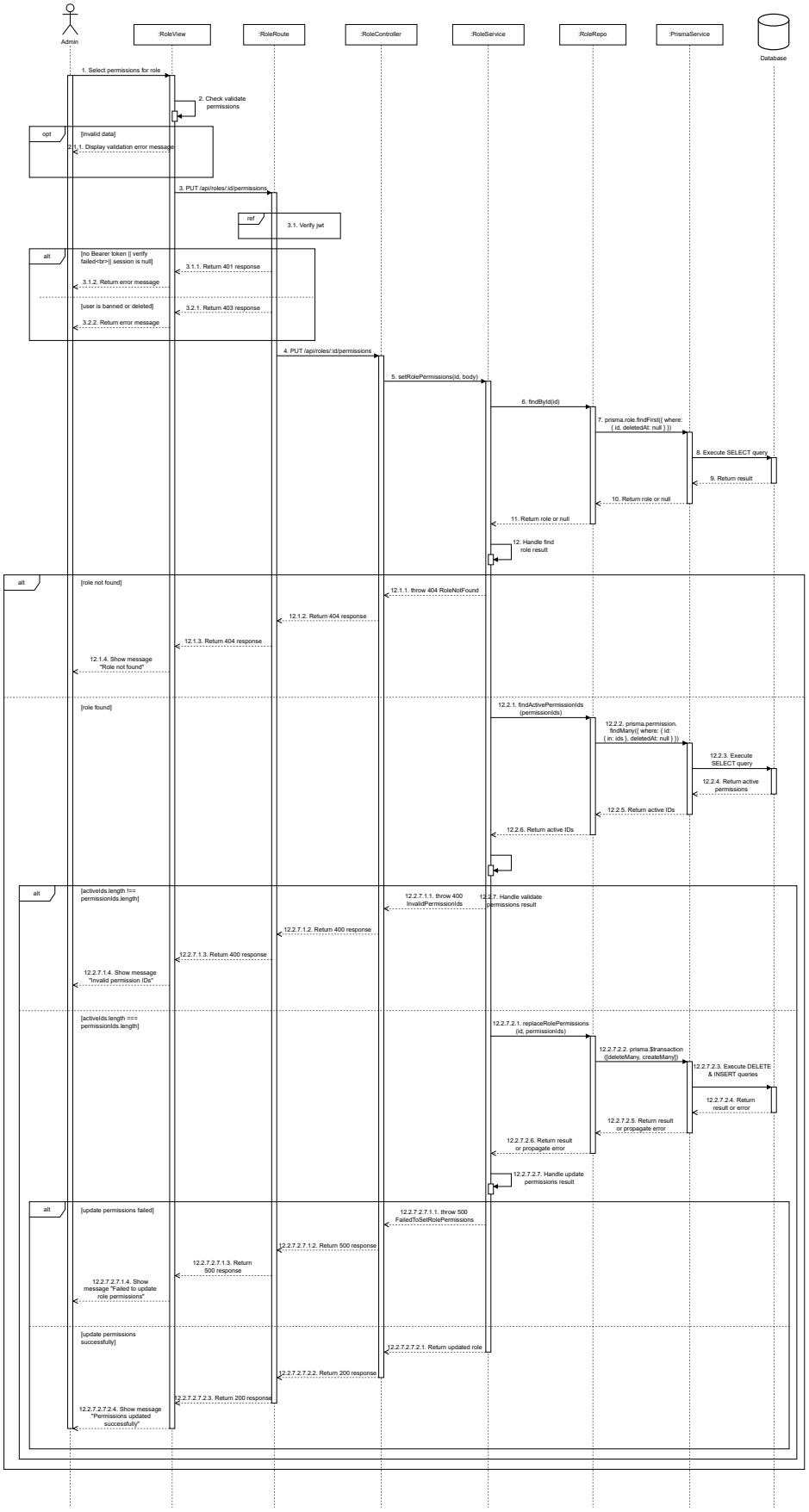

Figure 40: Sequence diagram of UC-06.06 Update Role Permissions

3.4 Job Browsing

Job Browsing allows guests and registered users to search, filter, and view the list of available jobs on the platform, as well as view comprehensive job details.

3.4.0.1 Class Diagram

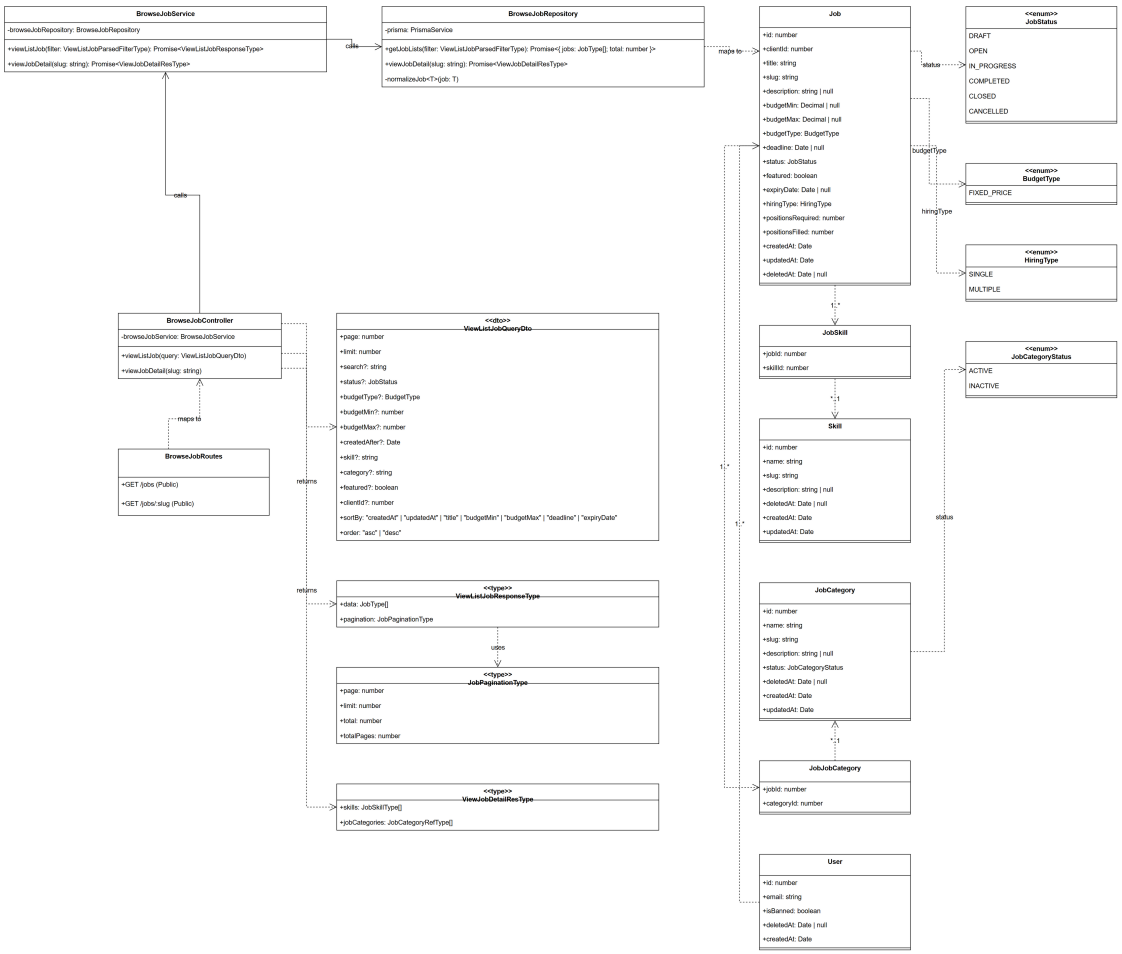

Figure 41: Class diagram of Job Browsing

3.4.0.2 UC-10.01 – View Job List

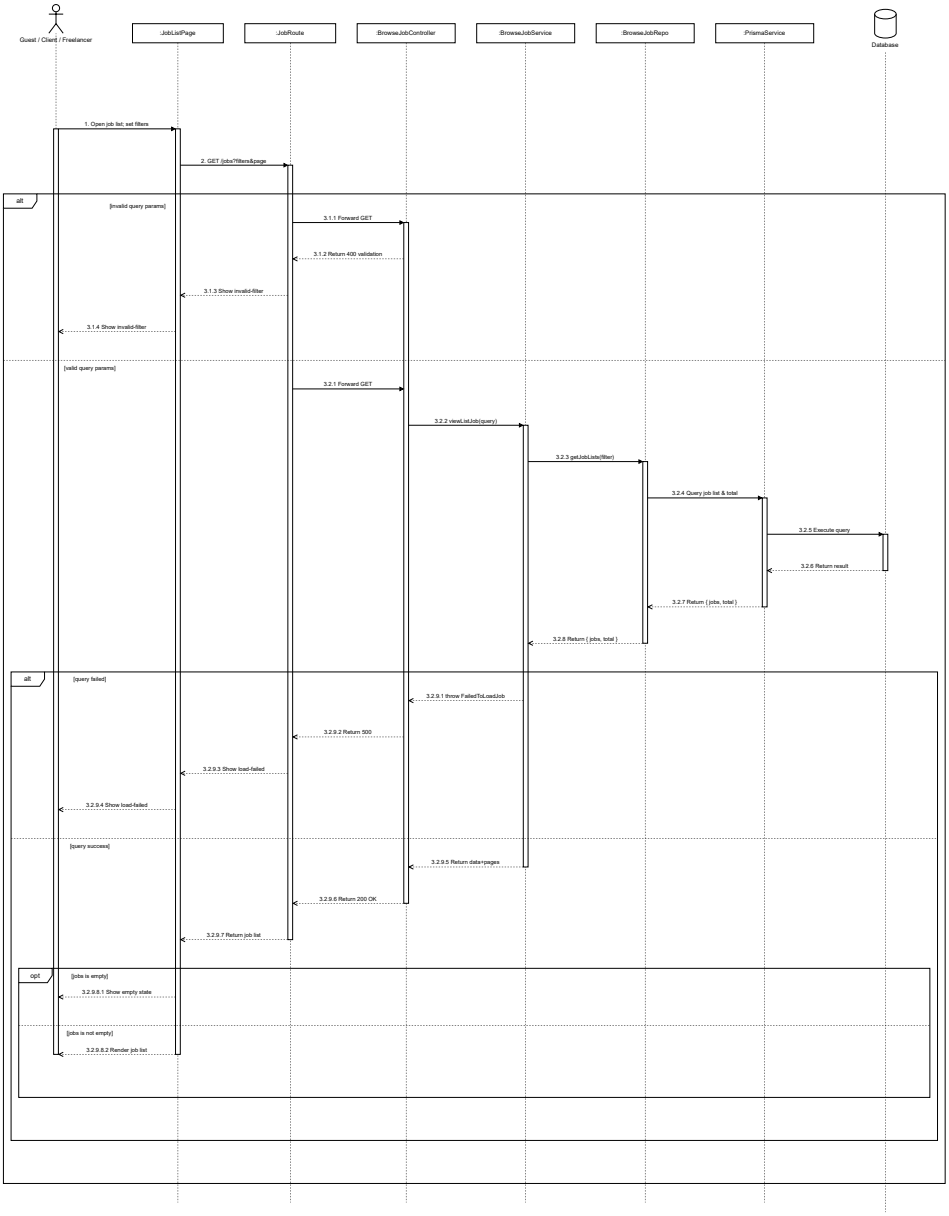

Figure 42: Sequence diagram of UC-10.01 View Job List

3.4.0.3 UC-10.02 – View Job Detail

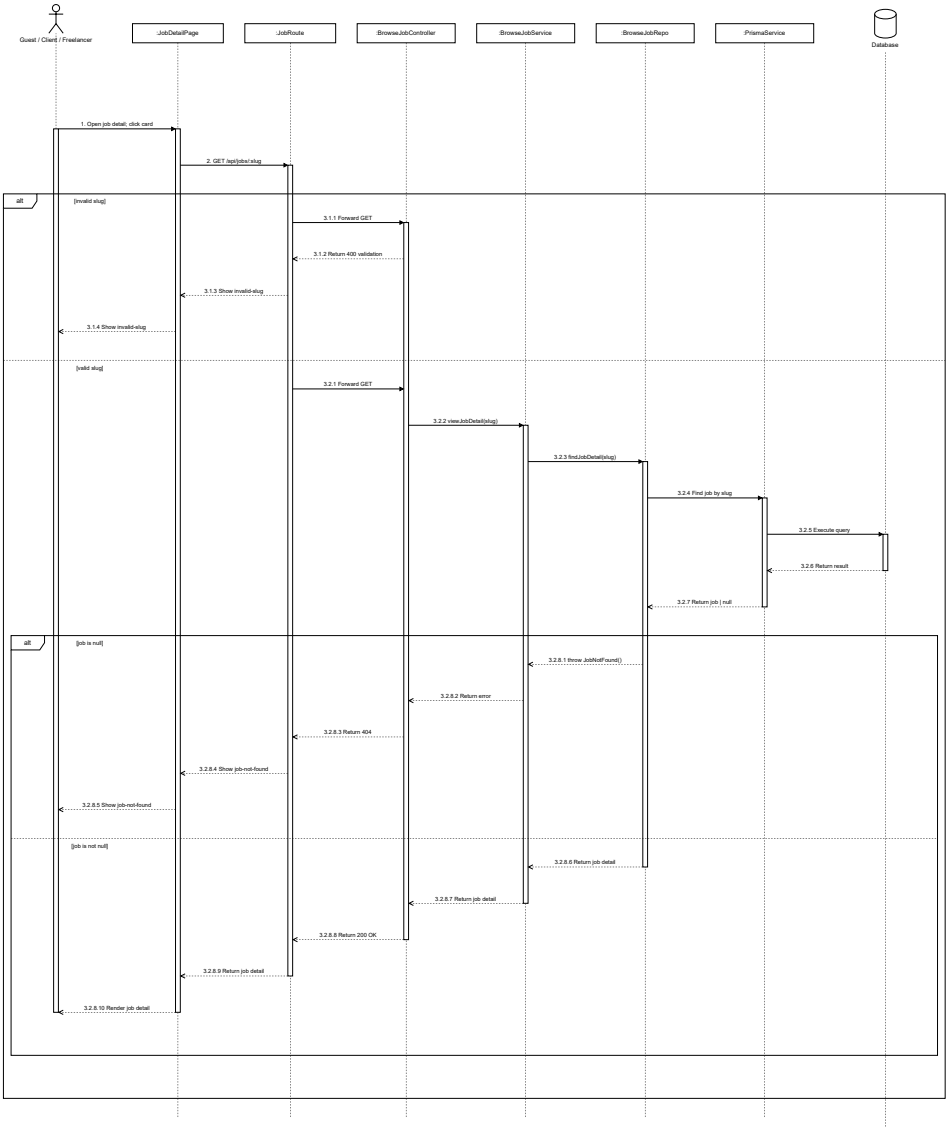

Figure 43: Sequence diagram of UC-10.02 View Job Detail

3.5 Job Management

Job Management enables clients to manage their posted jobs throughout their lifecycle, including creating new jobs, editing job parameters, publishing drafts, closing completed or cancelled jobs, and soft-deleting job postings.

3.5.0.1 Class Diagram

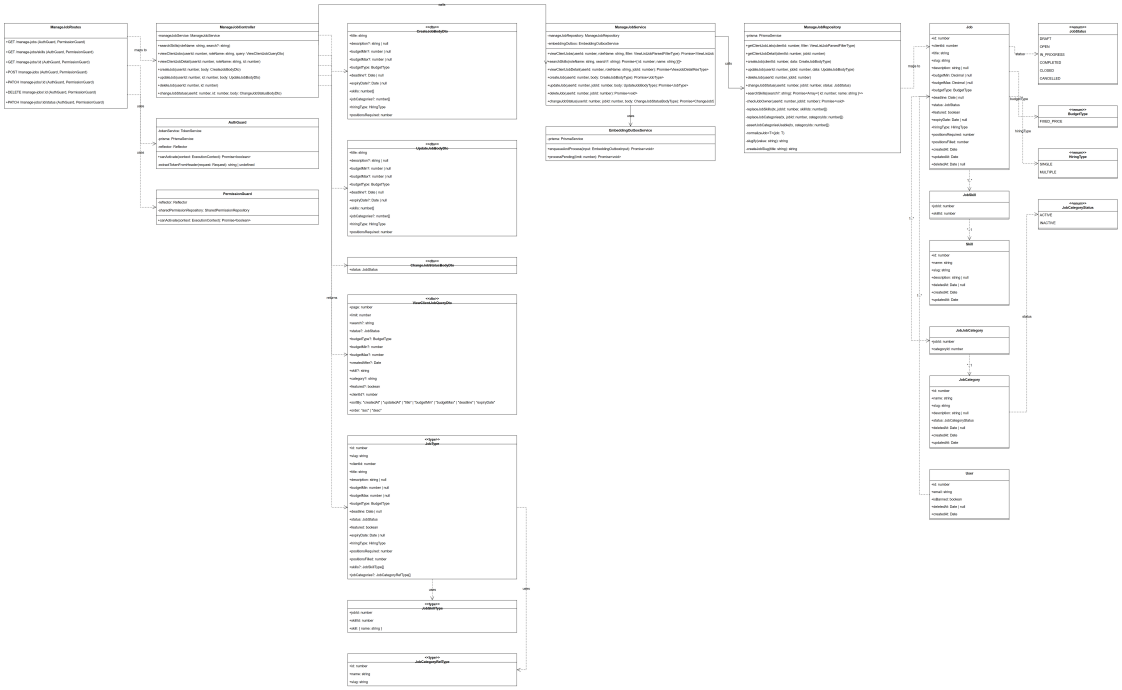

Figure 44: Class diagram of Job Management

3.5.0.2 UC-11.01 – View My Jobs

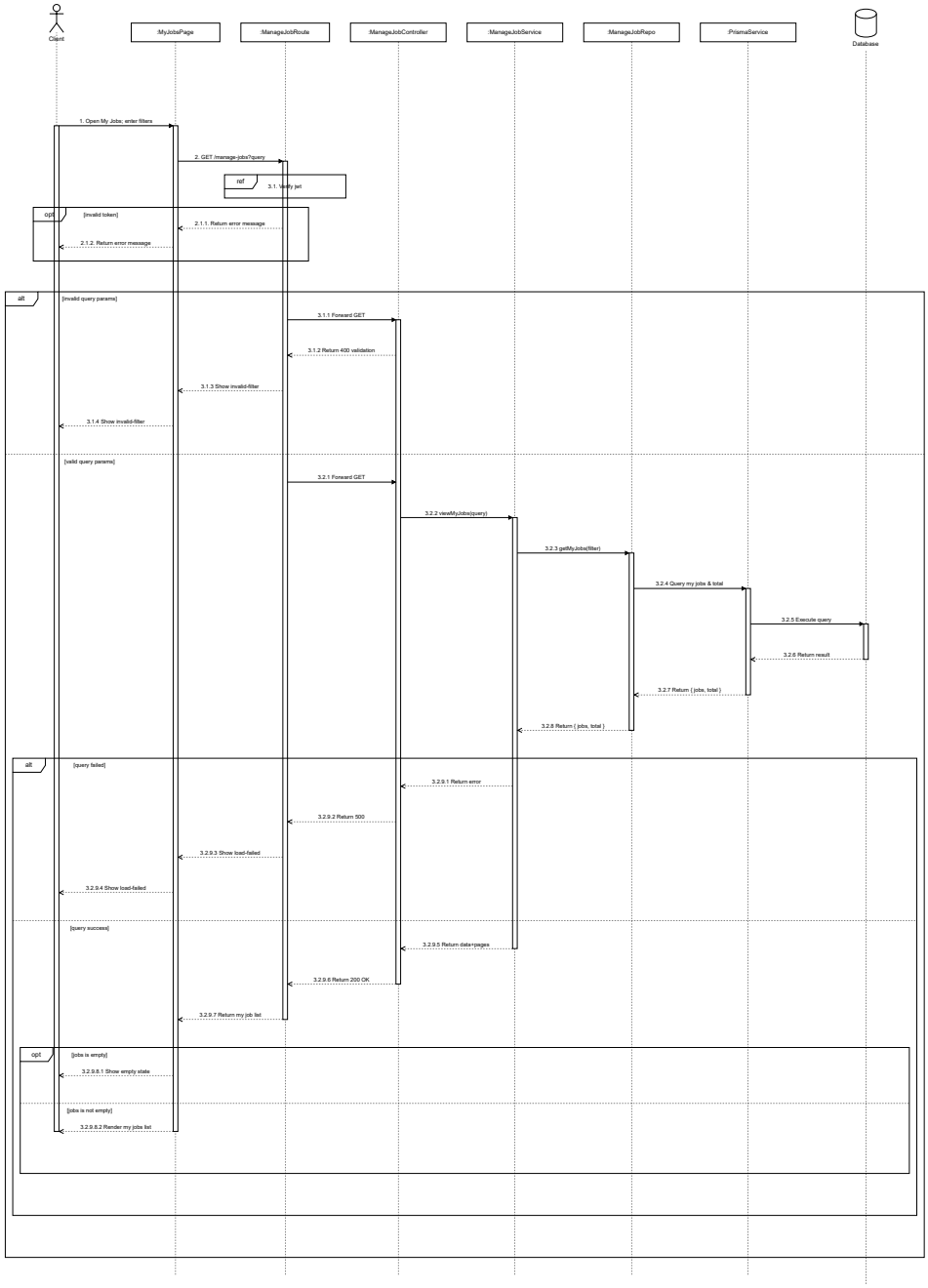

Figure 45: Sequence diagram of UC-11.01 View My Jobs

3.5.0.3 UC-11.02 – Create Job

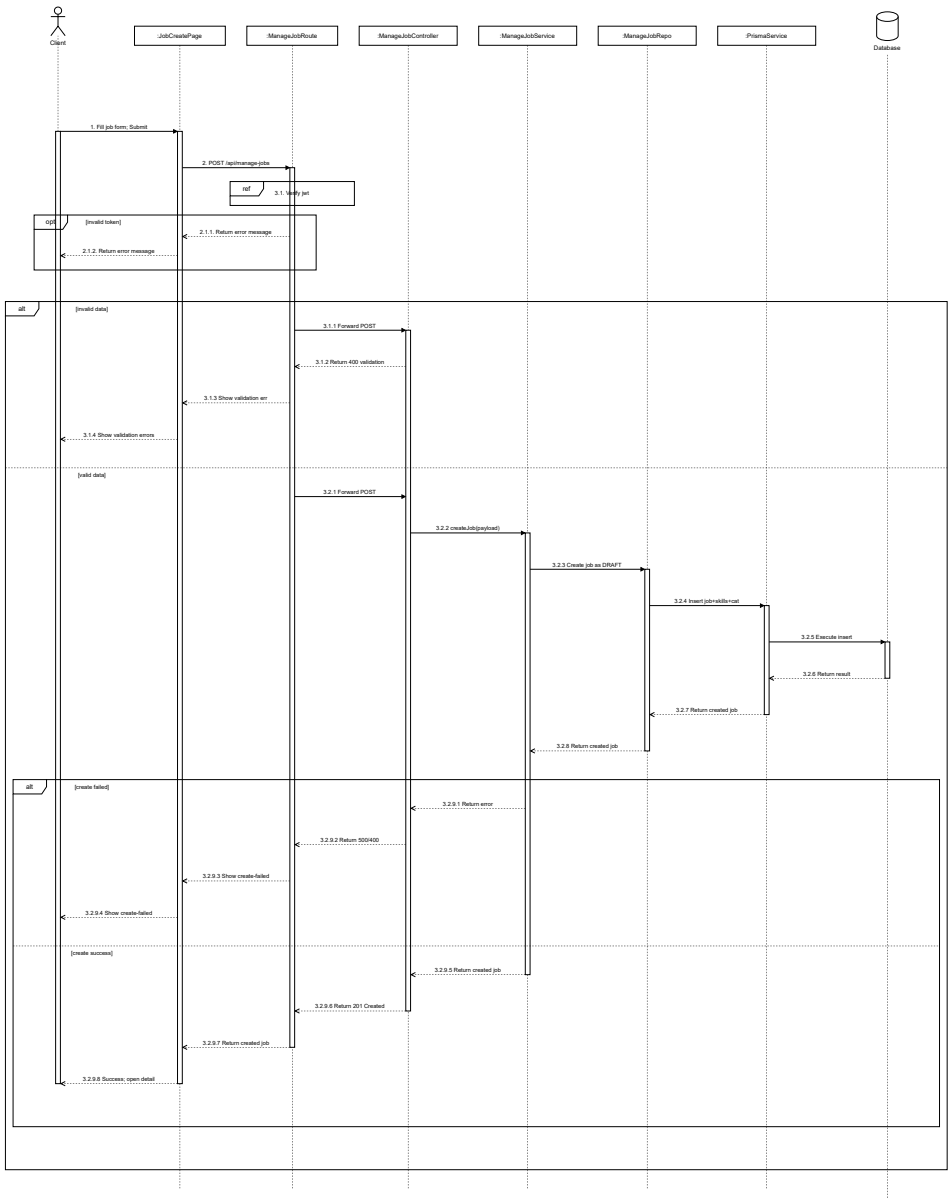

Figure 46: Sequence diagram of UC-11.02 Create Job

3.5.0.4 UC-11.03 – Publish Job

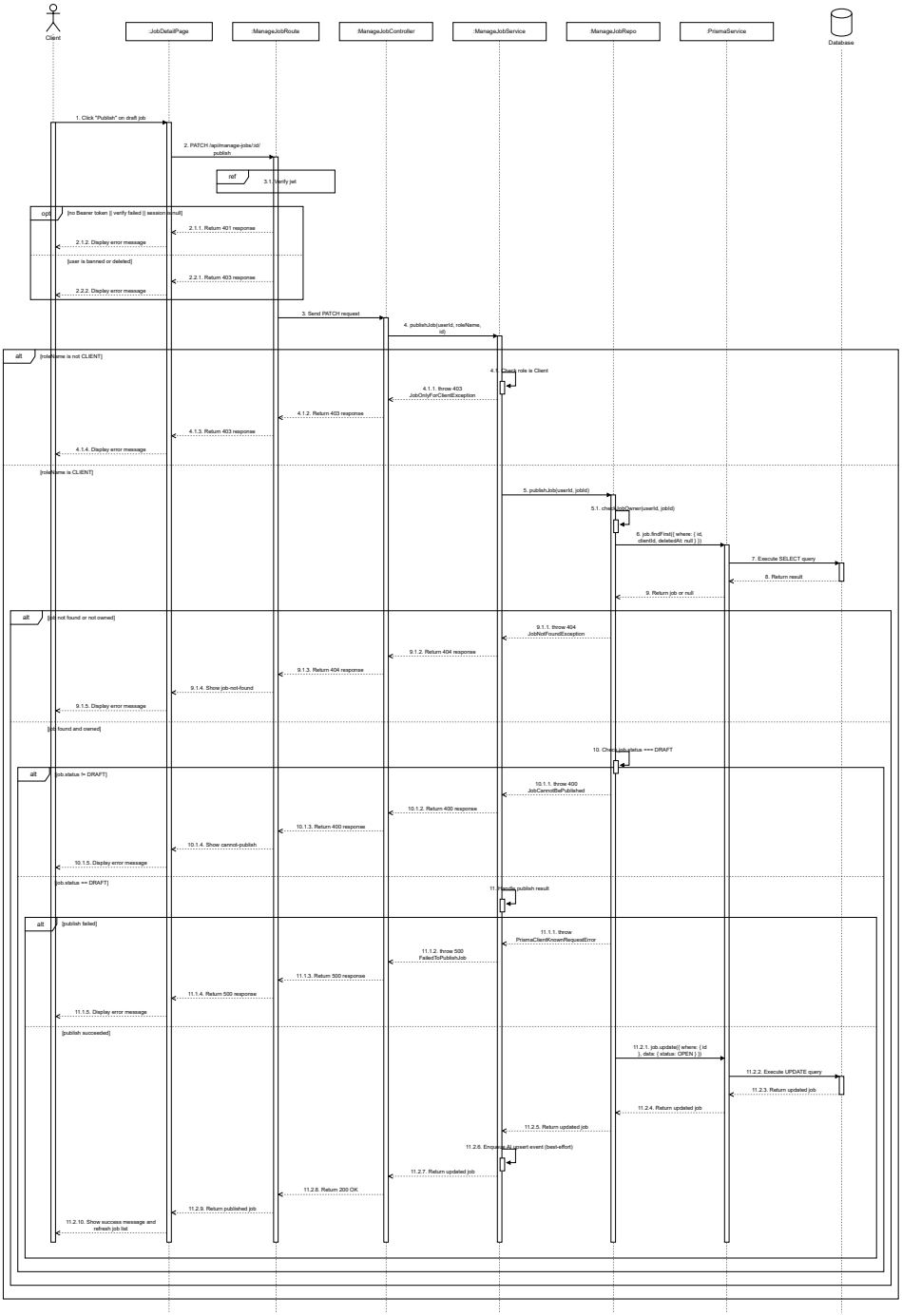

Figure 47: Sequence diagram of UC-11.03 Publish Job

3.5.0.5 UC-11.04 – Update Job

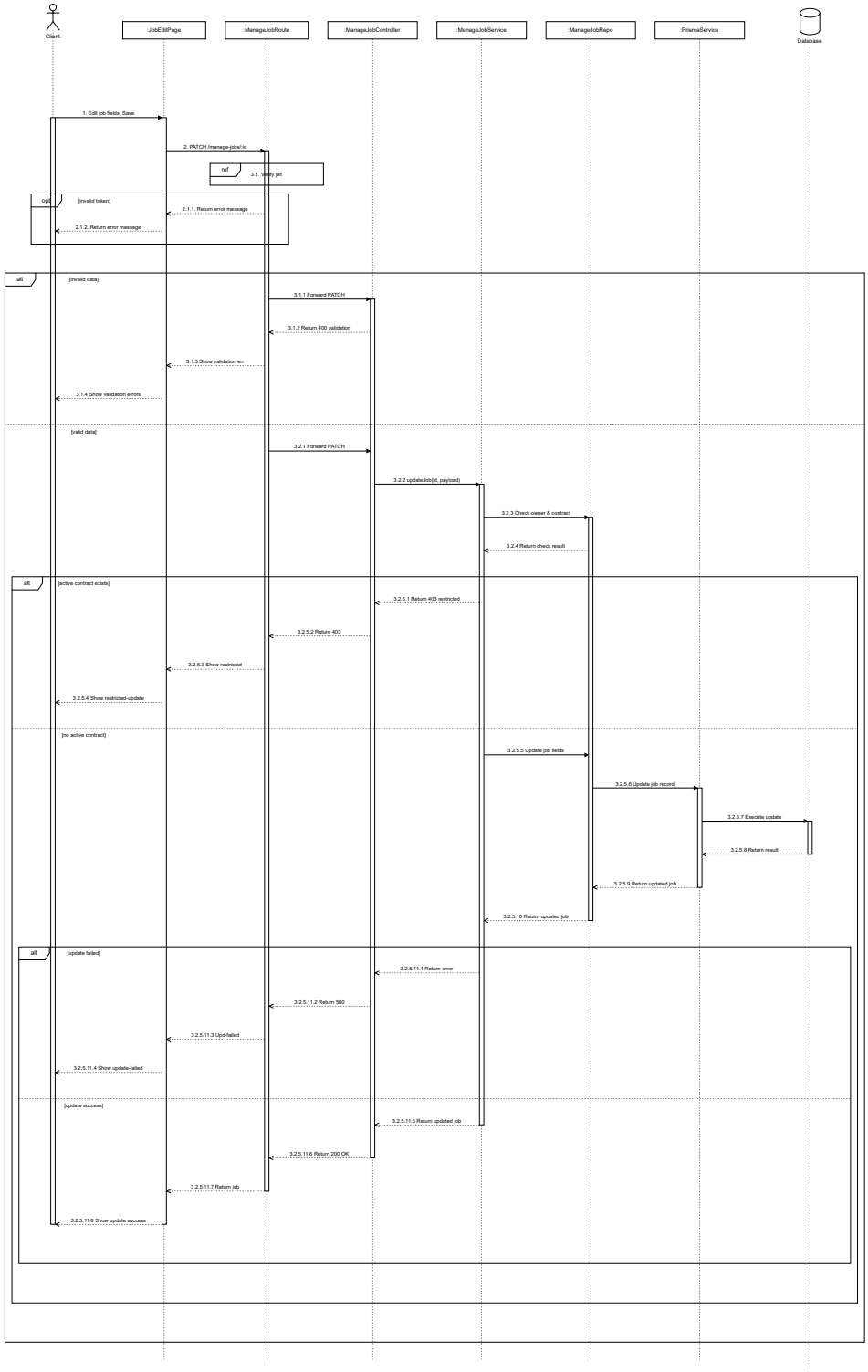

Figure 48: Sequence diagram of UC-11.04 Update Job

3.5.0.6 UC-11.05 – Delete Job

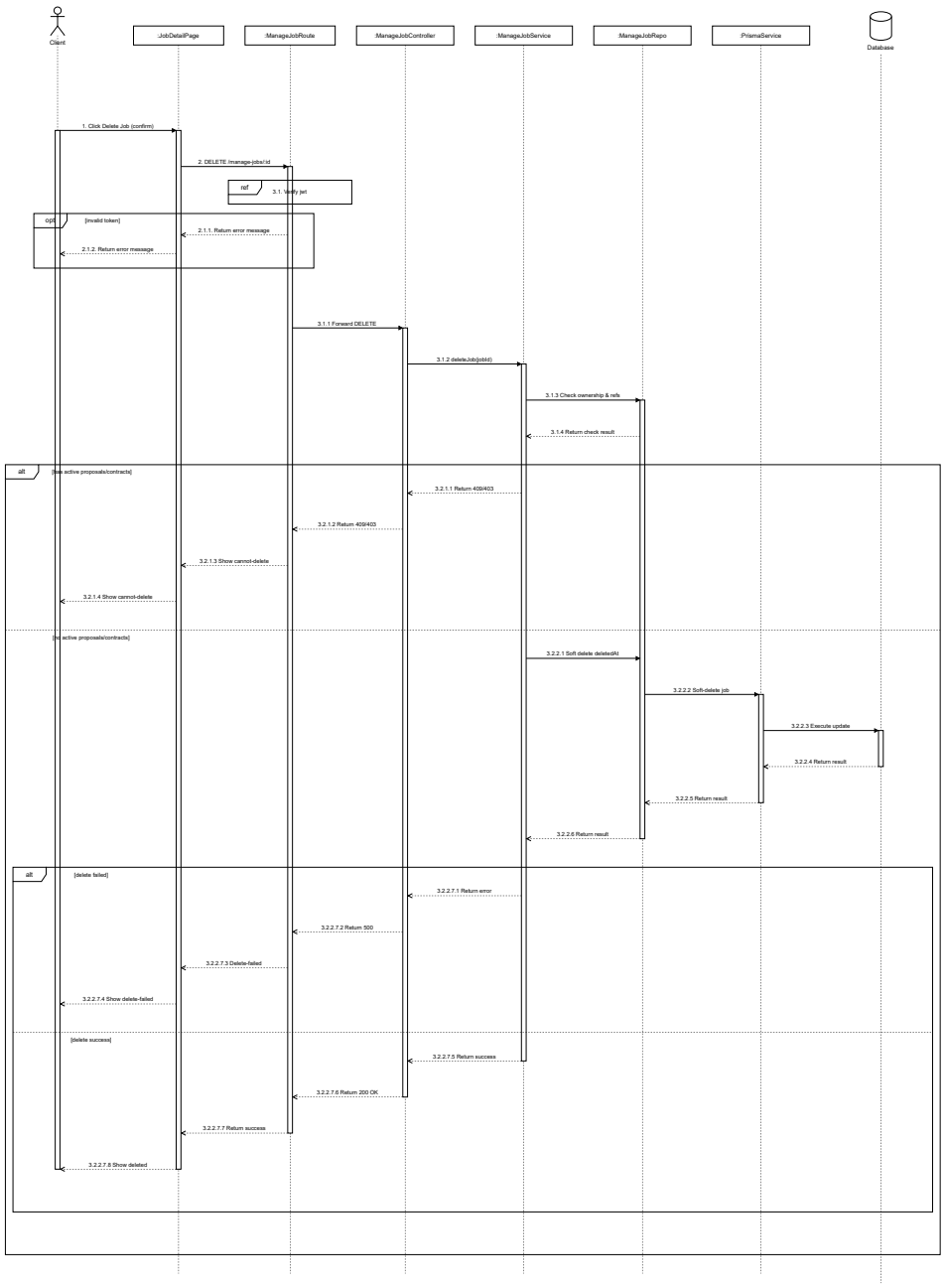

Figure 49: Sequence diagram of UC-11.05 Delete Job

3.5.0.7 UC-11.06 – Change Job Status

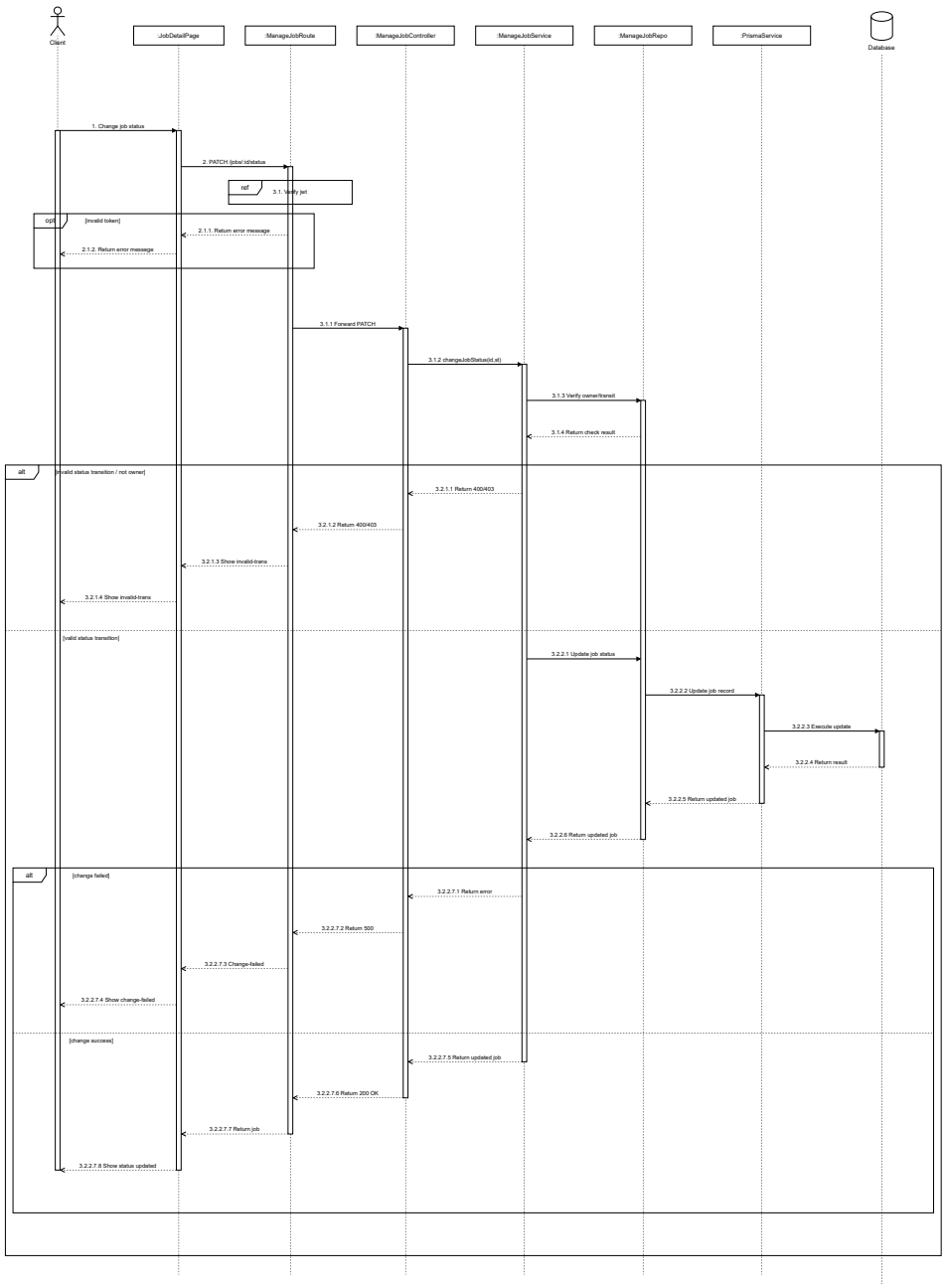

Figure 50: Sequence diagram of UC-11.06 Change Job Status

3.6 Bookmarked Jobs Management

Bookmarked Jobs Management allows freelancers to save interesting job postings for later review, inspect their list of bookmarked jobs, and remove bookmarks when no longer needed.

3.6.0.1 Class Diagram

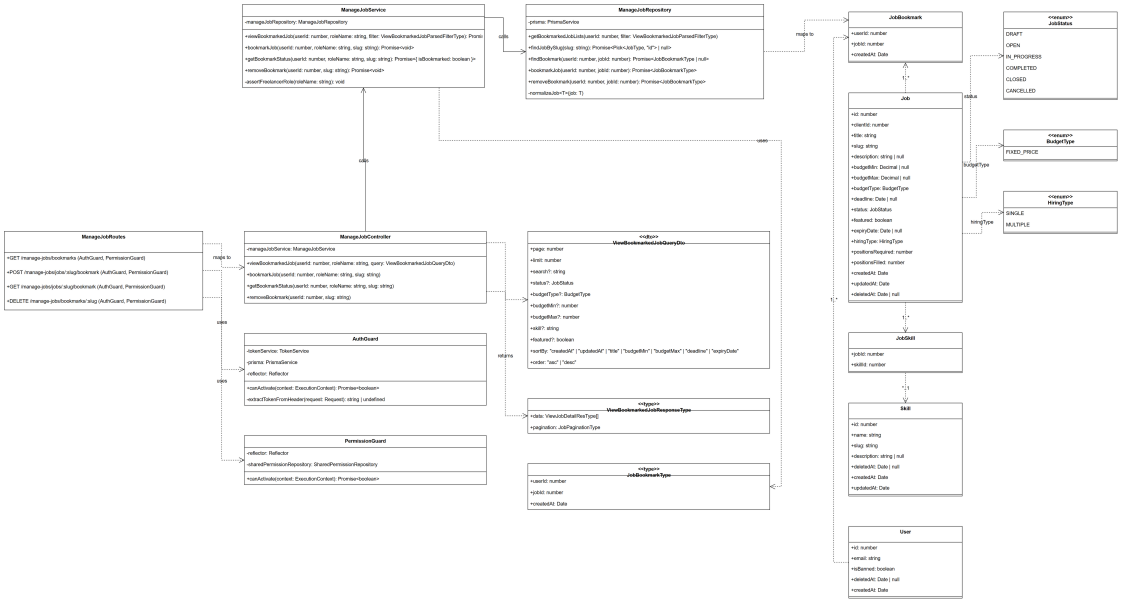

Figure 51: Class diagram of Bookmarked Jobs Management

3.6.0.2 UC-12.01 – View Bookmarked Job List

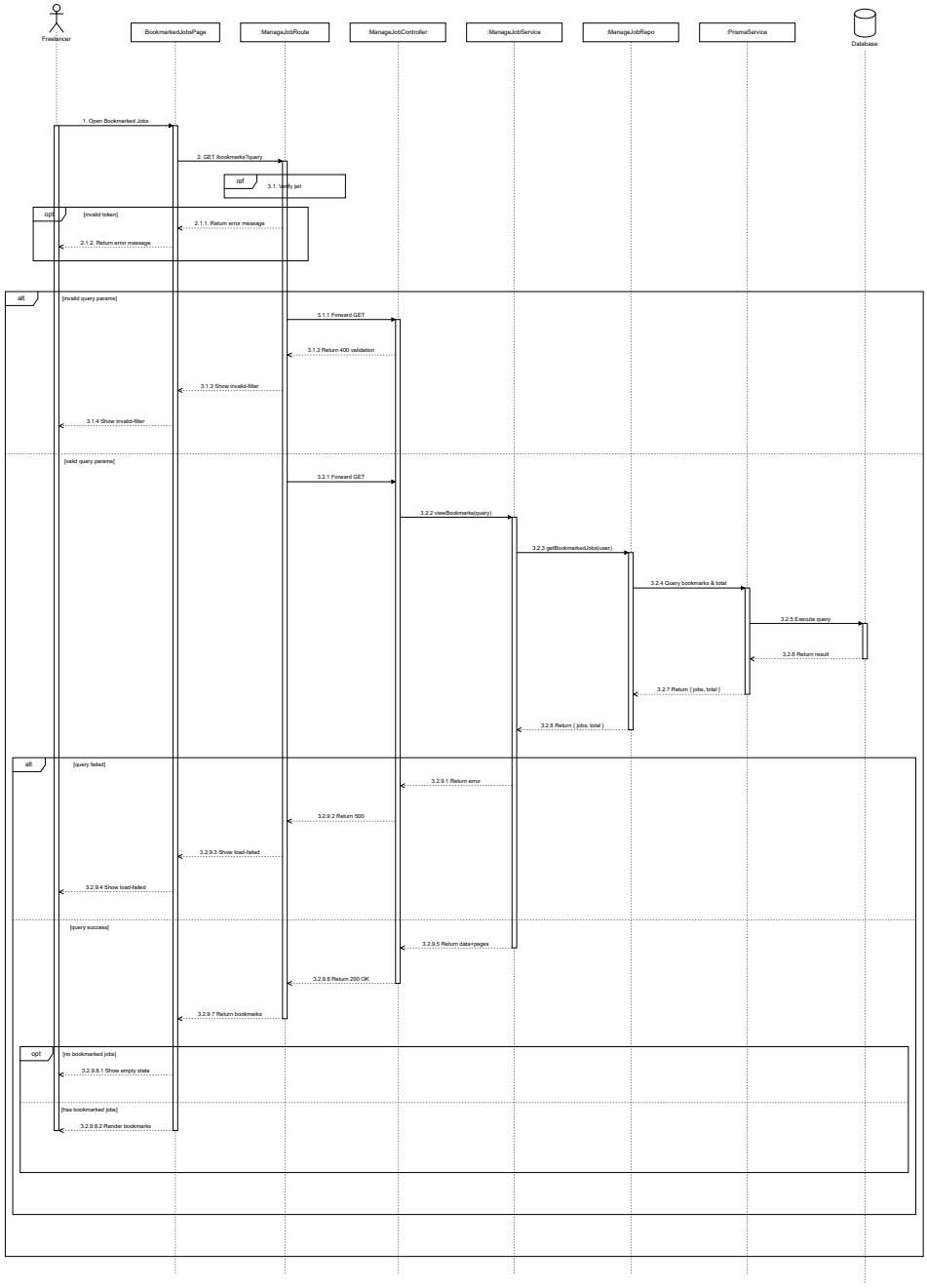

Figure 52: Sequence diagram of UC-12.01 View Bookmarked Job List

3.6.0.3 UC-12.02 – Bookmark Job

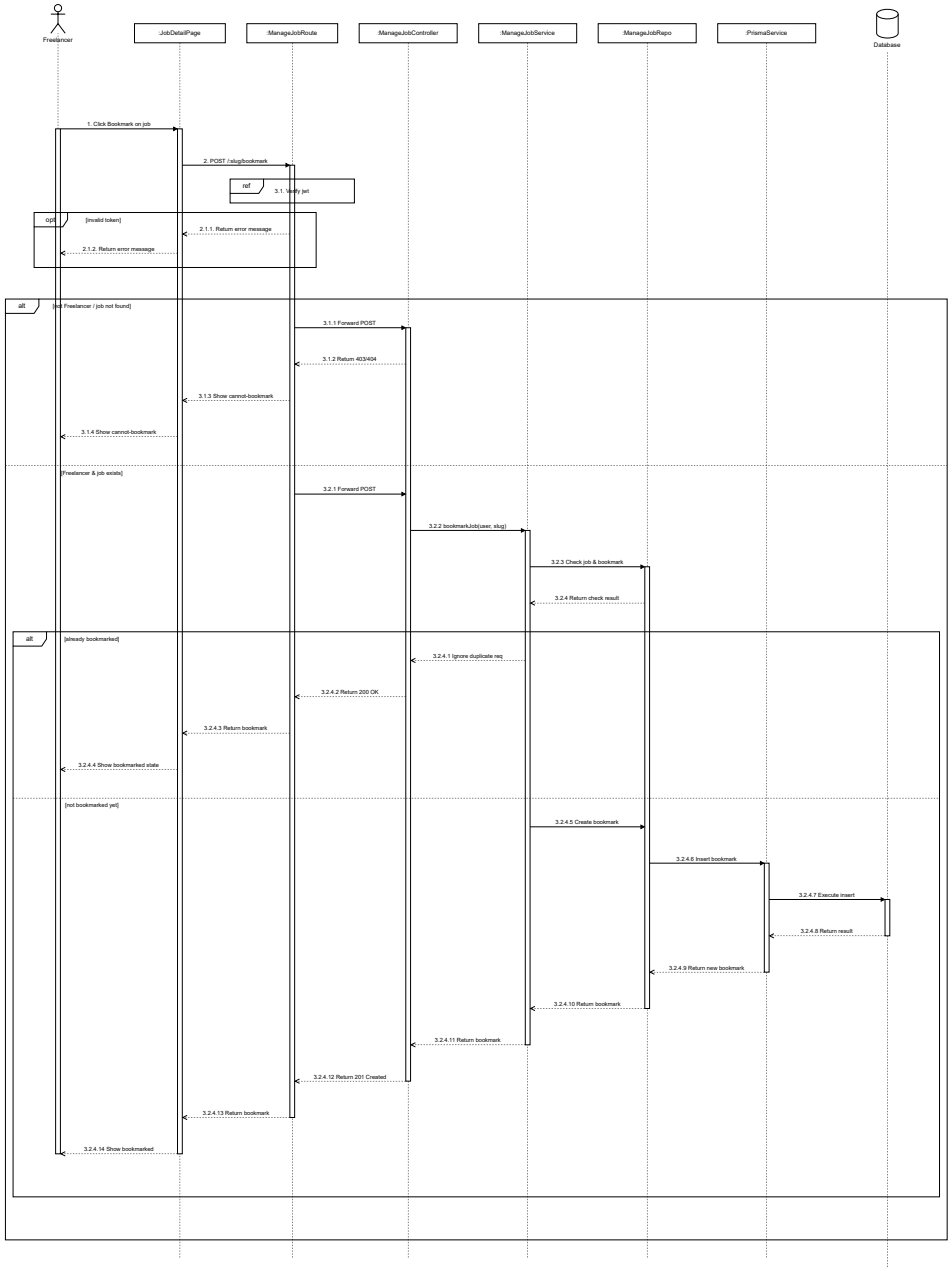

Figure 53: Sequence diagram of UC-12.02 Bookmark Job

3.6.0.4 UC-12.03 – Remove Bookmark

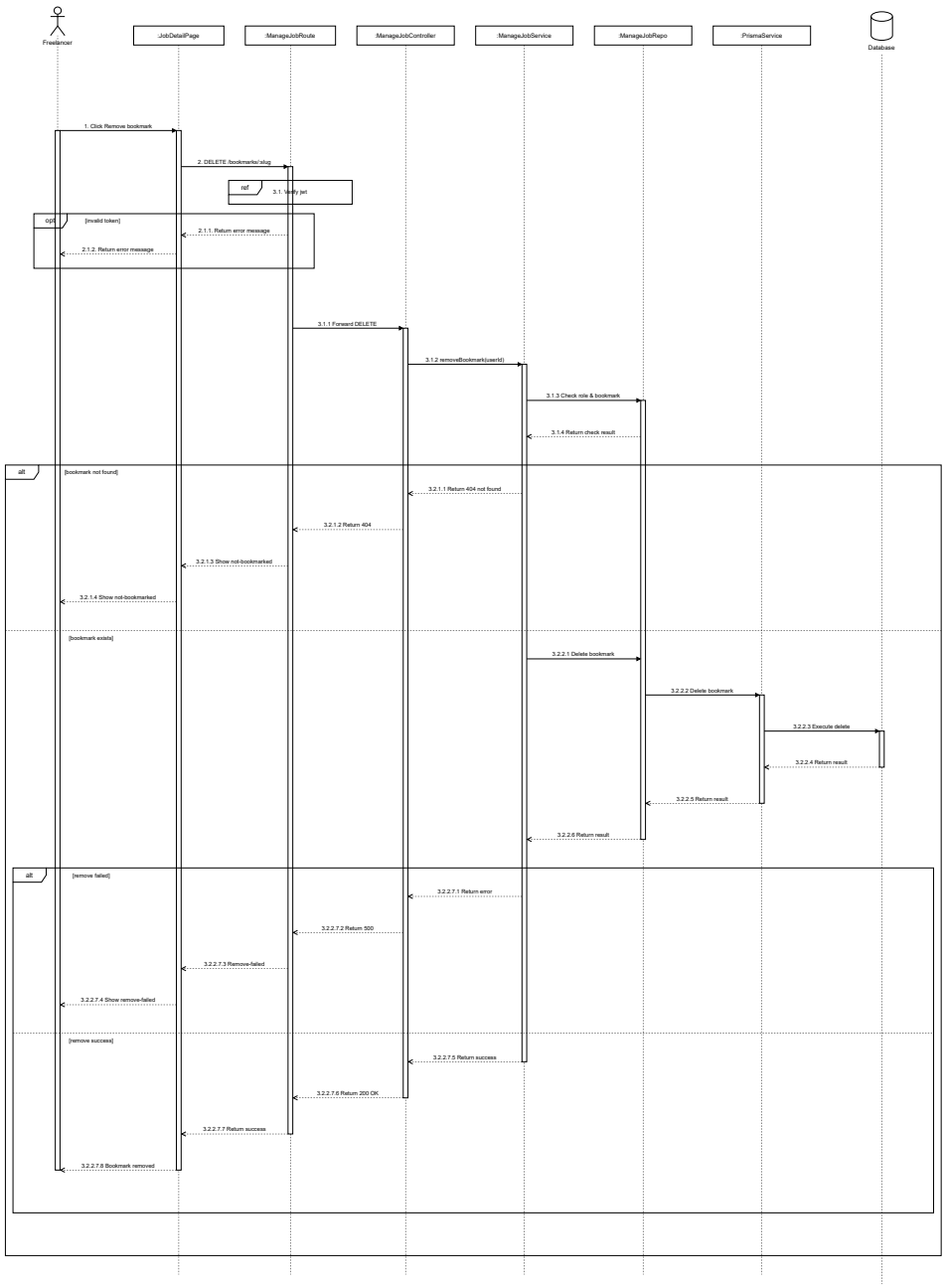

Figure 54: Sequence diagram of UC-12.03 Remove Bookmark

3.7 Forum Management

Forum Management covers the UC-14 community forum: browsing forum categories and posts, publishing and maintaining posts and comments, toggling likes on posts and comments, and reporting content that breaks the forum rules.

3.7.0.1 Class Diagram

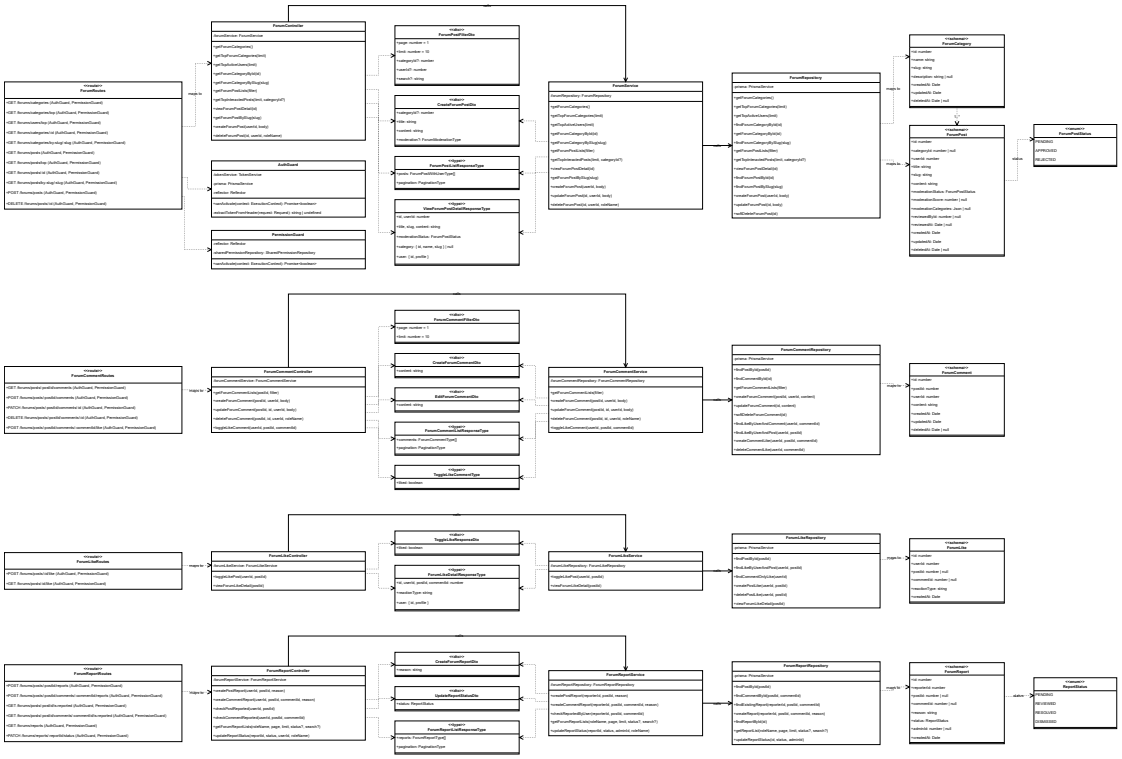

Figure 55: Class diagram of Forum Management

3.7.0.2 UC-14.01 – View Forum Category List

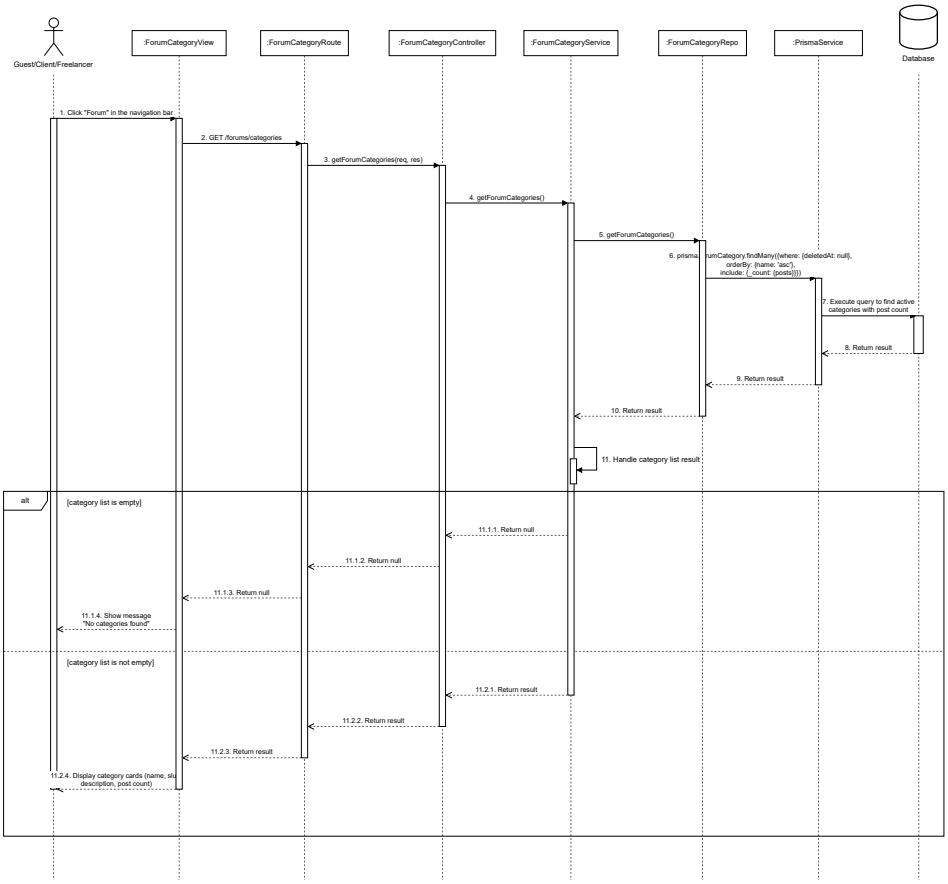

Figure 56: Sequence diagram of UC-14.01 View Forum Category List

3.7.0.3 UC-14.02 – View Forum Category Detail

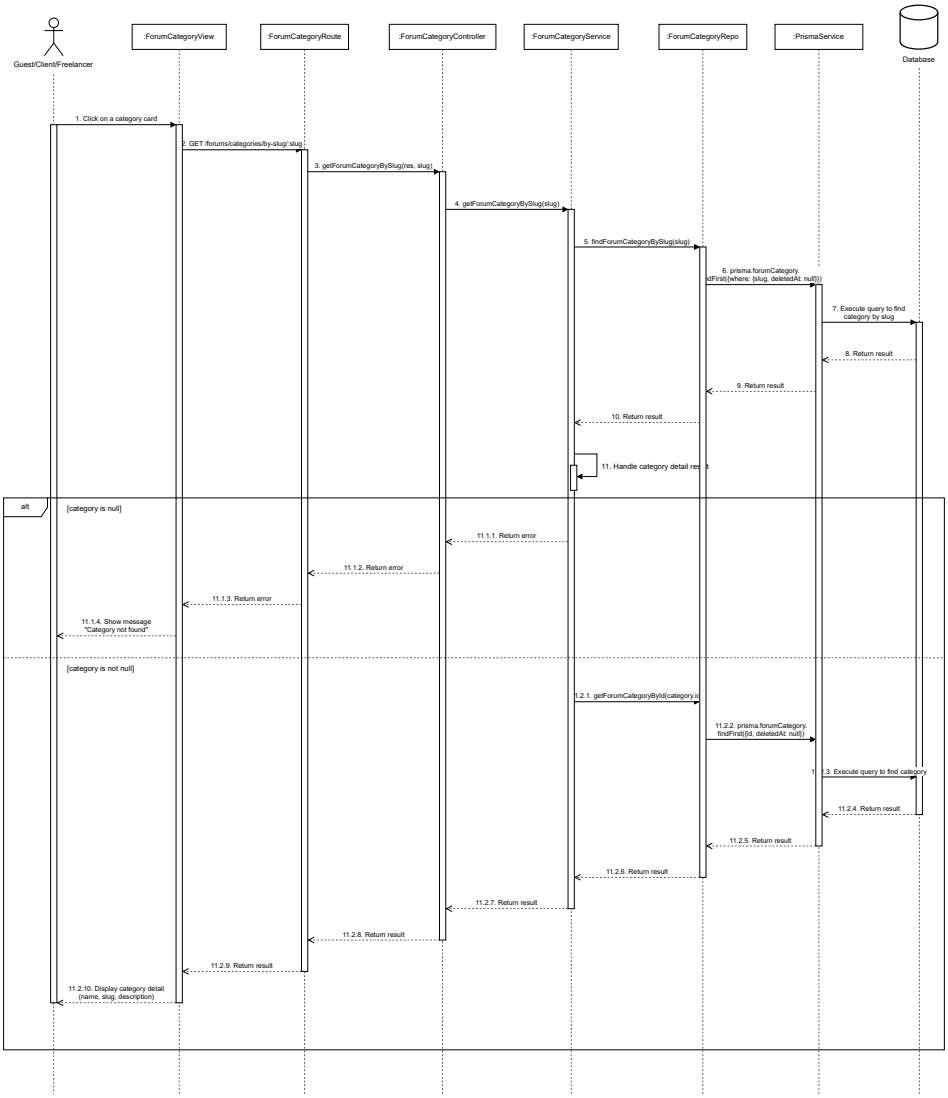

Figure 57: Sequence diagram of UC-14.02 View Forum Category Detail

3.7.0.4 UC-14.03 – View Post List

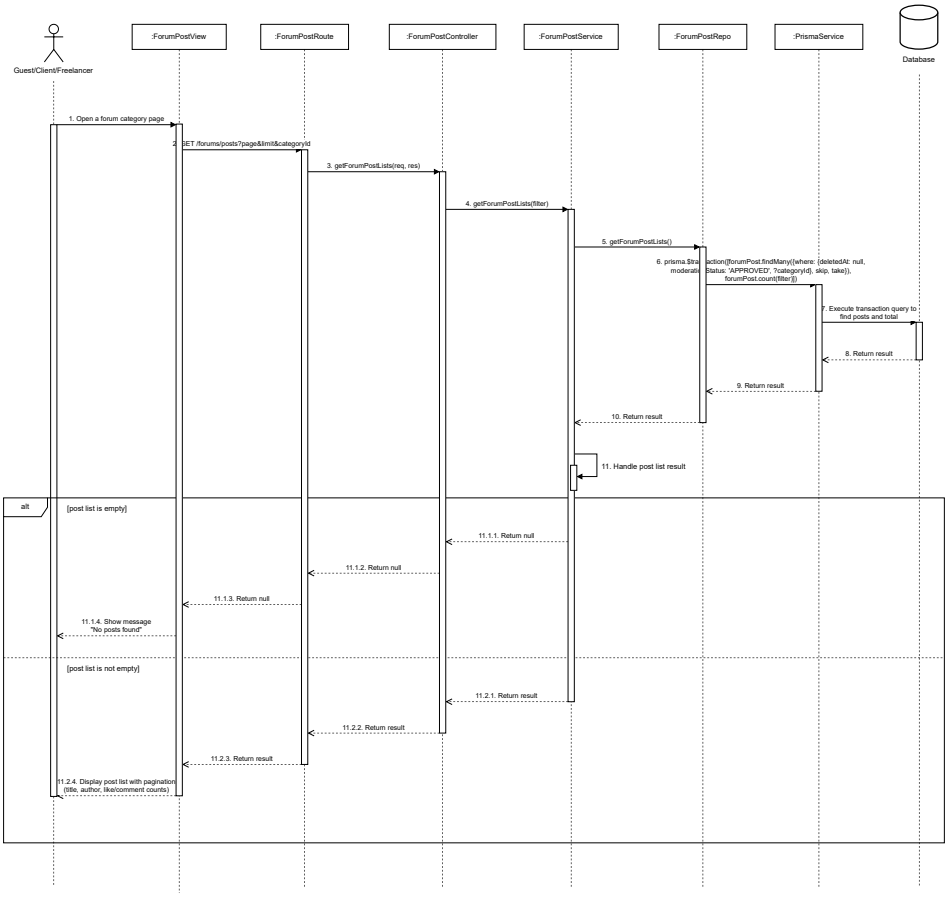

Figure 58: Sequence diagram of UC-14.03 View Post List

3.7.0.5 UC-14.04 – View Post Detail

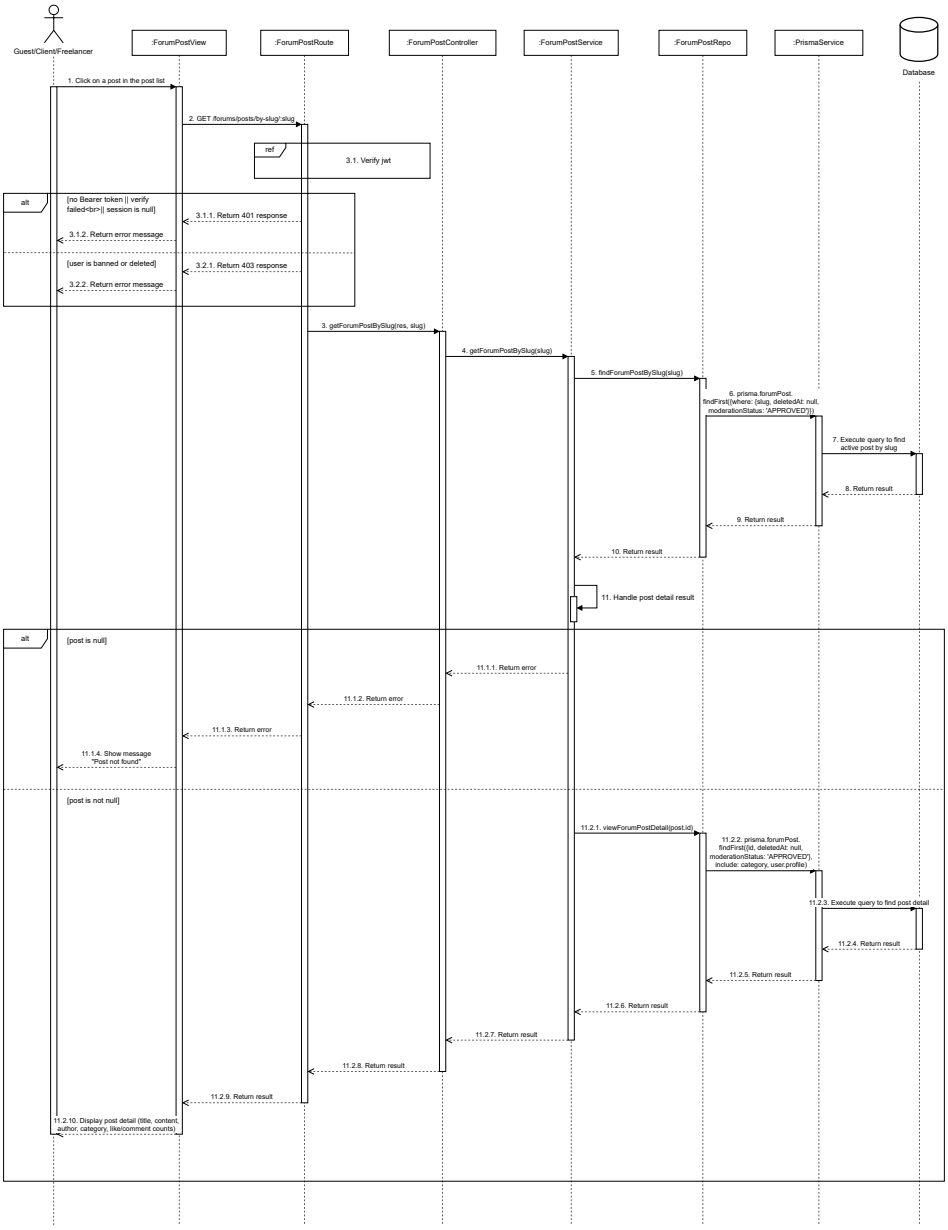

Figure 59: Sequence diagram of UC-14.04 View Post Detail

3.7.0.6 UC-14.05 – Create Post

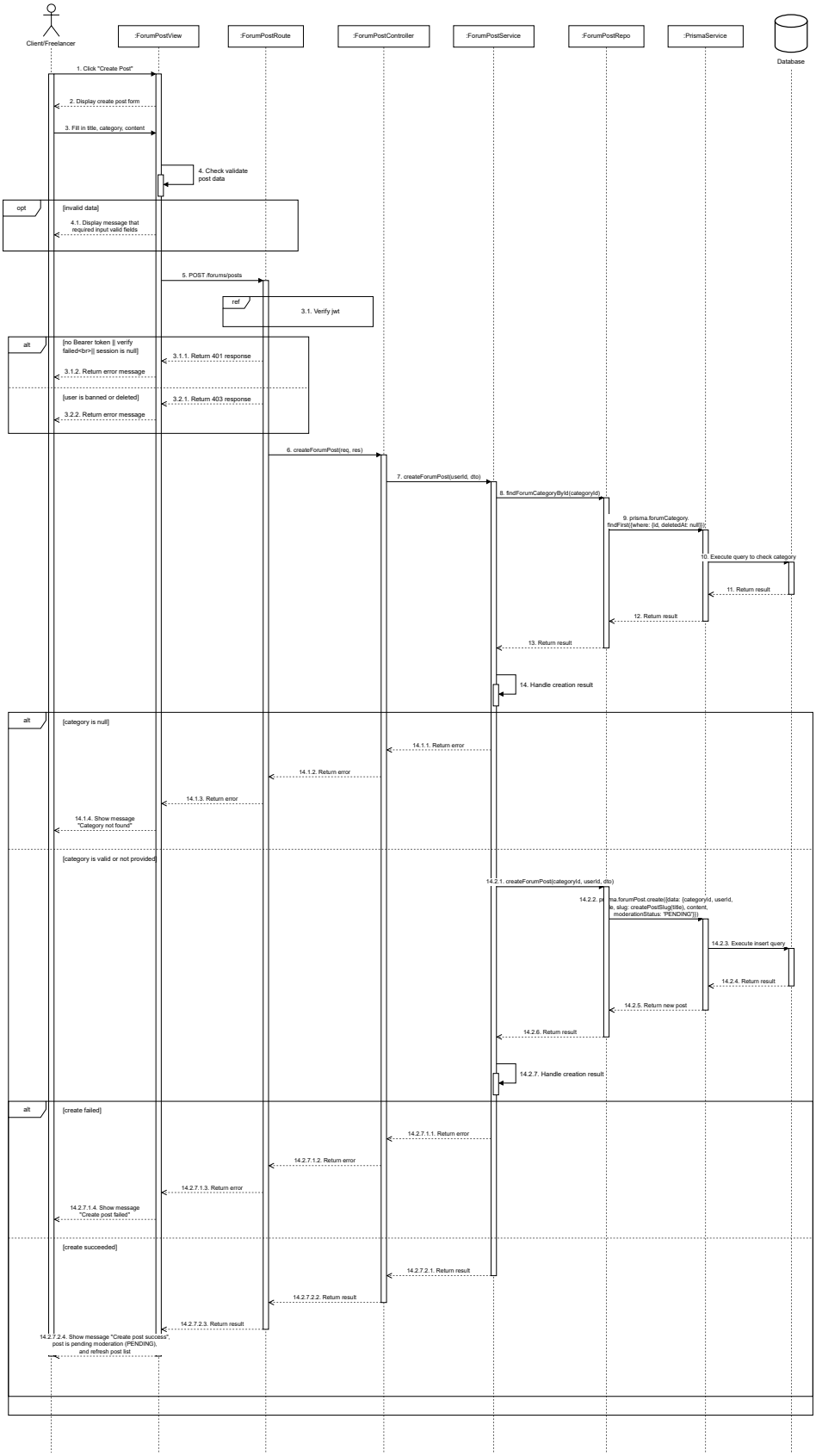

Figure 60: Sequence diagram of UC-14.05 Create Post

3.7.0.7 UC-14.06 – Update Post

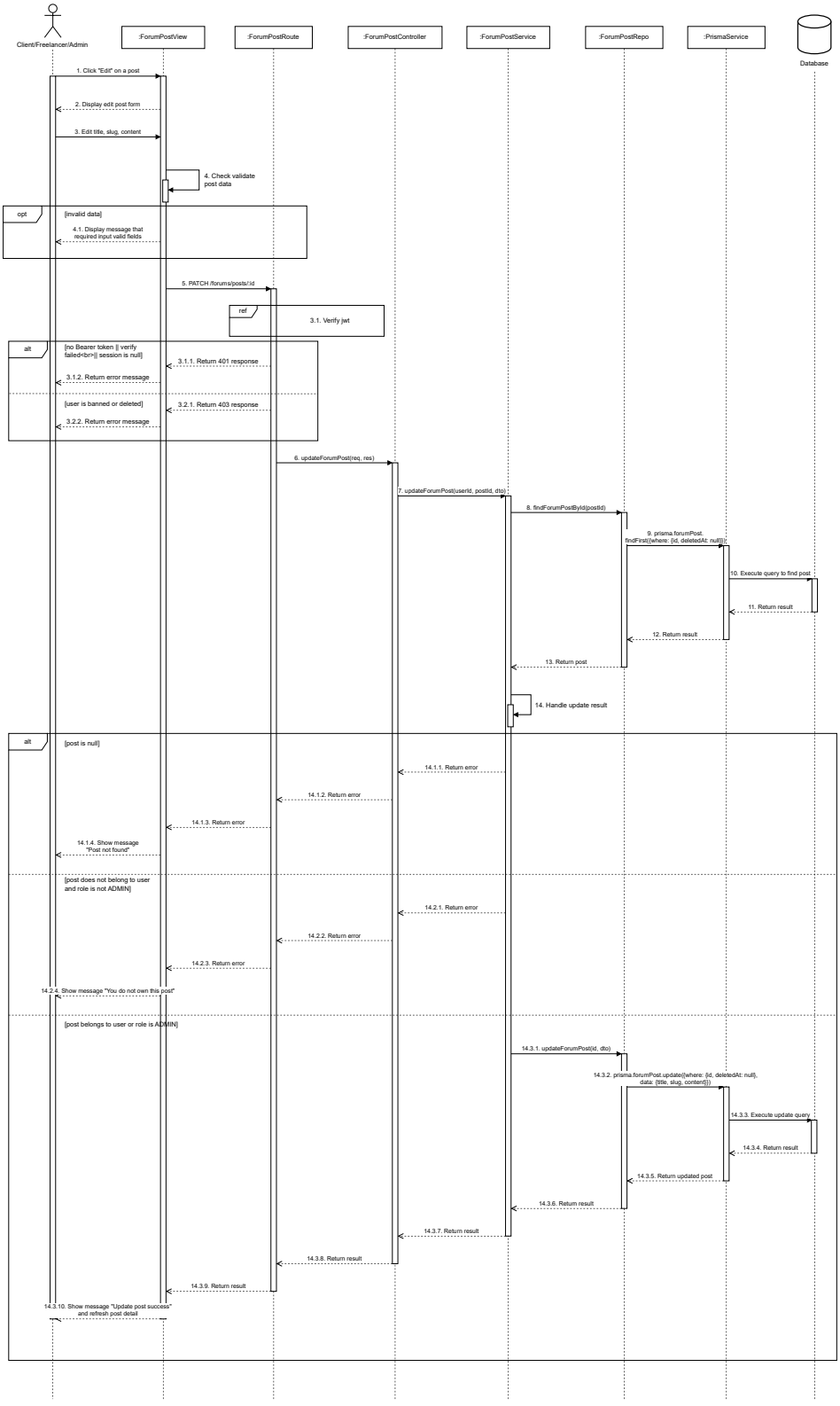

Figure 61: Sequence diagram of UC-14.06 Update Post

3.7.0.8 UC-14.07 – Delete Post

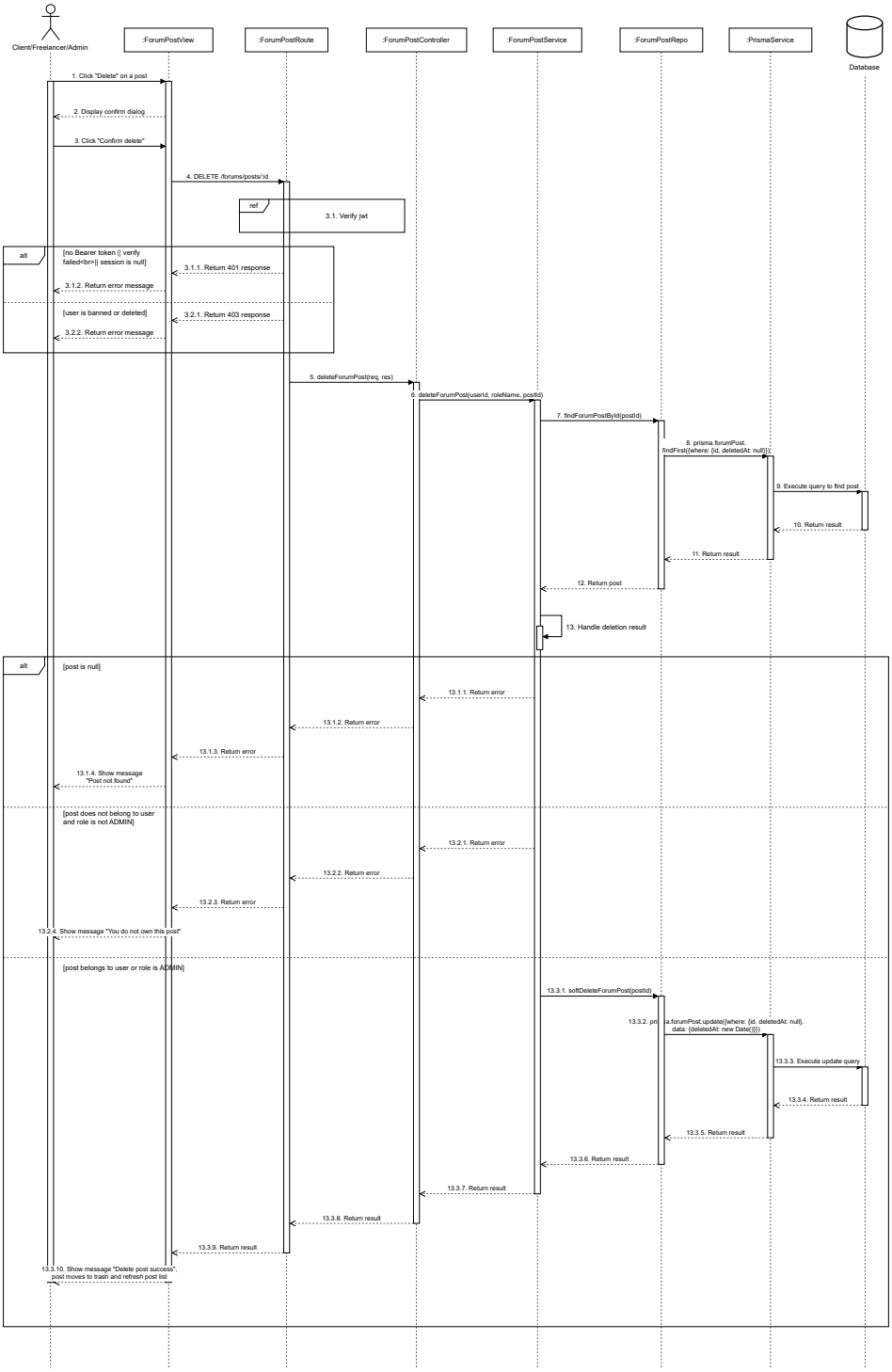

Figure 62: Sequence diagram of UC-14.07 Delete Post

3.7.0.9 UC-14.08 – Like Post

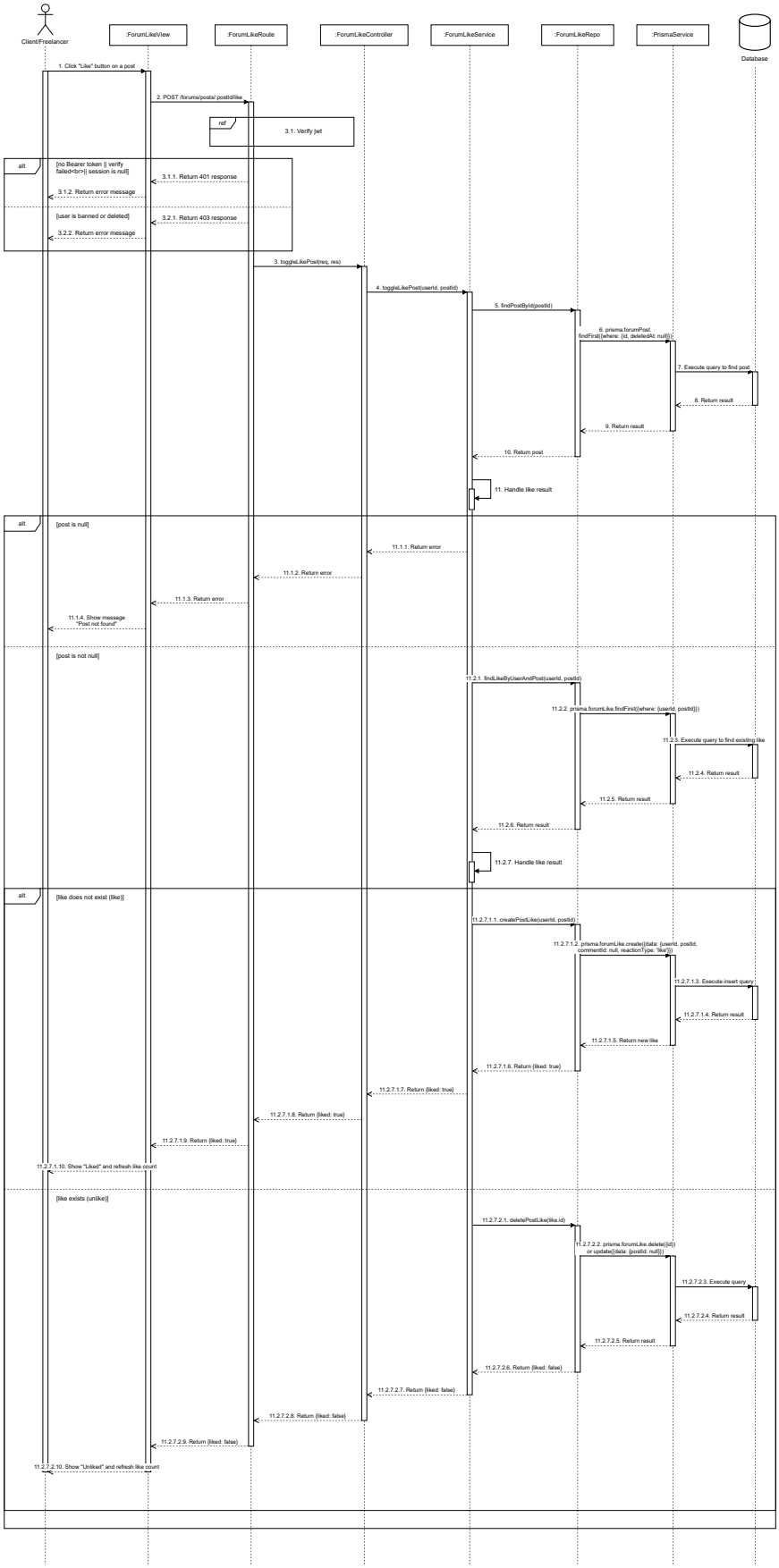

Figure 63: Sequence diagram of UC-14.08 Like Post

3.7.0.10 UC-14.09 – Unlike Post

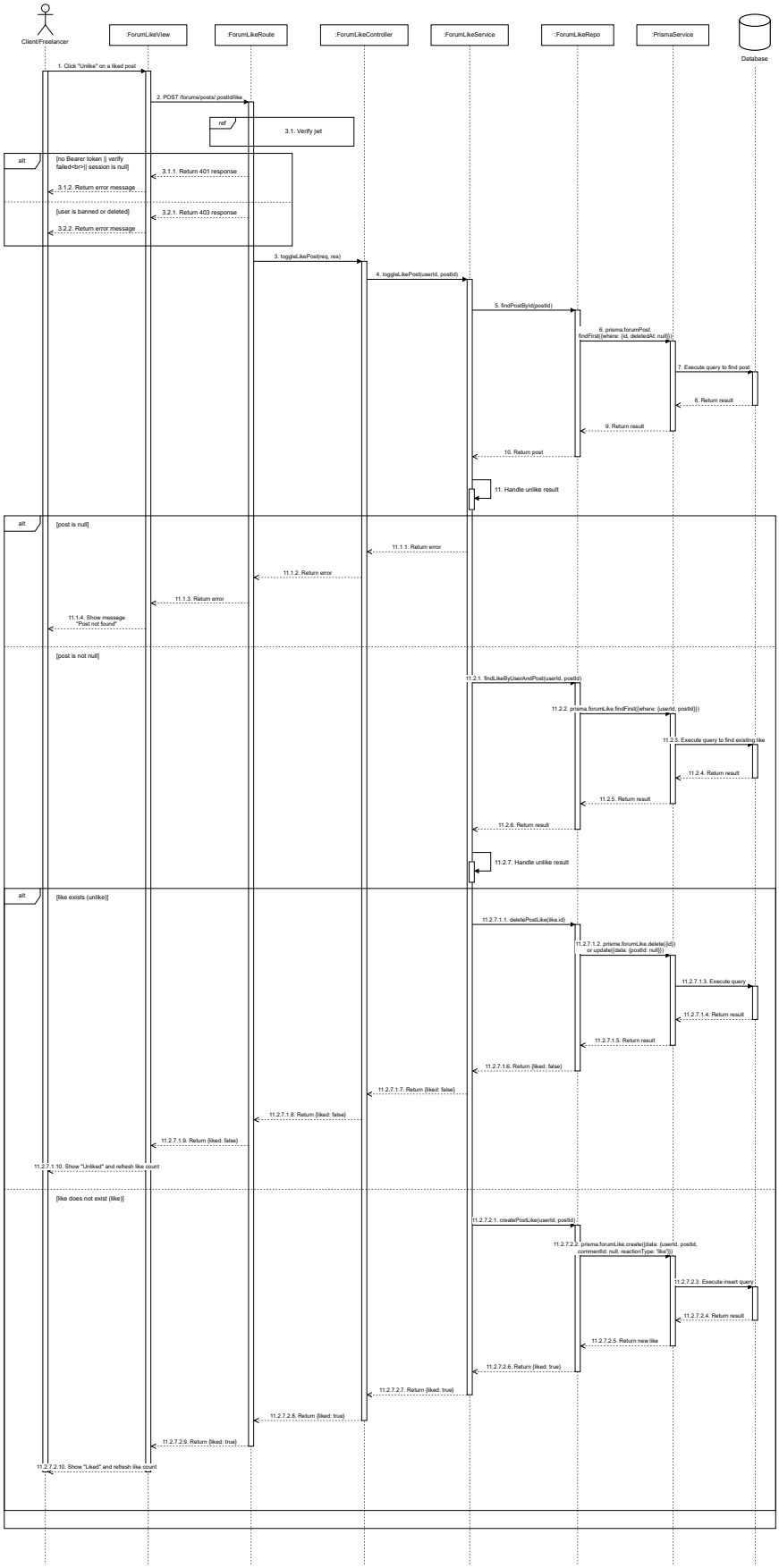

Figure 64: Sequence diagram of UC-14.09 Unlike Post

3.7.0.11 UC-14.10 – View Post Likes

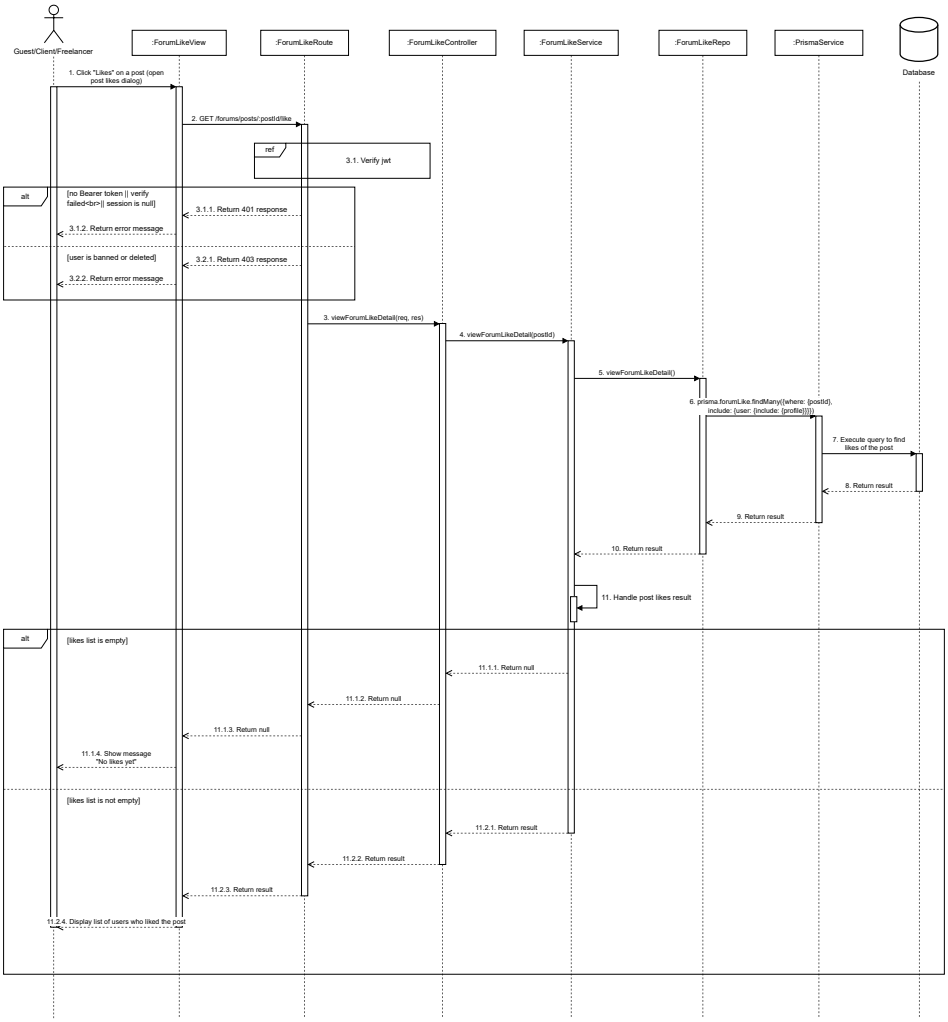

Figure 65: Sequence diagram of UC-14.10 View Post Likes

3.7.0.12 UC-14.11 – View Post Comments

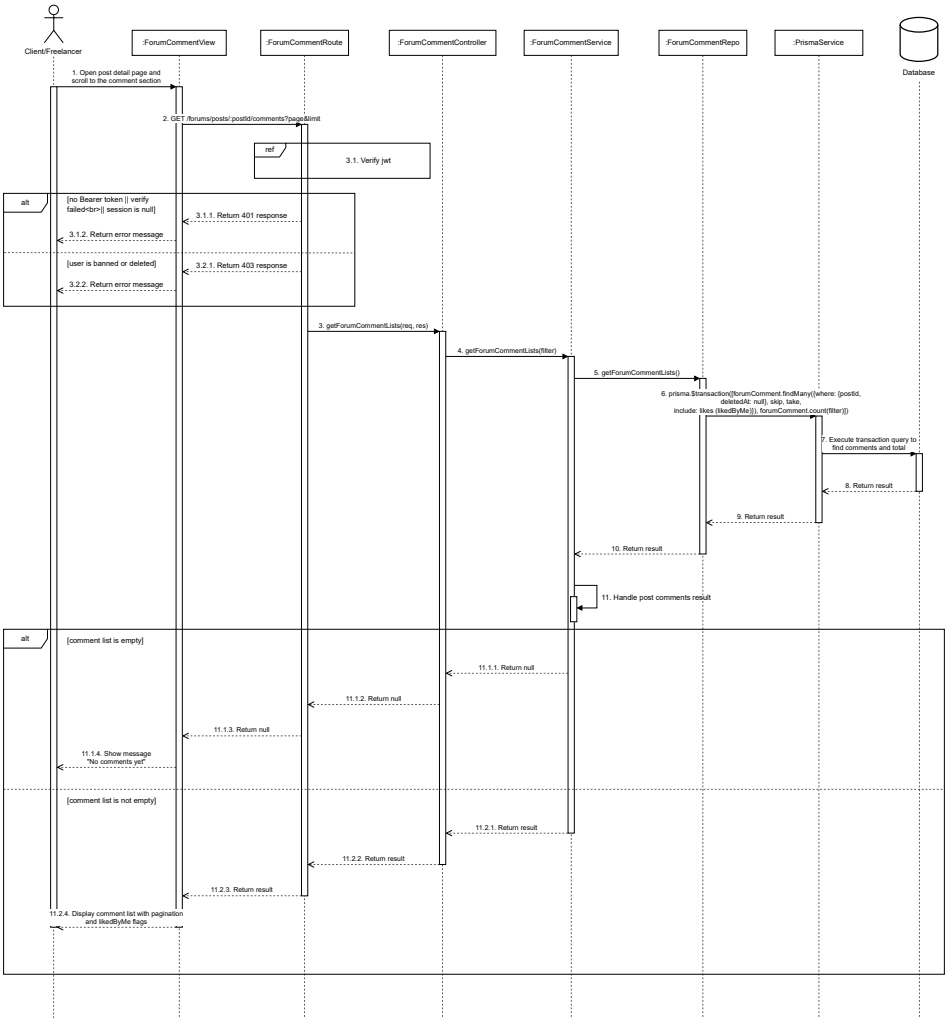

Figure 66: Sequence diagram of UC-14.11 View Post Comments

3.7.0.13 UC-14.12 – Create Comment

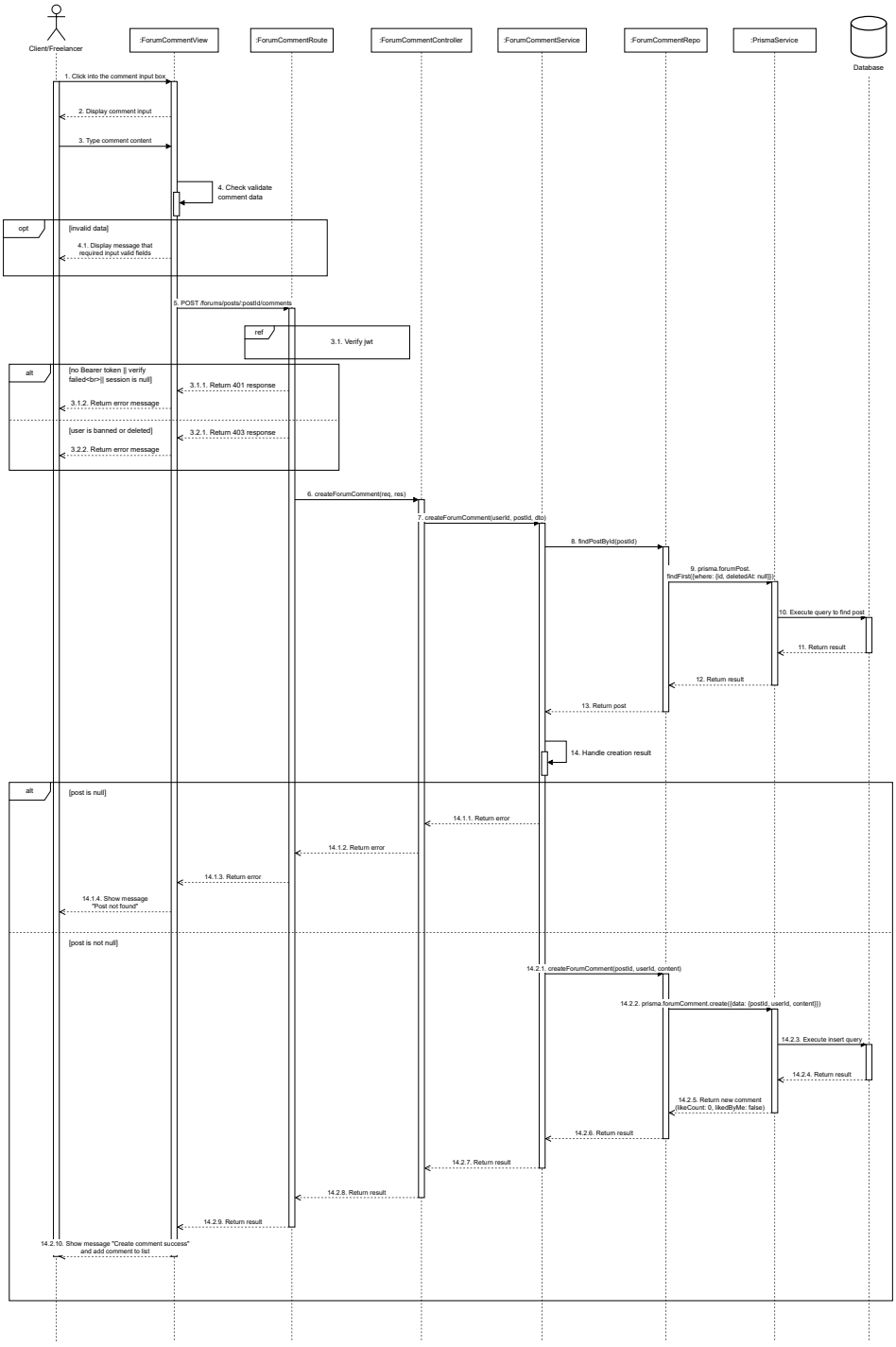

Figure 67: Sequence diagram of UC-14.12 Create Comment

3.7.0.14 UC-14.13 – Update Comment

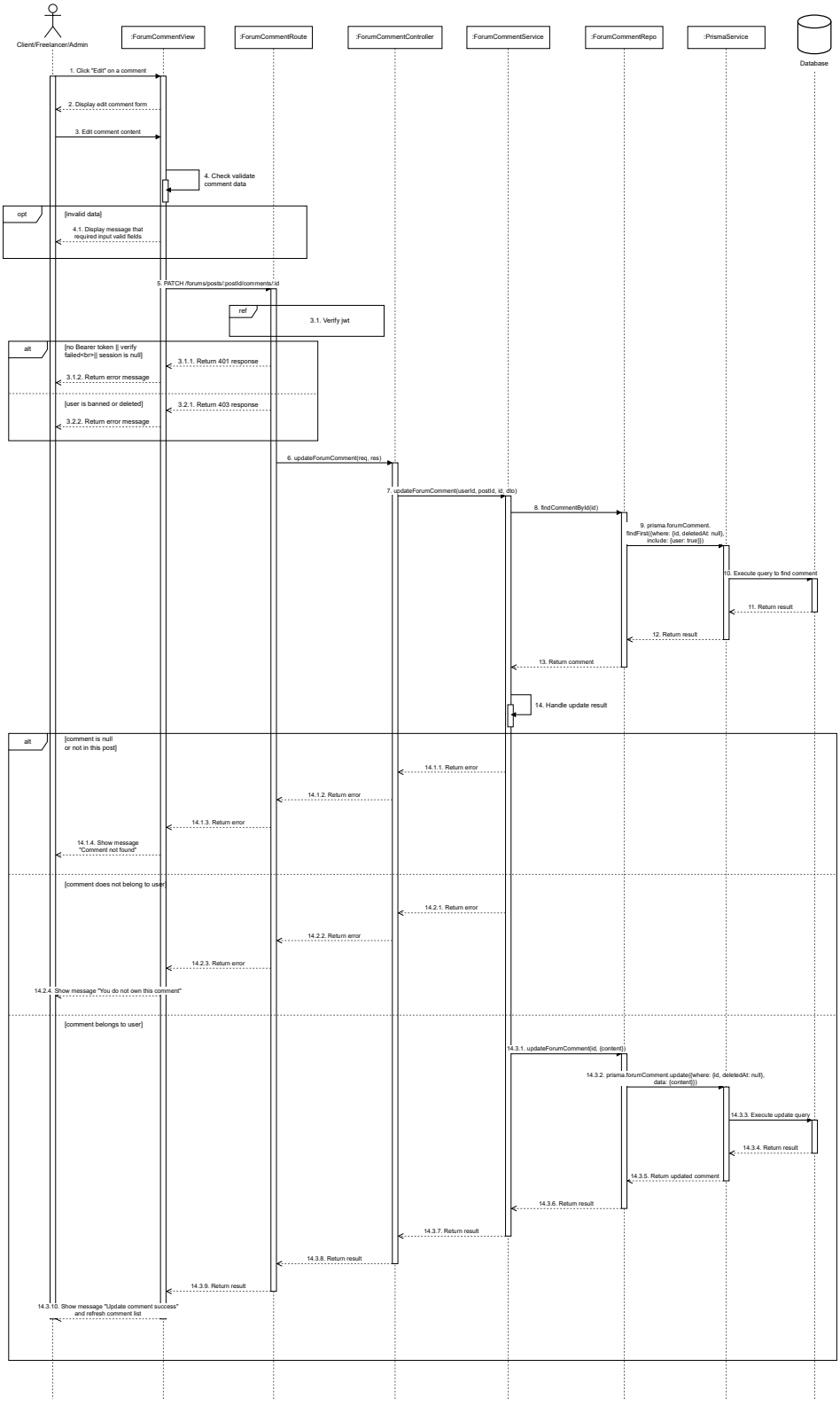

Figure 68: Sequence diagram of UC-14.13 Update Comment

3.7.0.15 UC-14.14 – Delete Comment

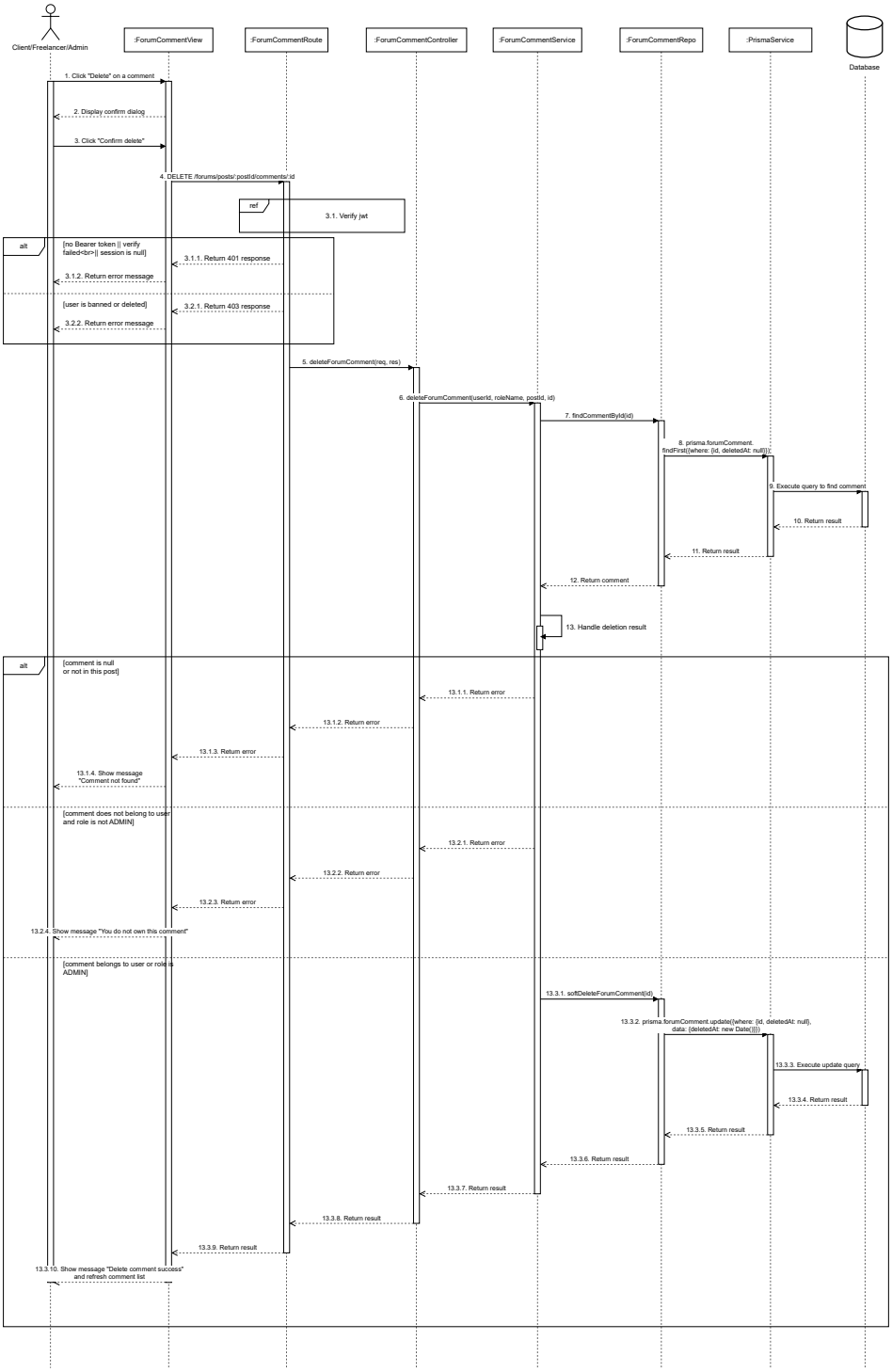

Figure 69: Sequence diagram of UC-14.14 Delete Comment

3.7.0.16 UC-14.15 – Like Comment

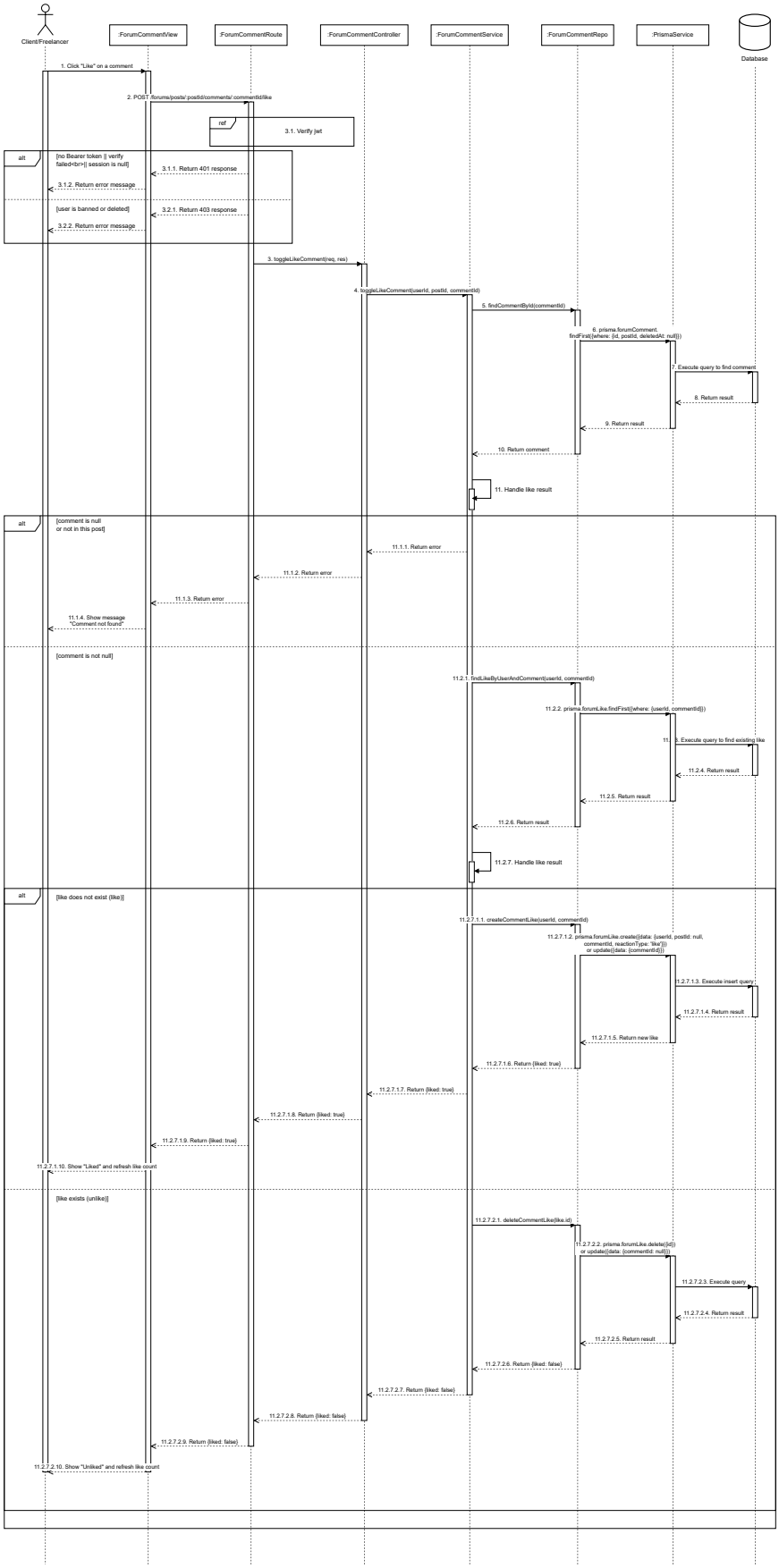

Figure 70: Sequence diagram of UC-14.15 Like Comment

3.7.0.17 UC-14.16 – Unlike Comment

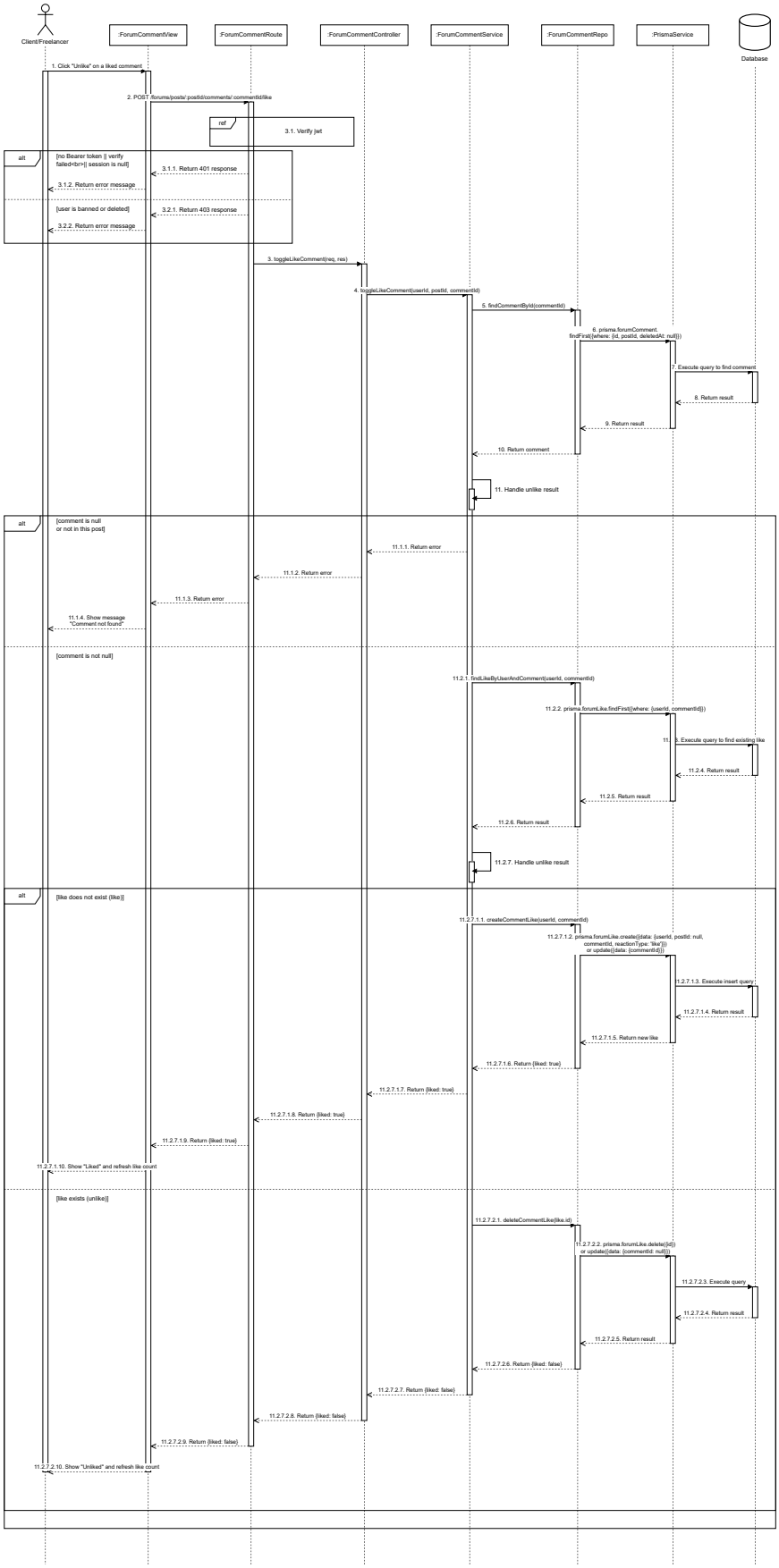

Figure 71: Sequence diagram of UC-14.16 Unlike Comment

3.7.0.18 UC-14.17 – Report Content

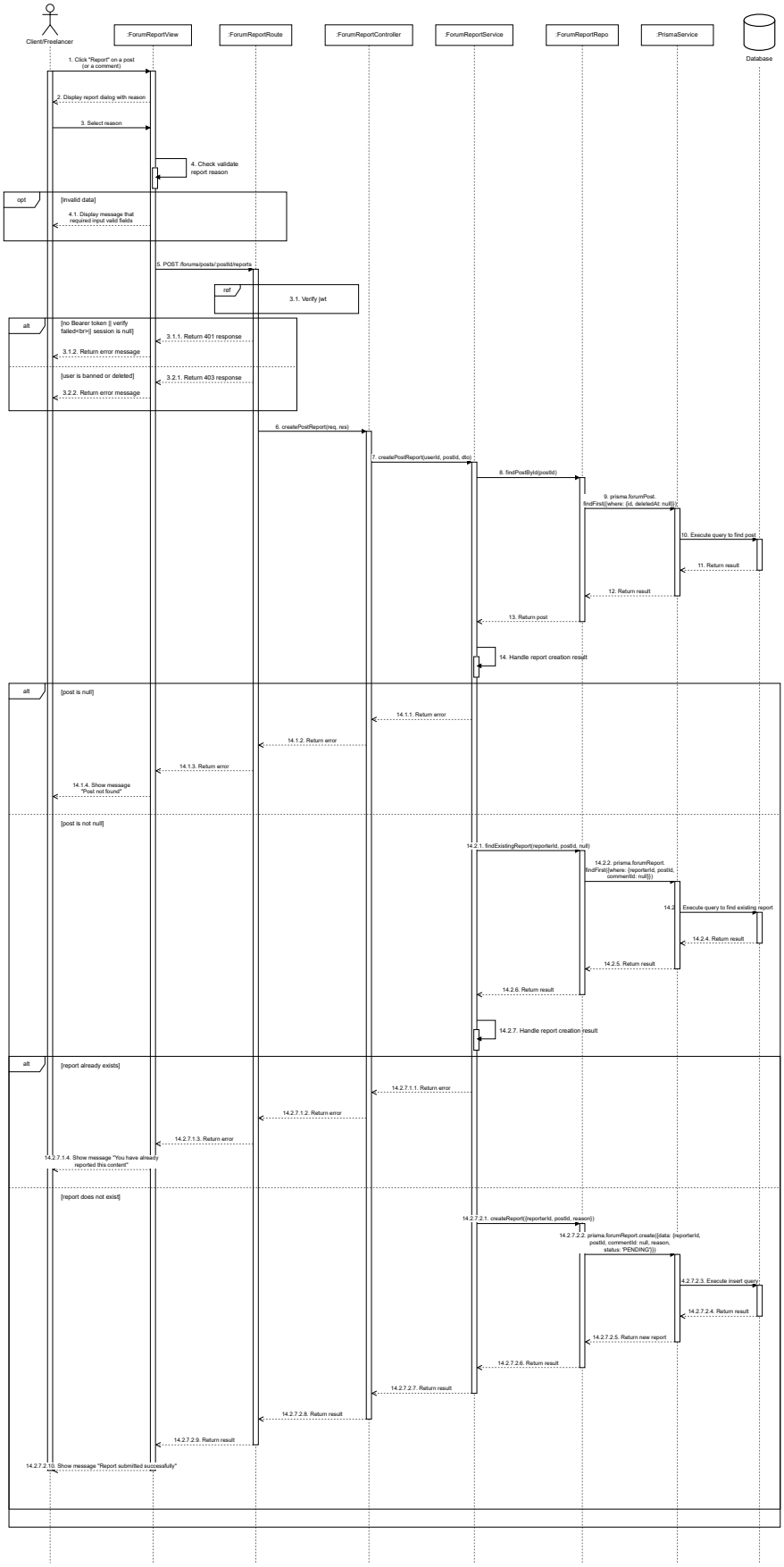

Figure 72: Sequence diagram of UC-14.17 Report Content

3.8 Manage Conversation

Manage Conversation covers the UC-27 private messaging module: listing and starting one-to-one conversations, opening a conversation, sending text messages and files, marking messages as read, recalling a message, and pinning or unpinning a conversation. Delivery is served both by the REST endpoints and by the realtime gateway in the same namespace.

3.8.0.1 Class Diagram

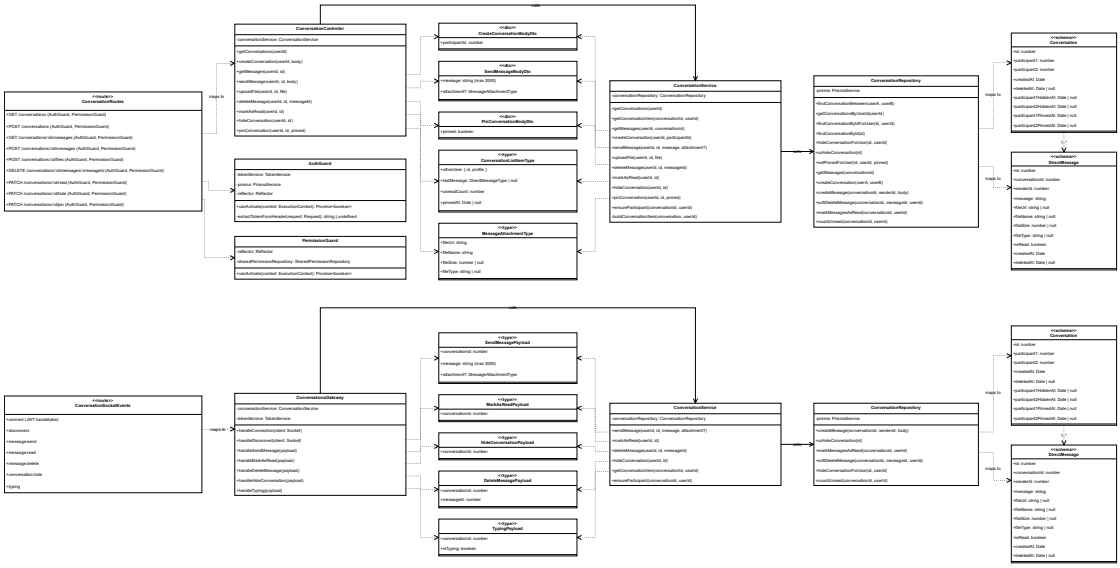

Figure 73: Class diagram of Manage Conversation

3.8.0.2 UC-27.01 – View Conversation List

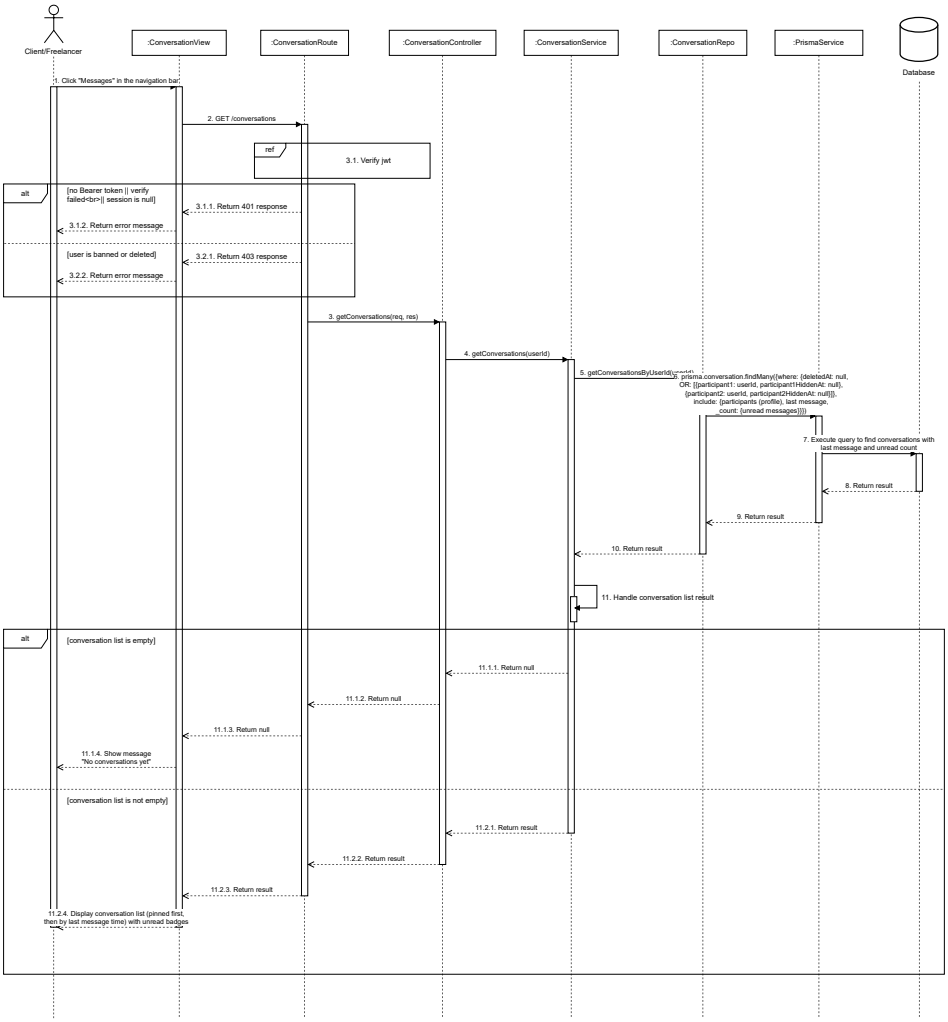

Figure 74: Sequence diagram of UC-27.01 View Conversation List

3.8.0.3 UC-27.02 – Start Conversation

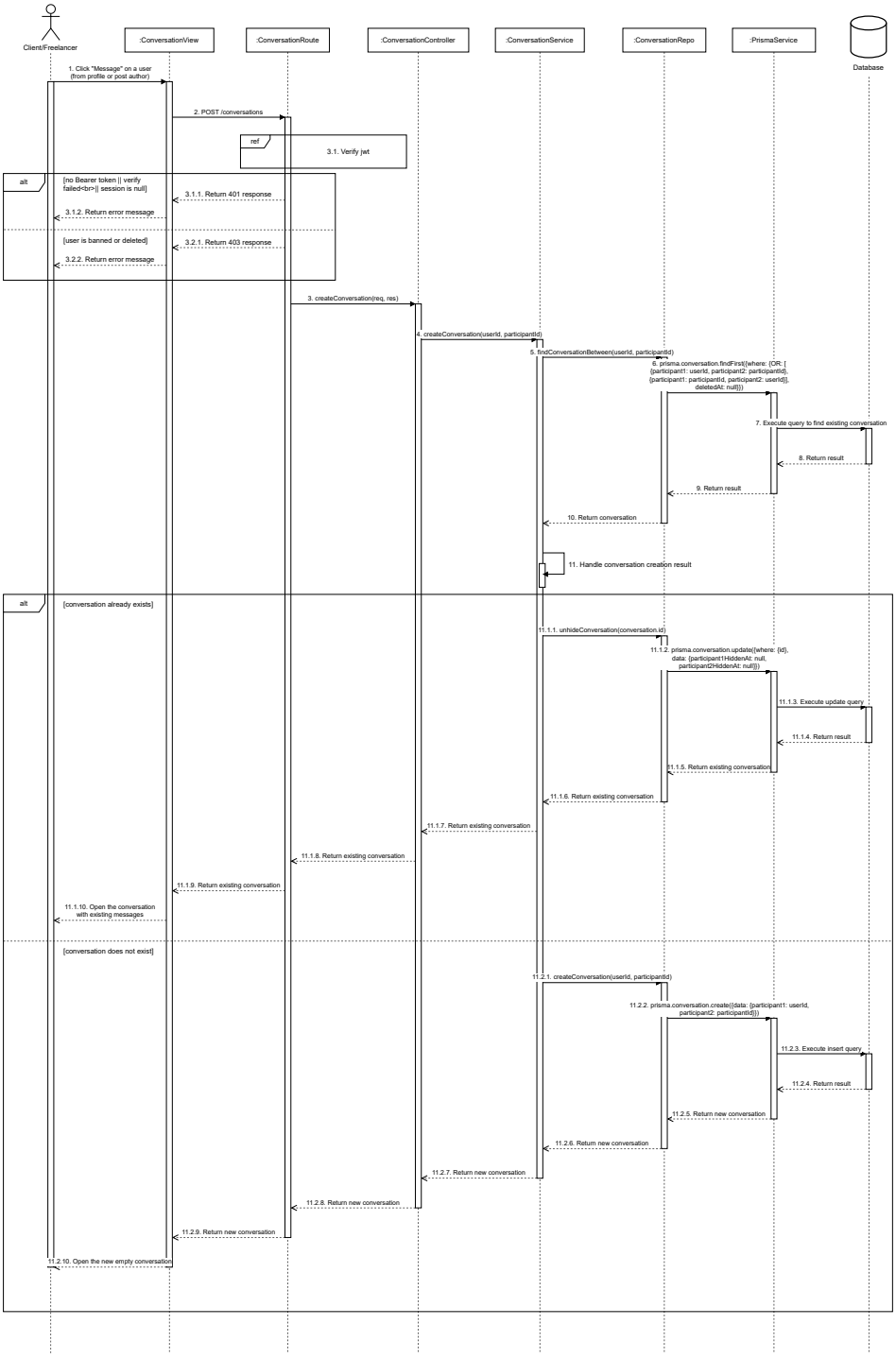

Figure 75: Sequence diagram of UC-27.02 Start Conversation

3.8.0.4 UC-27.03 – View Conversation Detail

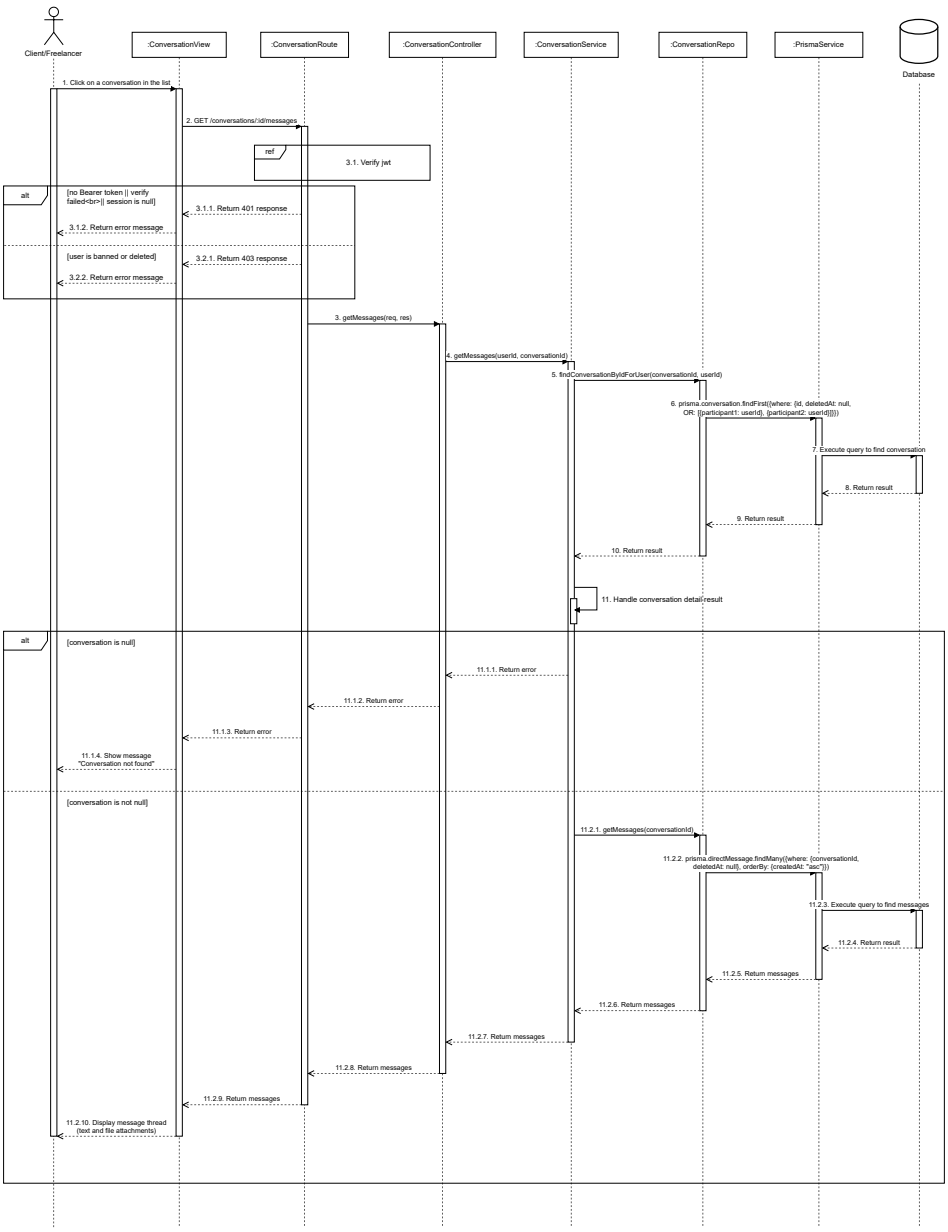

Figure 76: Sequence diagram of UC-27.03 View Conversation Detail

3.8.0.5 UC-27.04 – Send Message

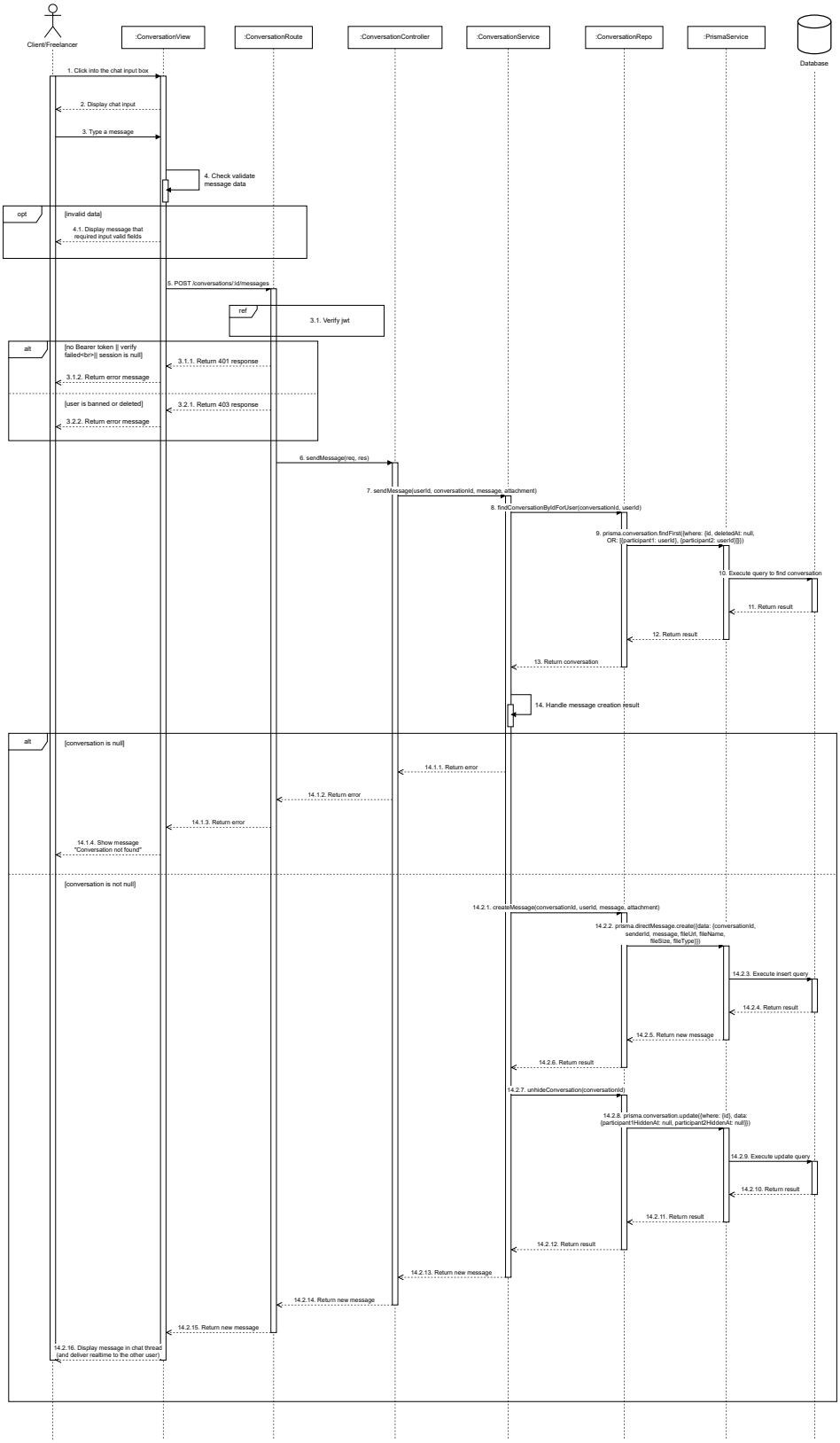

Figure 77: Sequence diagram of UC-27.04 Send Message

3.8.0.6 UC-27.05 – Mark Message as Read

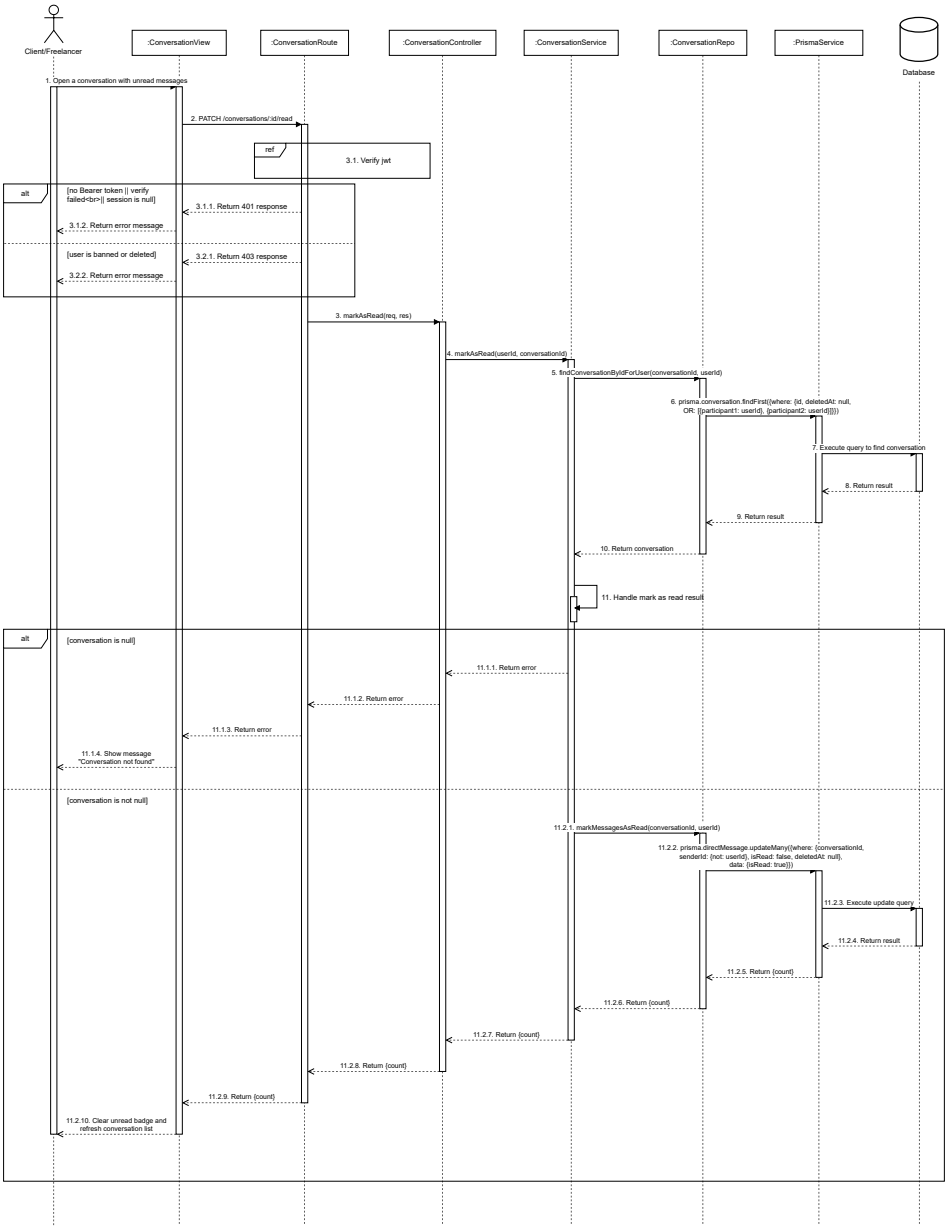

Figure 78: Sequence diagram of UC-27.05 Mark Message as Read

3.8.0.7 UC-27.06 – Delete/Recall Message

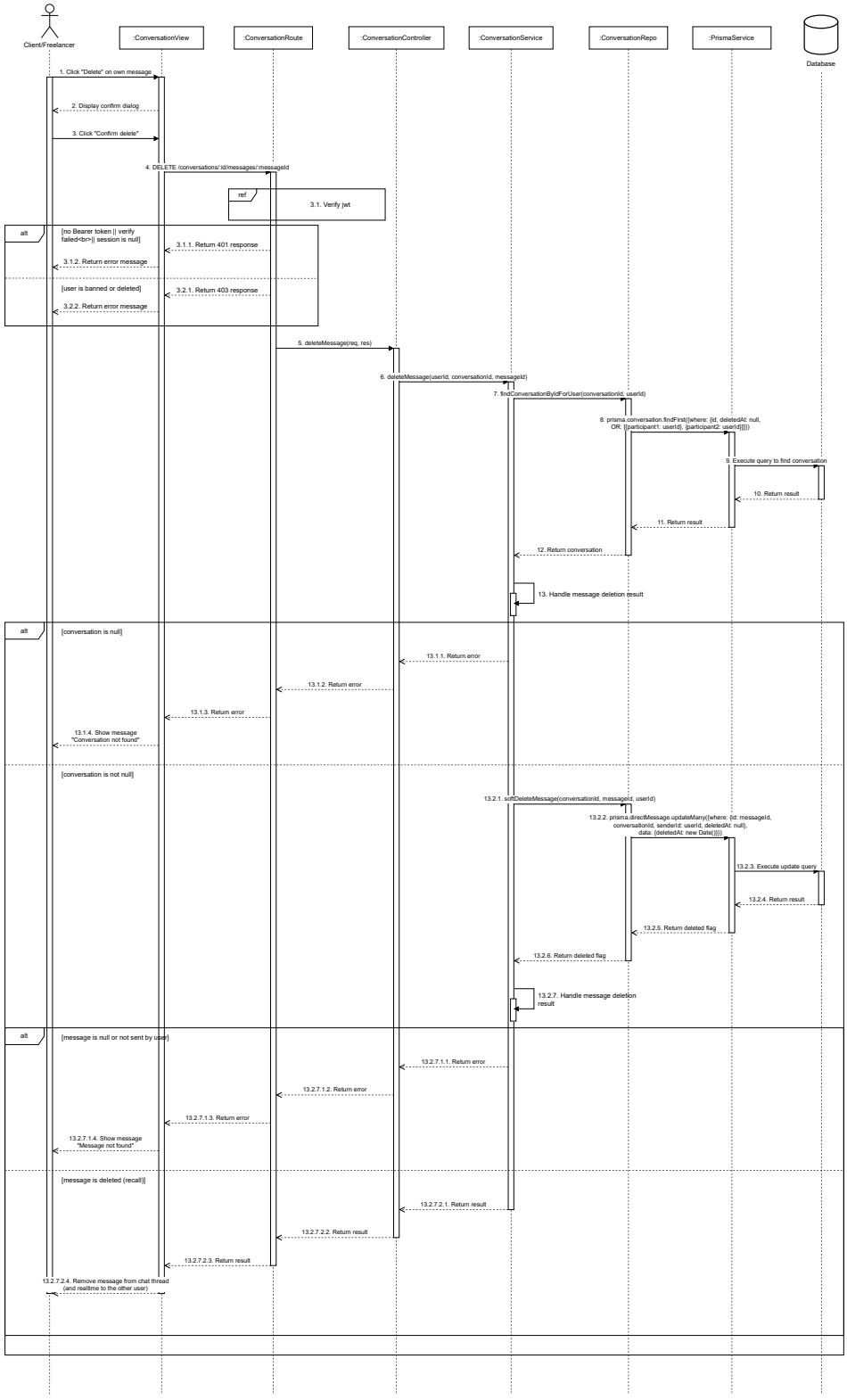

Figure 79: Sequence diagram of UC-27.06 Delete/Recall Message

3.8.0.8 UC-27.07 – Pin/Unpin Conversation

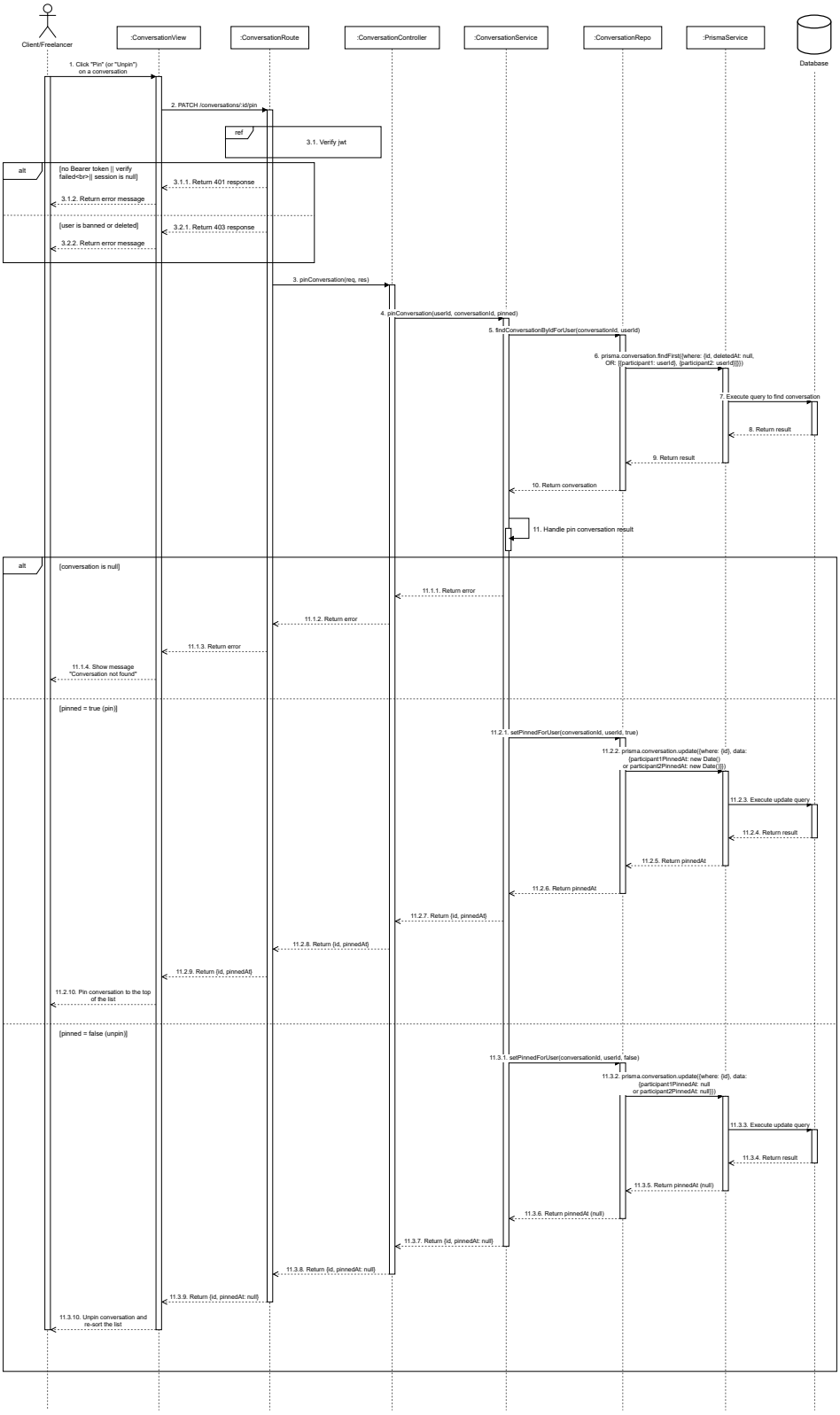

Figure 80: Sequence diagram of UC-27.07 Pin/Unpin Conversation

3.8.0.9 UC-27.08 – Send File

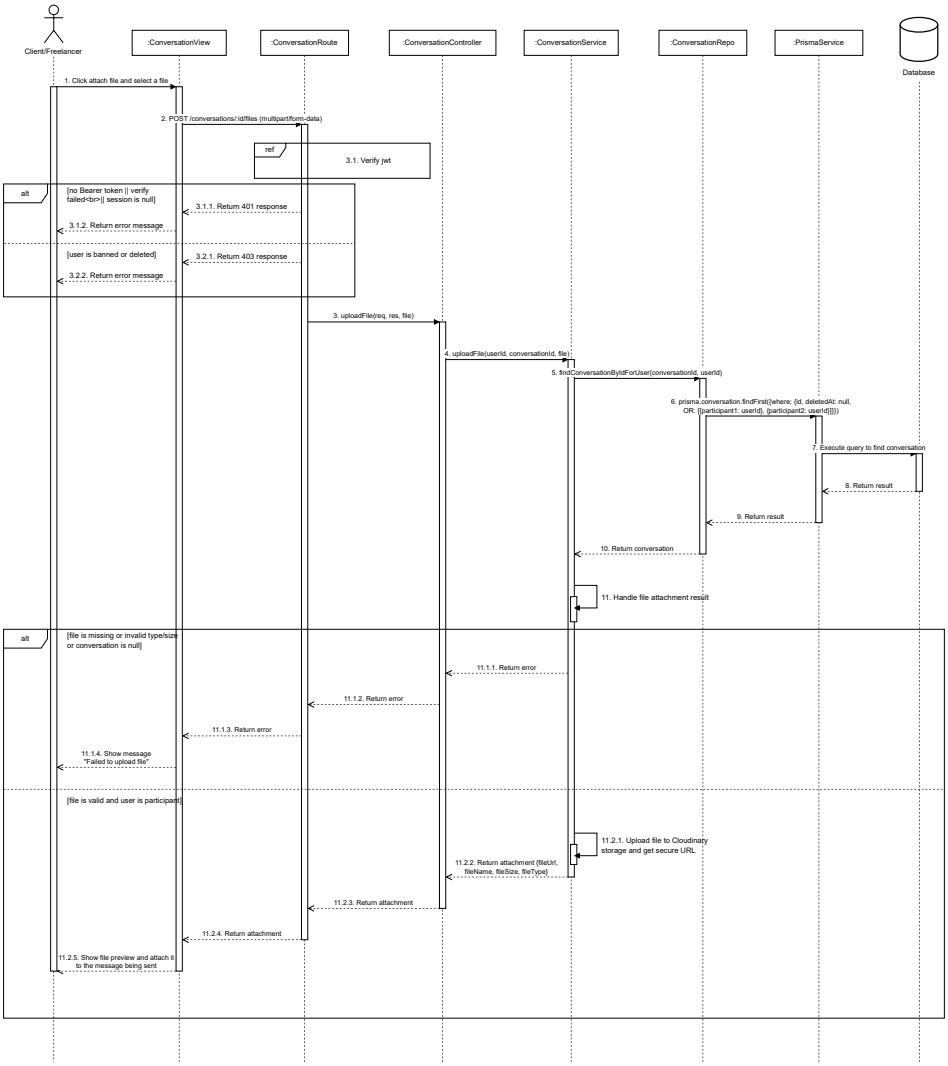

Figure 81: Sequence diagram of UC-27.08 Send File

3.9 Saved Search Management

Saved Search Management allows freelancers to save complex job filter criteria, view their list of saved searches, inspect search details, update filter parameters, and remove obsolete searches.

3.9.0.1 Class Diagram

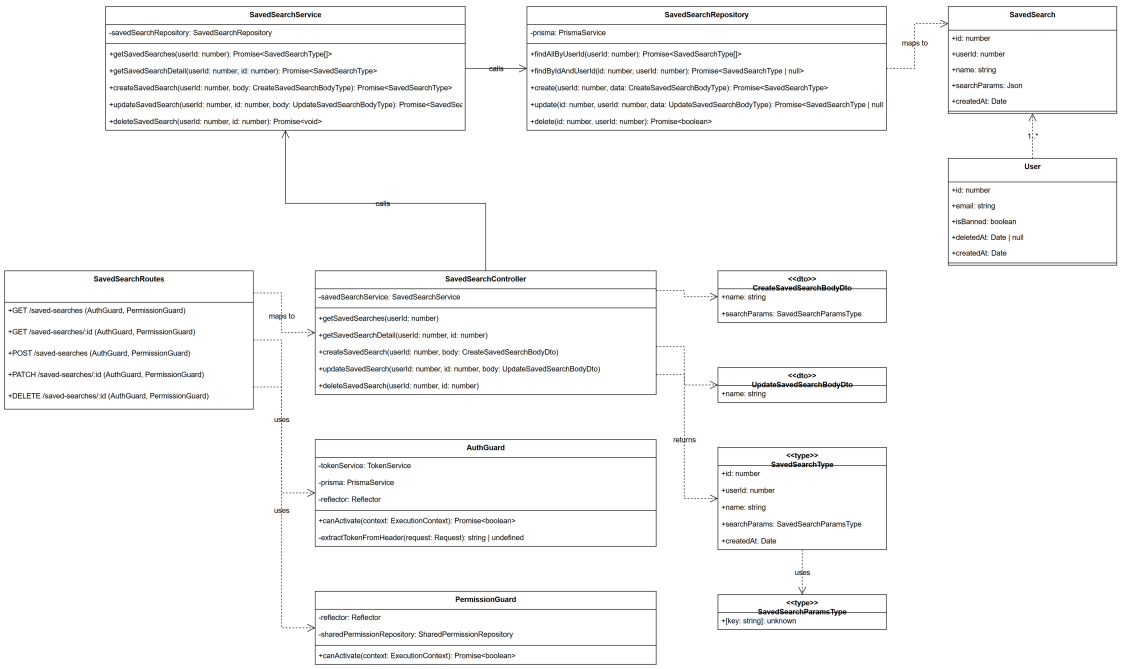

Figure 82: Class diagram of Saved Search Management

3.9.0.2 UC-22.01 – View Saved Search List

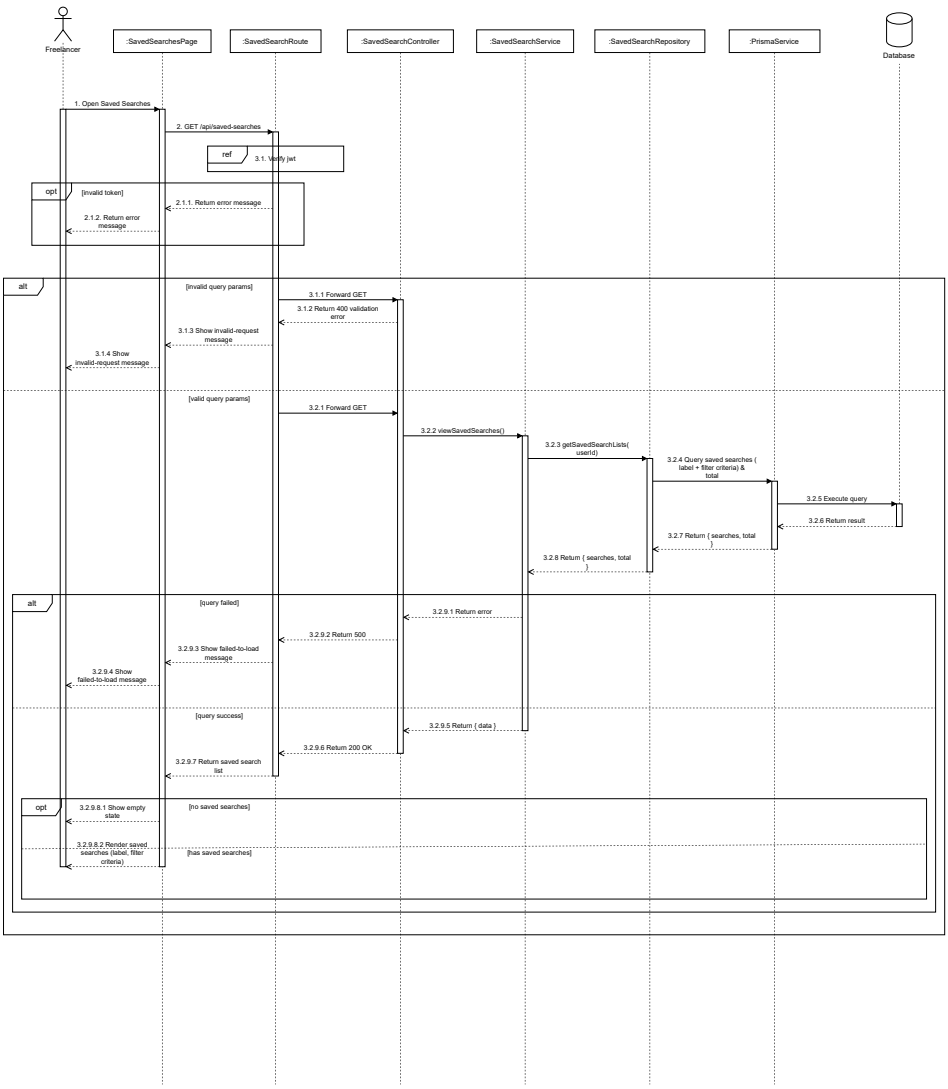

Figure 83: Sequence diagram of UC-22.01 View Saved Search List

3.9.0.3 UC-22.02 – View Saved Search Detail

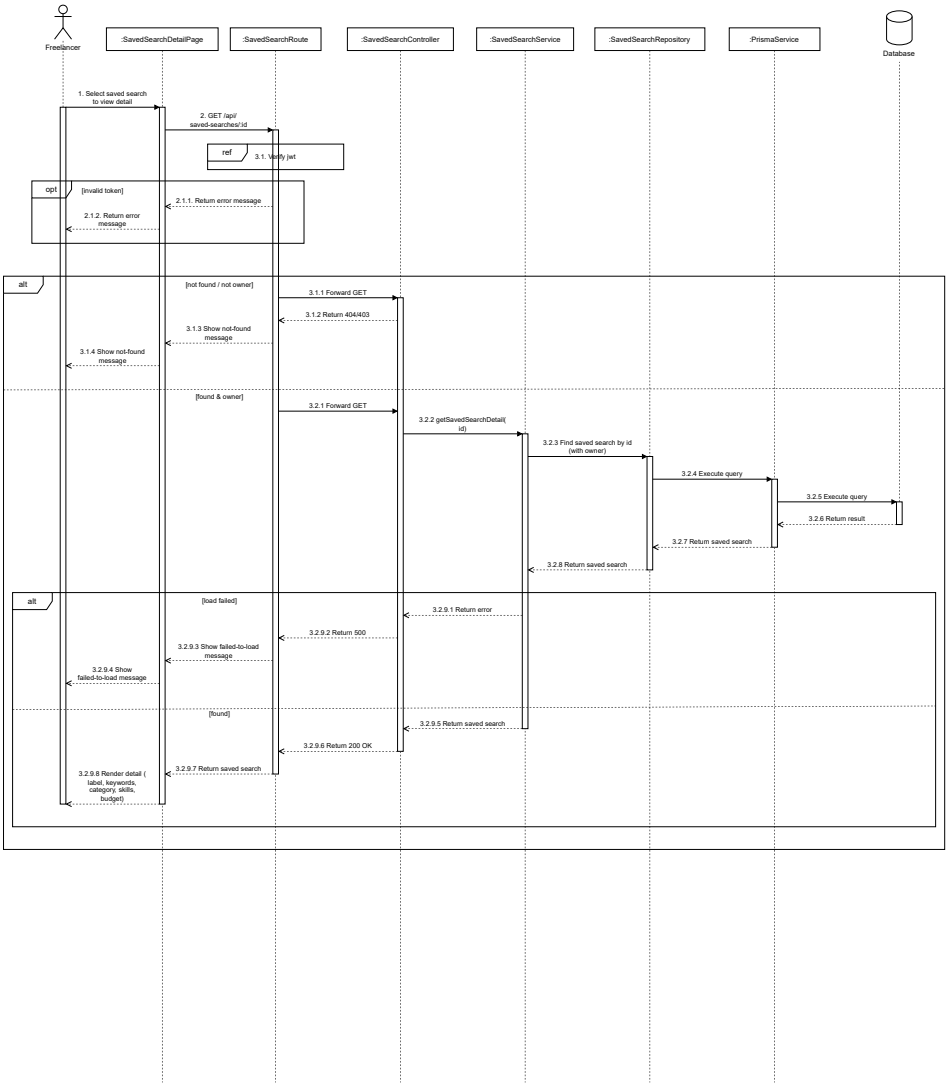

Figure 84: Sequence diagram of UC-22.02 View Saved Search Detail

3.9.0.4 UC-22.03 – Create Saved Search

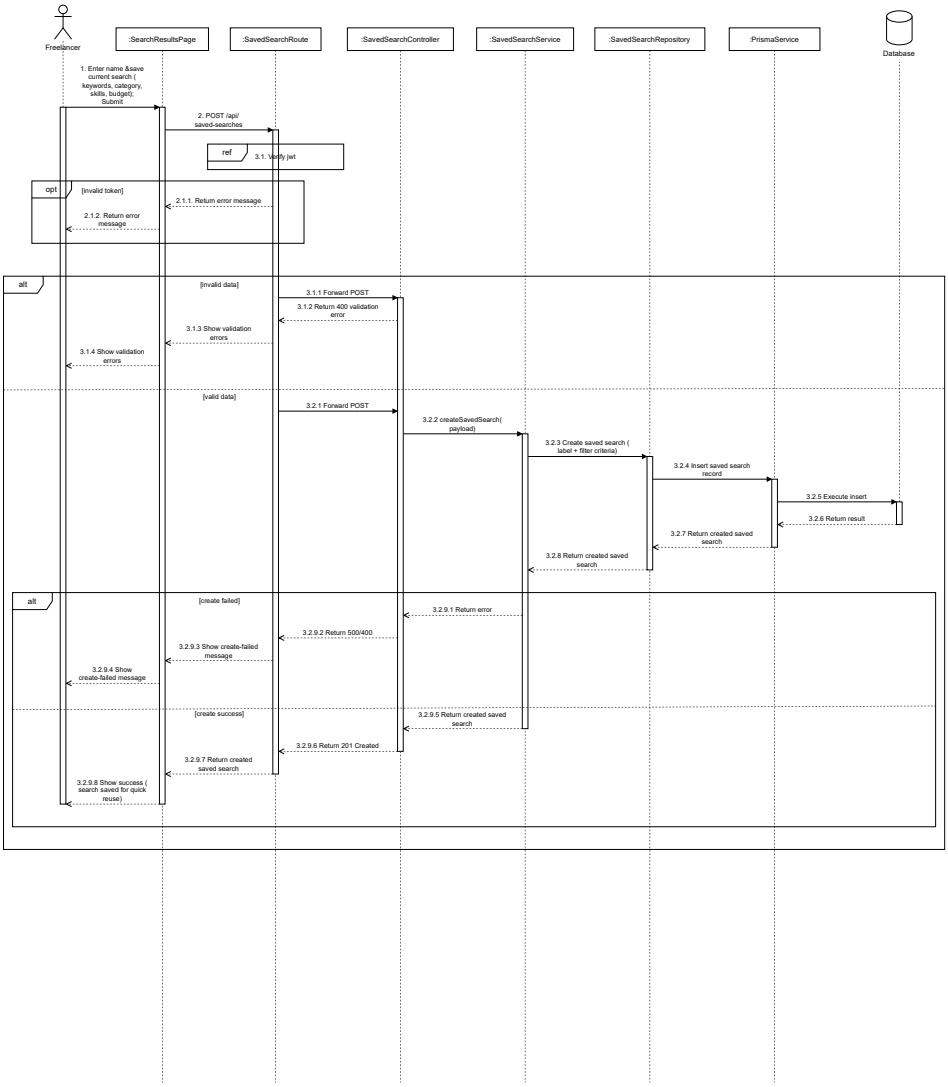

Figure 85: Sequence diagram of UC-22.03 Create Saved Search

3.9.0.5 UC-22.04 – Update Saved Search

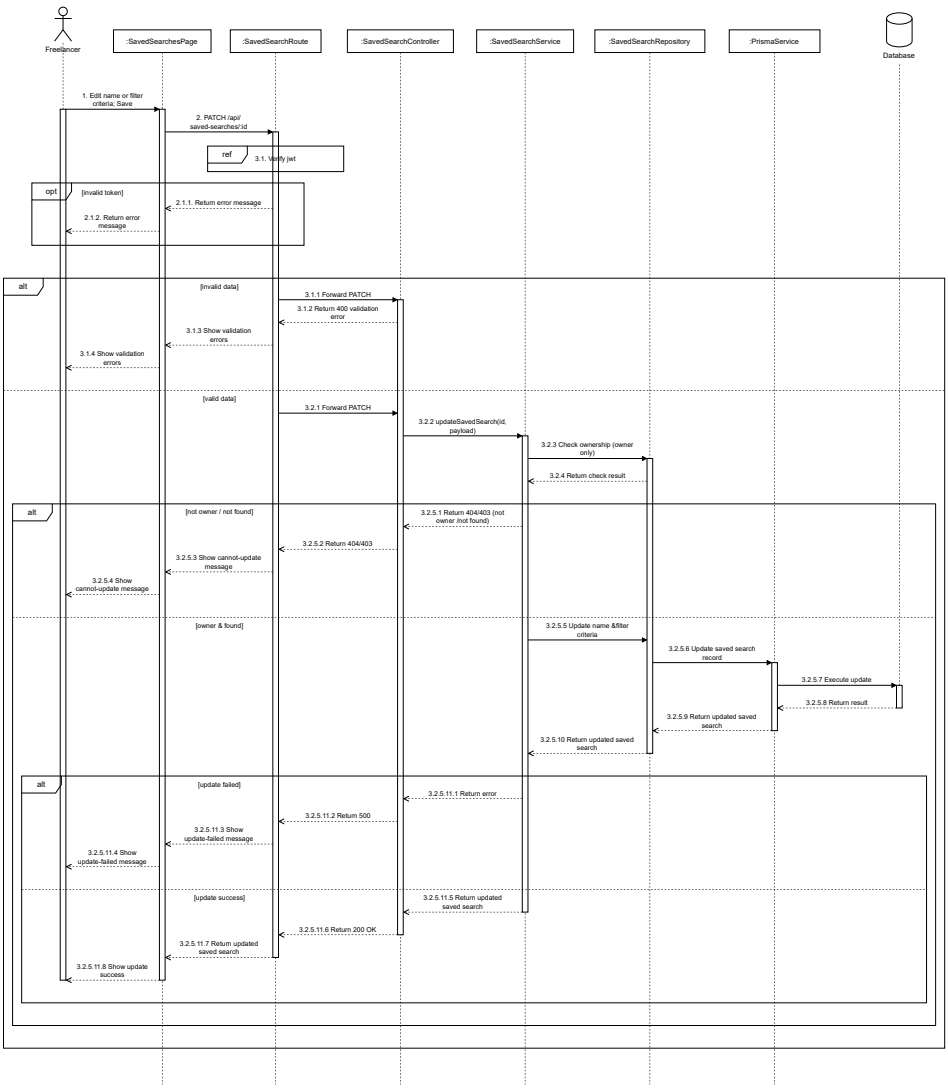

Figure 86: Sequence diagram of UC-22.04 Update Saved Search

3.9.0.6 UC-22.05 – Delete Saved Search

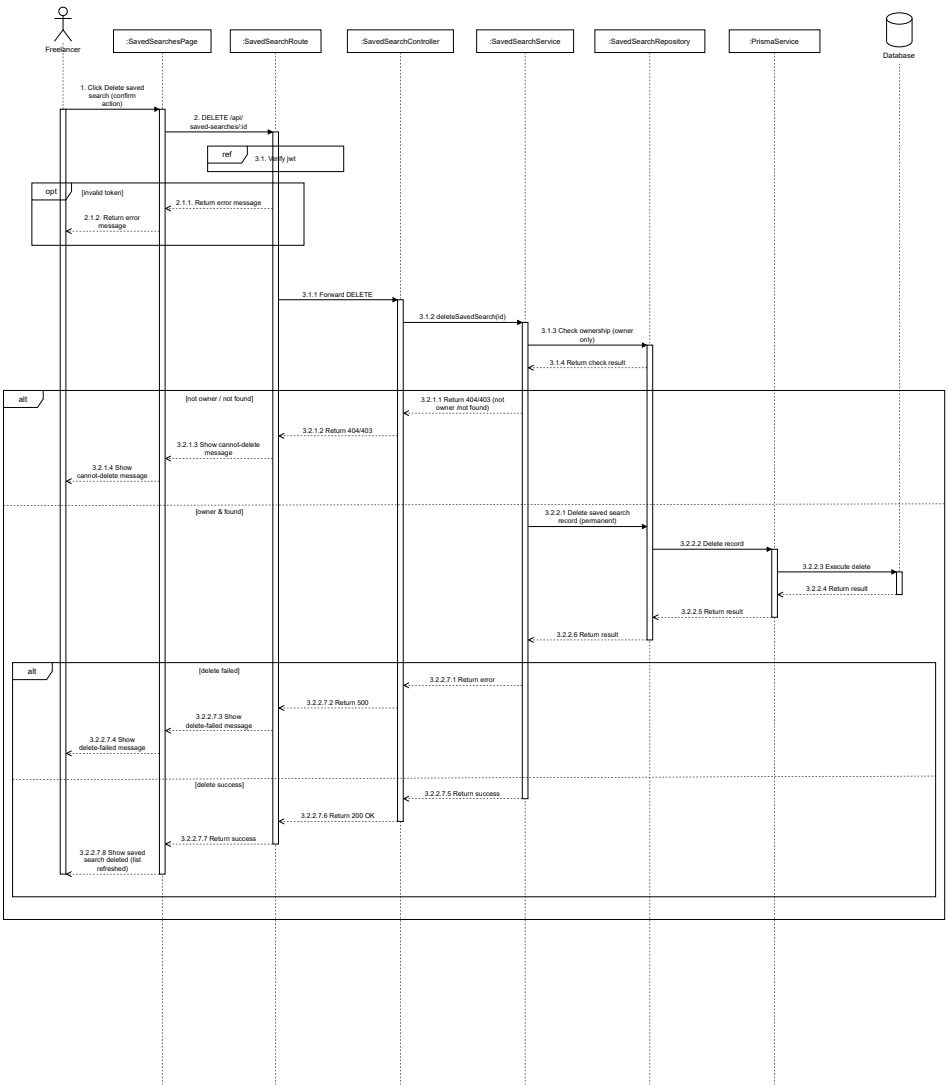

Figure 87: Sequence diagram of UC-22.05 Delete Saved Search

3.10 Job Alert Management

Job Alert Management enables freelancers to configure automated notifications based on job keywords, budget ranges, and category criteria, ensuring they receive prompt alerts when matching jobs are published.

3.10.0.1 Class Diagram

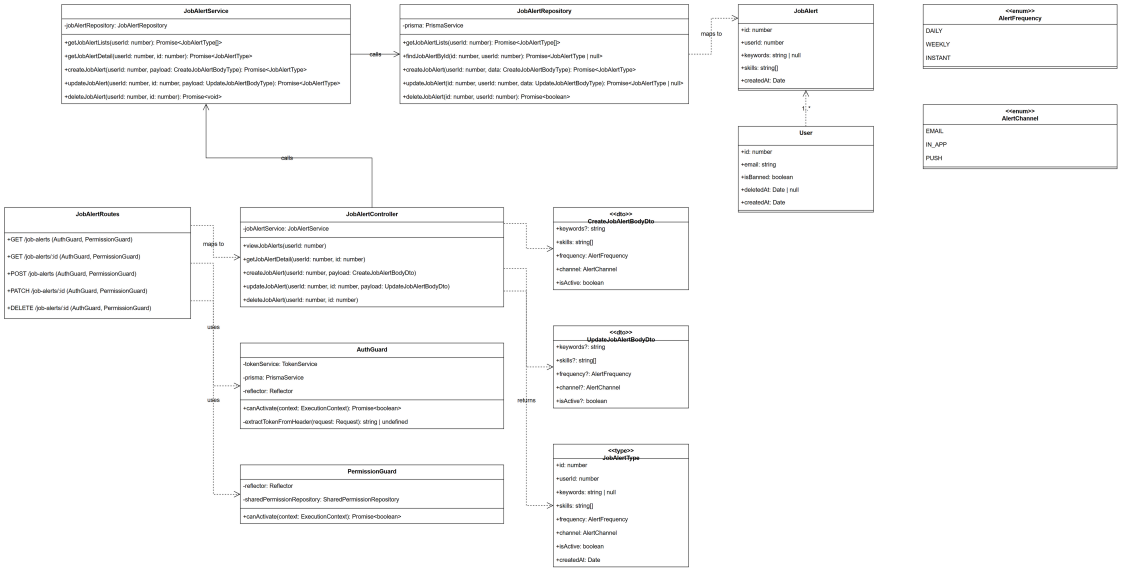

Figure 88: Class diagram of Job Alert Management

3.10.0.2 UC-23.01 – View Job Alert List

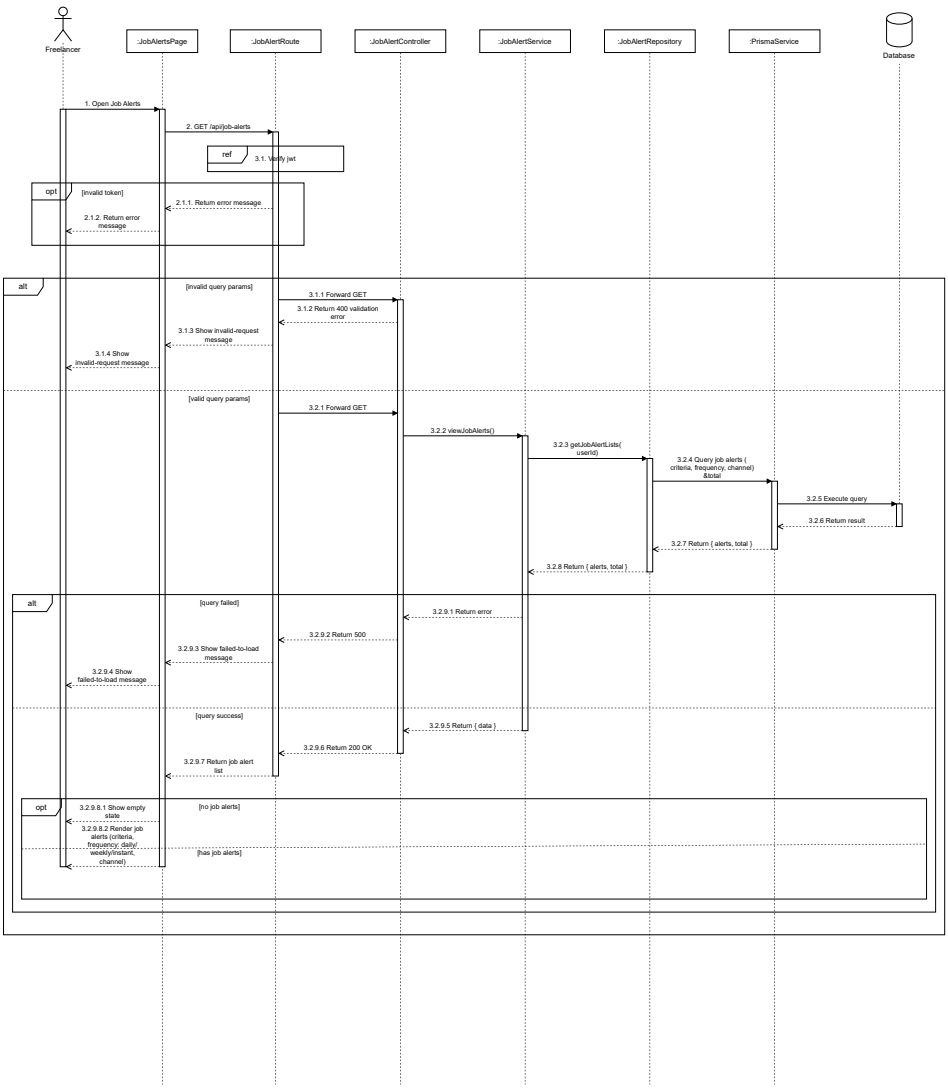

Figure 89: Sequence diagram of UC-23.01 View Job Alert List

3.10.0.3 UC-23.02 – View Job Alert Detail

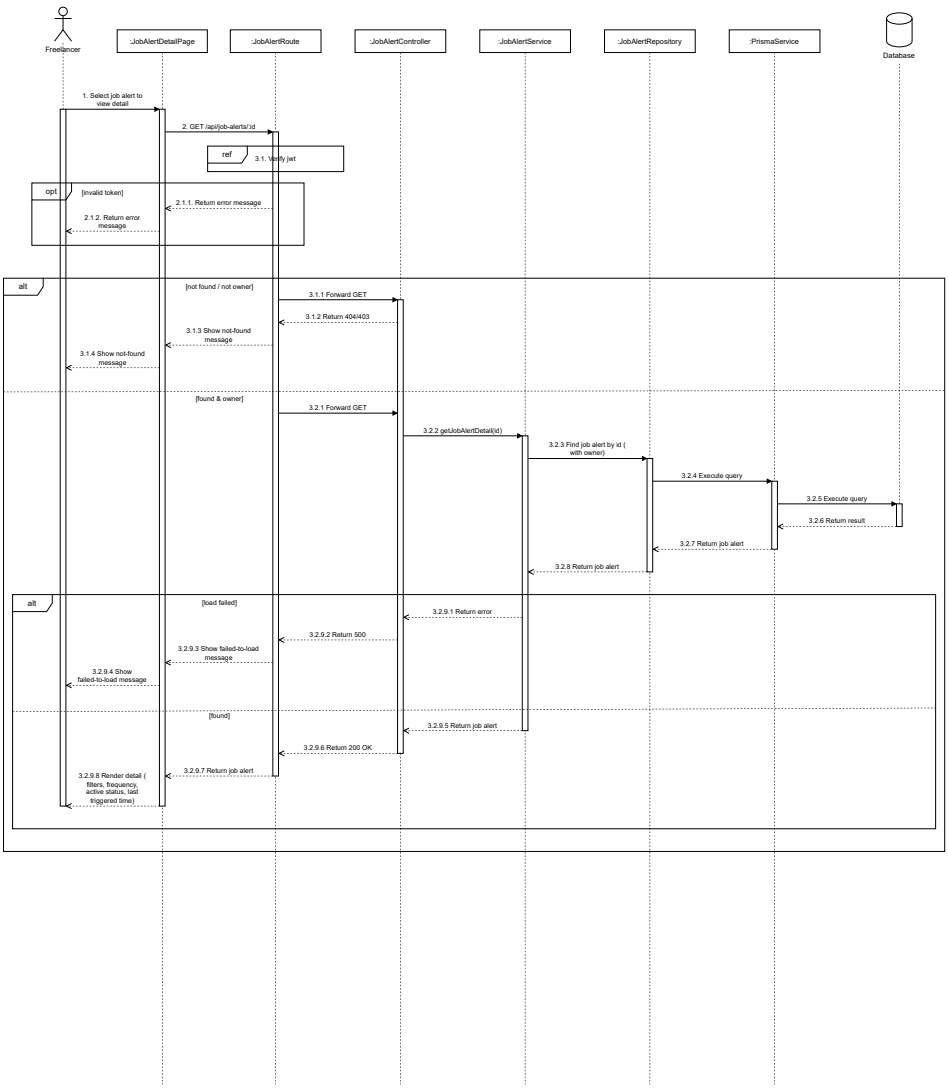

Figure 90: Sequence diagram of UC-23.02 View Job Alert Detail

3.10.0.4 UC-23.03 – Create Job Alert

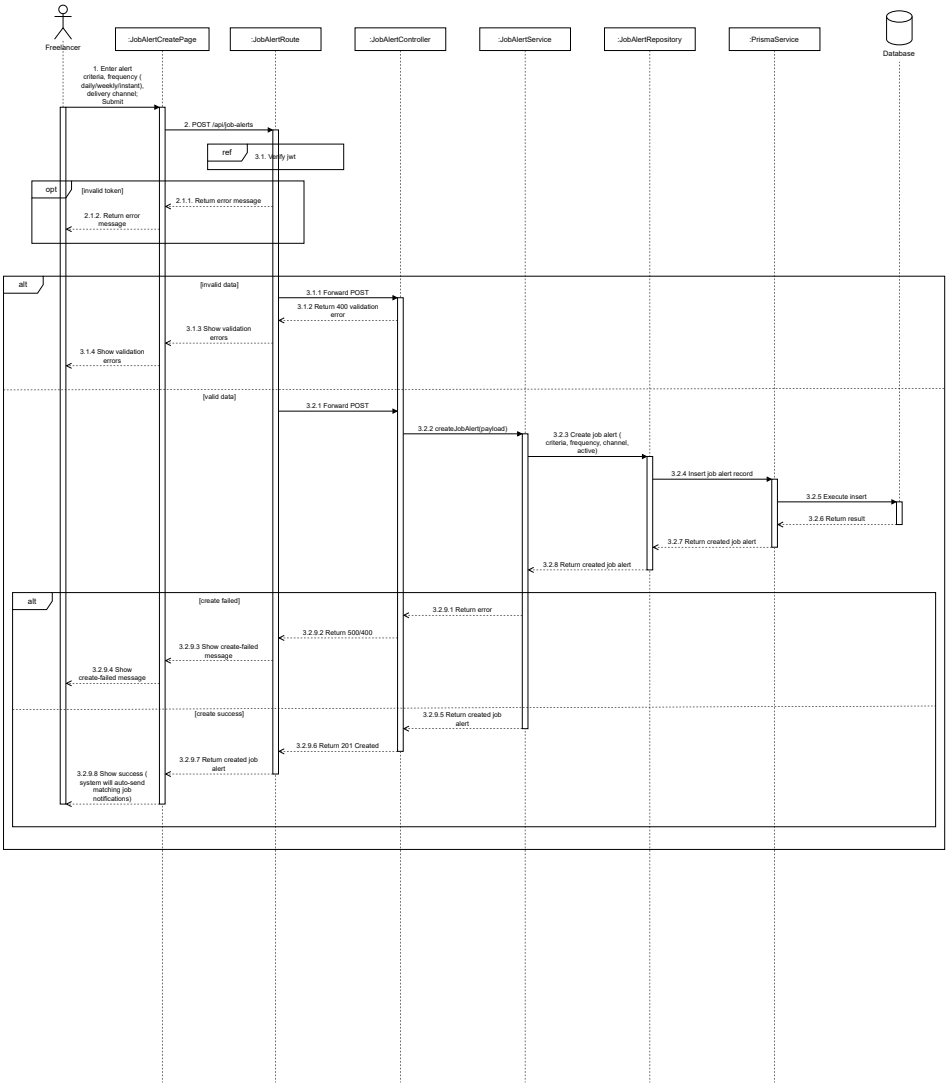

Figure 91: Sequence diagram of UC-23.03 Create Job Alert

3.10.0.5 UC-23.04 – Update Job Alert

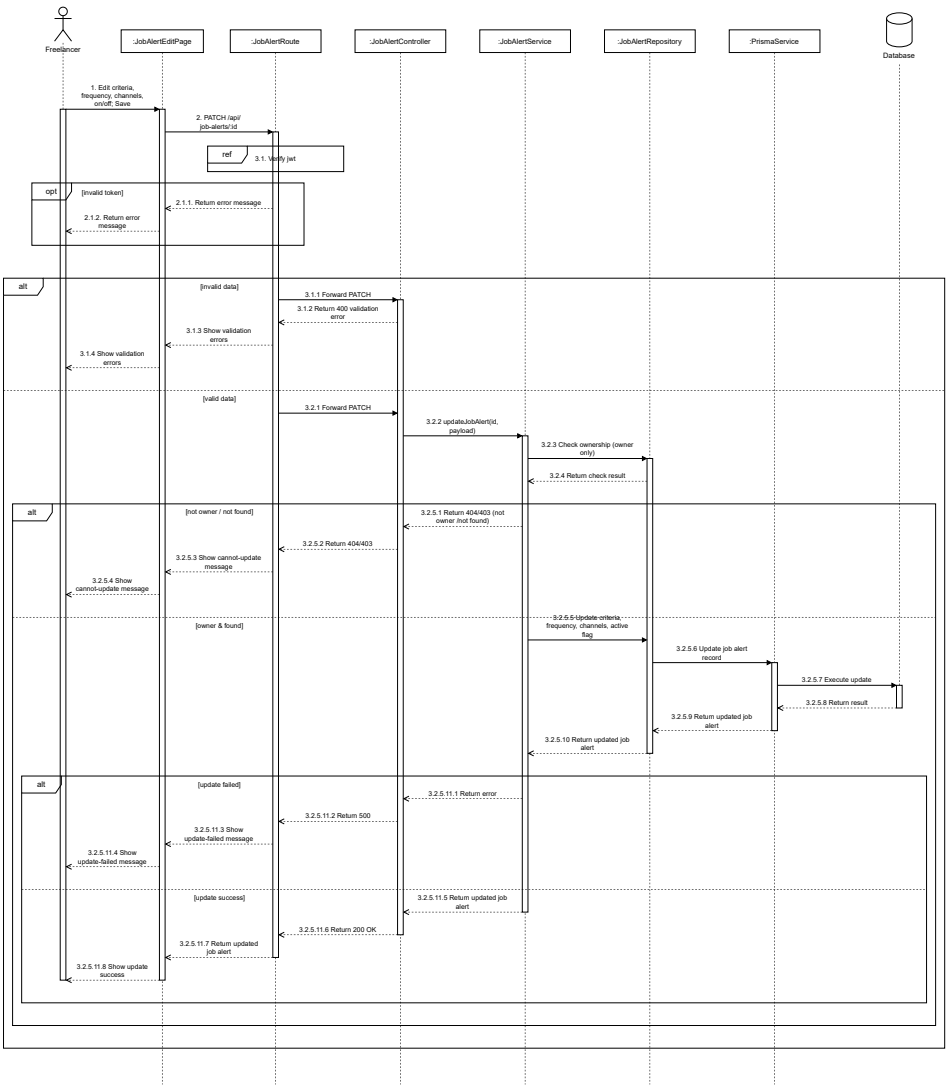

Figure 92: Sequence diagram of UC-23.04 Update Job Alert

3.10.0.6 UC-23.05 – Delete Job Alert

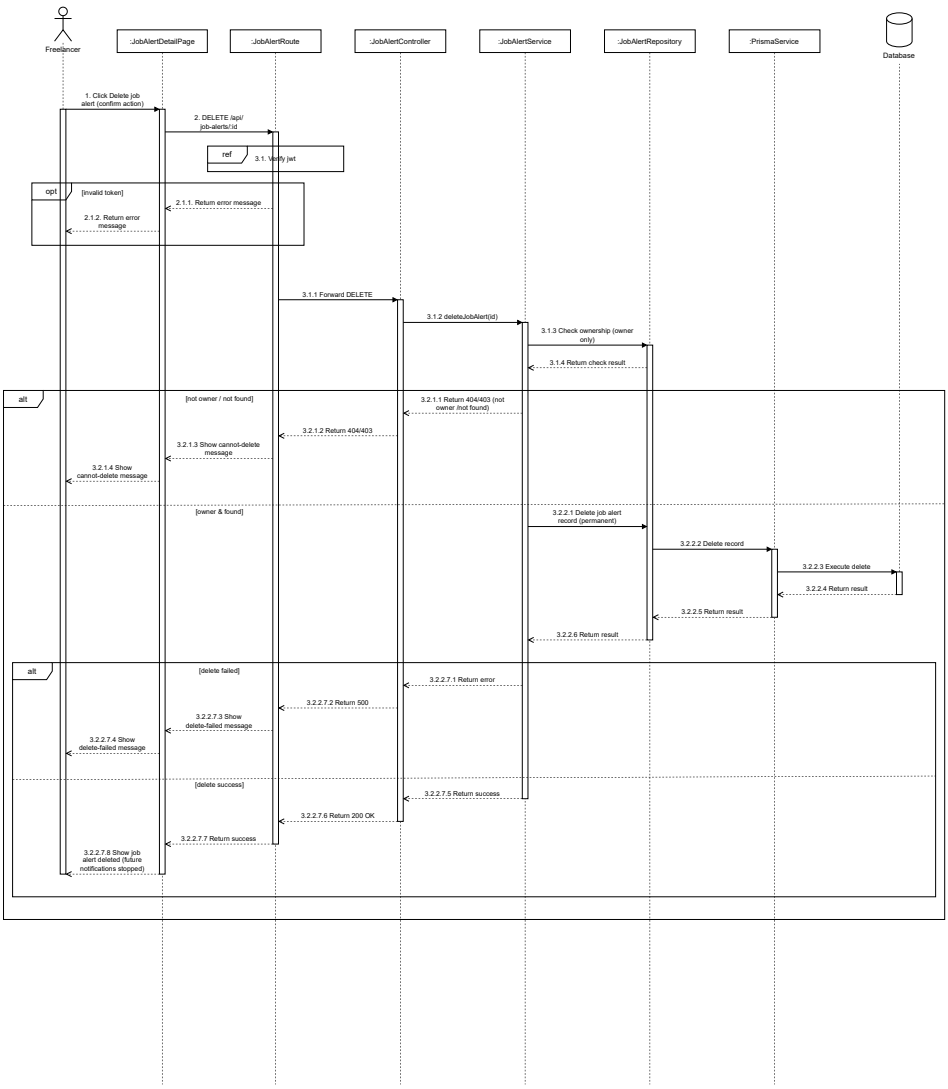

Figure 93: Sequence diagram of UC-23.05 Delete Job Alert

3.11 Permission Management

Permission Management exposes the route-level permission catalogue so that an administrator can inspect the permissions available for role assignment.

3.11.0.1 Class Diagram

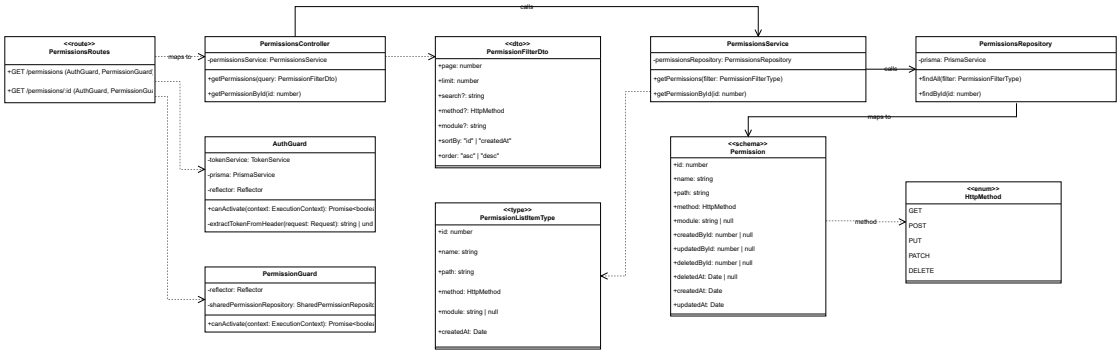

Figure 94: Class diagram of Permission Management

3.11.0.2 UC-15.01 – View Permission List

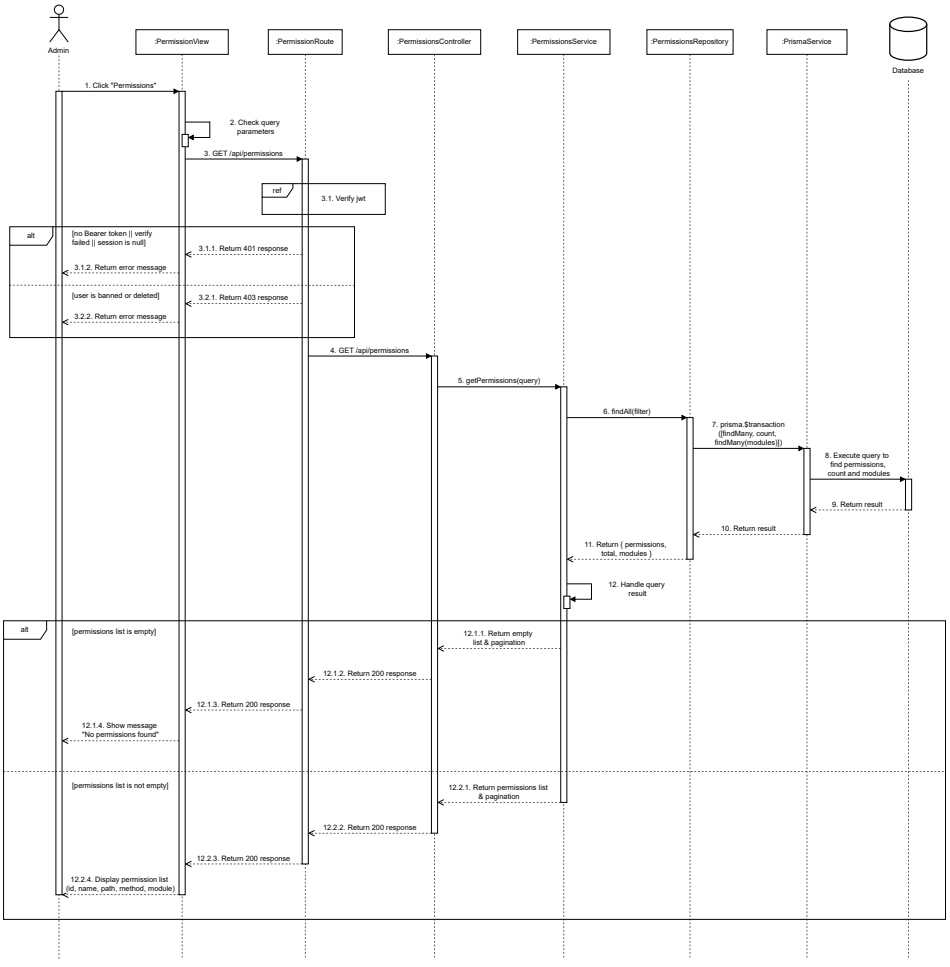

Figure 95: Sequence diagram of UC-15.01 View Permission List

3.11.0.3 UC-15.02 – View Permission Detail

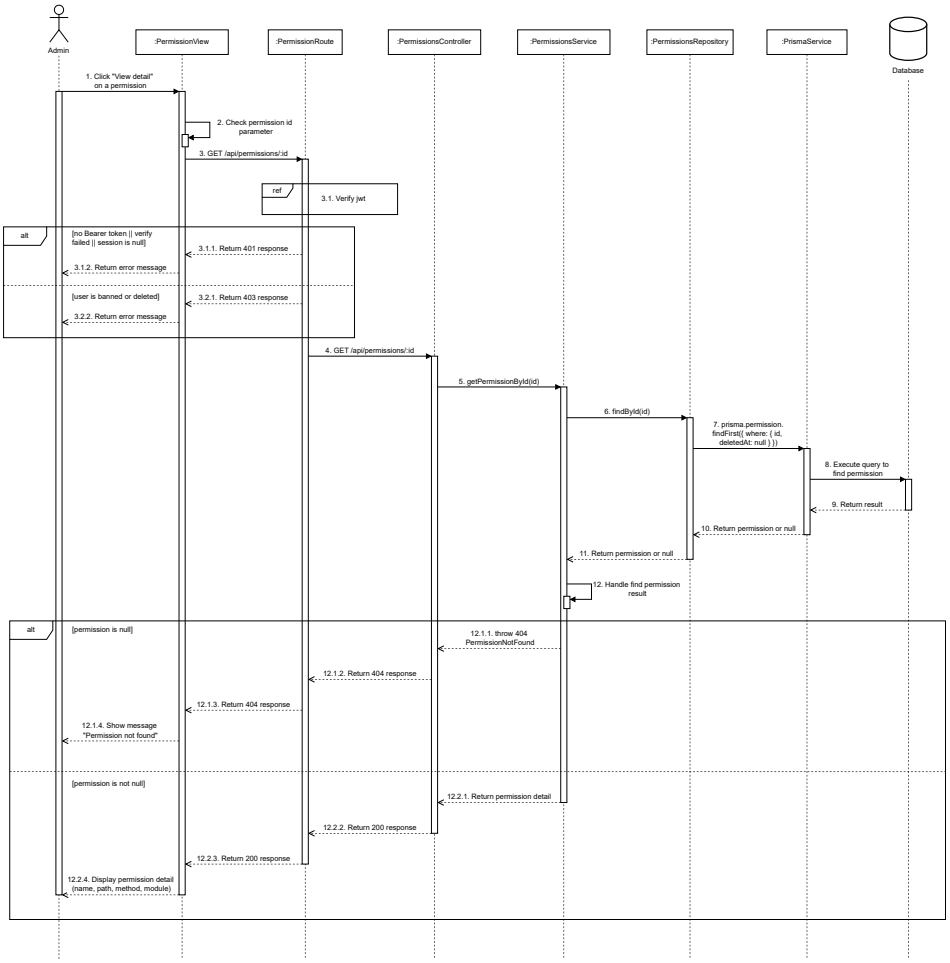

Figure 96: Sequence diagram of UC-15.02 View Permission Detail

3.12 Session Management

Session Management gives an authenticated user visibility over the devices that currently hold a refresh-token session and allows a session to be revoked remotely.

3.12.0.1 Class Diagram

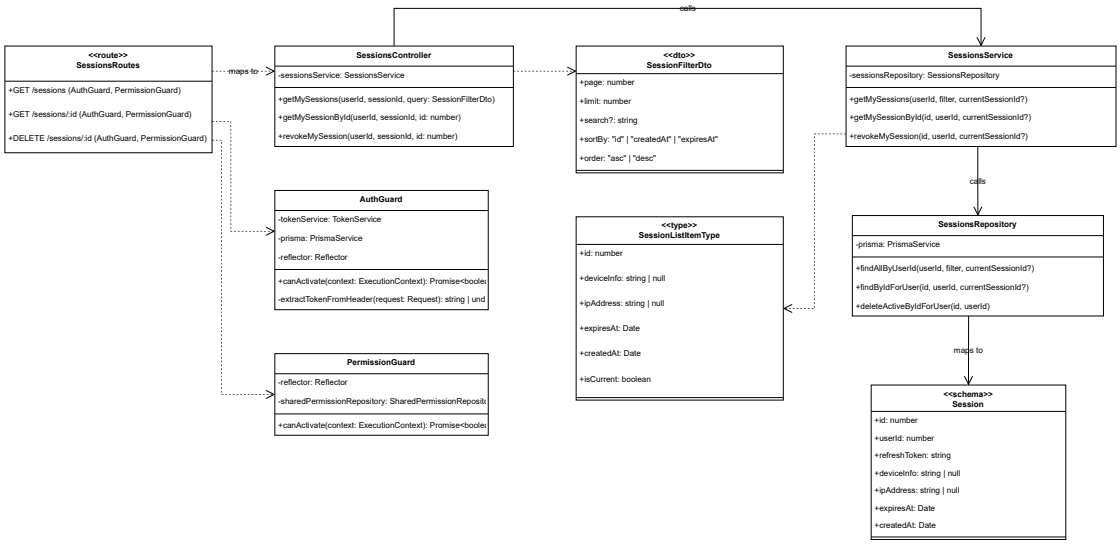

Figure 97: Class diagram of Session Management

3.12.0.2 UC-16.01 – View Session List

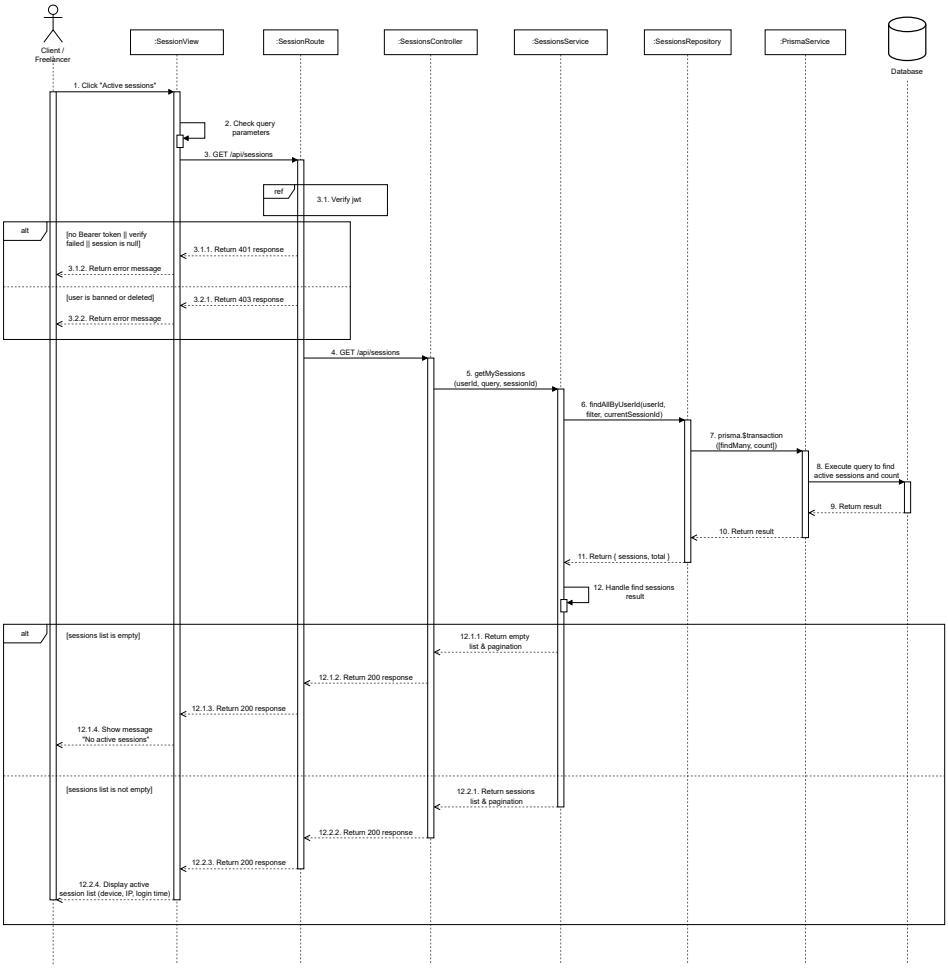

Figure 98: Sequence diagram of UC-16.01 View Session List

3.12.0.3 UC-16.02 – View Session Detail

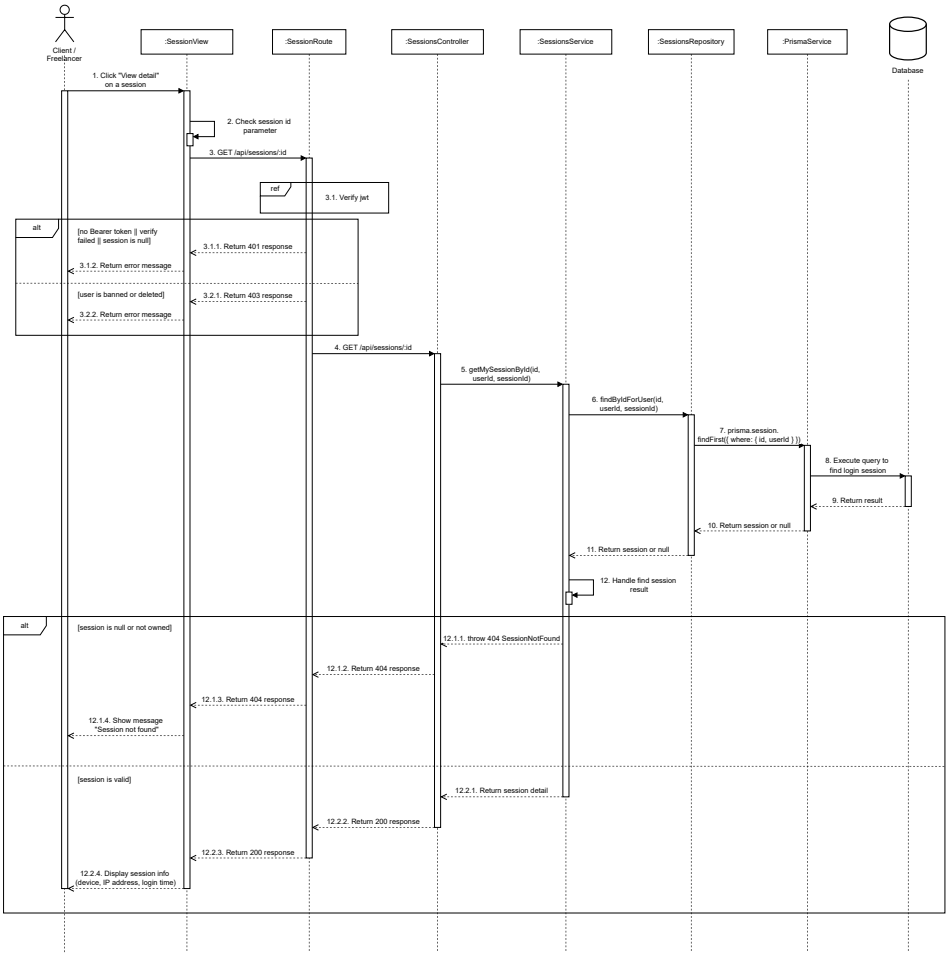

Figure 99: Sequence diagram of UC-16.02 View Session Detail

3.12.0.4 UC-16.03 – Revoke Session

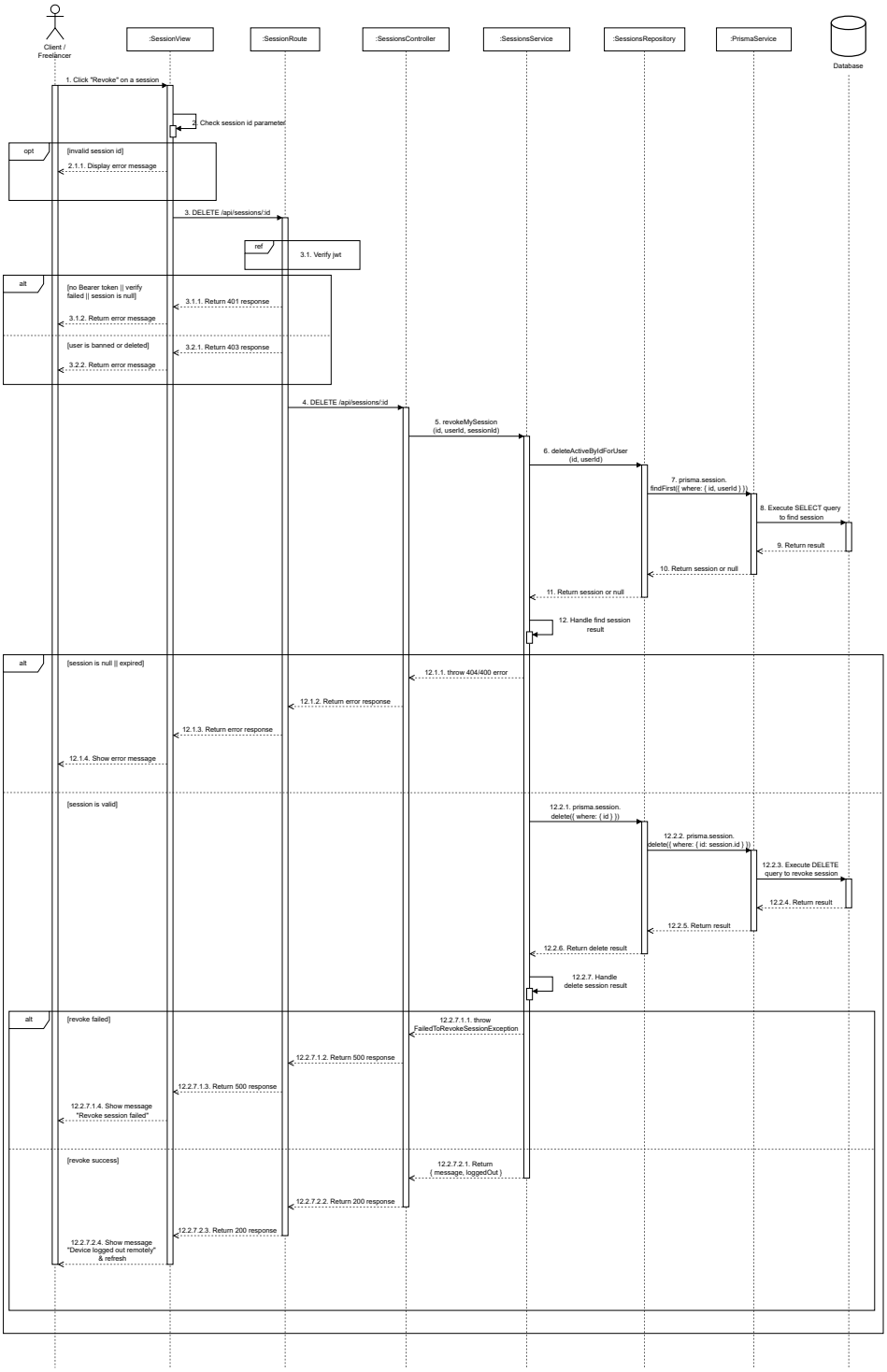

Figure 100: Sequence diagram of UC-16.03 Revoke Session

3.13 Forum Category Management

Forum Category Management covers the administrator-side category catalogue and the public category views used by the forum. Deletion is a soft delete, so a category can be restored later.

3.13.0.1 Class Diagram

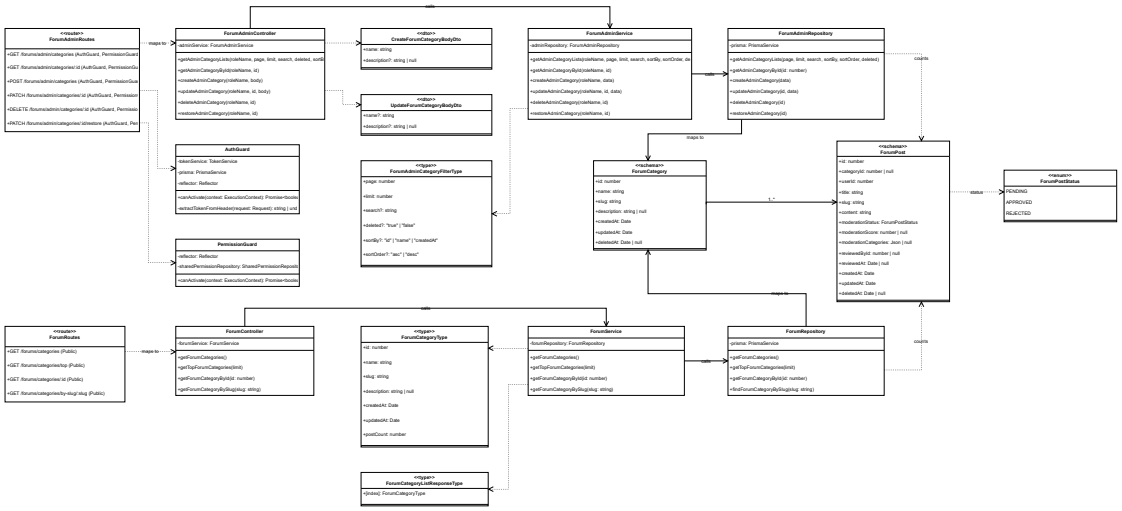

Figure 101: Class diagram of Forum Category Management

3.13.0.2 UC-17.01 – Create Forum Category

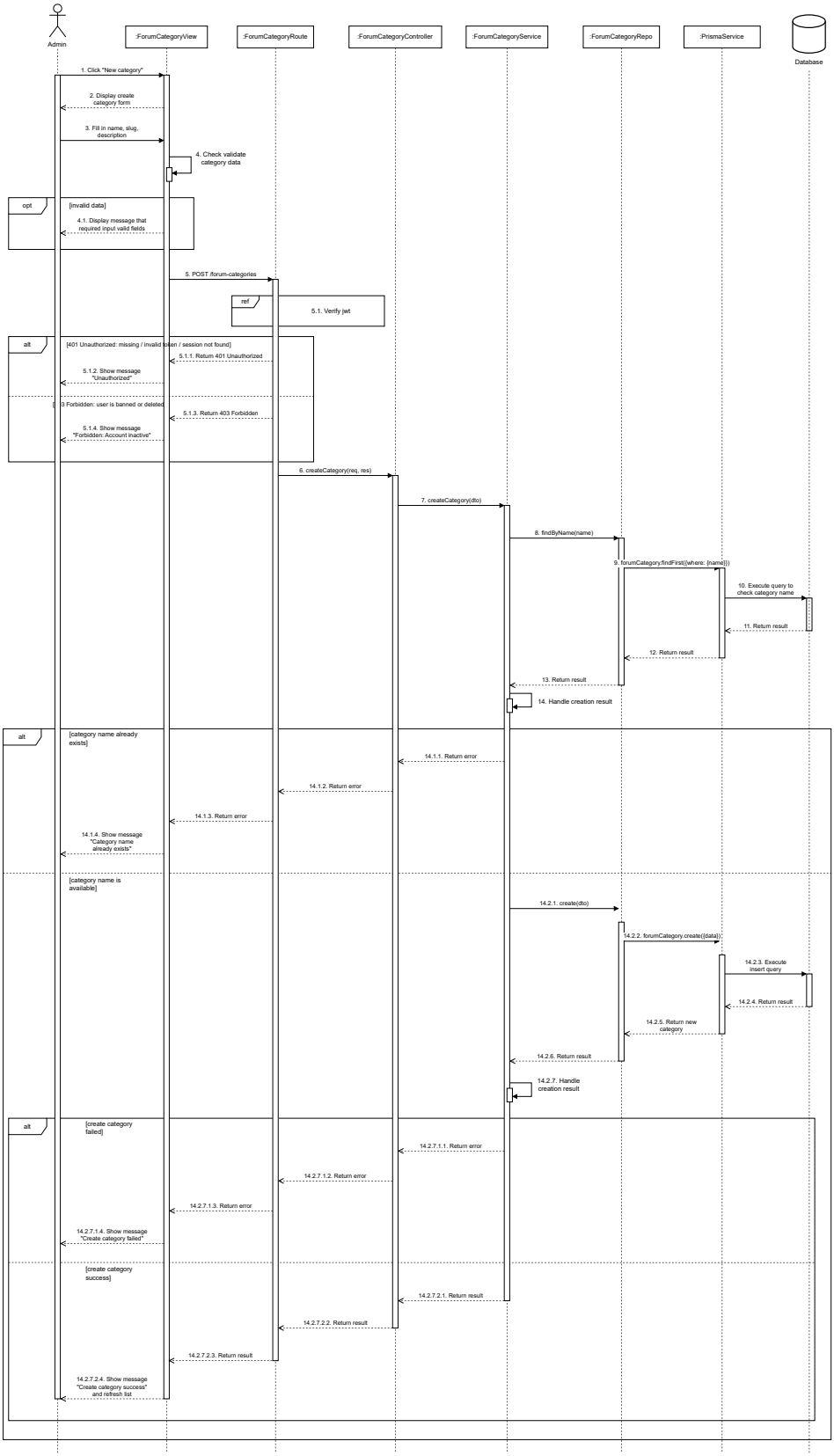

Figure 102: Sequence diagram of UC-17.01 Create Forum Category

3.13.0.3 UC-17.02 – Update Forum Category

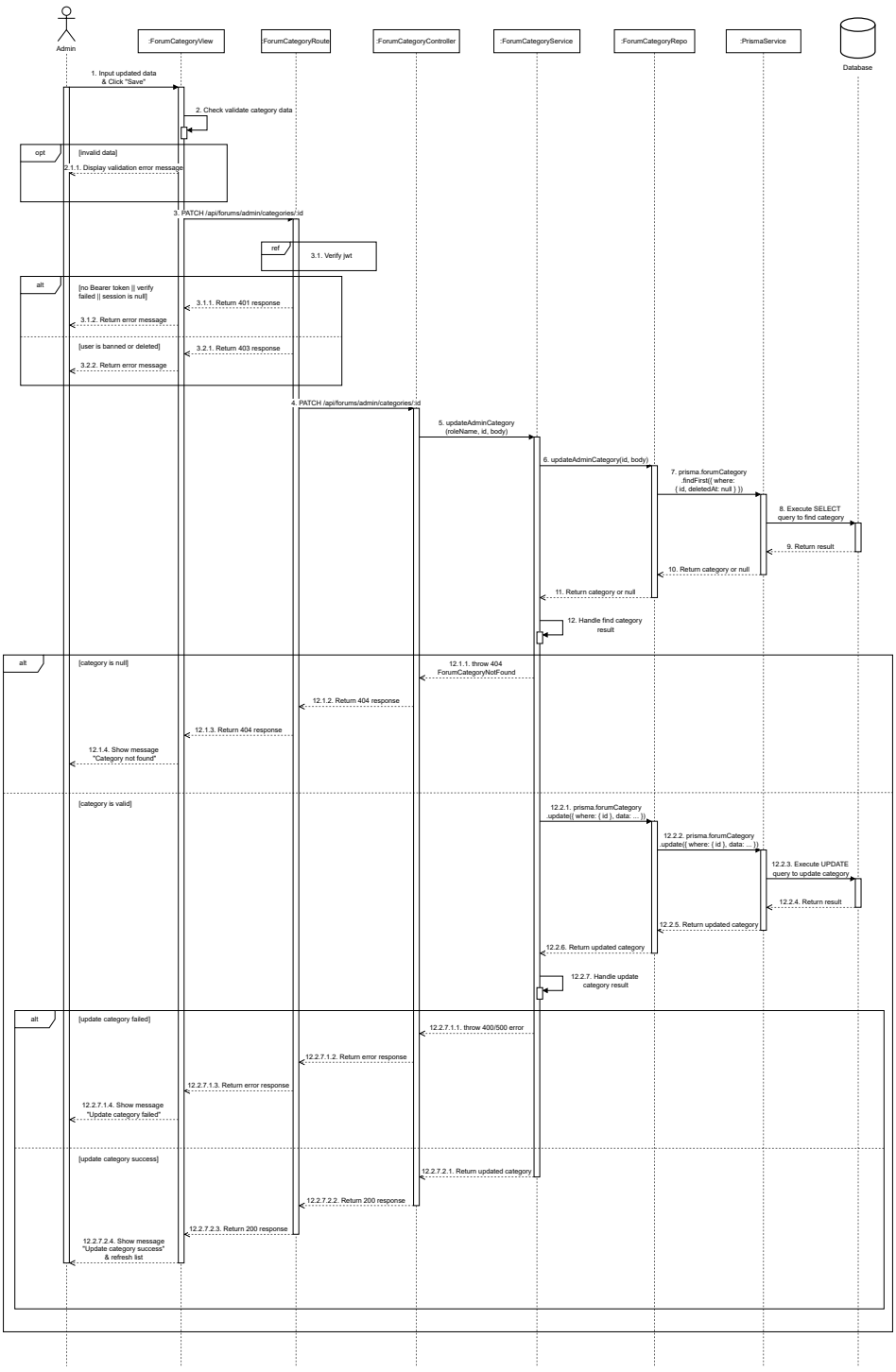

Figure 103: Sequence diagram of UC-17.02 Update Forum Category

3.13.0.4 UC-17.03 – Delete Forum Category

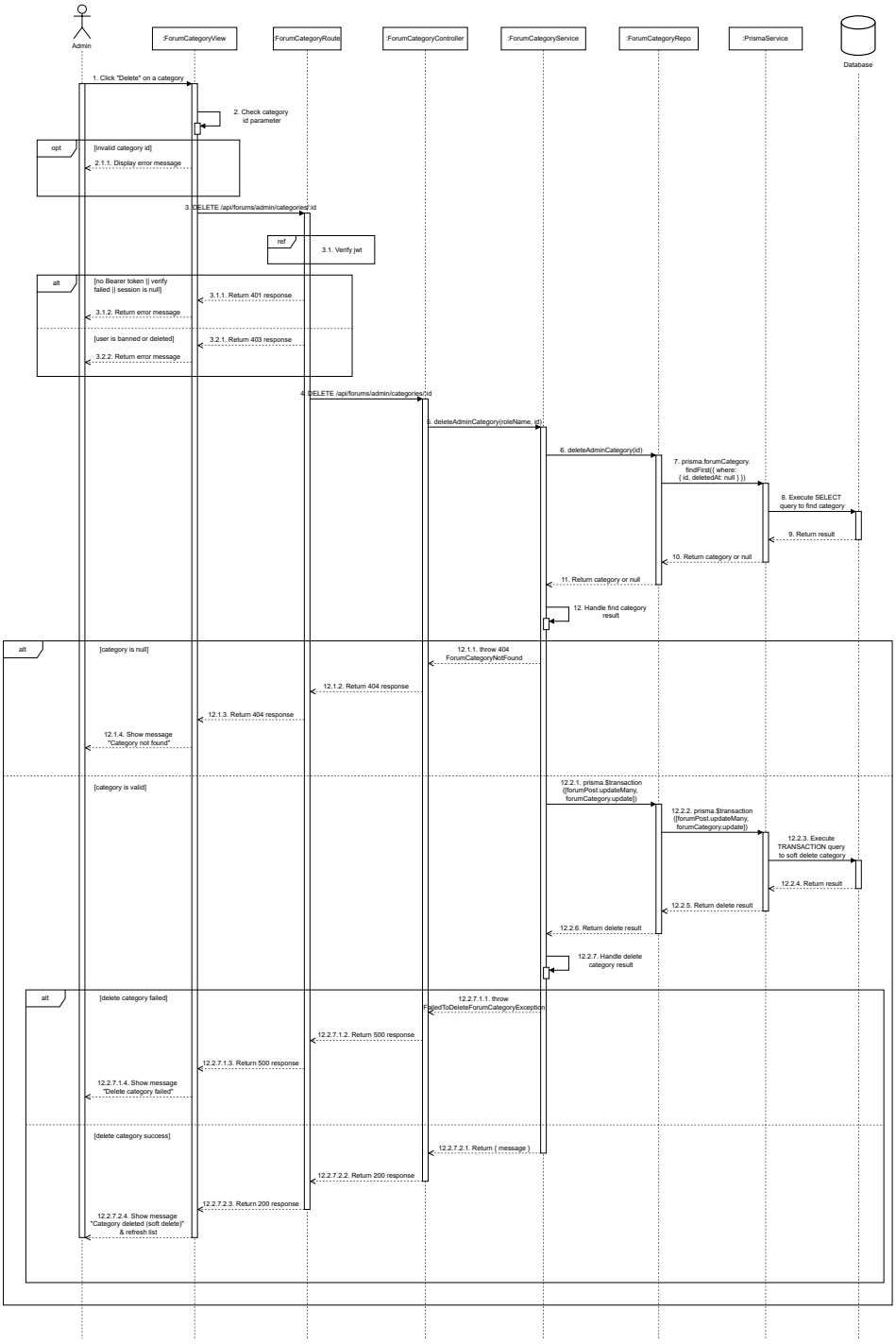

Figure 104: Sequence diagram of UC-17.03 Delete Forum Category

3.13.0.5 UC-17.06 – Restore Forum Category

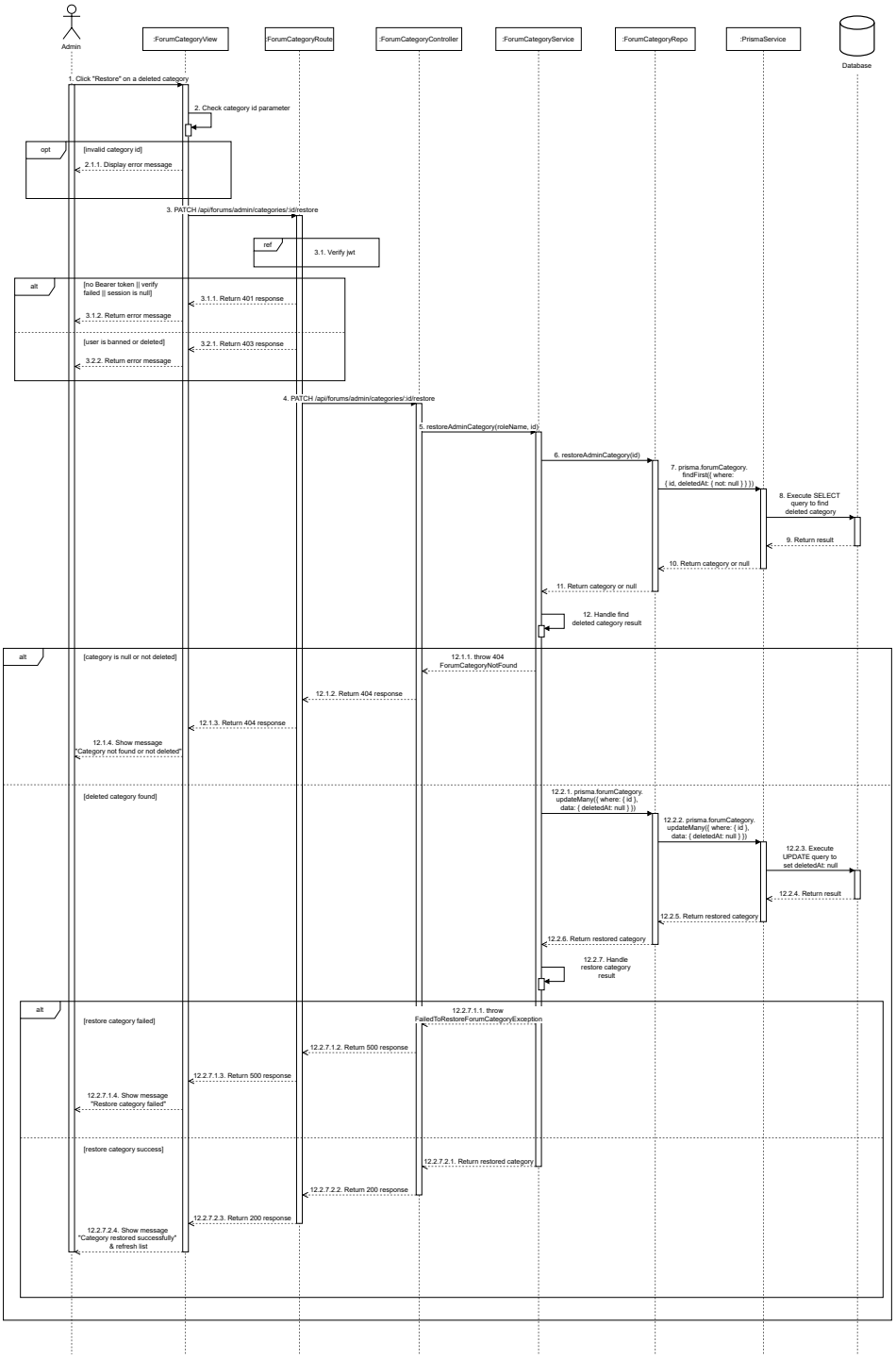

Figure 105: Sequence diagram of UC-17.06 Restore Forum Category

3.14 Manage Moderation

Manage Moderation covers the UC-34 review workflow of the administration console: listing the posts waiting for review, opening a flagged post and approving or rejecting it. A review writes the moderation status, score and reviewer back to the forum post.

3.14.0.1 Class Diagram

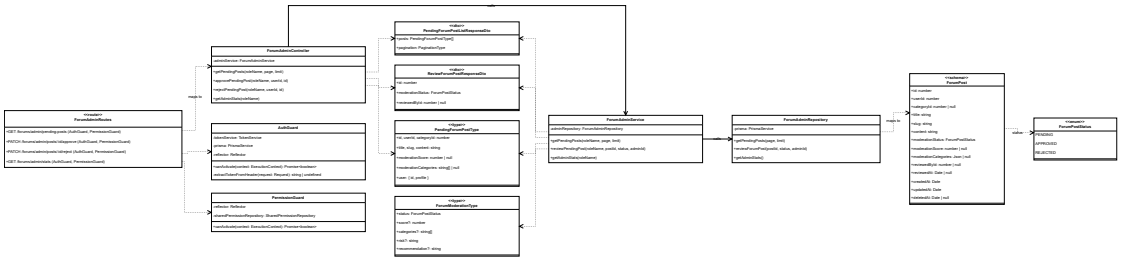

Figure 106: Class diagram of Manage Moderation

3.14.0.2 UC-34.01 – View Moderation Queue

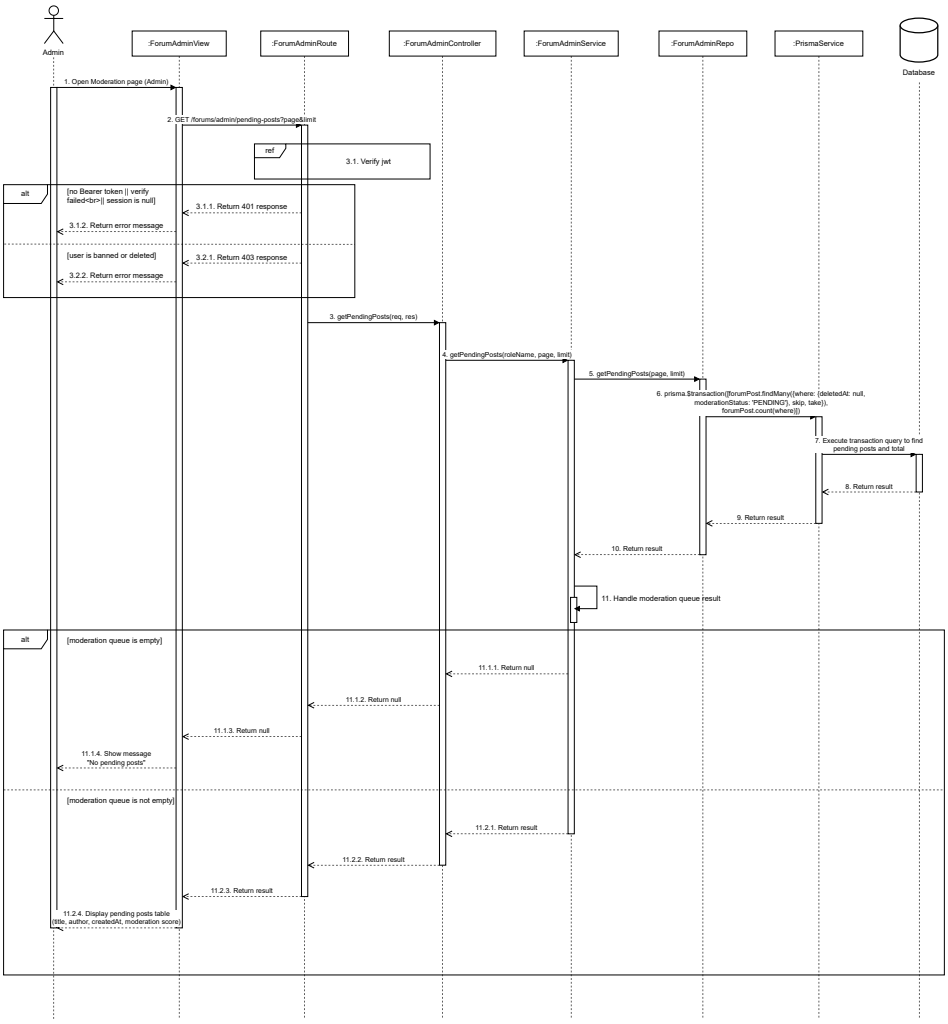

Figure 107: Sequence diagram of UC-34.01 View Moderation Queue

3.14.0.3 UC-34.02 – View Moderation Item Detail

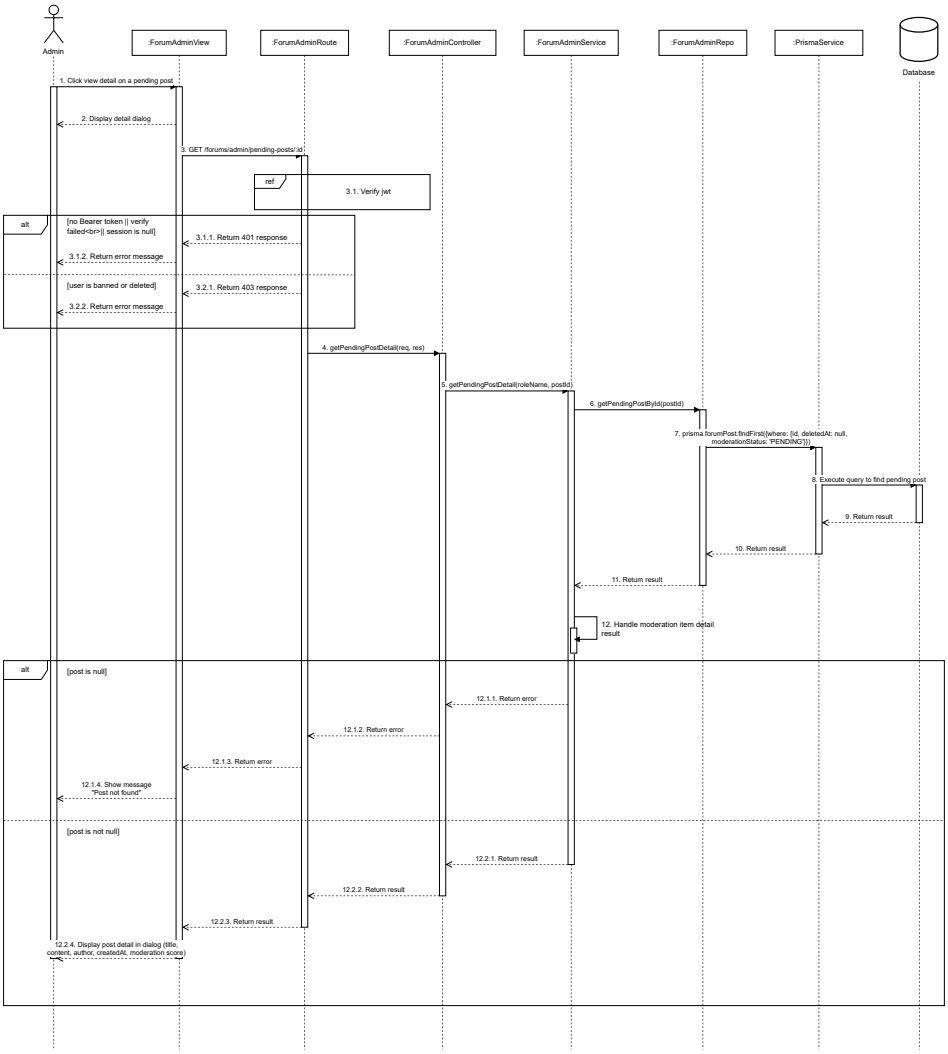

Figure 108: Sequence diagram of UC-34.02 View Moderation Item Detail

3.14.0.4 UC-34.03 – Resolve Moderation Item

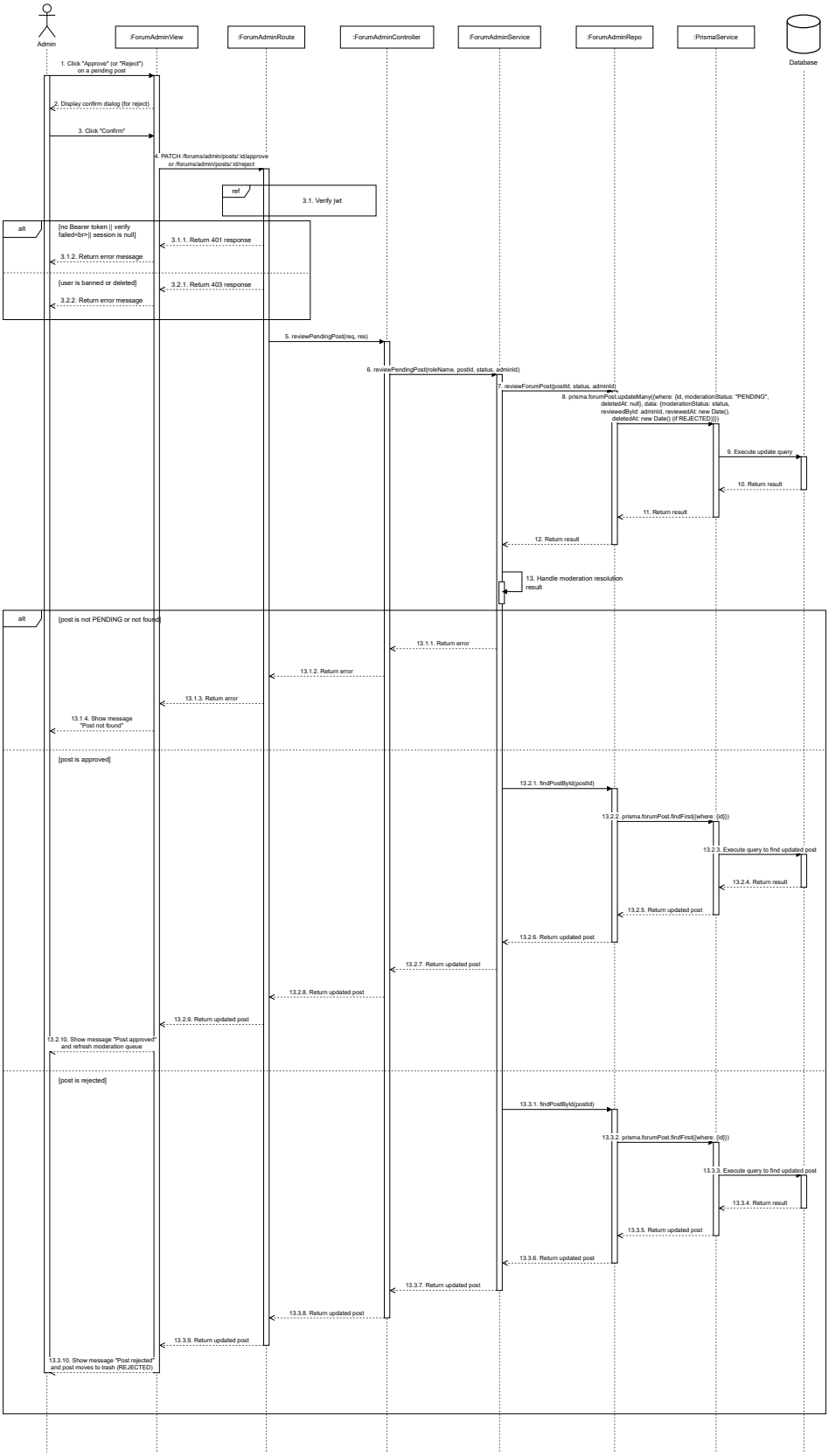

Figure 109: Sequence diagram of UC-34.03 Resolve Moderation Item

3.15 Manage ID Verification

Manage ID Verification covers the UC-35 identity review workflow: listing the submitted identity documents, opening a request with its uploaded file, and approving or rejecting the request. A decision updates the verification status and the reviewer of the identity document.

3.15.0.1 Class Diagram

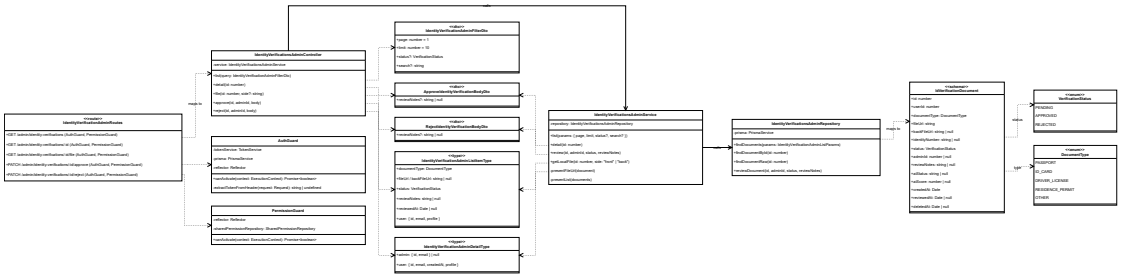

Figure 110: Class diagram of Manage ID Verification

3.15.0.2 UC-35.01 – View ID Verification Request List

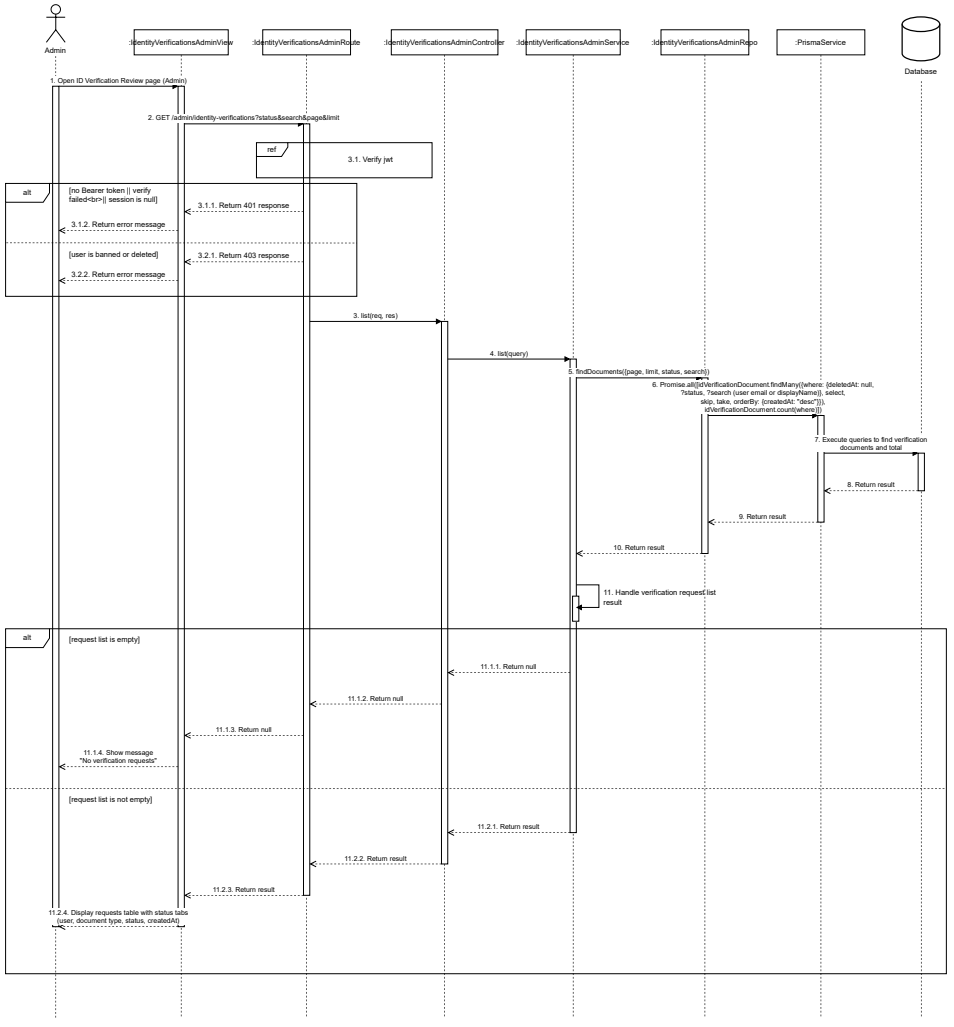

Figure 111: Sequence diagram of UC-35.01 View ID Verification Request List

3.15.0.3 UC-35.02 – View ID Verification Request Detail

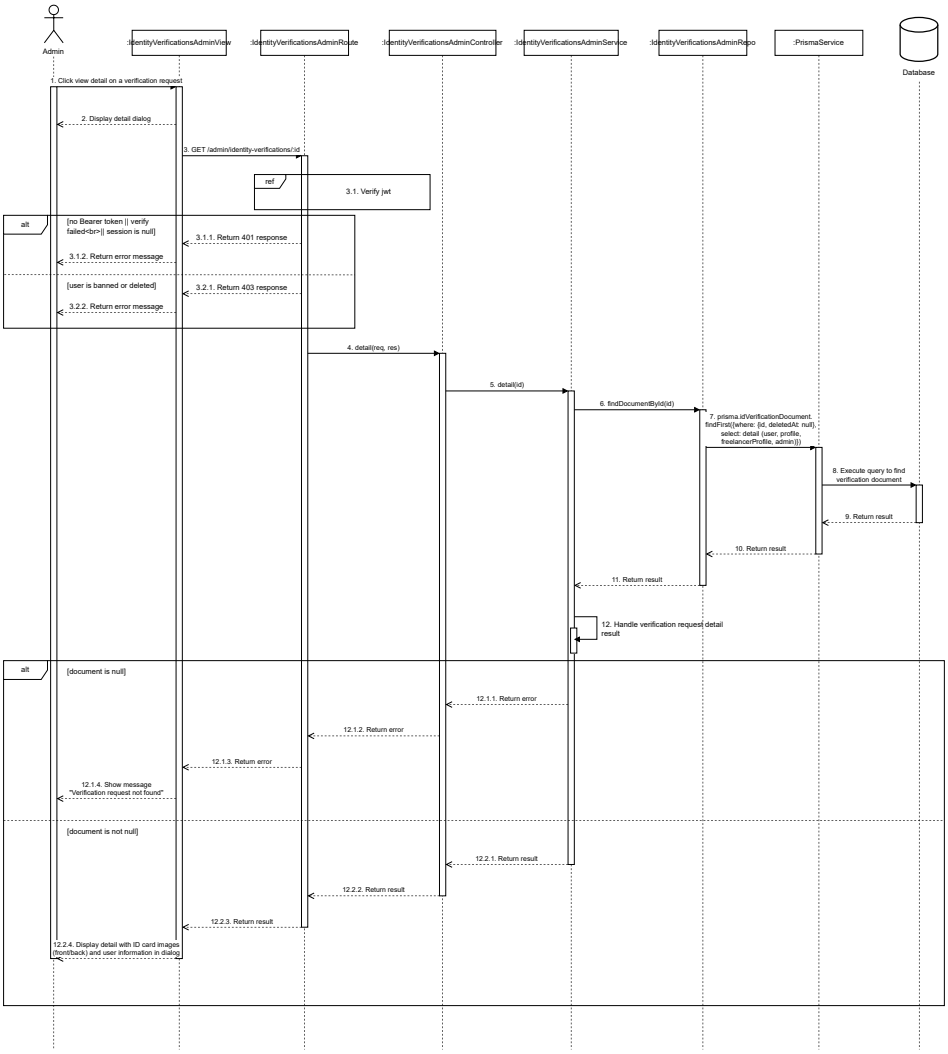

Figure 112: Sequence diagram of UC-35.02 View ID Verification Request Detail

3.15.0.4 UC-35.03 – Approve ID Verification

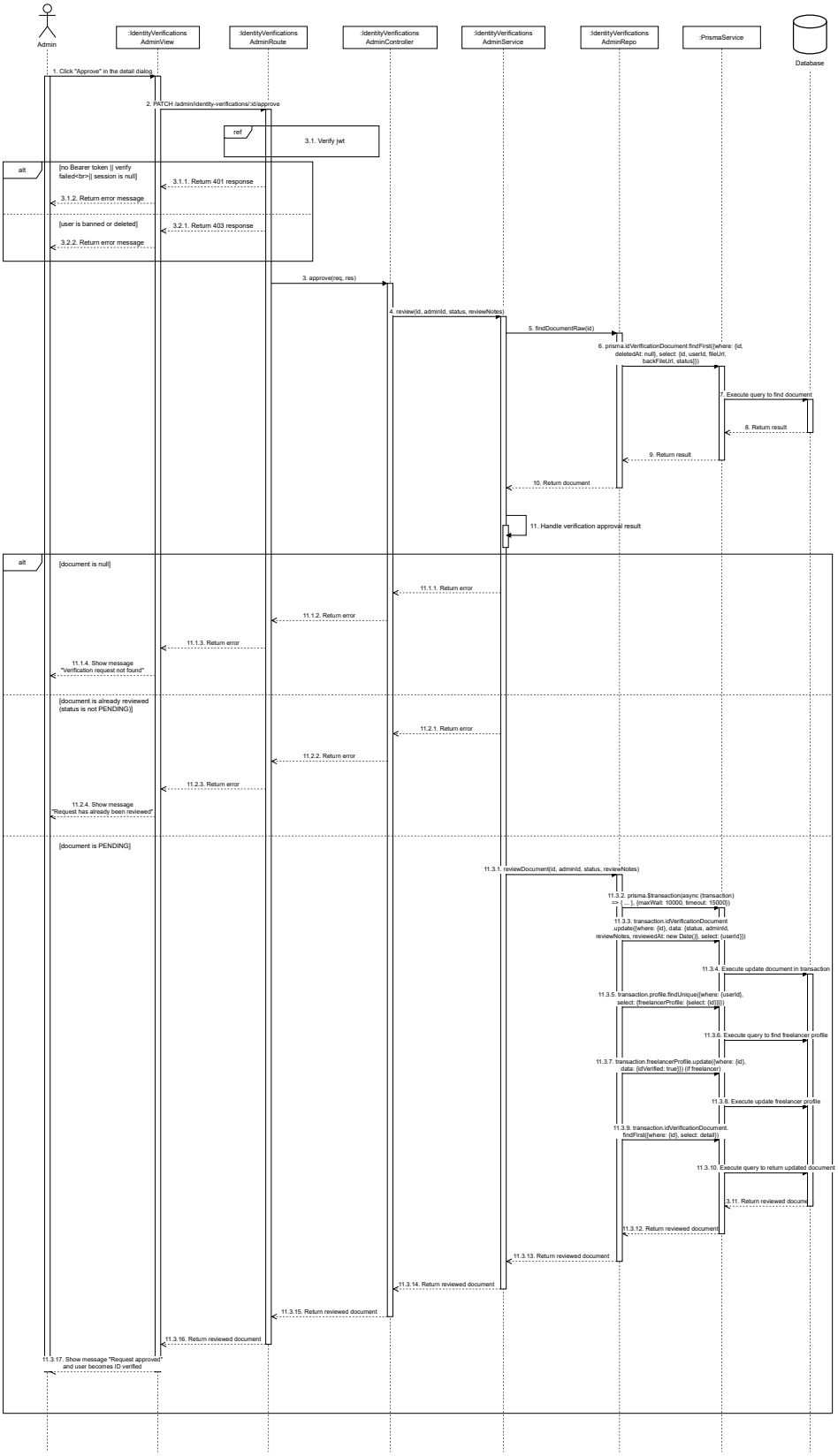

Figure 113: Sequence diagram of UC-35.03 Approve ID Verification

3.15.0.5 UC-35.04 – Reject ID Verification

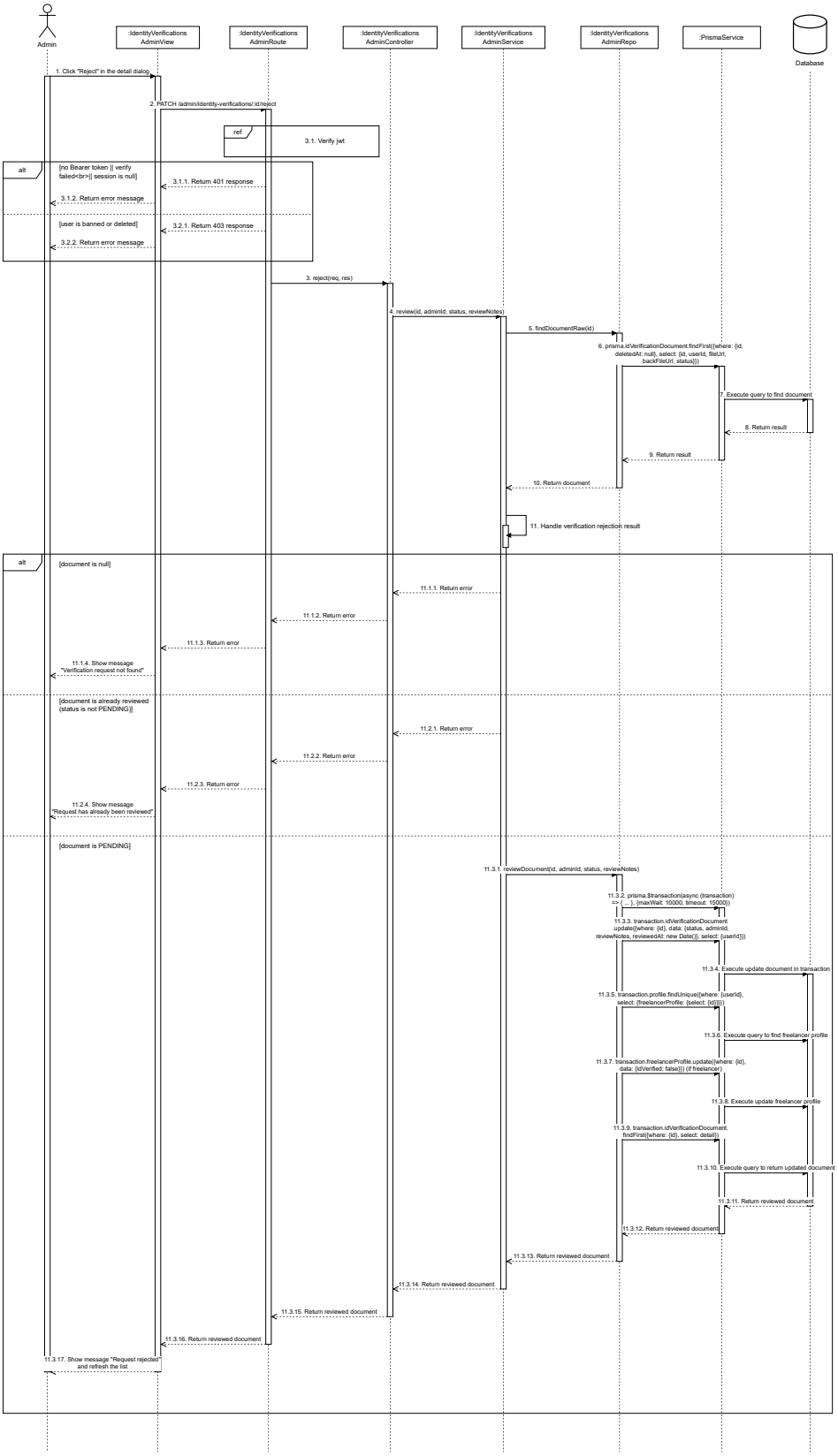

Figure 114: Sequence diagram of UC-35.04 Reject ID Verification

3.16 Manage Forum Trash

Manage Forum Trash covers the UC-36 recovery workflow: listing the soft-deleted posts and comments kept for review, opening one deleted item, and restoring it so that it becomes visible in the forum again.

3.16.0.1 Class Diagram

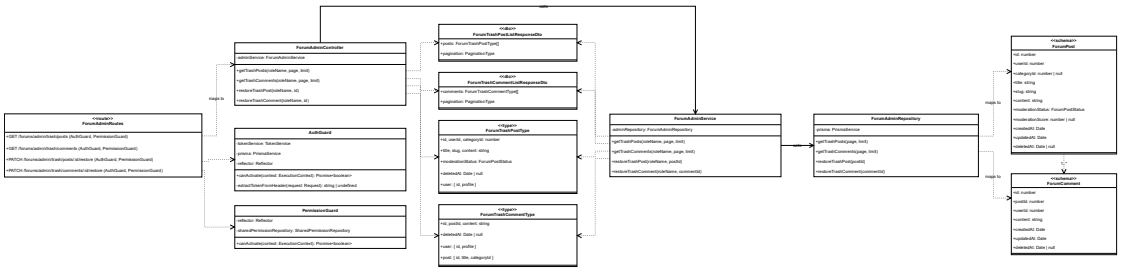

Figure 115: Class diagram of Manage Forum Trash

3.16.0.2 UC-36.01 – View Deleted Posts/Comments List

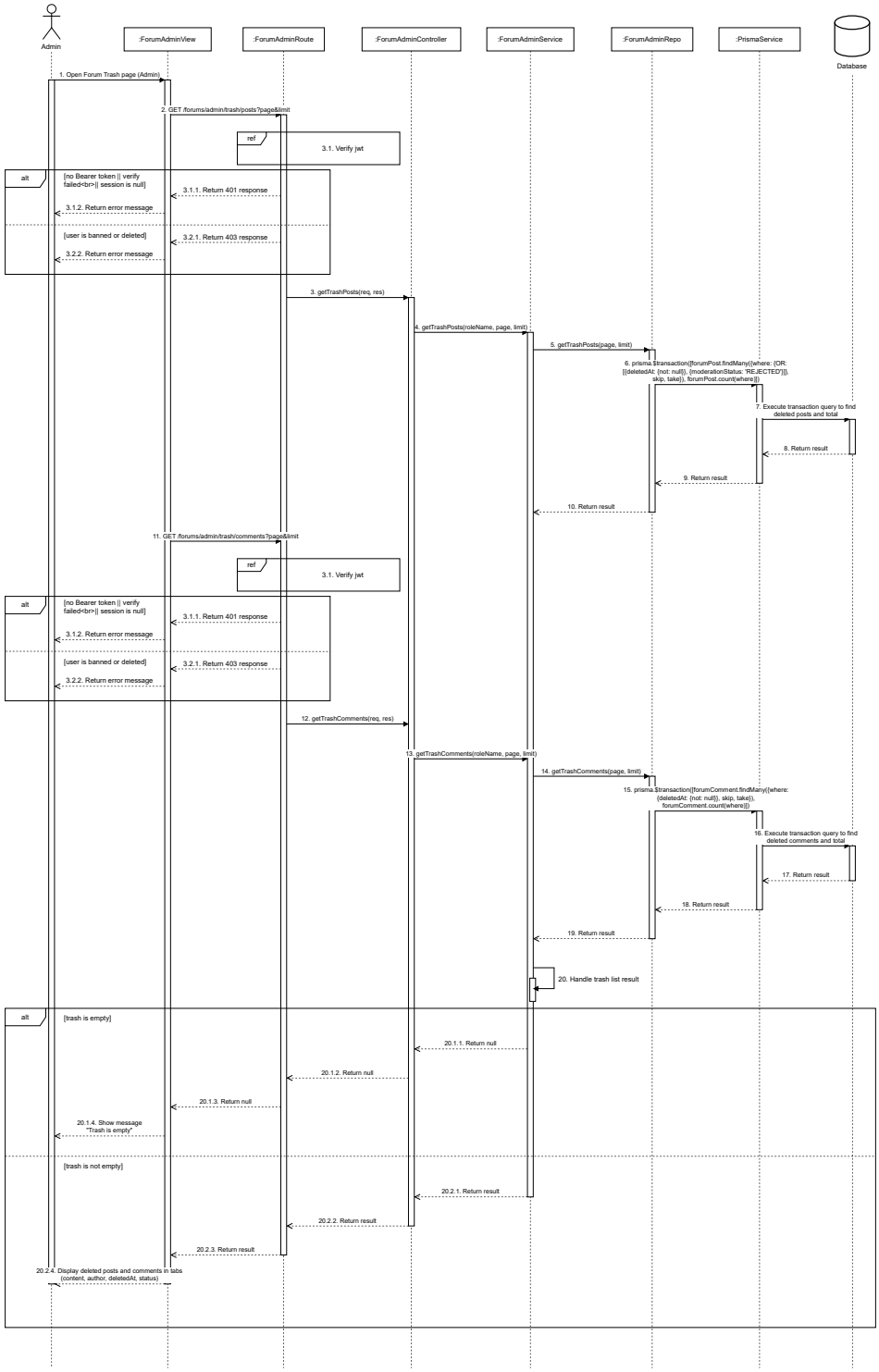

Figure 116: Sequence diagram of UC-36.01 View Deleted Posts/Comments List

3.16.0.3 UC-36.02 – View Deleted Posts/Comments Detail

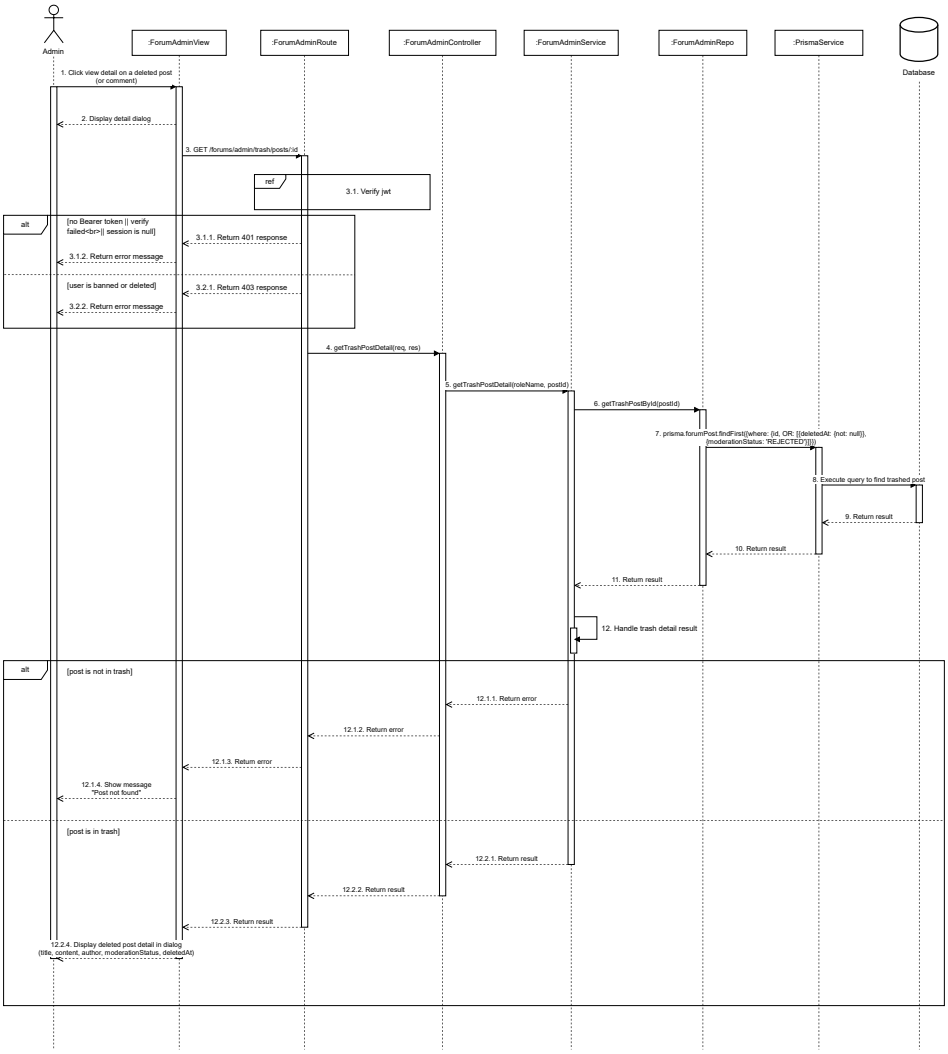

Figure 117: Sequence diagram of UC-36.02 View Deleted Posts/Comments Detail

3.16.0.4 UC-36.03 – Restore Deleted Posts/Comments

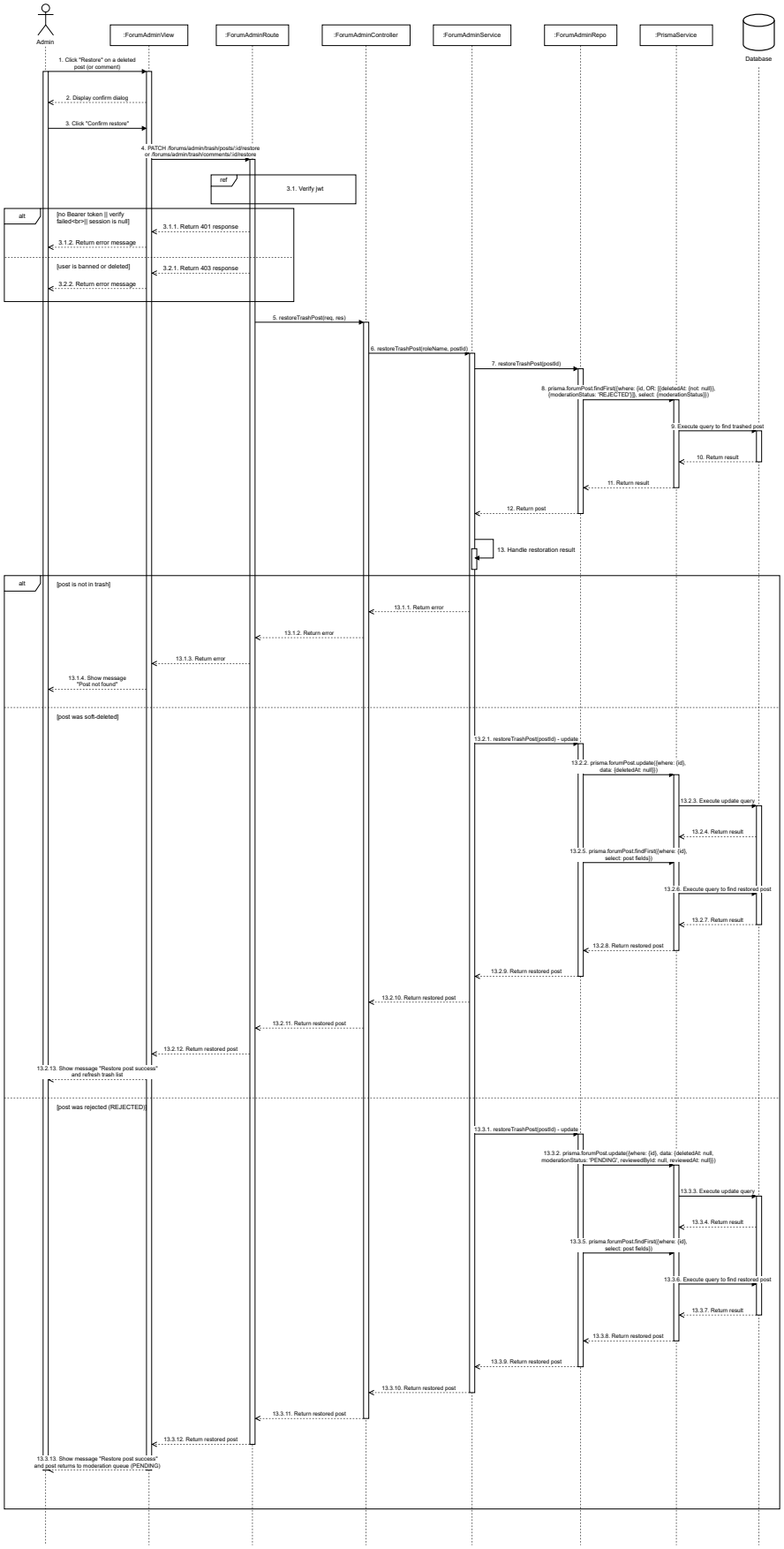

Figure 118: Sequence diagram of UC-36.03 Restore Deleted Posts/Comments

3.17 Proposal Management

Proposal Management covers the full proposal workflow between freelancers and clients. Freelancers can submit proposals with cover letters, bid amounts, and delivery estimates, view their submitted proposals, modify pending proposals, or withdraw them. Clients can view all proposals submitted for their jobs, inspect detailed freelancer bids, and accept or reject proposals to initiate contracting.

3.17.0.1 Class Diagram

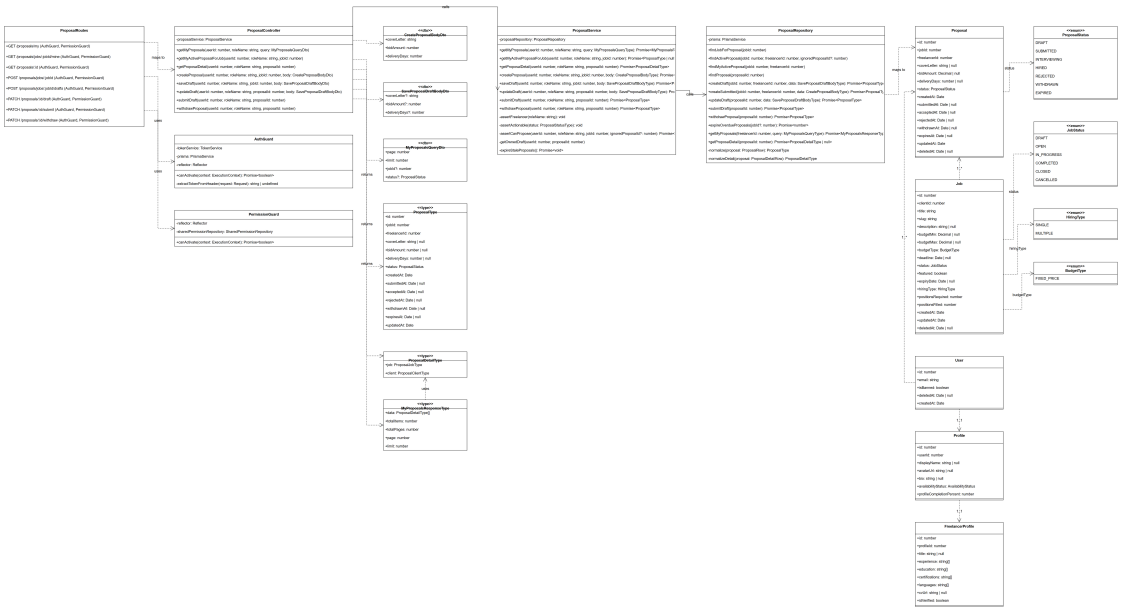

Figure 119: Class diagram of Freelancer Proposal Management

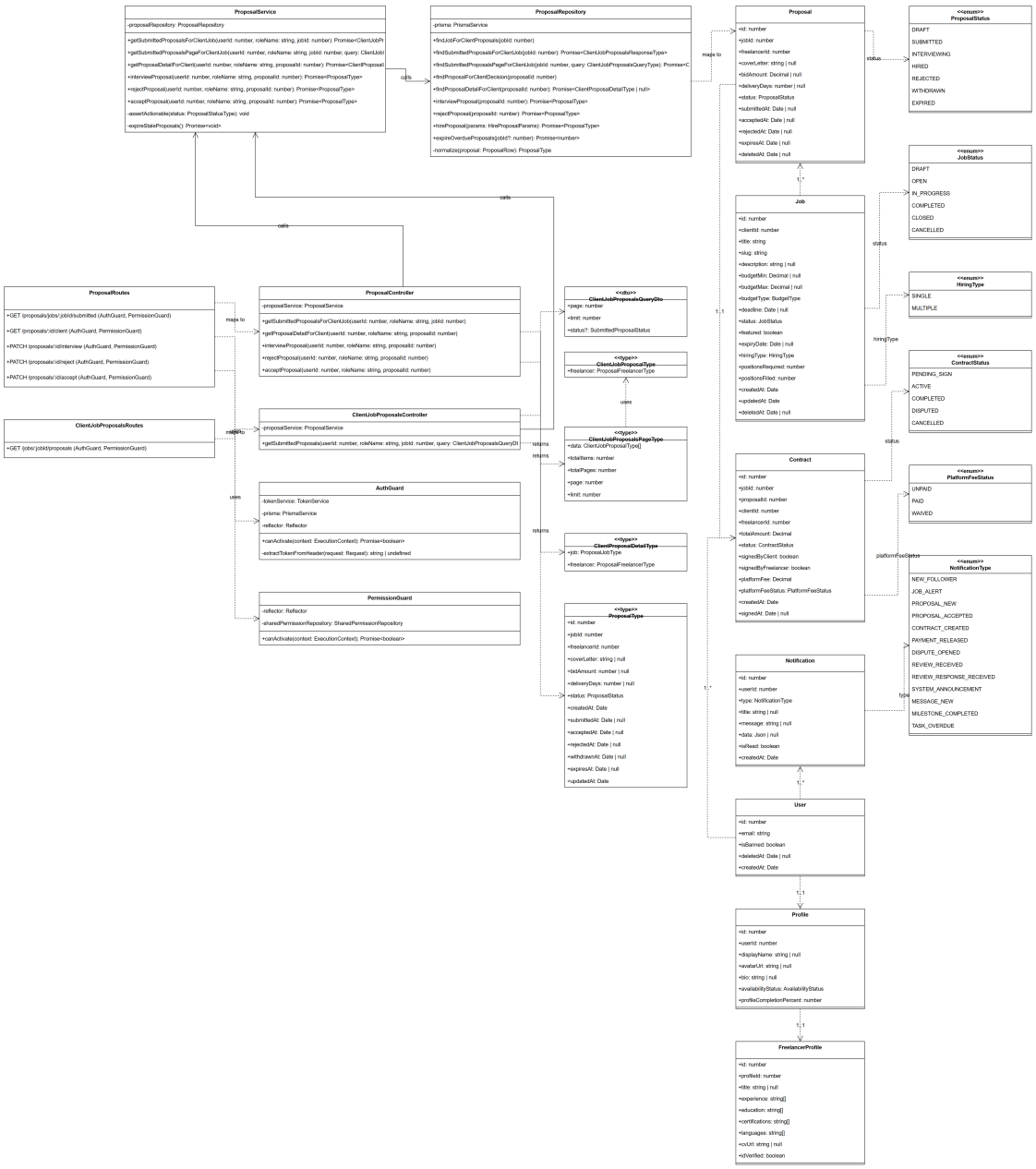

Figure 120: Class diagram of Client Proposal Management

3.17.0.2 UC-32.01 – View My Proposal List

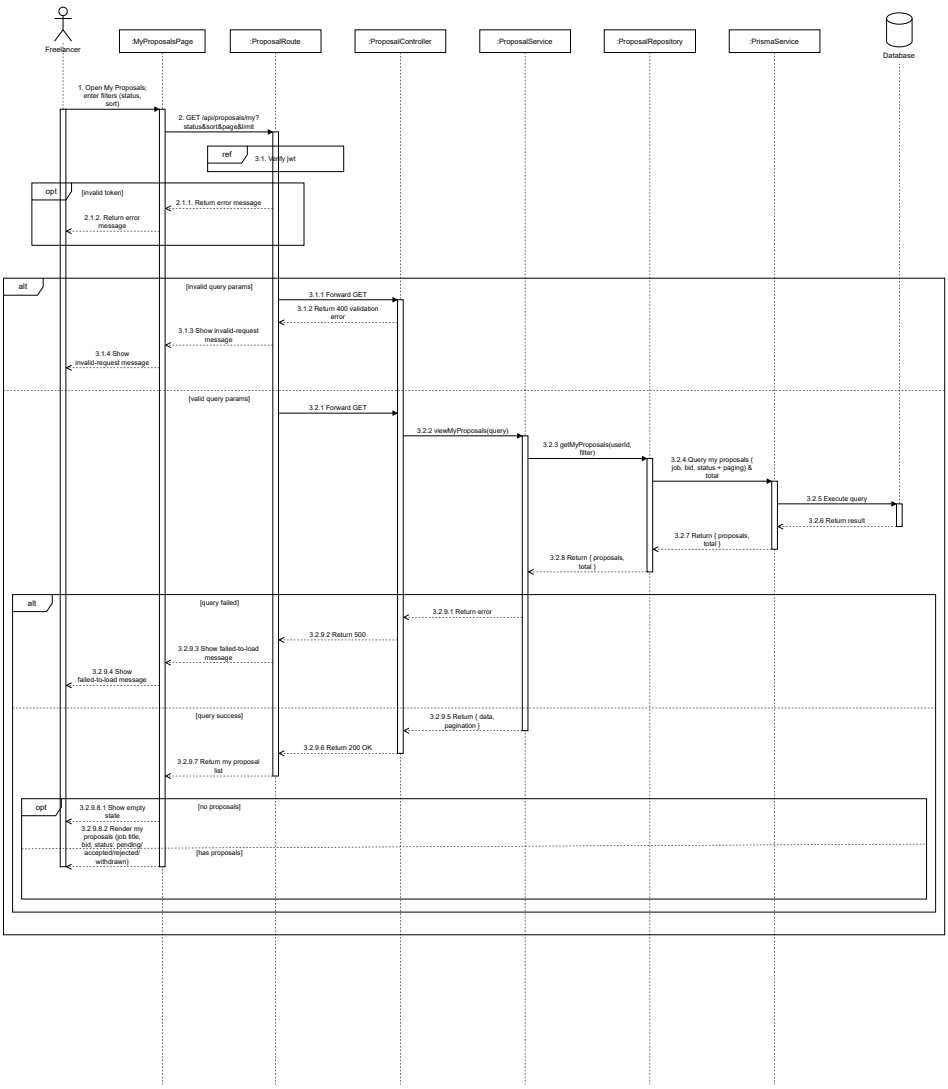

Figure 121: Sequence diagram of UC-32.01 View My Proposal List

3.17.0.3 UC-32.02 – View My Proposal Detail

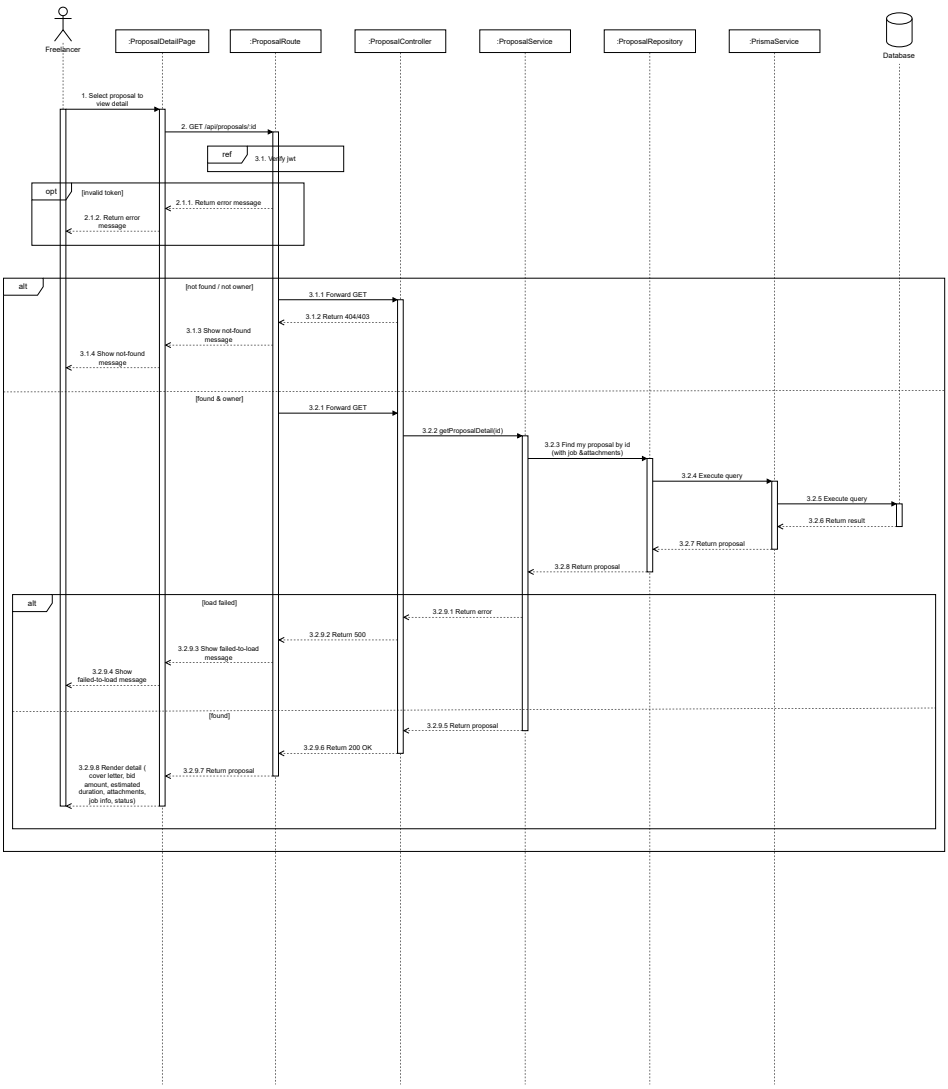

Figure 122: Sequence diagram of UC-32.02 View My Proposal Detail

3.17.0.4 UC-32.03 – Submit Proposal

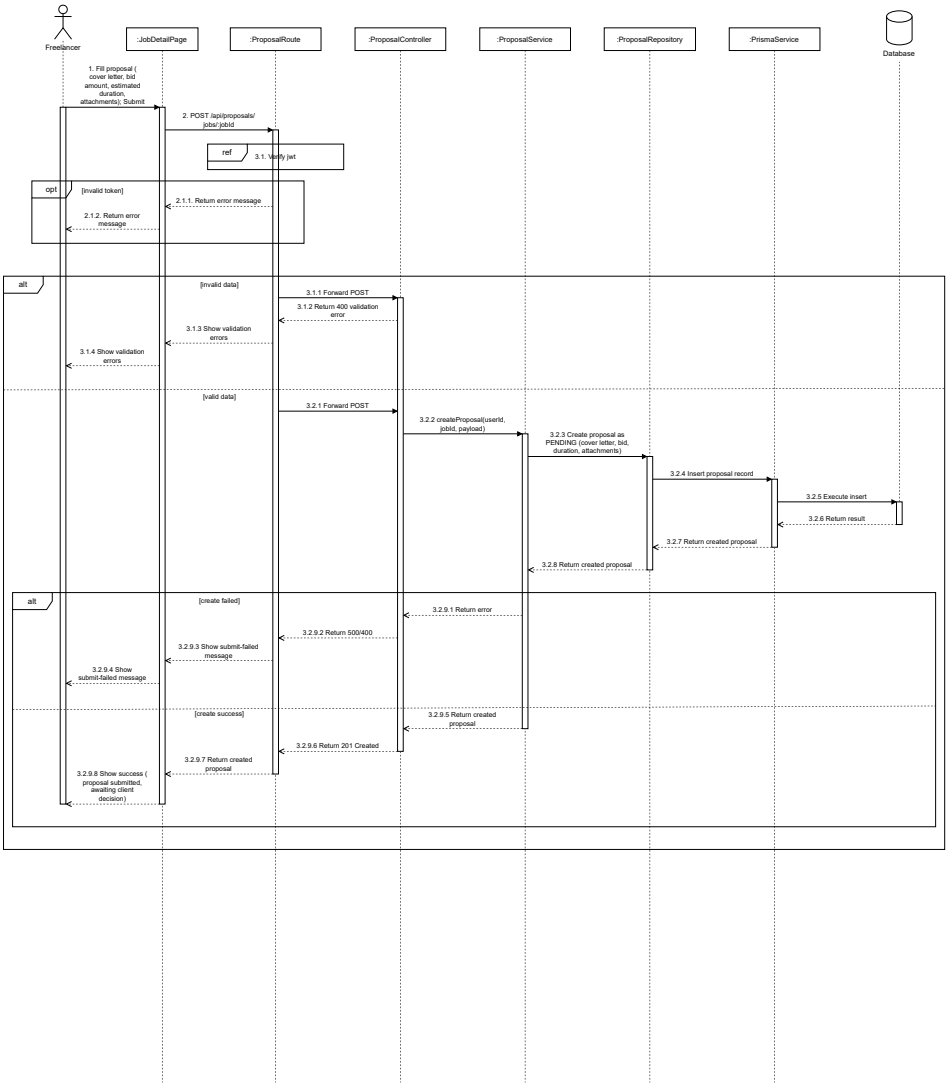

Figure 123: Sequence diagram of UC-32.03 Submit Proposal

3.17.0.5 UC-32.04 – Update Proposal

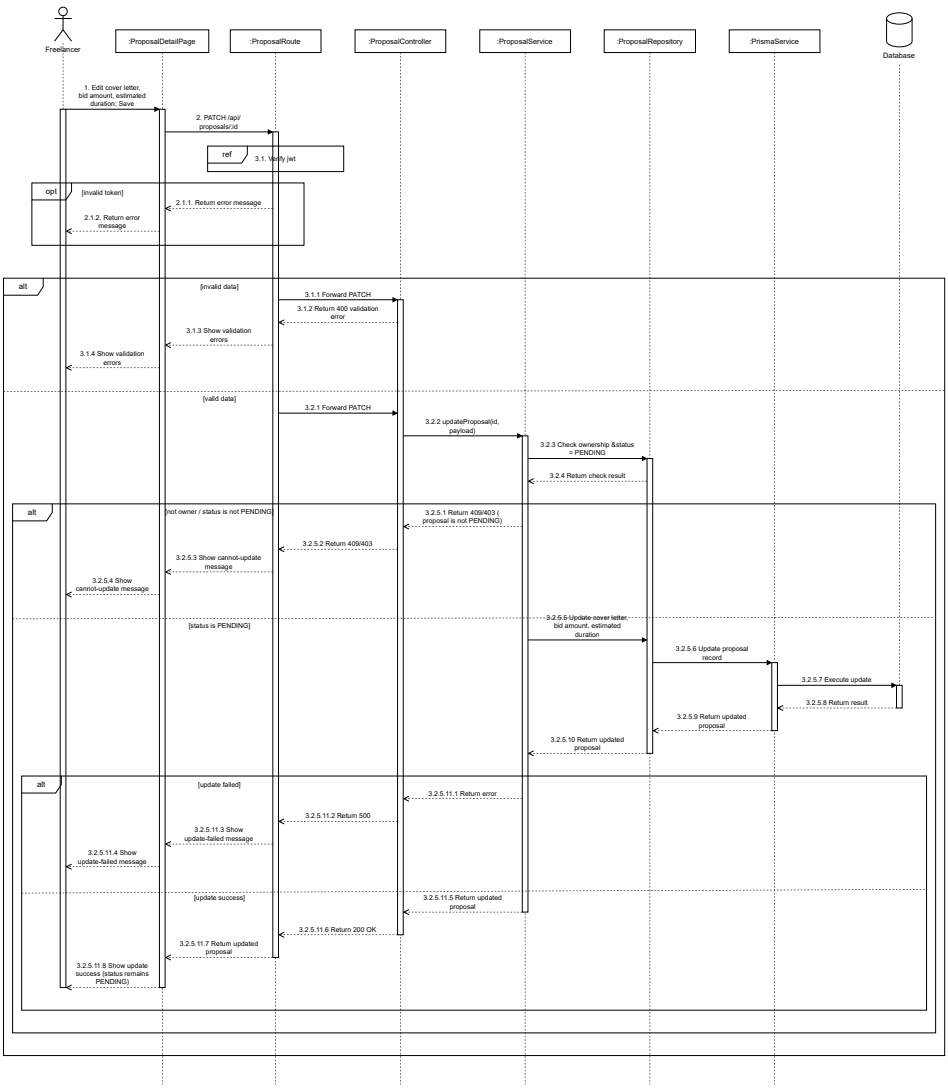

Figure 124: Sequence diagram of UC-32.04 Update Proposal

3.17.0.6 UC-32.05 – Withdraw Proposal

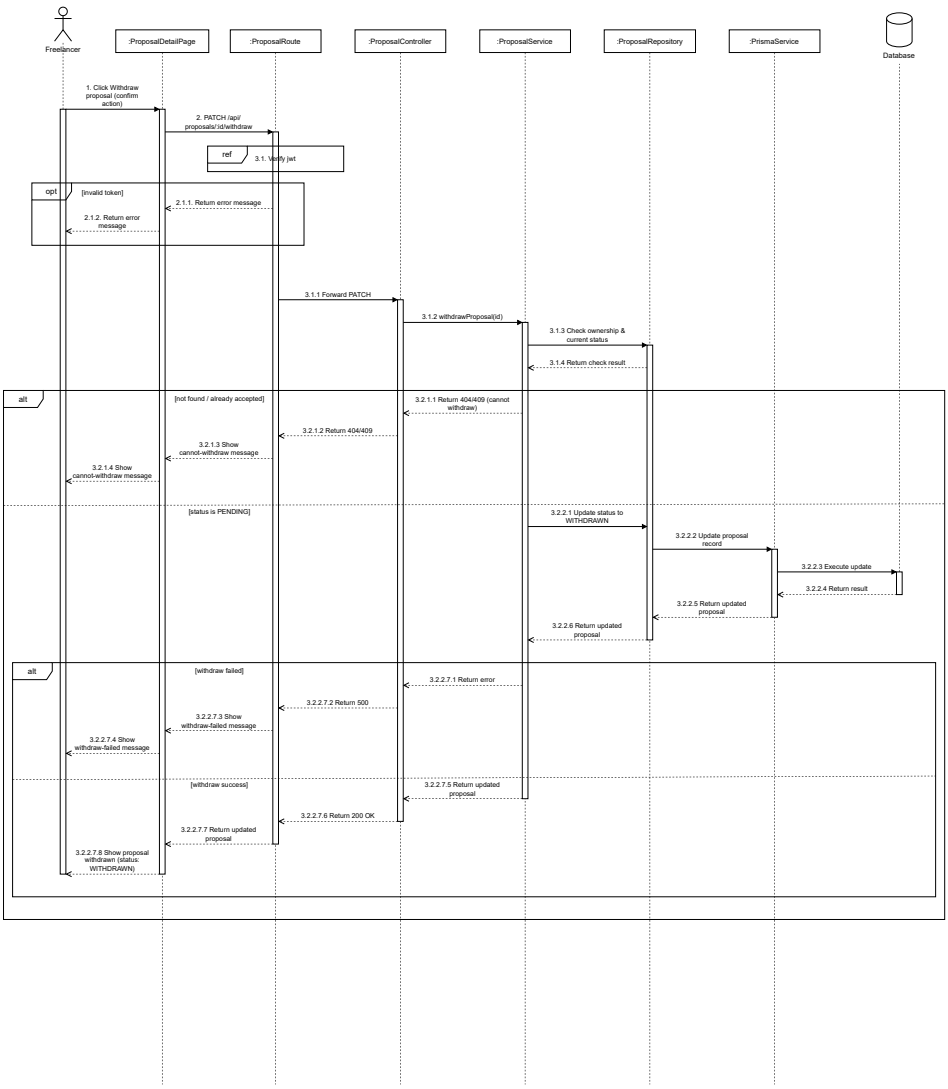

Figure 125: Sequence diagram of UC-32.05 Withdraw Proposal

3.17.0.7 UC-32.06 – Accept Proposal

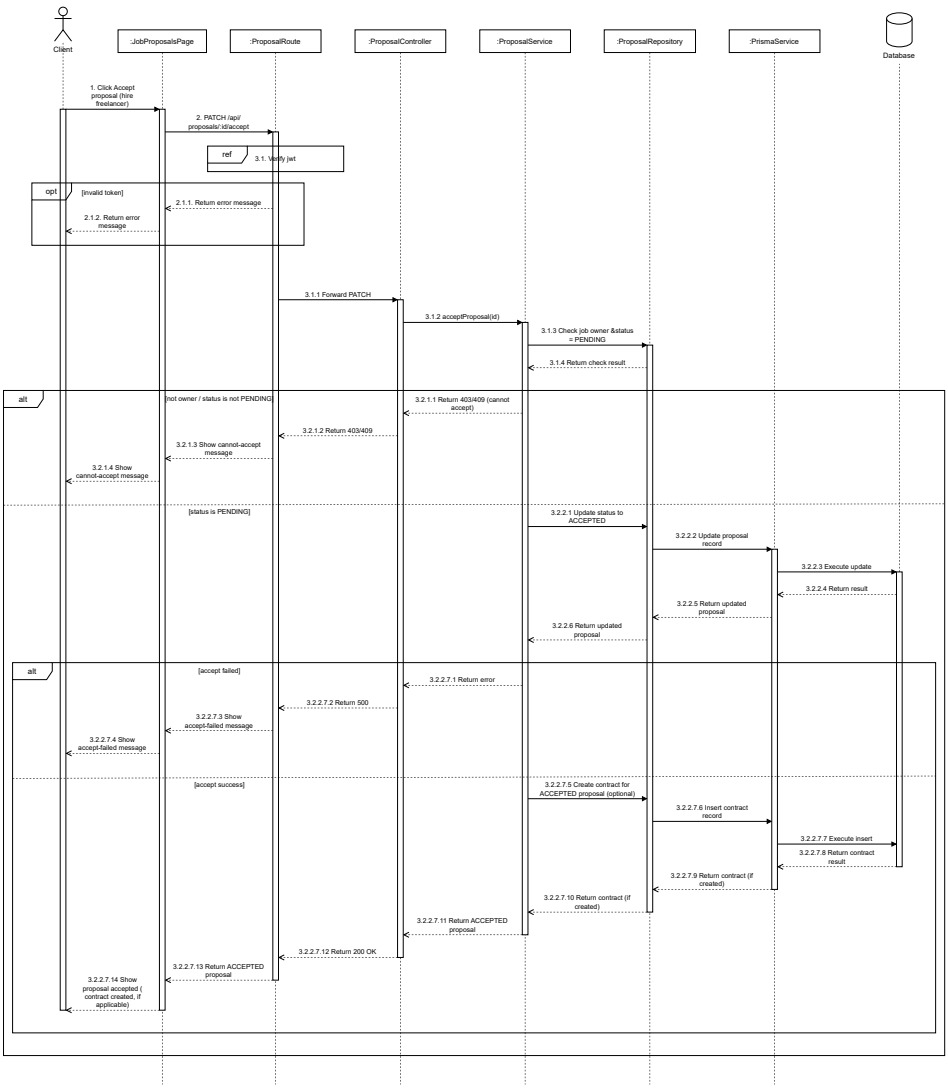

Figure 126: Sequence diagram of UC-32.06 Accept Proposal

3.17.0.8 UC-32.07 – Reject Proposal

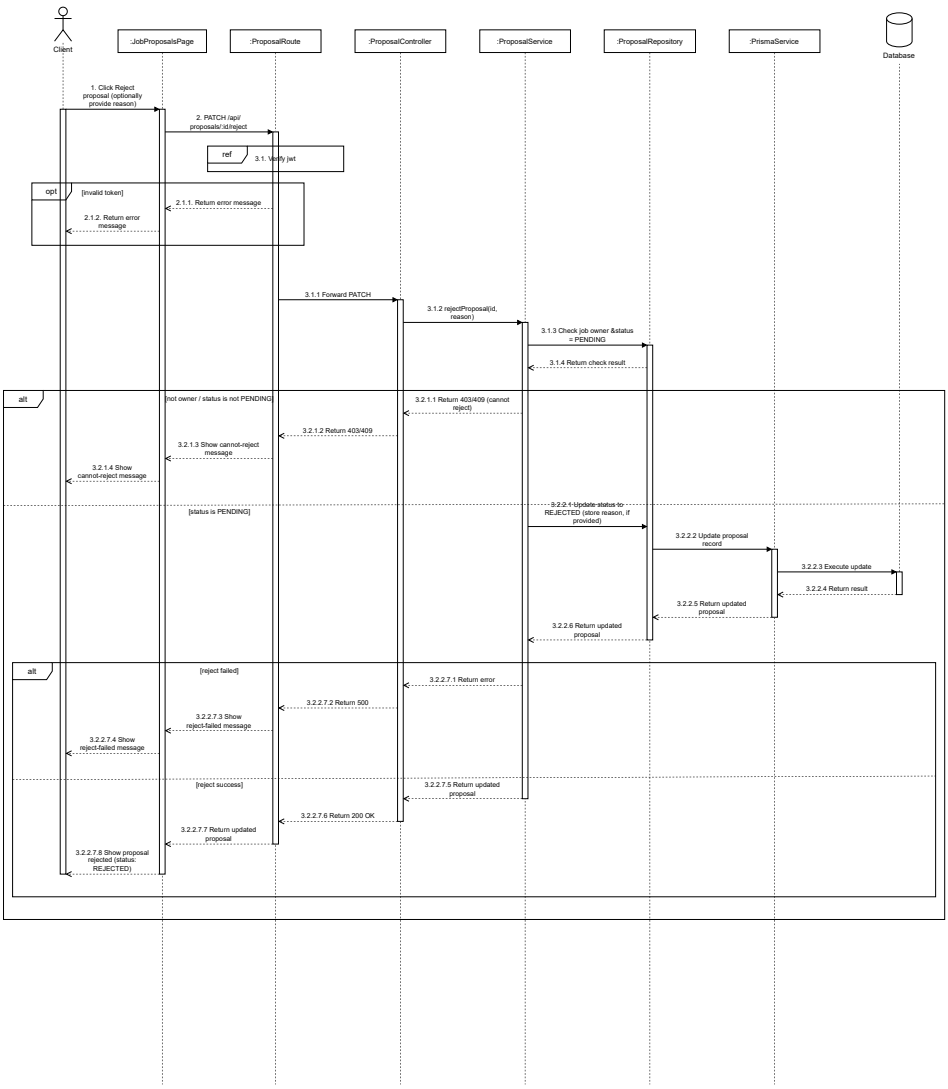

Figure 127: Sequence diagram of UC-32.07 Reject Proposal

3.17.0.9 UC-32.08 – View Job Proposal List

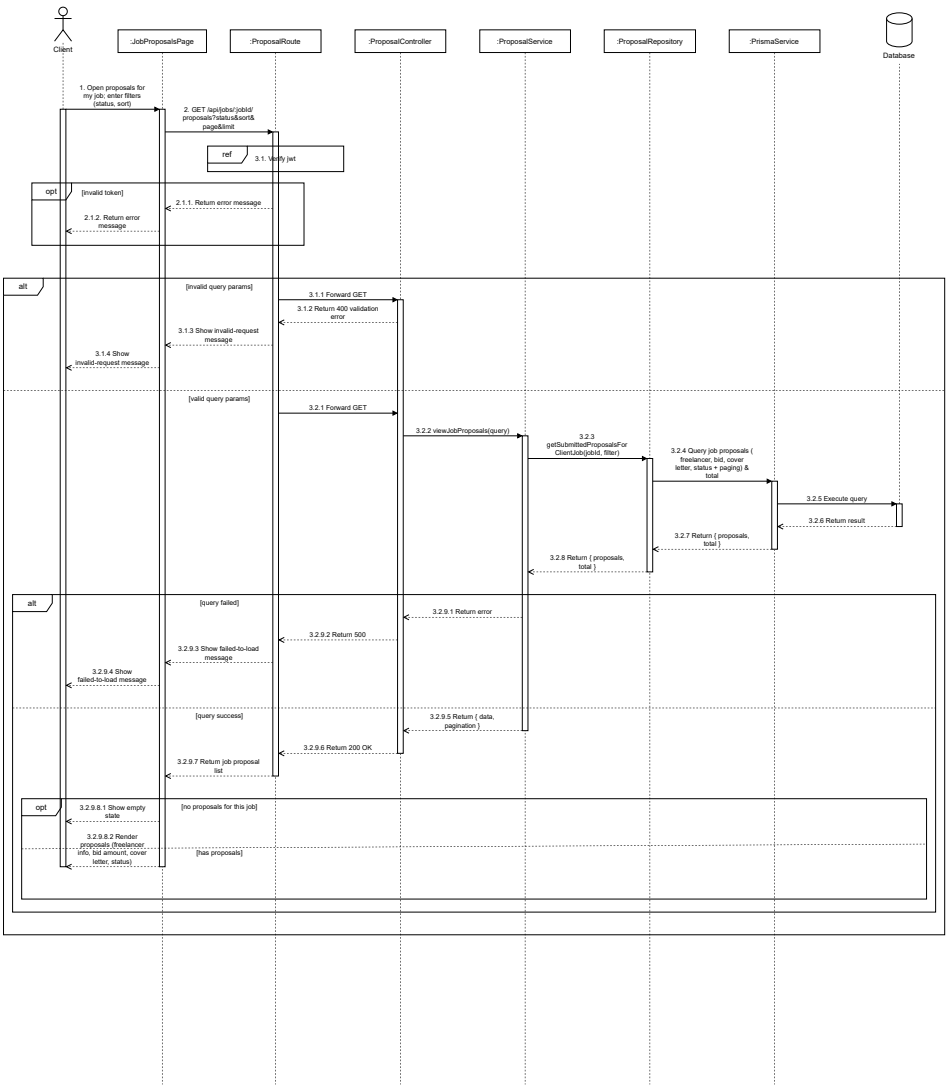

Figure 128: Sequence diagram of UC-32.08 View Job Proposal List

3.17.0.10 UC-32.09 – View Proposal Detail

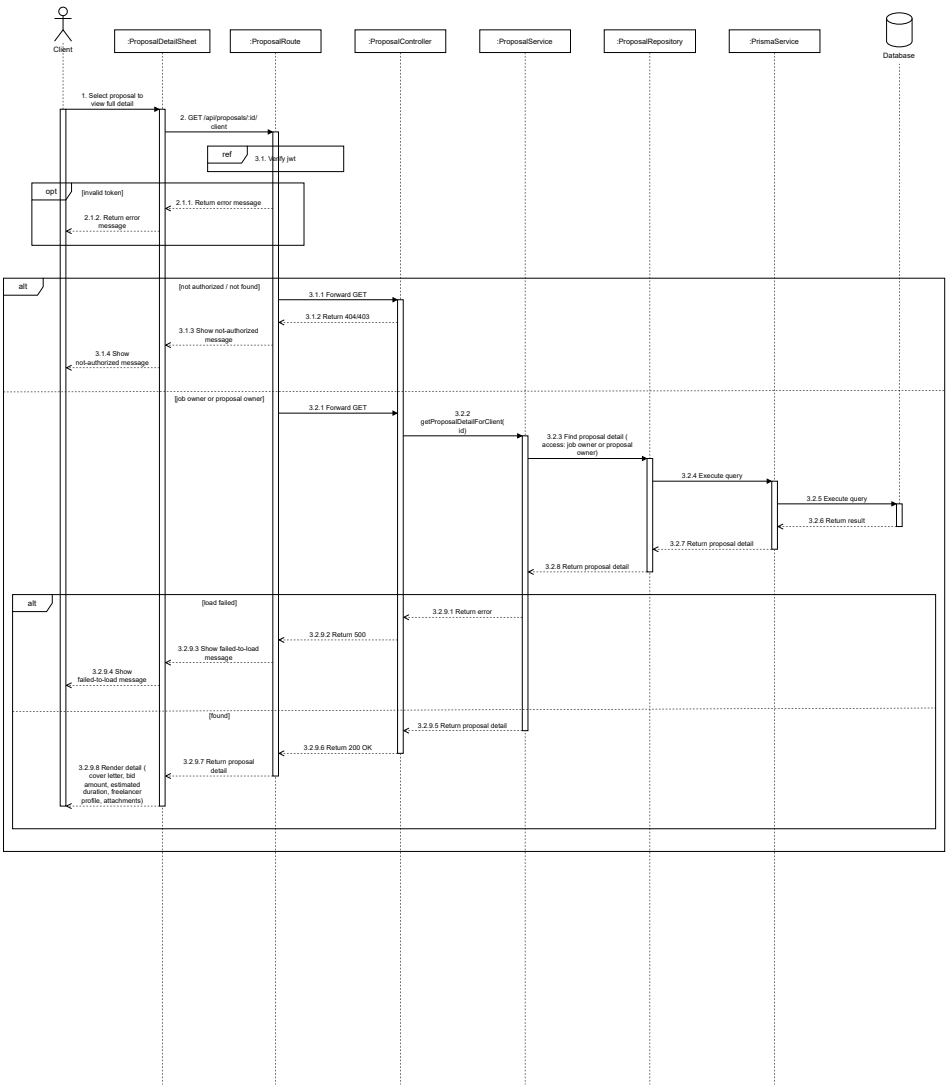

Figure 129: Sequence diagram of UC-32.09 View Proposal Detail

3.18 User Management

User Management provides the administrator with a searchable user catalogue, a consolidated account detail view, and dedicated editors for the Client and Freelancer profiles attached to an account.

3.18.0.1 Class Diagram

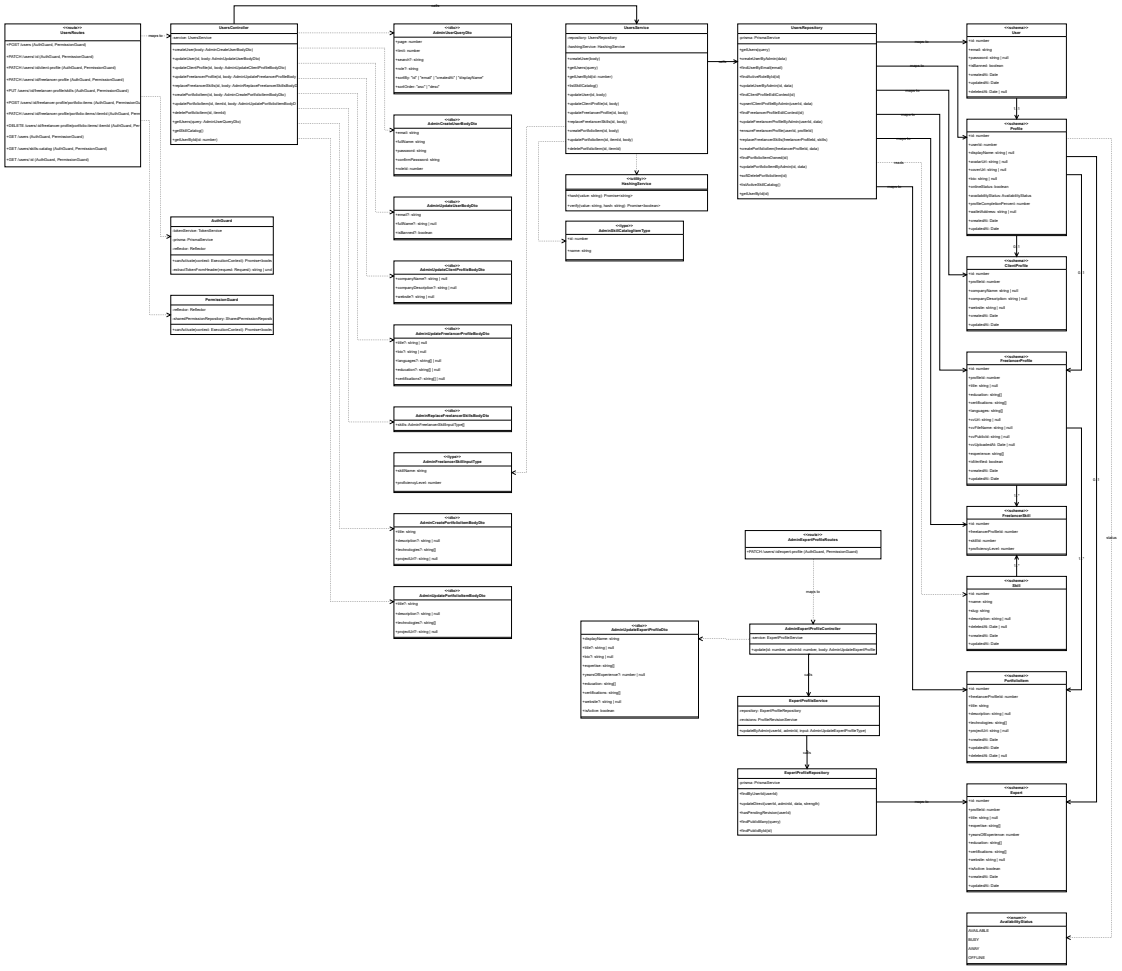

Figure 130: Class diagram of User Management

3.18.0.2 UC-28.01 – View User List

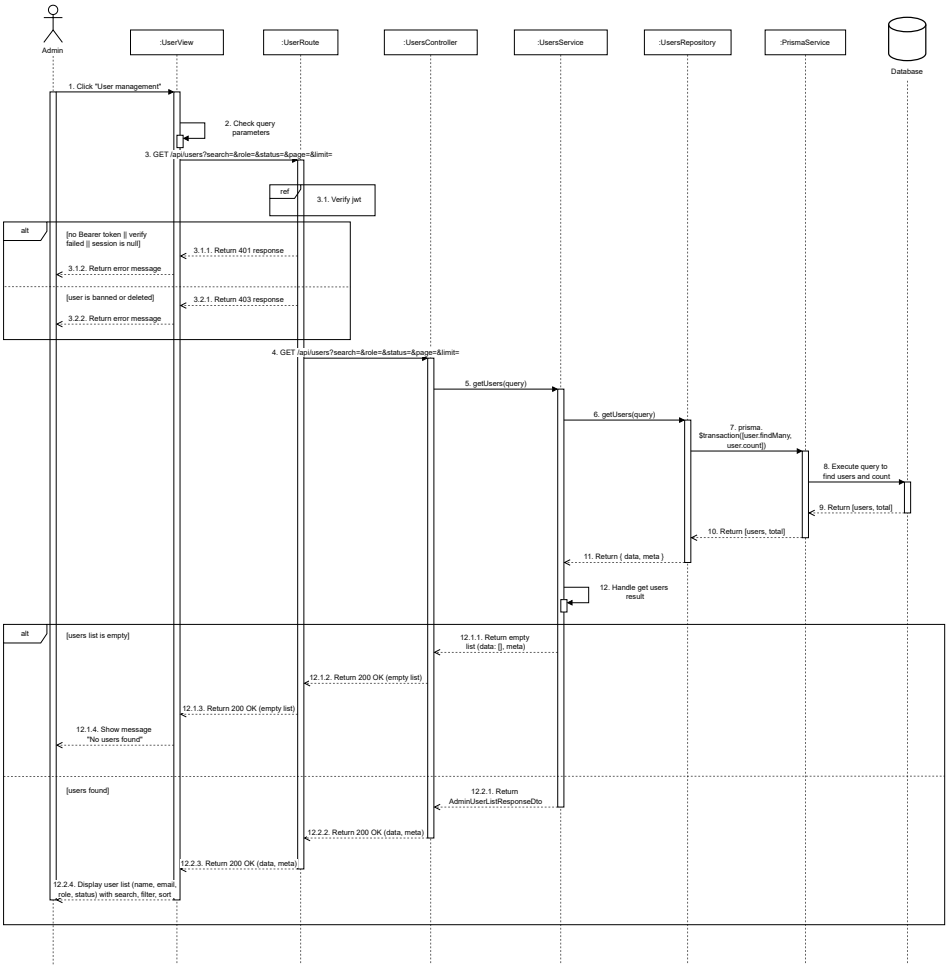

Figure 131: Sequence diagram of UC-28.01 View User List

3.18.0.3 UC-28.02 – View User Detail

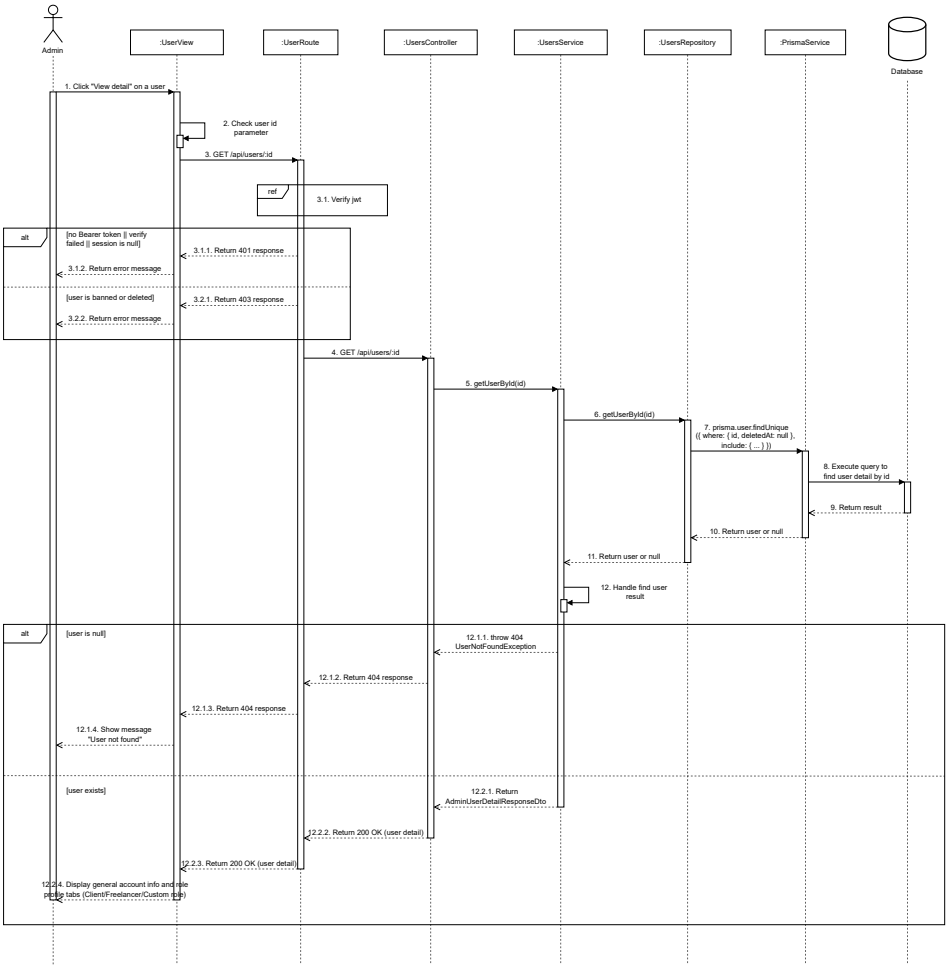

Figure 132: Sequence diagram of UC-28.02 View User Detail

3.18.0.4 UC-28.03 – Create User

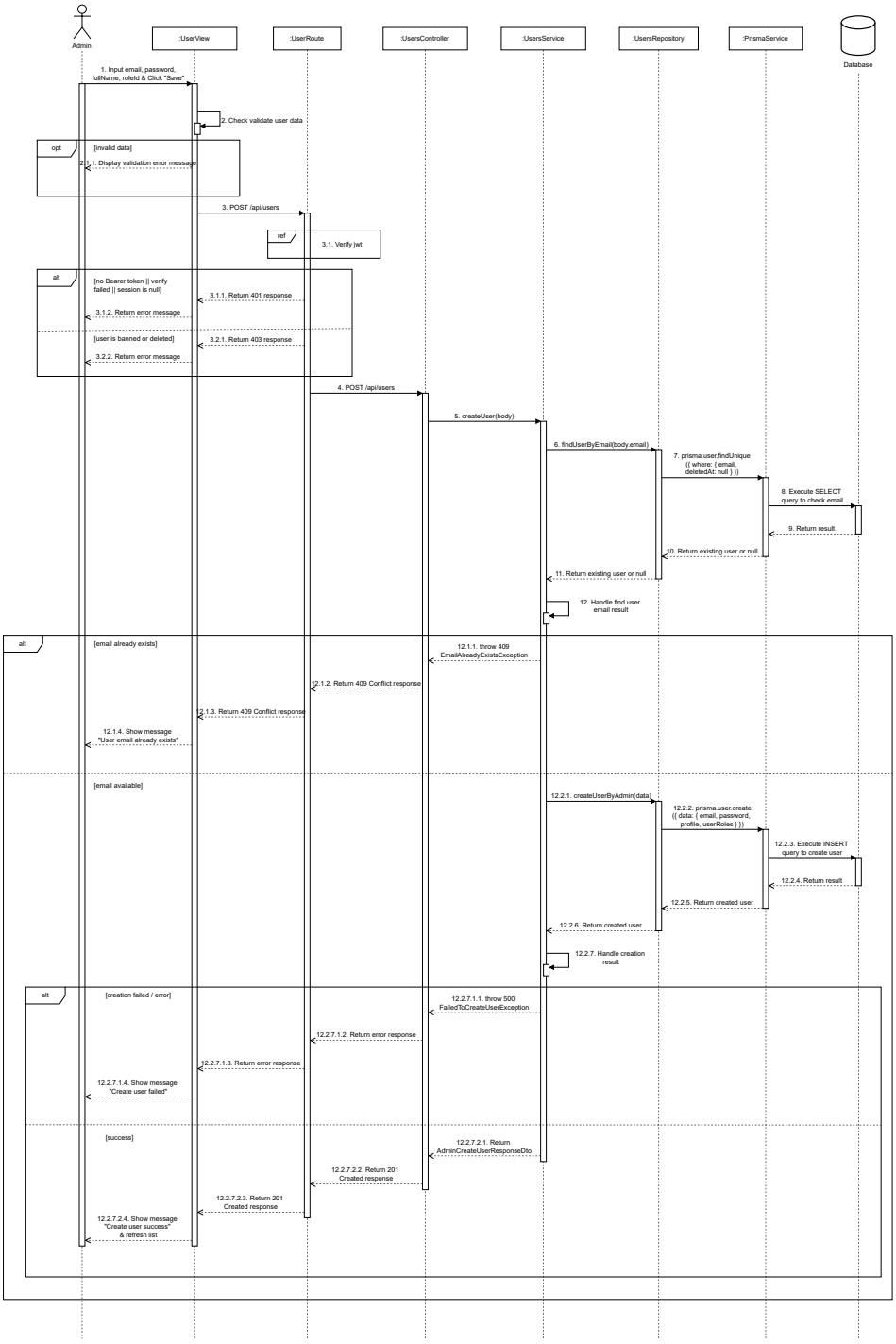

Figure 133: Sequence diagram of UC-28.03 Create User

3.18.0.5 UC-28.04 – Update User

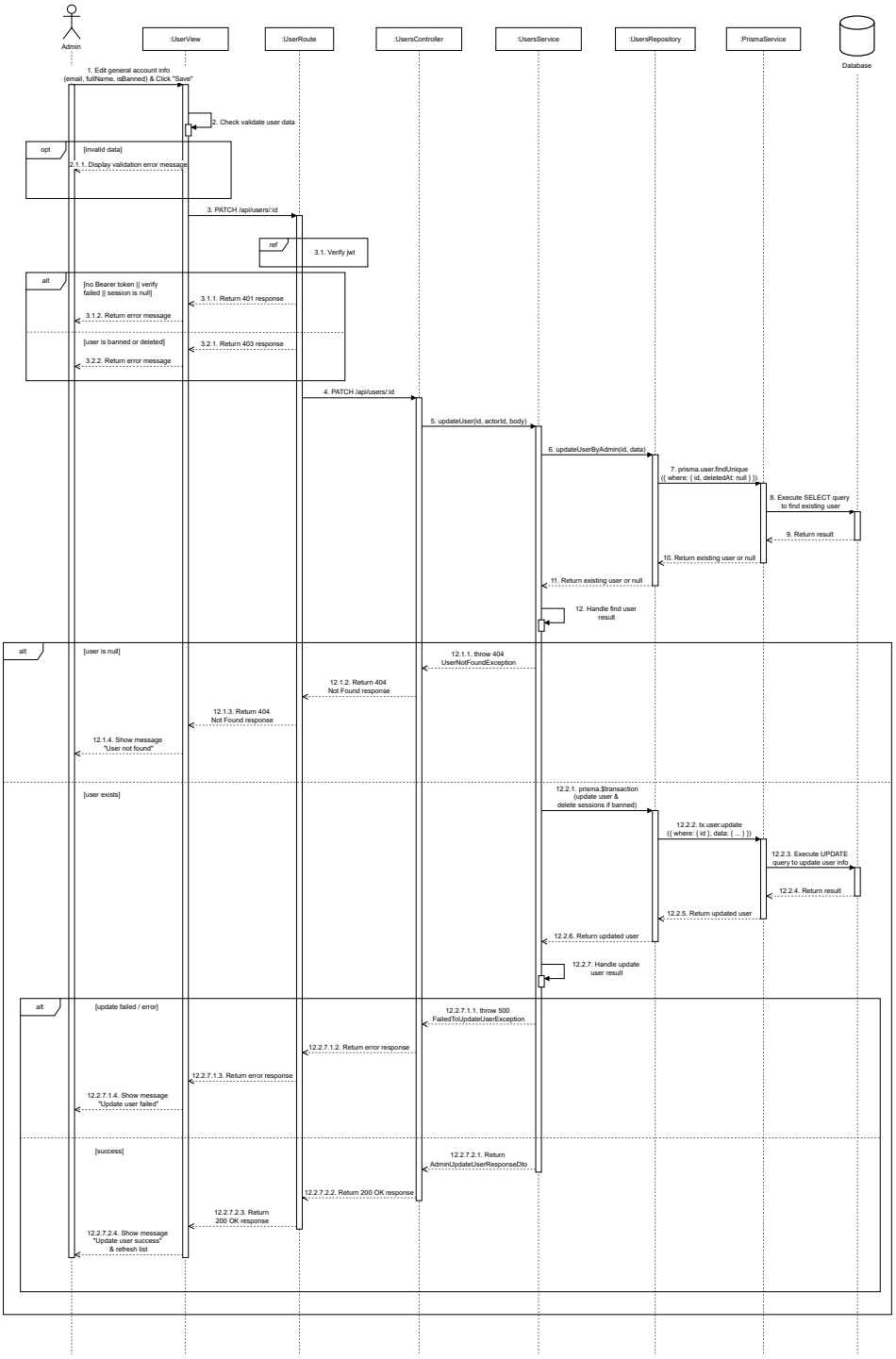

Figure 134: Sequence diagram of UC-28.04 Update User

3.18.0.6 UC-28.05 – Update Client Profile

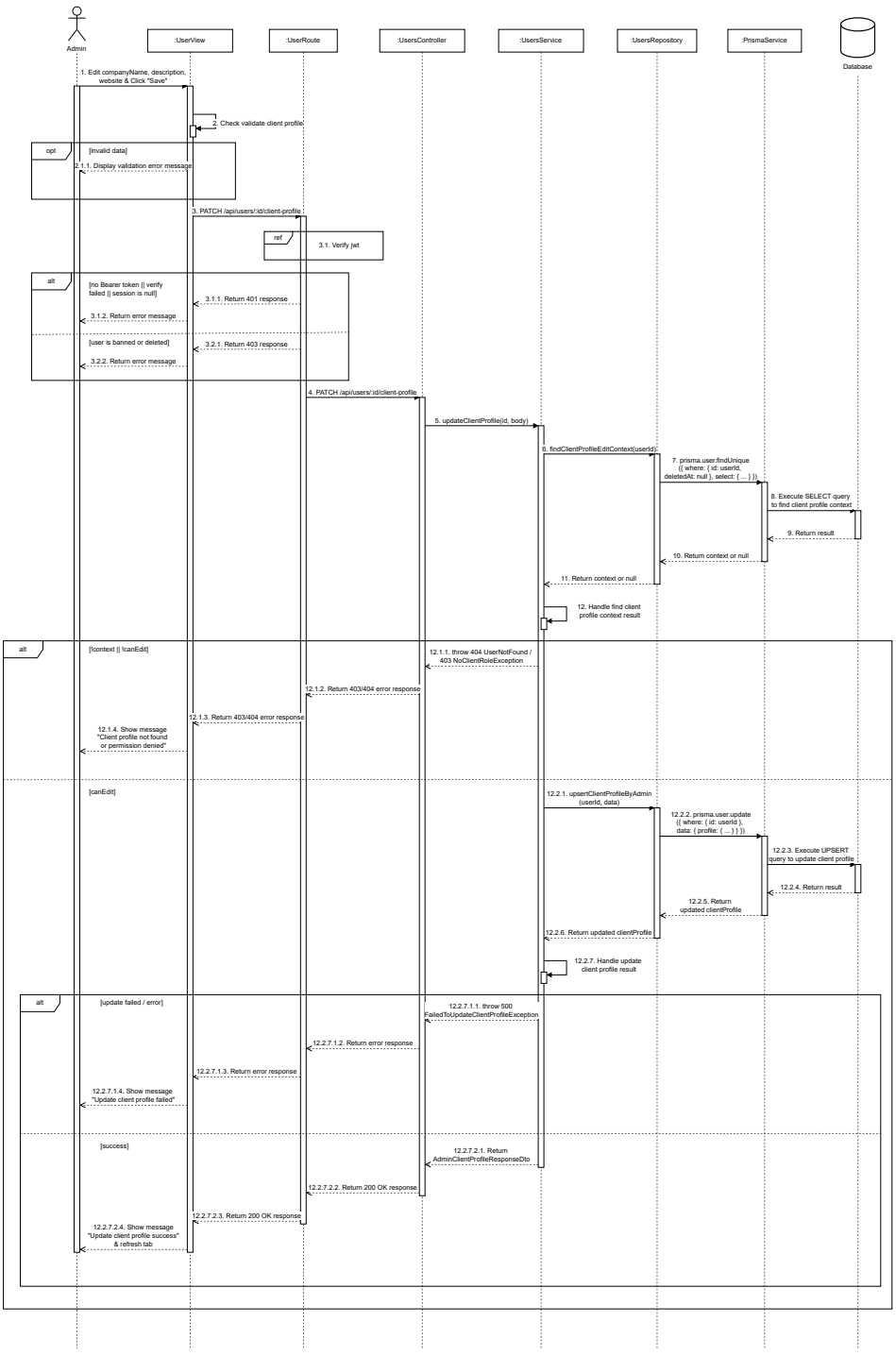

Figure 135: Sequence diagram of UC-28.05 Update Client Profile

3.18.0.7 UC-28.06 – Update Freelancer Basic Info

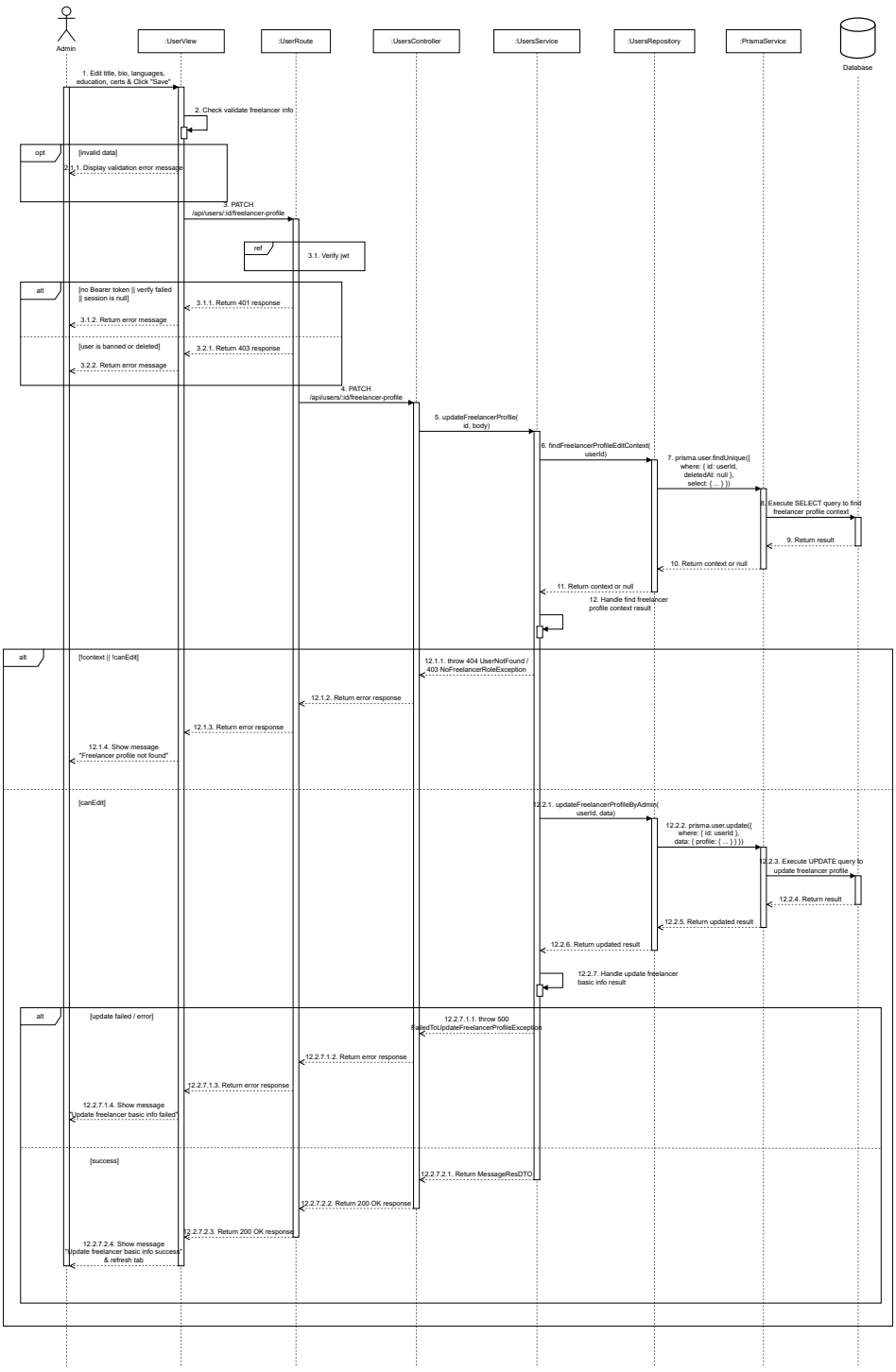

Figure 136: Sequence diagram of UC-28.06 Update Freelancer Basic Info

3.18.0.8 UC-28.07 – Update Freelancer Skills

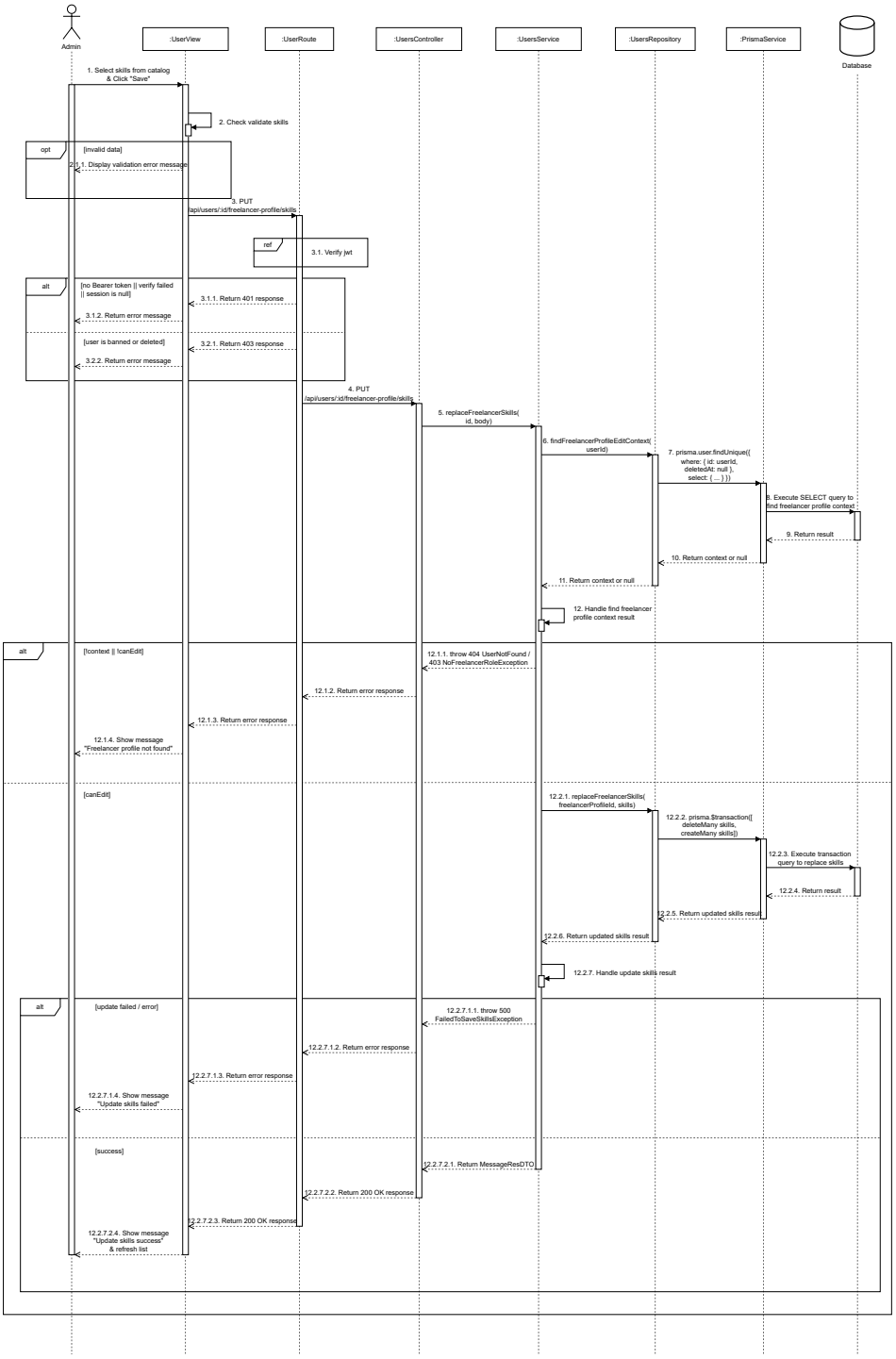

Figure 137: Sequence diagram of UC-28.07 Update Freelancer Skills

3.18.0.9 UC-28.08 – Update Freelancer Portfolio

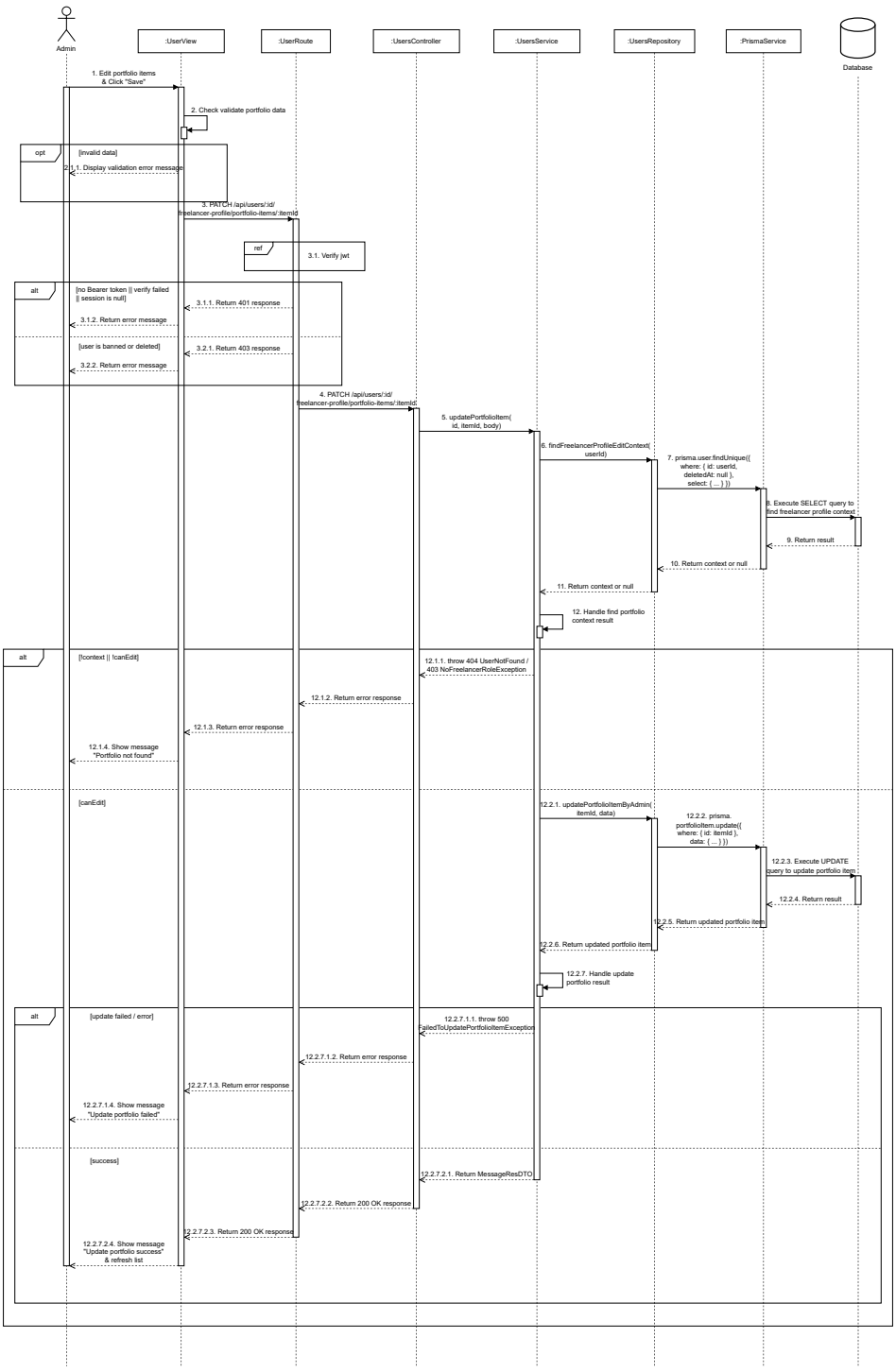

Figure 138: Sequence diagram of UC-28.08 Update Freelancer Portfolio

3.19 Banner Management

Banner Management maintains the advertisement banners shown in configured page positions. The module supports image upload, lifecycle dates, activation state, and hard deletion.

3.19.0.1 Class Diagram

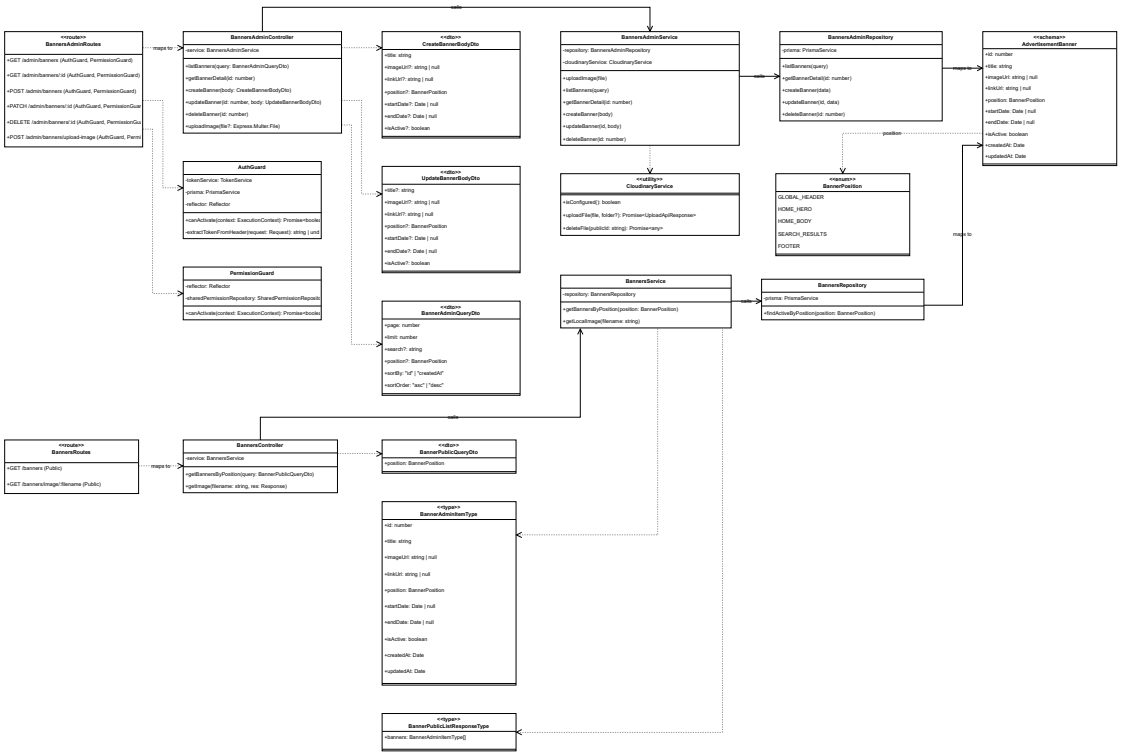

Figure 139: Class diagram of Banner Management

3.19.0.2 UC-37.01 – View Banner List

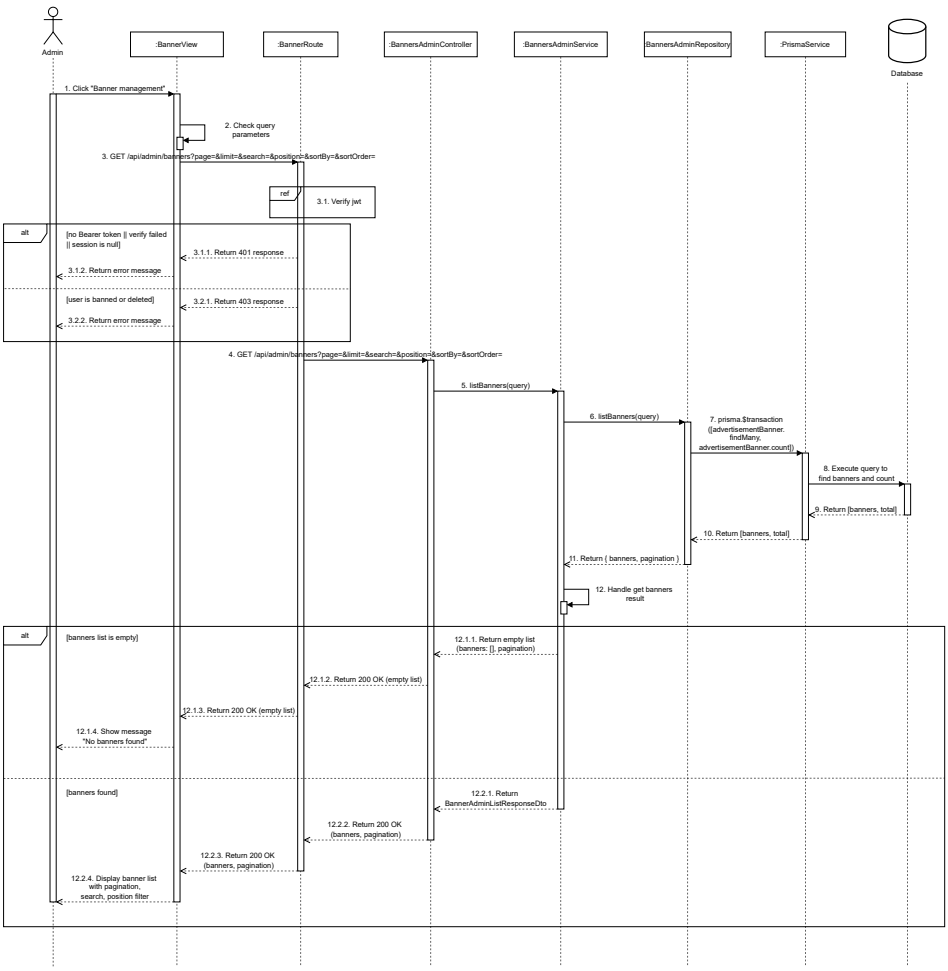

Figure 140: Sequence diagram of UC-37.01 View Banner List

3.19.0.3 UC-37.02 – View Banner Detail

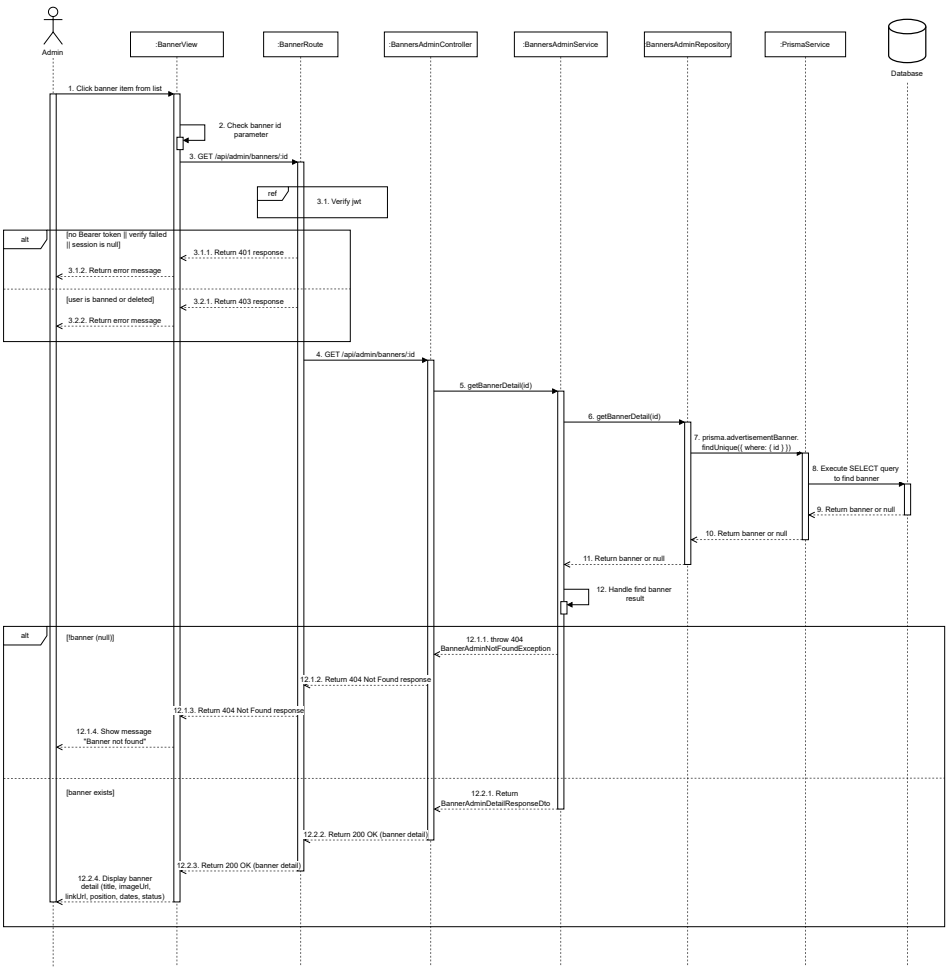

Figure 141: Sequence diagram of UC-37.02 View Banner Detail

3.19.0.4 UC-37.03 – Create Banner

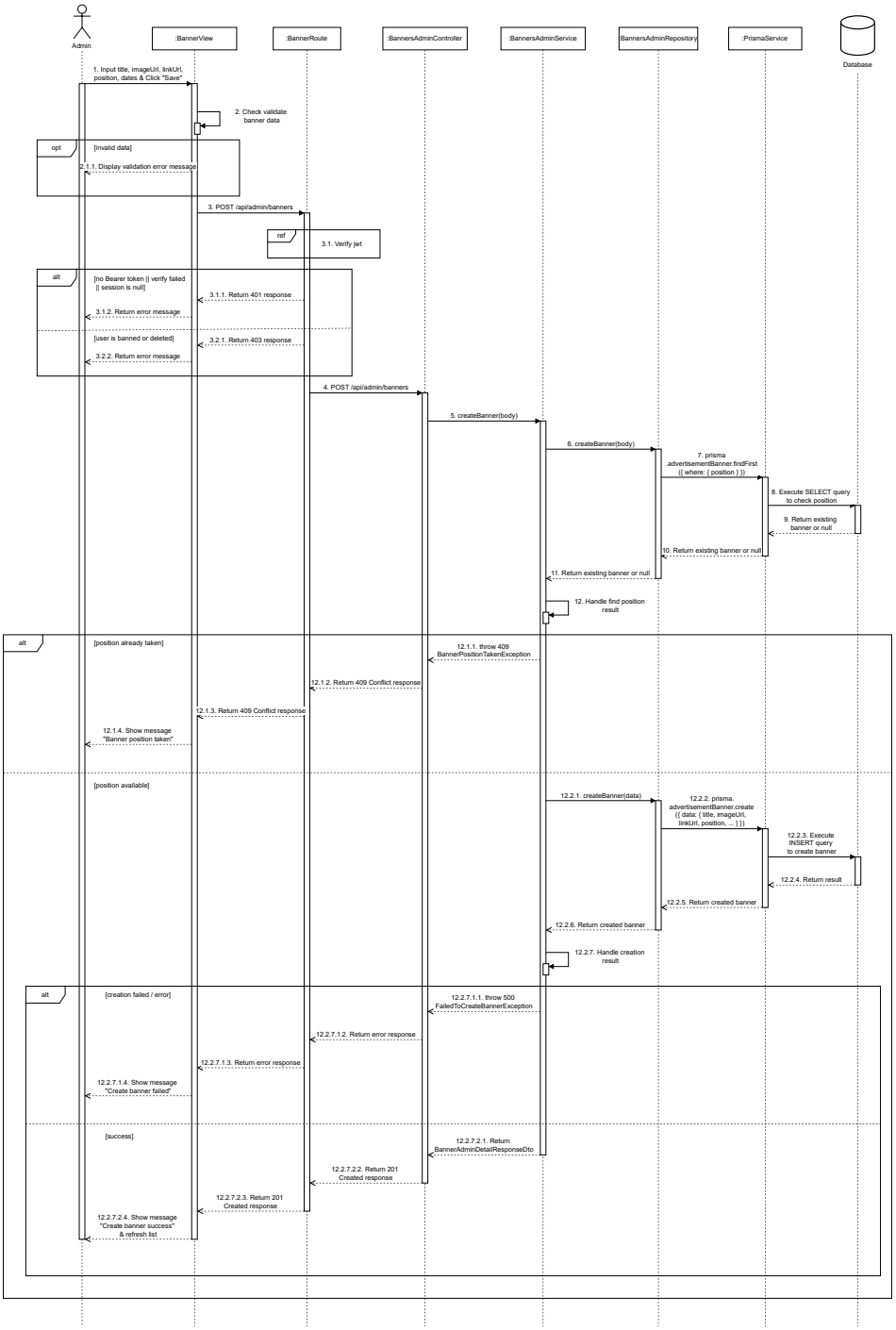

Figure 142: Sequence diagram of UC-37.03 Create Banner

3.19.0.5 UC-37.04 – Update Banner

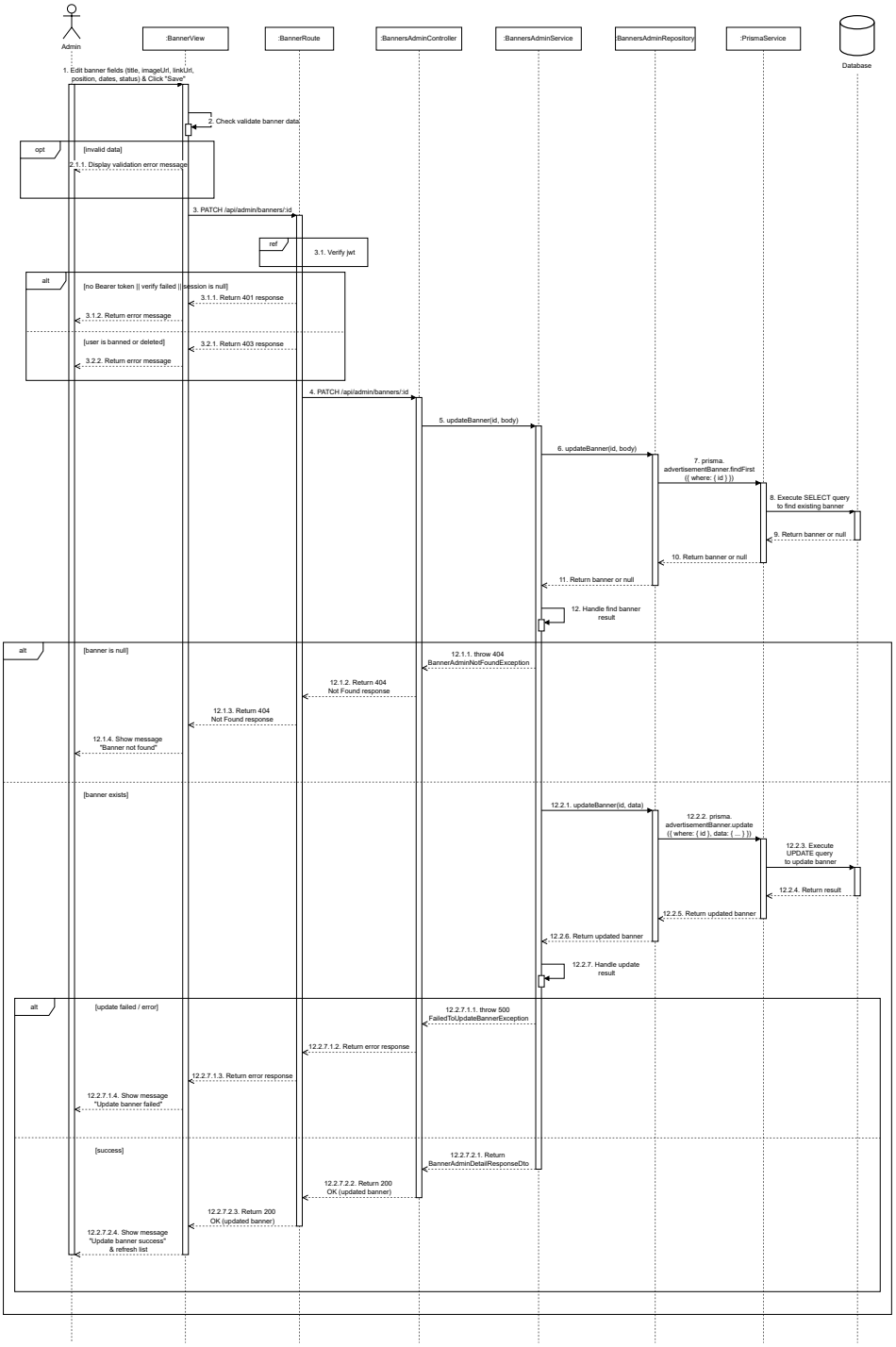

Figure 143: Sequence diagram of UC-37.04 Update Banner

3.19.0.6 UC-37.05 – Delete Banner

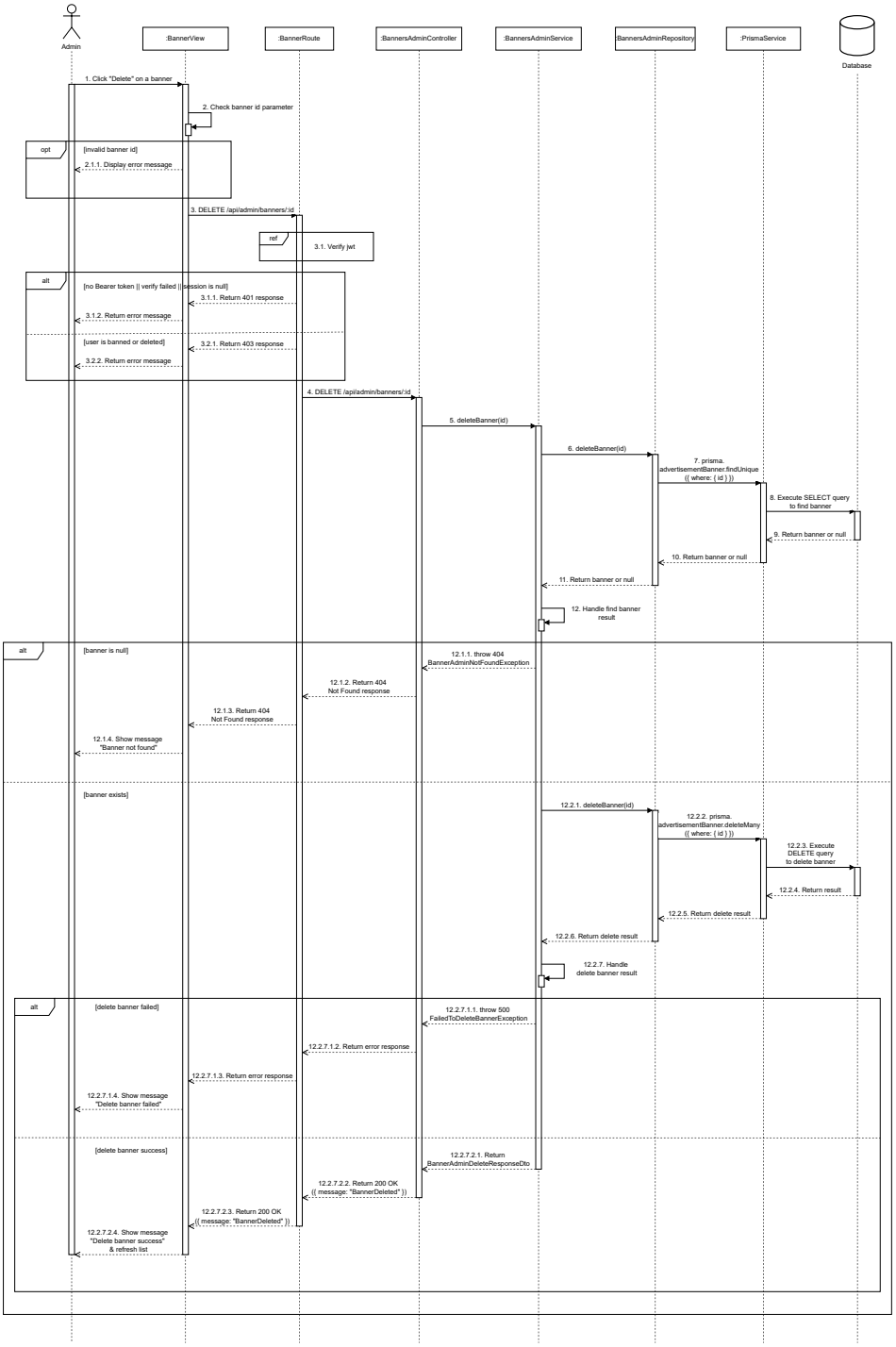

Figure 144: Sequence diagram of UC-37.05 Delete Banner

3.20 Skill Management

Skill Management owns the master skill catalogue used by freelancer profiles and job postings. Skills are soft-deleted so that existing references remain valid, and a deleted skill can be restored.

3.20.0.1 Class Diagram

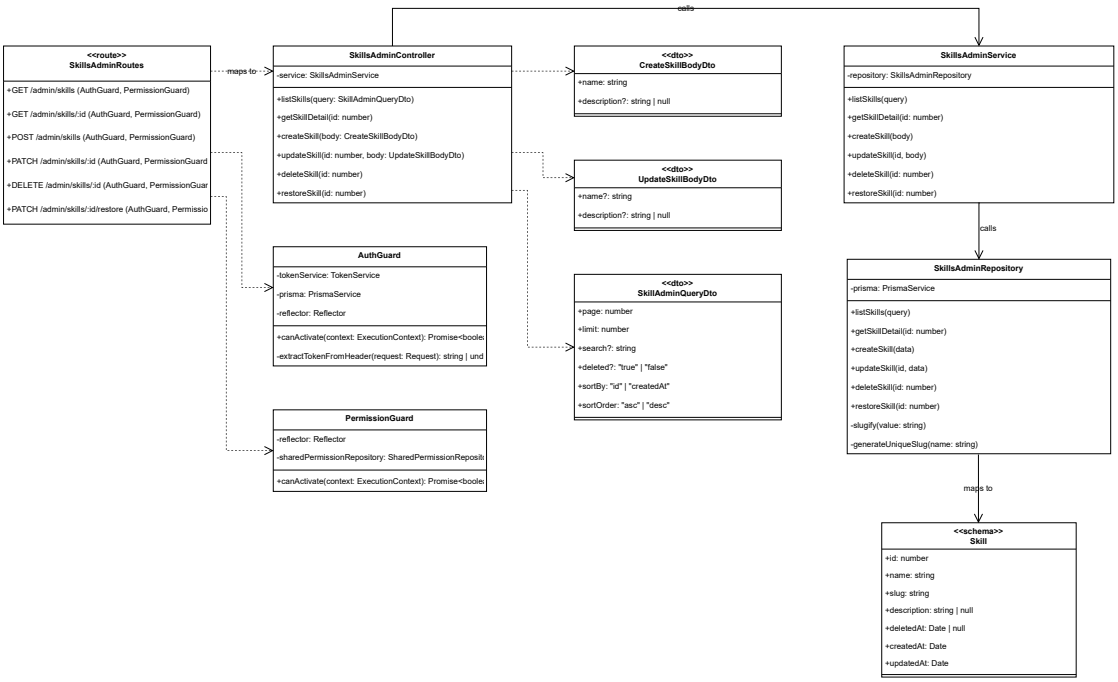

Figure 145: Class diagram of Skill Management

3.20.0.2 UC-38.01 – View Skill List

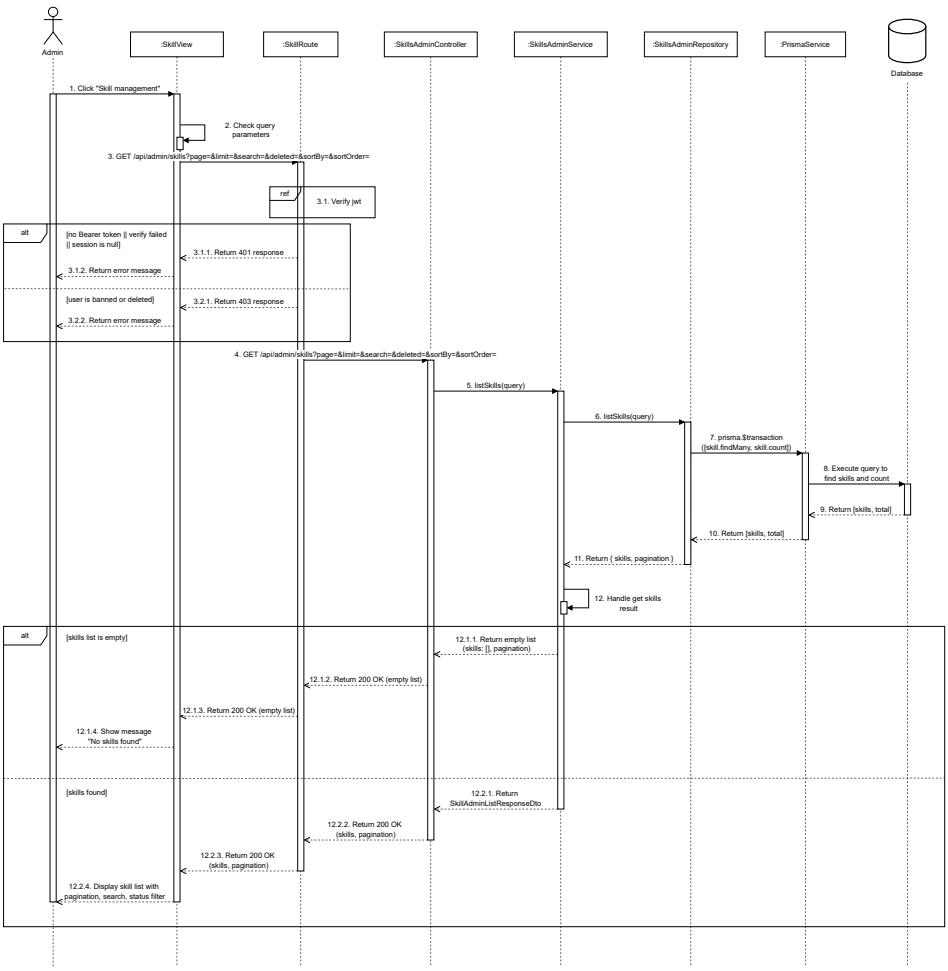

Figure 146: Sequence diagram of UC-38.01 View Skill List

3.20.0.3 UC-38.02 – View Skill Detail

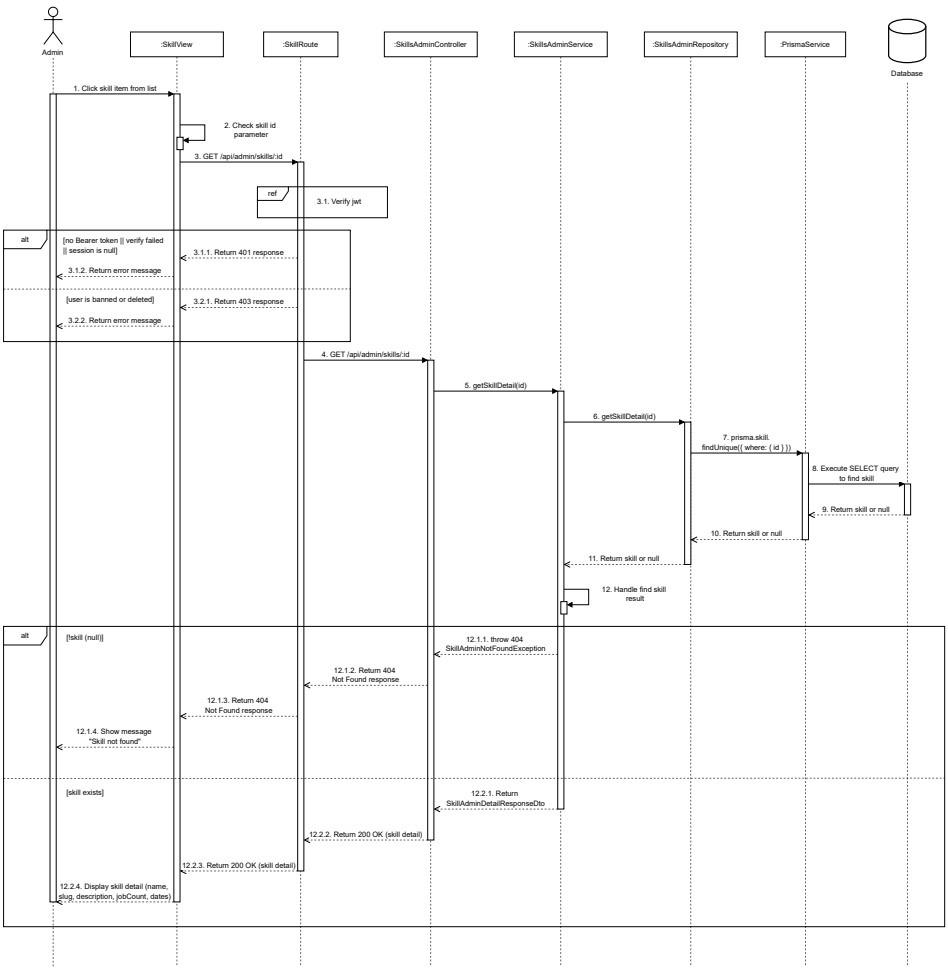

Figure 147: Sequence diagram of UC-38.02 View Skill Detail

3.20.0.4 UC-38.03 – Create Skill

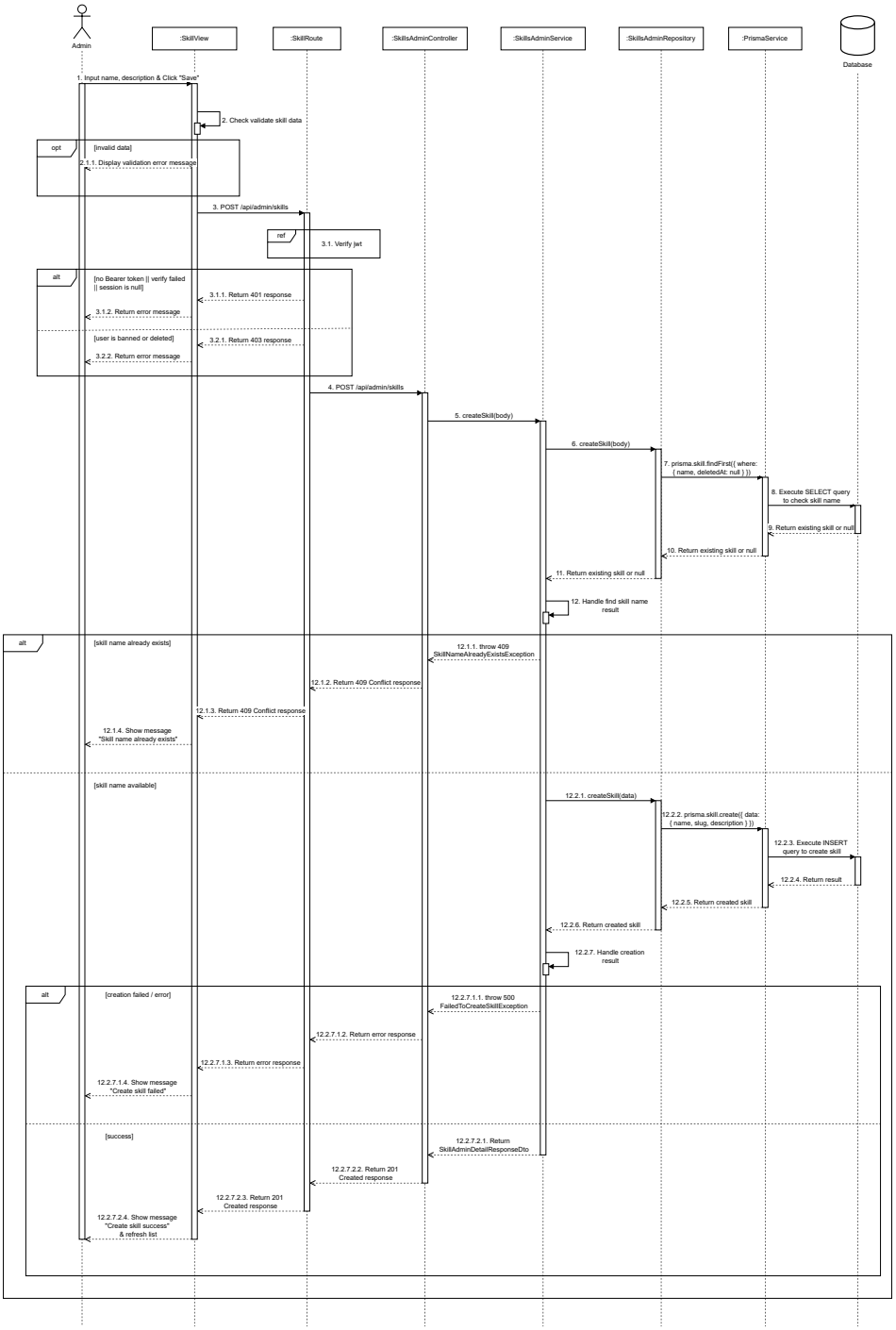

Figure 148: Sequence diagram of UC-38.03 Create Skill

3.20.0.5 UC-38.04 – Update Skill

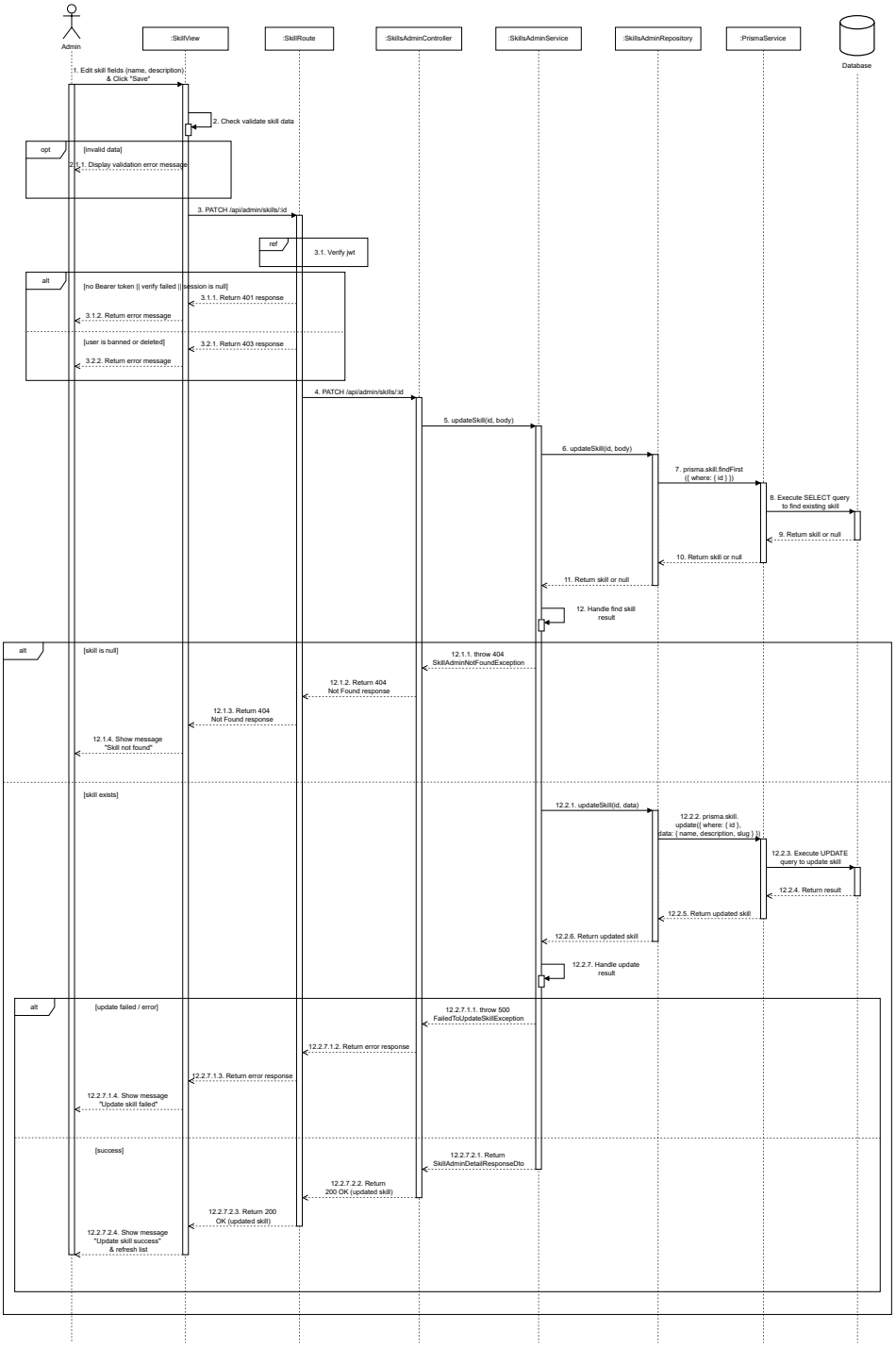

Figure 149: Sequence diagram of UC-38.04 Update Skill

3.20.0.6 UC-38.05 – Delete Skill

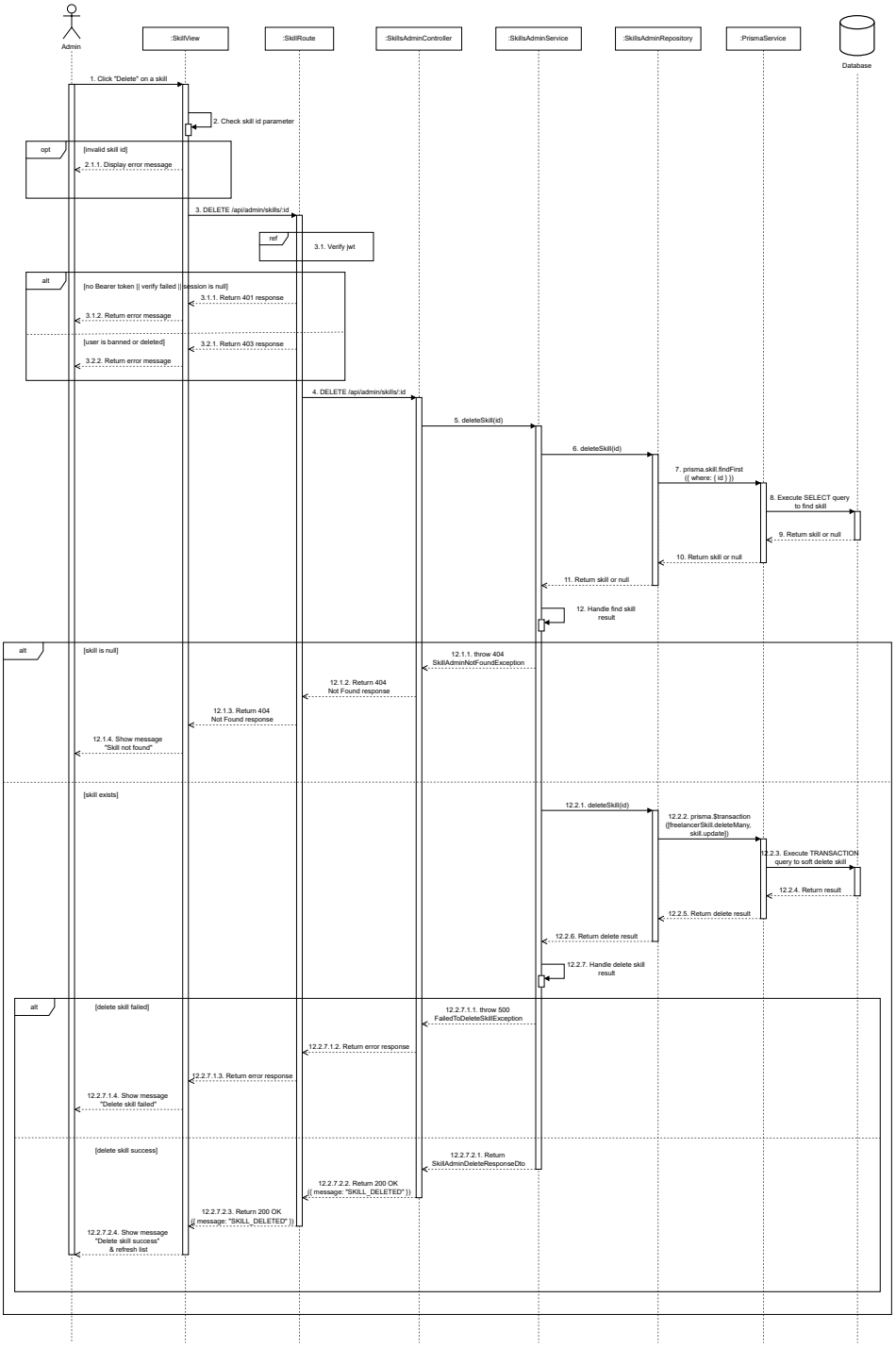

Figure 150: Sequence diagram of UC-38.05 Delete Skill

3.20.0.7 UC-38.06 – Restore Skill

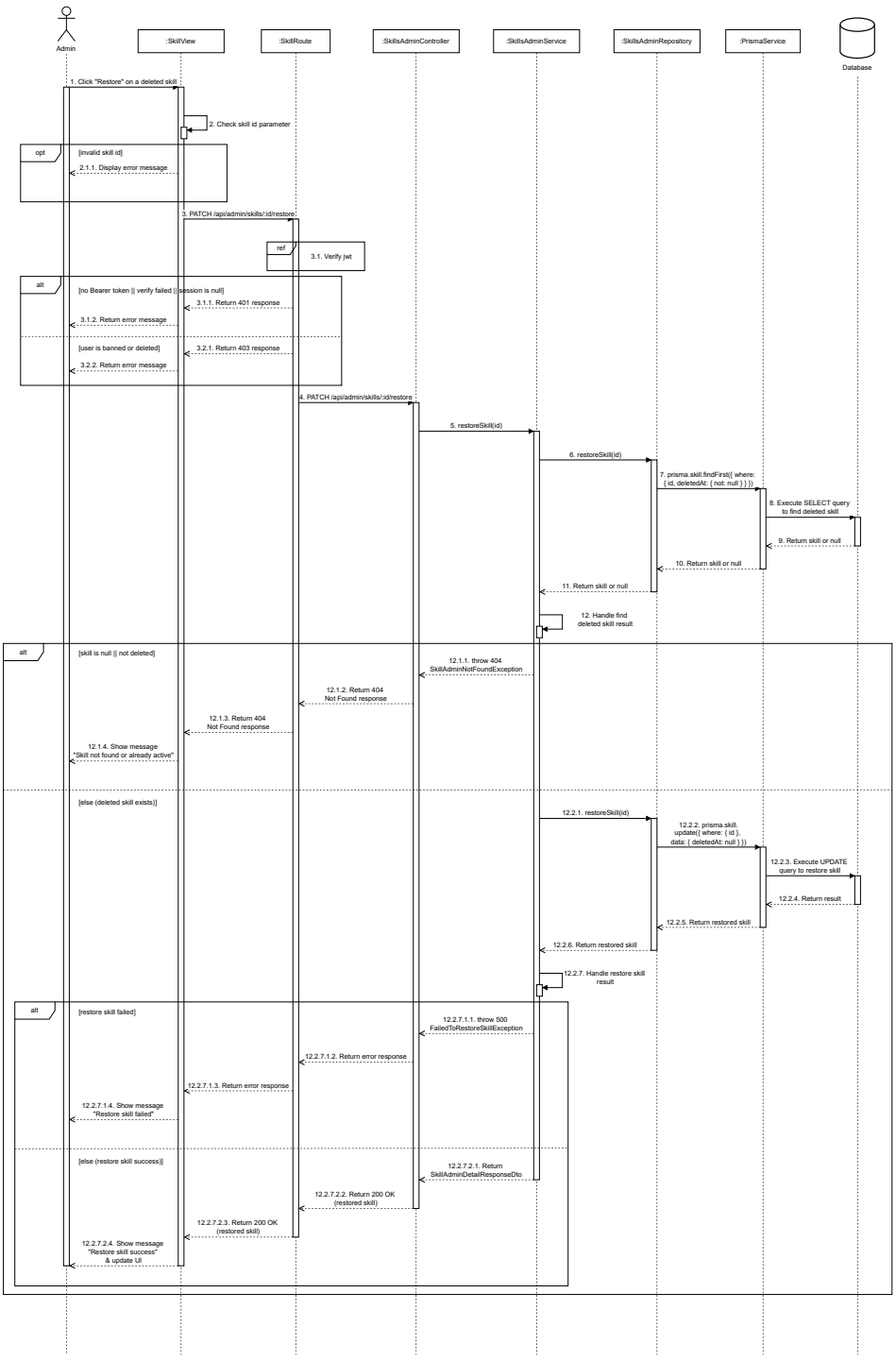

Figure 151: Sequence diagram of UC-38.06 Restore Skill

3.21 Expert Profile Management

The administrator-side Expert Profile editor updates an expert account directly. Any pending self-service revision is superseded, and the account owner is notified of the administrative change.

3.21.0.1 Class Diagram

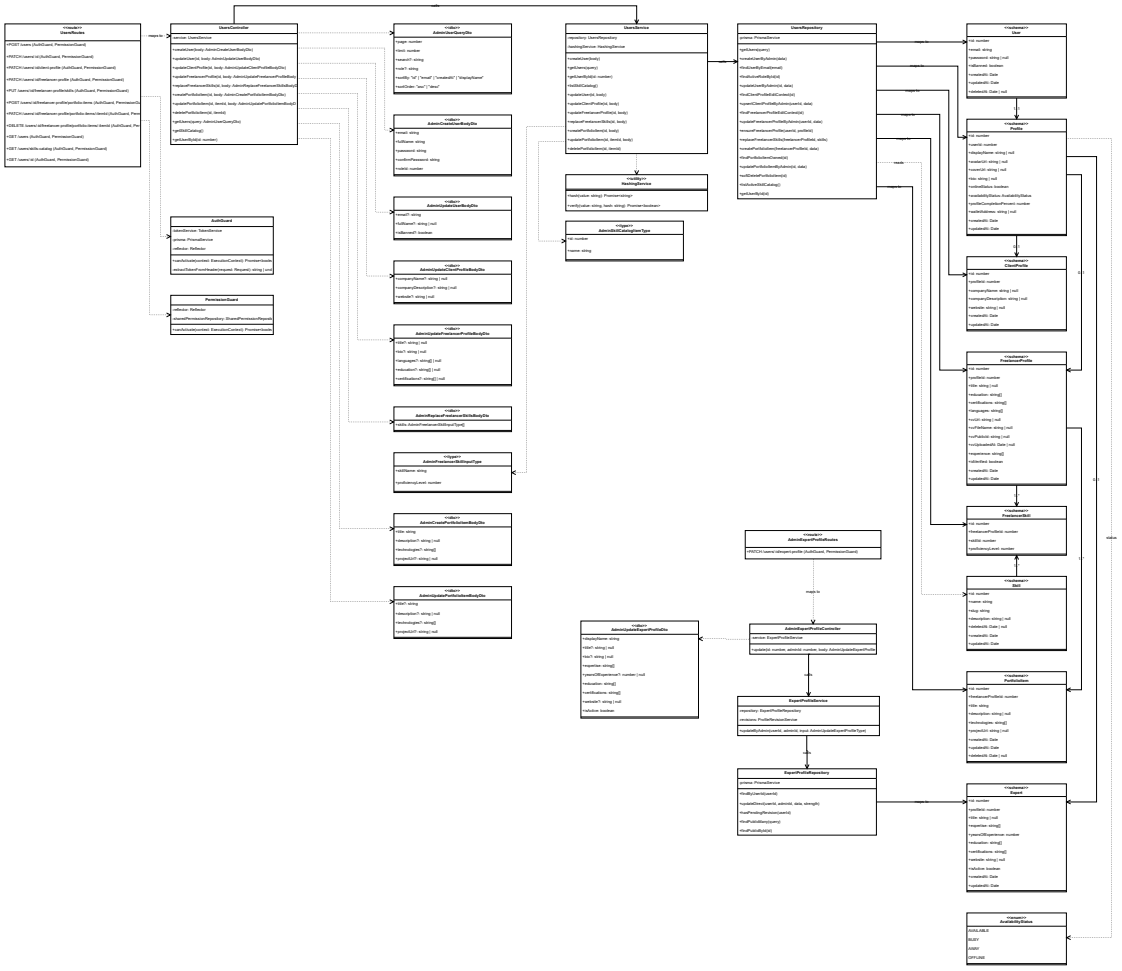

Figure 152: Class diagram of Expert Profile Management

3.21.0.2 UC-28.09 – Update Expert Profile

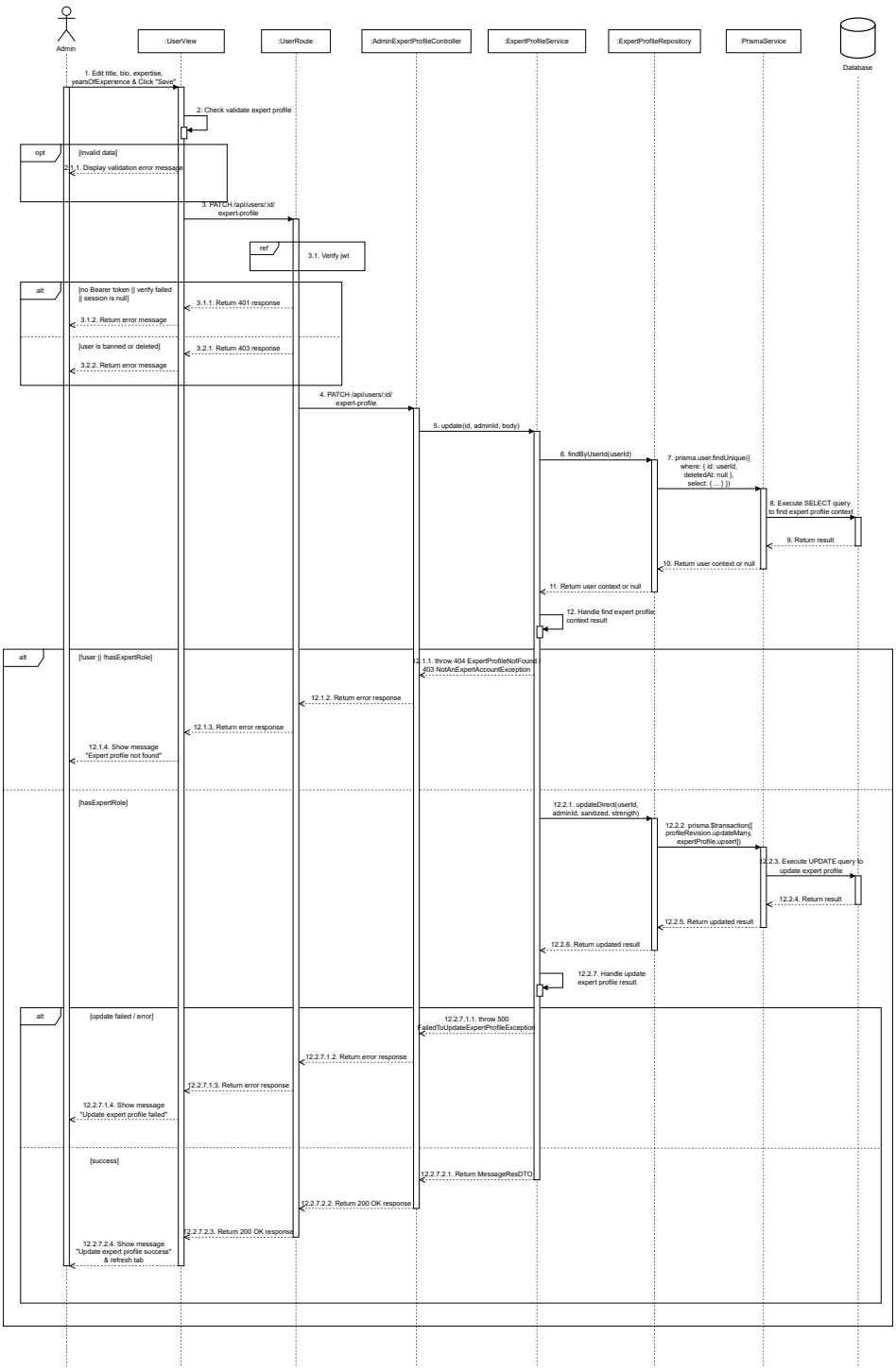

Figure 153: Sequence diagram of UC-28.09 Update Expert Profile

3.22 Job Category Management

Job Category Management maintains the taxonomy used by job postings. A category can be created, updated, soft-deleted, and restored; deletion is blocked while active jobs still reference the category.

3.22.0.1 Class Diagram

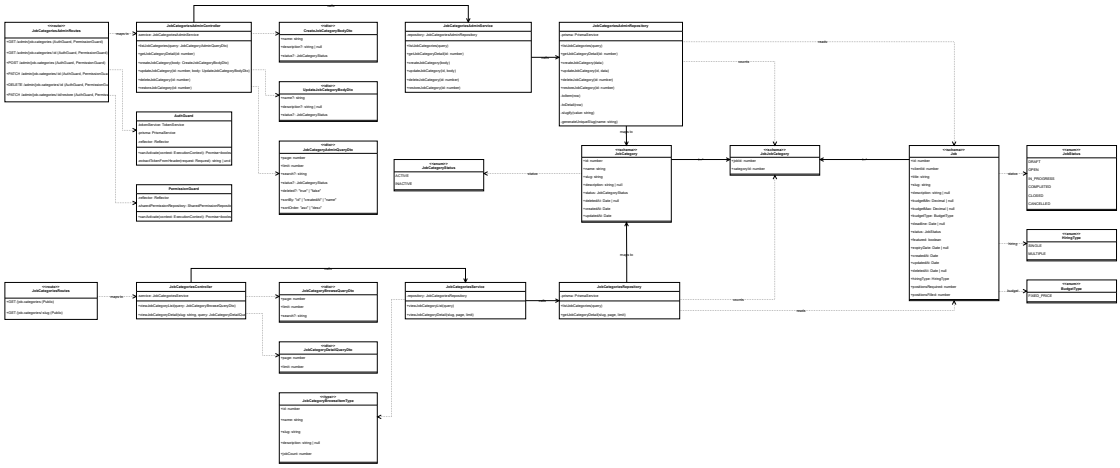

Figure 154: Class diagram of Job Category Management

3.22.0.2 UC-46.01 – View Job Category List

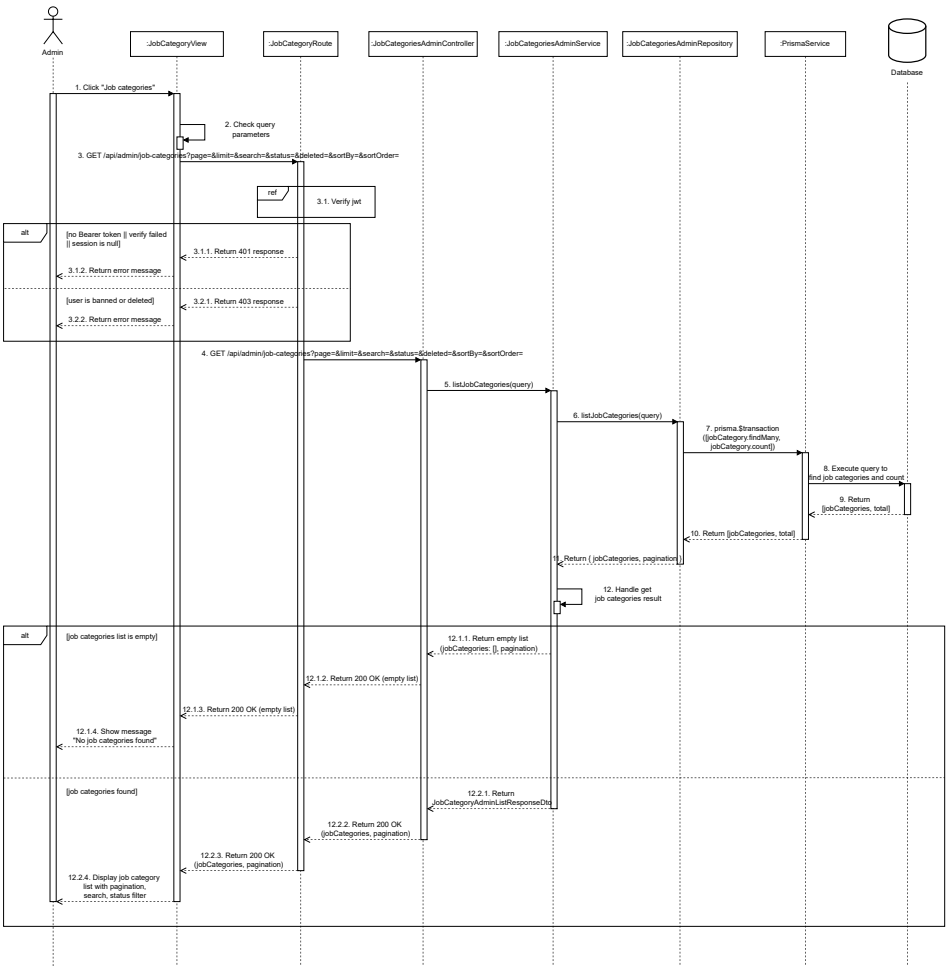

Figure 155: Sequence diagram of UC-46.01 View Job Category List

3.22.0.3 UC-46.02 – View Job Category Detail

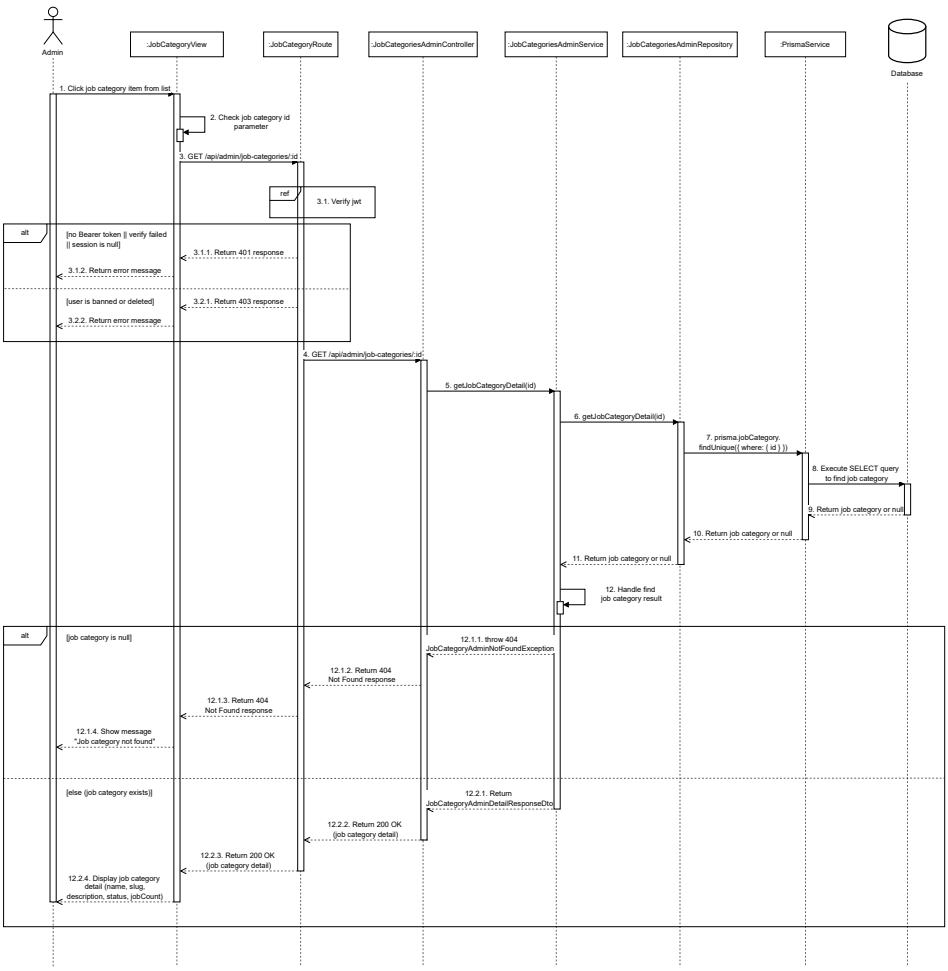

Figure 156: Sequence diagram of UC-46.02 View Job Category Detail

3.22.0.4 UC-46.03 – Create Job Category

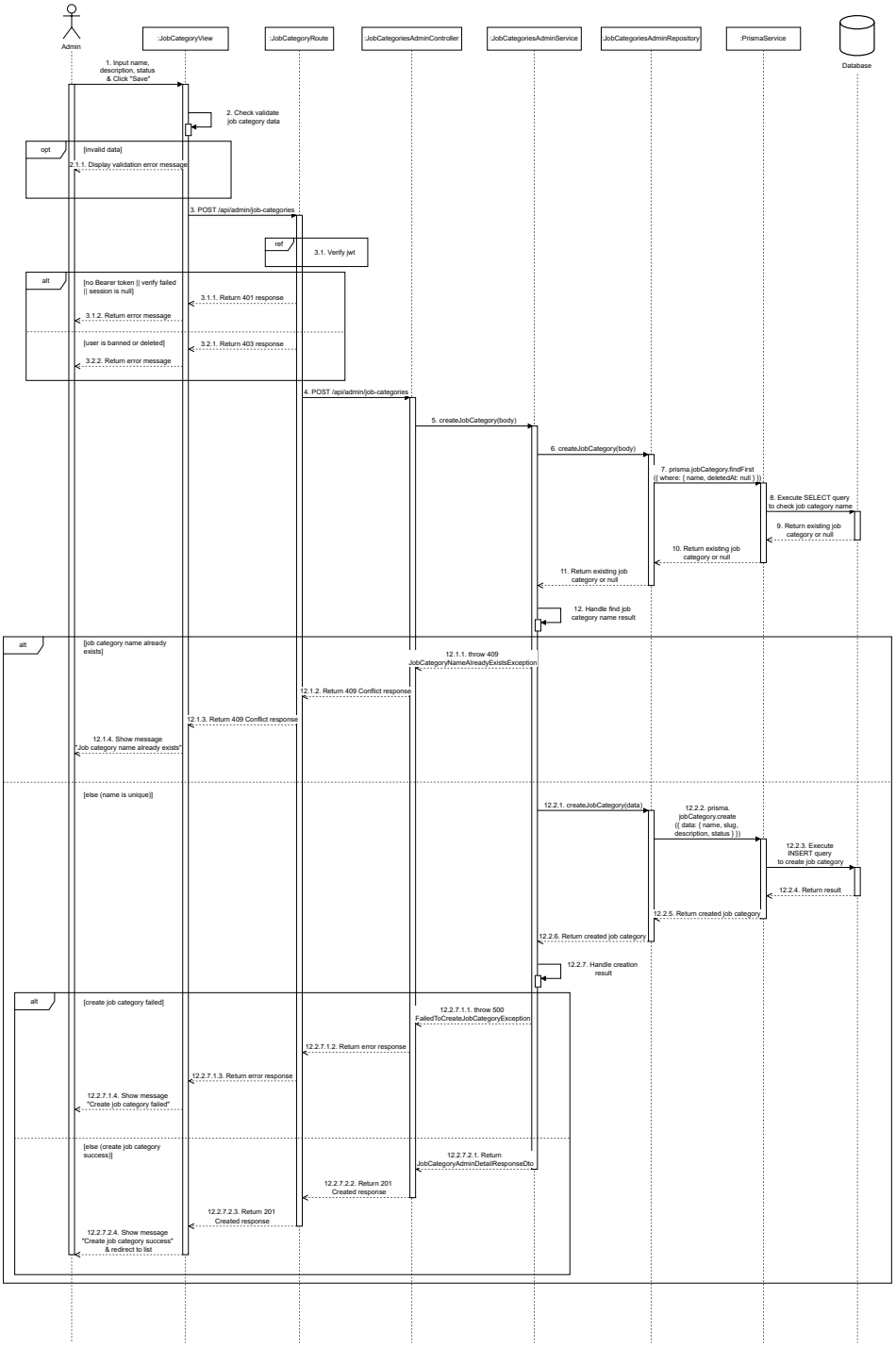

Figure 157: Sequence diagram of UC-46.03 Create Job Category

3.22.0.5 UC-46.04 – Update Job Category

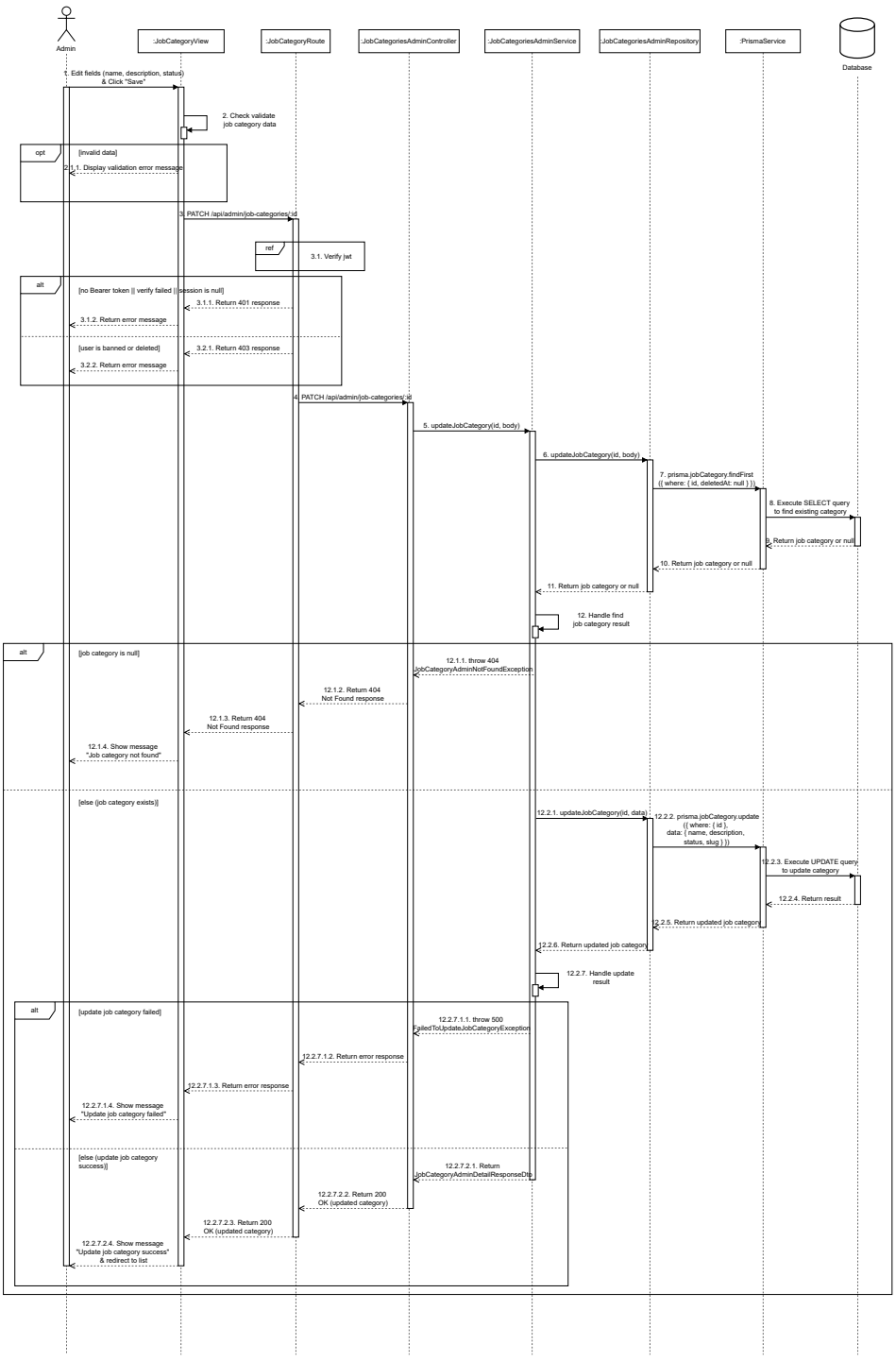

Figure 158: Sequence diagram of UC-46.04 Update Job Category

3.22.0.6 UC-46.05 – Delete Job Category

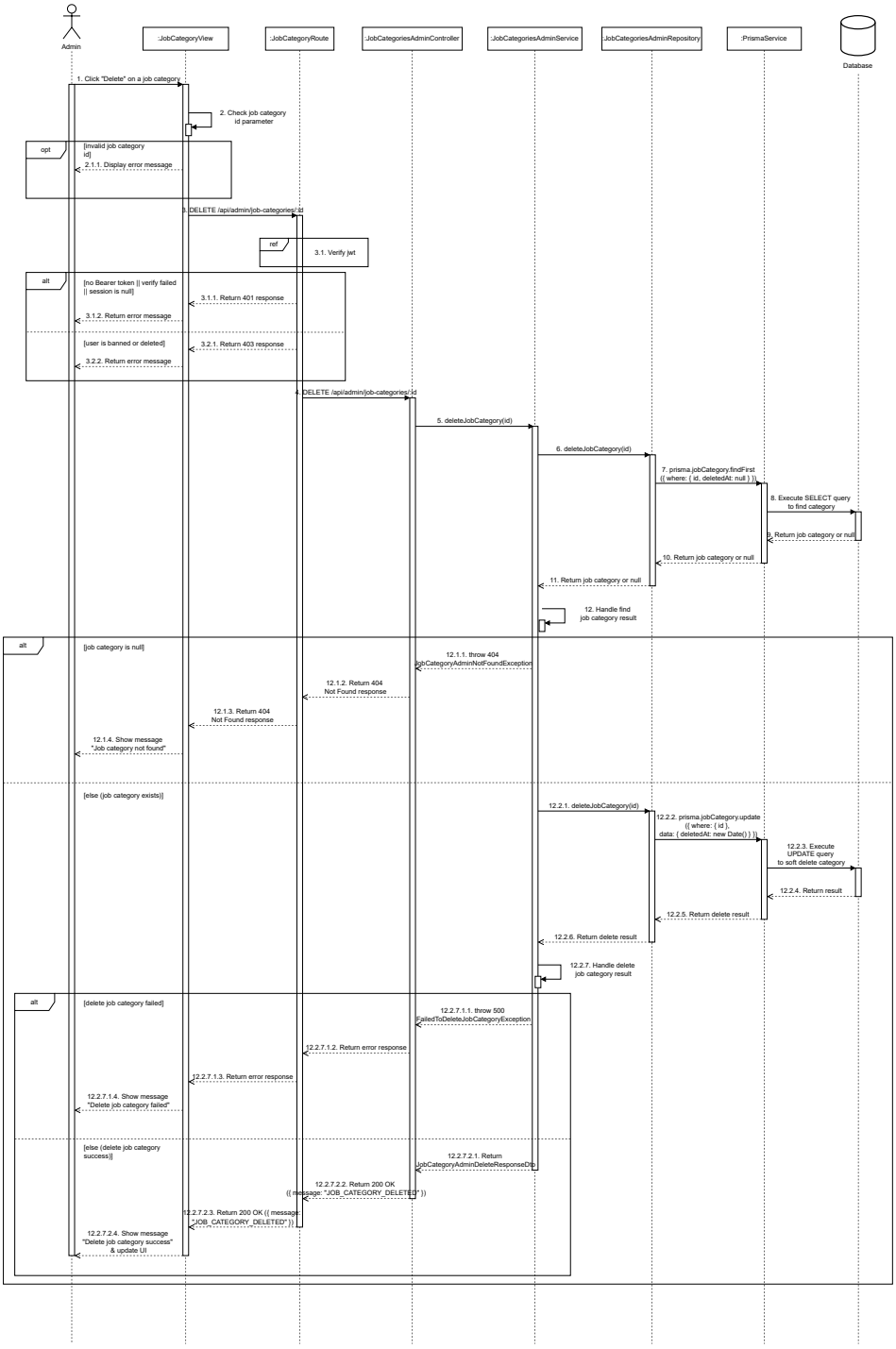

Figure 159: Sequence diagram of UC-46.05 Delete Job Category

3.22.0.7 UC-46.06 – Restore Job Category

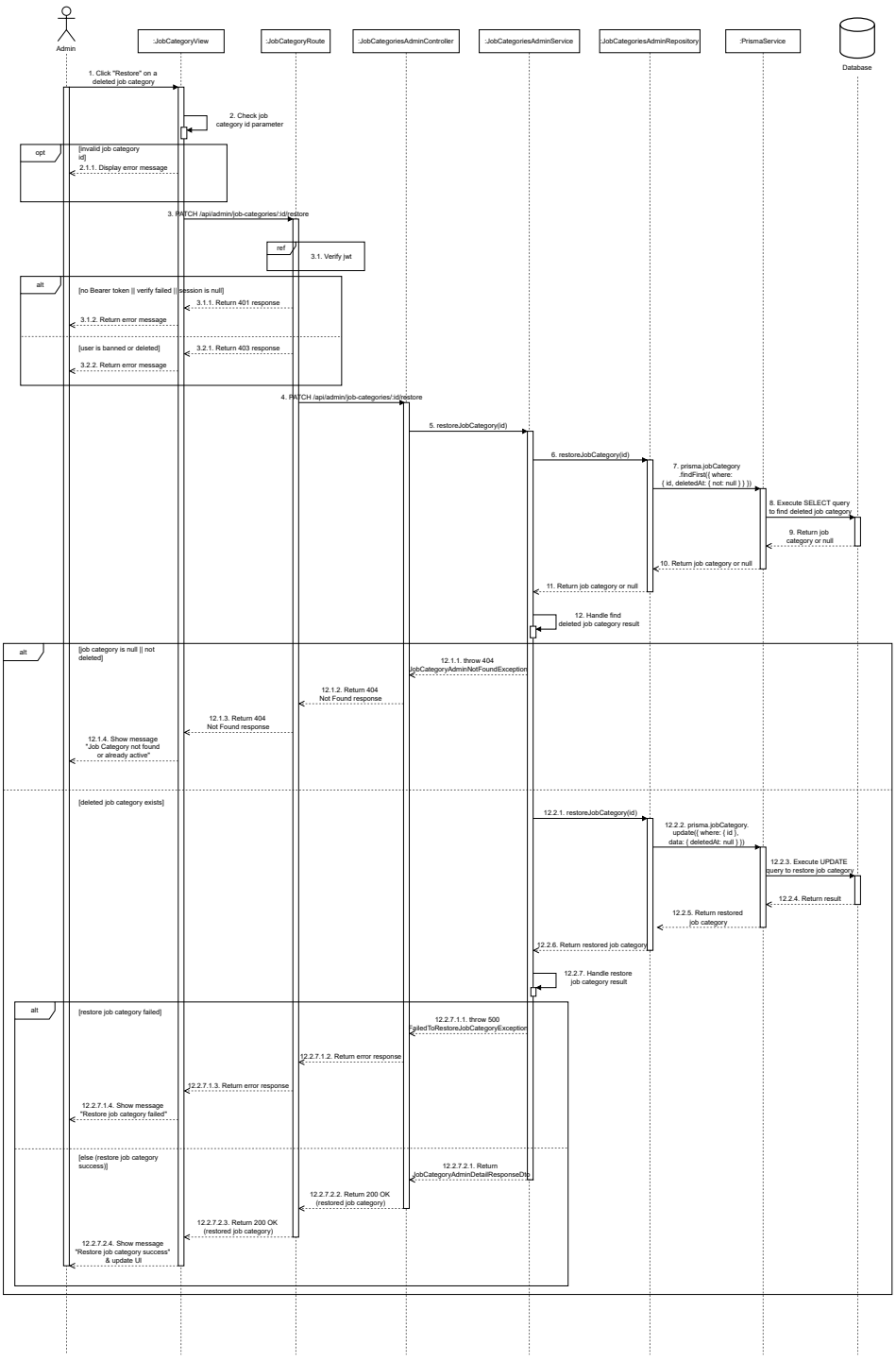

Figure 160: Sequence diagram of UC-46.06 Restore Job Category

4. Reference

[1] OMG, “About the Unified Modeling Language Specification Version 2.5.1,” Object Management Group, Dec. 2017. [Online]. Available: https://www.omg.org/spec/UML/2.5.1/

[2] NestJS, “NestJS Documentation - A progressive Node.js framework,” 2024. [Online]. Available: https://docs.nestjs.com/

[3] Next.js, “Next.js by Vercel - The React Framework for the Web,” 2024. [Online]. Available: https://nextjs.org/docs

[4] Prisma, “Prisma - Next-generation ORM for Node.js and TypeScript,” 2024. [Online]. Available: https://www.prisma.io/docs

[5] PostgreSQL Global Development Group, “PostgreSQL 16 Documentation,” 2024. [Online]. Available: https://www.postgresql.org/docs/16/index.html

[6] Redis, “Redis Documentation and Command Reference,” 2024. [Online]. Available: https://redis.io/docs/

[7] Stripe, “Stripe API Reference - Escrow Payments and Connect Payouts,” 2024. [Online]. Available: https://stripe.com/docs/api

[8] Google Developers, “Google Identity - Using OAuth 2.0 for Web Server Applications,” 2024. [Online]. Available: https://developers.google.com/identity/protocols/oauth2

[9] Tailwind CSS, “Tailwind CSS v4 Documentation,” 2024. [Online]. Available: https://tailwindcss.com/docs